Learn Being Forward

COURSES · CAMPUS PARTNERSHIPS · PLACEMENTS

COURSE NOTES · BASIC TO ADVANCED

HTML5
Web Foundations

Complete notes for the Learn Being Forward HTML5 syllabus: every module, topic and subtopic explained step by step, with diagrams, verified examples and a practice checkpoint for each module.

17	3	28	197
modules	levels	minimum days	examples rendered

How to use these notes

These notes follow the **Learn Being Forward HTML5 syllabus** exactly: the same 17 modules, in the same order, with every topic and subtopic explained step by step. Each subtopic number (for example **3.7.2**) matches the syllabus, so you can always find where you are.

Level	Modules	Minimum	Outcome
1. Beginner	1 to 7	9 days	Build structured web pages with text, links, images, lists, tables and basic forms
2. Intermediate	8 to 12	9 days	Create validated forms, semantic layouts, multimedia pages, SEO-ready metadata and accessible markup
3. Advanced	13 to 17	10 days	Optimise performance, use modern interactive elements and HTML5 APIs, follow best practices and deploy a professional portfolio
Total	1 to 17	28 days	About 5 weeks at 6 study days per week

Read
Read the concept for each subtopic and note the key points in your own words.

Type & run
Type out and run every example. Do not copy and paste code. Compare with the OUTPUT box.

Checkpoint
Finish the practice checkpoint at the end of each module before moving on.

Revise
At the end of each level, revise its modules and build one small project.

Every HTML example in these notes was opened in a real browser (Chromium) and the picture under the code, framed as a browser window, is a screenshot of what that browser displayed. Examples that use JavaScript (Module 15) were run in the same browser, and the BROWSER CONSOLE box shows what they printed. Pages are shown with the browser's default styles, exactly as plain HTML looks before any CSS is added; small differences in fonts and form controls between browsers and operating systems are normal.

HOW TO READ THE BOXES
Dark boxes with a gold top line are **HTML code**. Dashed cream boxes labelled **OUTPUT** show what the program prints. Gold-bordered boxes are notes and tips, red boxes warn about common mistakes, and navy boxes hold the key idea to remember.

Course plan at a glance

Module	Topic	Level	Min days
01	Introduction to the web and HTML	Beginner	1
02	Setup and HTML document structure	Beginner	1
03	Text content and formatting	Beginner	1
04	Links and navigation	Beginner	1
05	Images and basic media	Beginner	1
06	Lists and tables	Beginner	2

Module	Topic	Level	Min days
07	Forms basics	Beginner	2
08	Advanced forms and validation	Intermediate	2
09	Semantic HTML5 and page structure	Intermediate	2
10	Multimedia and embedded content	Intermediate	2
11	The head, metadata and SEO	Intermediate	1
12	Accessibility (a11y)	Intermediate	2
13	Responsive images and performance	Advanced	1
14	Interactive elements and modern HTML	Advanced	2
15	HTML5 APIs with JavaScript	Advanced	3
16	Best practices, validation and tooling	Advanced	1
17	Projects and interview preparation	Advanced	3

Table of contents

LEVEL 1 · BEGINNER LEVEL · MODULES 1 TO 7

Module 01 · Introduction to the web and HTML 7

1.1 How the web works 7
1.2 What is HTML? 10
1.3 History of HTML 13
1.4 Why do we need HTML? 15
1.5 What HTML5 added 15
1.6 Where is HTML used? 17

Module 02 · Setup and HTML document structure 19

2.1 Tools 19
2.2 Elements, tags and attributes 20
2.3 Basic page structure 24
2.4 Your first web page 26
2.5 Writing clean HTML 27
2.6 Block vs inline elements 29

Module 03 · Text content and formatting 33

3.1 Headings 33
3.2 Paragraphs and breaks 35
3.3 Text-level semantics 37
3.4 Quotes and code 39
3.5 Contact and addresses 40
3.6 Character entities 41
3.7 Bidirectional text and language 43

Module 04 · Links and navigation 46

4.1 The anchor element 46
4.2 Link attributes 48
4.3 Types of links 49
4.4 Navigation menus 51
4.5 Link best practices 52

Module 05 · Images and basic media 58

5.1 The img element 58
5.2 Image formats 60
5.3 Figures and captions 63
5.4 Image links and image maps 63
5.5 Favicons 65

Module 06 · Lists and tables 68

6.1 Lists 68
6.2 Table basics 73
6.3 Table structure 74
6.4 Merging cells 75
6.5 Accessible tables 76
6.6 When to use tables 78

Module 07 · Forms basics 80

7.1 The form element 80

7.2 Labels and inputs 81
7.3 Common input types 83
7.4 Other form controls 85
7.5 Grouping 87

LEVEL 2 · INTERMEDIATE LEVEL · MODULES 8 TO 12

Module 08 · Advanced forms and validation 92
8.1 HTML5 input types 92
8.2 Input attributes 94
8.3 Suggestions and output 98
8.4 Built-in validation 100
8.5 Form attributes 103
8.6 Form usability and security 105

Module 09 · Semantic HTML5 and page structure 109
9.1 Why semantics matter 109
9.2 Sectioning elements 111
9.3 Choosing the right element 115
9.4 Common page layouts 116
9.5 Microdata and structured data 120

Module 10 · Multimedia and embedded content 124
10.1 Audio 124
10.2 Video 126
10.3 Embedding external content 129
10.4 Graphics 132
10.5 Media accessibility 134

Module 11 · The head, metadata and SEO 138
11.1 Essential meta tags 138
11.2 Linking resources 141
11.3 Social sharing 143
11.4 SEO fundamentals 145

Module 12 · Accessibility (a11y) 149
12.1 Accessibility basics 149
12.2 Accessible HTML 151
12.3 Keyboard accessibility 154
12.4 ARIA 156
12.5 Testing accessibility 159

LEVEL 3 · ADVANCED LEVEL · MODULES 13 TO 17

Module 13 · Responsive images and performance 164
13.1 Responsive images 164
13.2 Loading strategies 167
13.3 Measuring performance 169

Module 14 · Interactive elements and modern HTML 173
14.1 Disclosure widgets 173
14.2 Dialogs 175
14.3 Popovers 177
14.4 Other modern attributes 179
14.5 Templates and Web Components 181

Module 15 · HTML5 APIs with JavaScript 185

15.1 Working with the DOM (overview) 185
15.2 Canvas API 188
15.3 Web Storage 191
15.4 Geolocation API 194
15.5 Drag and Drop API 194
15.6 Media APIs 196
15.7 History API 197
15.8 Other APIs (overview) 198

Module 16 · Best practices, validation and tooling 204

16.1 Validation 204
16.2 Code quality 206
16.3 Security in HTML 209
16.4 Cross-browser compatibility 211
16.5 Version control and deployment 212

Module 17 · Projects and interview preparation 217

17.1 Capstone projects 217
17.2 Interview preparation 224
17.3 Next steps 227

LEVEL 1

Beginner level

Outcome: build structured web pages with text, links, images, lists, tables and basic forms.

7 MODULES · 9 DAYS MINIMUM

Module 01 · Introduction to the web and HTML	**1 day**
Module 02 · Setup and HTML document structure	**1 day**
Module 03 · Text content and formatting	**1 day**
Module 04 · Links and navigation	**1 day**
Module 05 · Images and basic media	**1 day**
Module 06 · Lists and tables	**2 days**
Module 07 · Forms basics	**2 days**

MODULE 01 · BEGINNER

Introduction to the web and HTML

Minimum duration **1 day** | Level **Beginner**

This module explains how the web works and what HTML is, before any markup is written. It covers the difference between the Internet and the World Wide Web, clients, servers and browsers, URLs, domain names and DNS, HTTP and HTTPS, what happens when you type a URL, what HTML is and how it differs from a programming language, how HTML, CSS and JavaScript work together, the history of HTML from 1991 to the Living Standard, why every web page needs HTML, what HTML5 added, and where HTML is used.

IN THIS MODULE

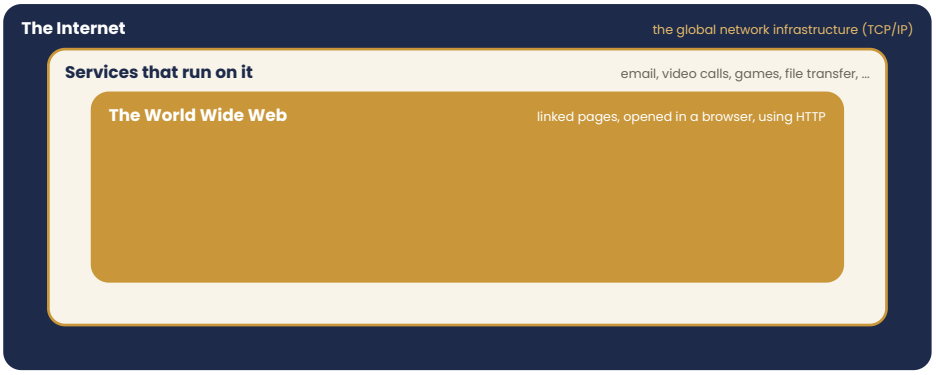
1.1 How the web works
1.2 What is HTML?
1.3 History of HTML
1.4 Why do we need HTML?
1.5 What HTML5 added
1.6 Where is HTML used?

1.1 How the web works

Before writing HTML it helps to know where a web page comes from and how it reaches your screen. Every time you open a website, several systems cooperate in a fraction of a second: your device, your internet connection, name servers, web servers and your browser. Understanding this journey explains many things you will meet later, such as why file paths matter, why pages load slowly, and why HTTPS is required.

1.1.1 Internet vs World Wide Web

The two terms are often used as if they meant the same thing, but they do not. The **Internet** is the global **network of networks**: millions of computers, phones, routers and cables (including undersea fibre-optic cables) connected using the TCP/IP protocols. It is the infrastructure. The **World Wide Web** (WWW, or simply "the web") is **one service that runs on top of the Internet**: a huge collection of documents and applications (web pages) that are linked to each other and accessed with a browser. Email, video calls, online games and file transfer also use the Internet, but they are not the web.



THE WEB IS ONE OF MANY SERVICES THAT USE THE INTERNET

	Internet	World Wide Web
What it is	a worldwide network of connected devices	a system of linked documents and apps
Made of	cables, routers, servers, wireless links	web pages, websites, links, web apps
Main protocols	TCP/IP	HTTP/HTTPS, plus HTML and URLs
Started	1969 (ARPANET), TCP/IP in 1983	1989 to 1991 (Tim Berners-Lee at CERN)
Example	sending an email, a WhatsApp call	opening learnbeingforward.in in Chrome

1.1.2 Clients, servers and browsers

The web follows the **client-server model**. A **server** is a computer that stores websites and waits for requests; it runs web server software such as Nginx, Apache or Node.js. A **client** is the device that asks for something: your laptop or phone. On the client, the program that makes those requests and shows the results is the **browser** (Chrome, Edge, Firefox, Safari, Brave). The browser sends a request, the server sends back a response containing files (HTML, CSS, JavaScript, images), and the browser's **rendering engine** (Blink in Chrome and Edge, Gecko in Firefox, WebKit in Safari) turns those files into the page you see.



THE CLIENT ASKS, THE SERVER ANSWERS

1.1.3 URLs, domain names and DNS

Every resource on the web has an address called a **URL** (Uniform Resource Locator). A URL tells the browser **how** to fetch the resource (the scheme), **where** (the host), and **which** resource (the path), optionally with extra information. People remember **domain names** such as `learnbeingforward.in`, but computers on the Internet find each other with numeric **IP addresses** such as `142.250.183.36`. The **Domain Name System (DNS)** is the Internet's phone book: it translates a domain name into the IP address of the server. Domains are bought from registrars (GoDaddy, Hostinger, Namecheap) and organised in levels: `.in` is the top-level domain, `learnbeingforward` the second-level domain, and `www` or `app` a subdomain.



THE PARTS OF A URL

Part	Example	Meaning
scheme (protocol)	`https`	how to talk to the server: http, https, mailto, tel, file, ftp
host	`www.learnbeingforward.in`	the server, as a domain name or an IP address

Part	Example	Meaning
port	`:443`	the "door" on the server; usually omitted (80 for http, 443 for https)
path	`/courses/html.html`	which file or page on that server
query string	`?batch=weekend&city=mysuru`	extra name=value data, often from forms or filters
fragment	`#syllabus`	a section inside the page; never sent to the server

1.1.4 HTTP and HTTPS requests and responses

Browsers and servers talk using **HTTP** (HyperText Transfer Protocol), a simple text-based conversation of **requests** and **responses**. A request has a **method** (GET to read a page, POST to send form data, and others), the path, and **headers** with extra information (the browser type, accepted languages, cookies). A response has a **status code** (200 OK, 301 Moved Permanently, 404 Not Found, 500 Internal Server Error), headers (such as `Content-Type: text/html`) and the **body**: the HTML file itself. **HTTPS** is HTTP sent through an encrypted TLS connection, so nobody between you and the server can read or change the data; the padlock icon shows it. Today every site should use HTTPS: browsers mark plain HTTP pages as "Not secure", and many features only work on HTTPS.

AN HTTP REQUEST (WHAT THE BROWSER SENDS) text

```
GET /courses/html.html HTTP/1.1
Host: www.learnbeingforward.in
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/140.0
Accept: text/html
Accept-Language: en-IN,en;q=0.9
```

THE HTTP RESPONSE (WHAT THE SERVER SENDS BACK) text

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 5120
Cache-Control: max-age=600

<!DOCTYPE html>
<html lang="en">
<head><title>HTML5 Course | Learn Being Forward</title></head>
<body>...</body>
</html>
```

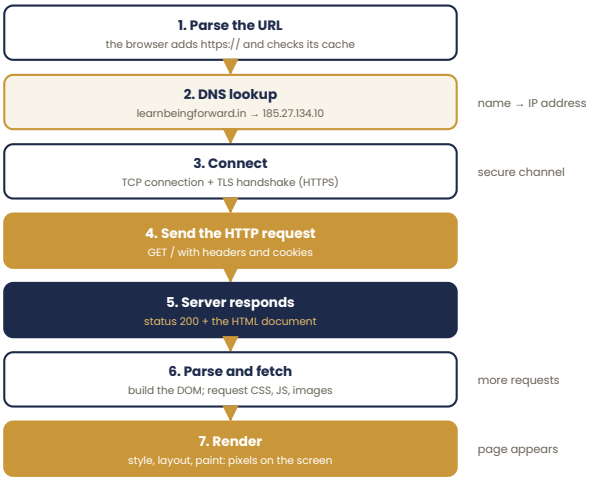
Status code	Meaning	When you see it
200 OK	success	the page loaded normally
301 / 302	moved (permanently / temporarily)	old URLs redirected to new ones, http to https
304 Not Modified	use your cached copy	revisiting a page whose files have not changed
403 Forbidden	not allowed	private folders, blocked access
404 Not Found	no such resource	a typo in the URL or a broken link
500 Internal Server Error	the server crashed	a bug in the server code
503 Service Unavailable	server overloaded or down	during maintenance or heavy traffic

TIP

Open DevTools (F12) in Chrome, go to the **Network** tab and reload any page: you will see every request the browser makes, with its method, status code, type, size and timing. It is the best way to watch HTTP in action.

1.1.5 What happens when you type a URL

Typing `learnbeingforward.in` and pressing Enter starts a chain of steps that usually completes in well under a second:



FROM URL TO PAGE, STEP BY STEP

1. The browser checks whether the text is a URL or a search, adds `https://`, and looks in its cache for a recent copy.
2. It asks **DNS** for the server's IP address (checking the browser, operating system and router caches first, then the ISP's DNS resolver).
3. It opens a **TCP connection** to the server and, for HTTPS, performs a **TLS handshake** to agree on encryption keys and verify the site's certificate.
4. It sends an **HTTP GET request** for the page.
5. The server finds or generates the HTML and sends a **response** with a status code, headers and the HTML.
6. The browser **parses** the HTML from top to bottom, building the **DOM** (Document Object Model, a tree of elements). Whenever it meets a stylesheet, script or image, it requests that file too.
7. Finally it combines the DOM with the CSS, calculates the **layout** (where each box goes) and **paints** the pixels. JavaScript can then change the page in response to user actions.

1.2 What is HTML?

1.2.1 Definition: HyperText Markup Language, the standard language for structuring web pages

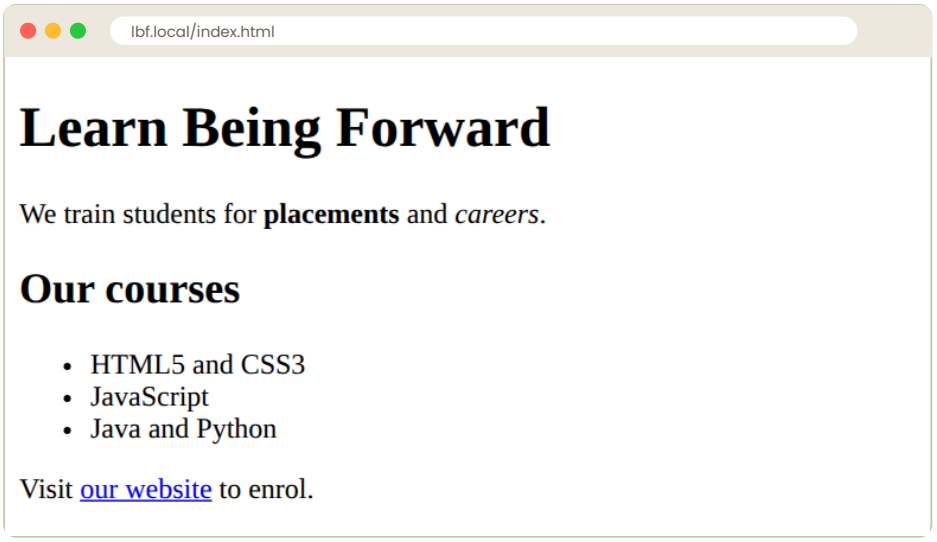
HTML stands for **HyperText Markup Language**. **HyperText** means text containing **links** (hyperlinks) to other text, which is what makes the web a web. **Markup** means you take ordinary content and surround parts of it with **tags** that say what each part is: this is a heading, this is a paragraph, this is a list, this is a link, this is an image. A **language** here means a set of standard rules that every browser understands.

HTML describes the **structure and meaning** of the content; it is not concerned with colours or fonts (that is CSS) or with behaviour (that is JavaScript).

A TINY HTML DOCUMENT: TAGS MARK UP EACH PIECE OF CONTENT

HTML

```
<h1>Learn Being Forward</h1>
<p>We train students for <strong>placements</strong> and <em>careers</em>.</p>
<h2>Our courses</h2>
<ul>
  <li>HTML5 and CSS3</li>
  <li>JavaScript</li>
  <li>Java and Python</li>
</ul>
<p>Visit <a href="https://learnbeingforward.in">our website</a> to enrol.</p>
```



HOW THE BROWSER DISPLAYS IT

Each piece of content is wrapped in a pair of **tags**: `<h1>` … `</h1>` marks the main heading, `<p>` a paragraph, `<ul>` an unordered list with `<li>` list items, `<strong>` important text, `<em>` emphasis, and `<a>` a link. The browser reads the tags and displays each part in a suitable default style: big bold headings, bullets for the list, underlined blue text for the link. You will learn every one of these elements in the next modules.

1.2.2 Markup language vs programming language

HTML is a **markup language**, not a programming language. A programming language (JavaScript, Python, Java) gives the computer **instructions to execute**: variables, calculations, conditions (*if*), loops, functions. HTML has none of these: it cannot add two numbers or decide anything. It only **describes** content. Saying "I know HTML" therefore means knowing how to structure documents correctly, which is still an essential and surprisingly deep skill (accessibility, SEO and good forms all depend on it).

	Markup language (HTML, XML, Markdown)	Programming language (JavaScript, Python)
Purpose	describe and structure content	give instructions that compute and decide
Has variables, conditions, loops?	no	yes
Output	a structured document	results of a computation, behaviour

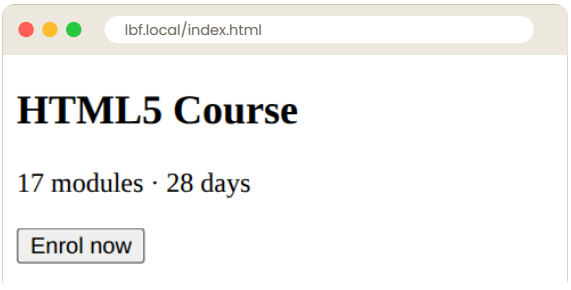
	Markup language (HTML, XML, Markdown)	Programming language (JavaScript, Python)
Errors	browsers forgive and guess	programs stop with errors
Example	`<h1>Welcome</h1>`	`if (marks >= 40) pass();`

1.2.3 HTML, CSS and JavaScript: structure, style and behaviour

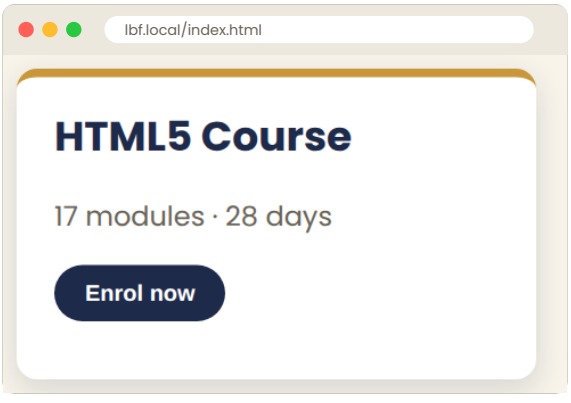
Every web page is built from three languages, each with one job. **HTML** provides the **structure** and meaning: the skeleton. **CSS** (Cascading Style Sheets) provides the **presentation**: colours, fonts, spacing, layout and animation, the skin and clothes. **JavaScript** provides the **behaviour**: reacting to clicks, validating input, loading data and updating the page without reloading, the muscles. Keeping the three separate (separation of concerns) makes pages easier to maintain. The three pages below contain the same HTML; the second adds CSS, and the third adds a line of JavaScript as well.

HTML ONLY HTML

```html
<div class="card">
  <h2>HTML5 Course</h2>
  <p>17 modules · 28 days</p>
  <button id="enrol">Enrol now</button>
  <p id="msg"></p>
</div>
```



HTML ONLY: STRUCTURE, BROWSER DEFAULT STYLES

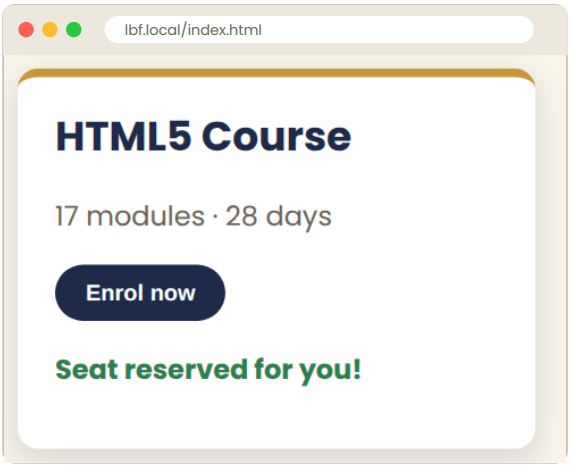


HTML + CSS: THE SAME STRUCTURE, STYLED

HTML + CSS + JAVASCRIPT

HTML

```html
<div class="card">
  <h2>HTML5 Course</h2>
  <p>17 modules · 28 days</p>
  <button id="enrol">Enrol now</button>
  <p id="msg"></p>
</div>
<script>
  document.querySelector("#enrol").addEventListener("click", () => {
    document.querySelector("#msg").textContent = "Seat reserved for you!";
  });
</script>
```



HTML + CSS + JAVASCRIPT: AFTER CLICKING THE BUTTON

HTML: structure

What the content **is**: headings, paragraphs, lists, links, images, forms, tables. Files end in `.html`.

CSS: presentation

How it **looks**: colours, fonts, spacing, layout, responsiveness, animation. Files end in `.css`.

JavaScript: behaviour

What it **does**: responds to events, validates, fetches data, updates the page. Files end in `.js`.

1.3 History of HTML

1.3.1 Tim Berners-Lee and the first web page (1991)

In 1989 **Tim Berners-Lee**, a British scientist at **CERN** (the European physics laboratory near Geneva), proposed a system to share documents between researchers using hypertext. By the end of 1990 he had written the three founding technologies: **HTML** to write documents, **HTTP** to transfer them, and **URLs** to address them, together with the first web browser (called WorldWideWeb) and the first web server. The first website, `info.cern.ch`, went public in **August 1991**; it explained what the web was and how to use it, and it is still online. The original HTML had only about 18 elements, many of which (`<title>`, `<p>`, `<a>`, `<h1>`, `<ul>`, `<li>`) you will still use every day. In 1994 Berners-Lee founded the **World Wide Web Consortium (W3C)** to develop web standards.

1.3.2 HTML 2.0 to HTML 4.01 and XHTML



MILESTONES IN THE HISTORY OF HTML

Version	Year	Highlights
HTML 2.0	1995	first formal standard (IETF): forms, images, the basic elements
HTML 3.2	1997	first W3C version: tables, applets; presentational tags such as `<font>` from the "browser wars"
HTML 4.01	1999	separation of structure and style: CSS for presentation, presentational tags deprecated; frames; better accessibility
XHTML 1.0	2000	HTML reformulated as XML: lowercase tags, all elements closed, quoted attributes; one error could break the page
XHTML 2.0	2002–2009	an incompatible redesign that browsers never adopted; abandoned
HTML5	2014 (W3C)	semantic elements, audio and video, canvas, new form controls, APIs; designed for web applications
HTML Living Standard	2019–today	one continuously updated specification maintained by WHATWG; no more version numbers

1.3.3 HTML5 (2014) and its new features

By the mid-2000s web pages had become applications (Gmail, YouTube, Google Maps), but HTML 4 had no native video, no drawing surface, weak forms, and relied on plug-ins such as Adobe Flash. In 2004 browser makers (Apple, Mozilla and Opera, later joined by Google and Microsoft) formed the **WHATWG** (Web Hypertext Application Technology Working Group) to evolve HTML for applications instead of switching to XHTML 2. Their work became **HTML5**, published as a W3C Recommendation in October 2014. It defined precise parsing rules so every browser builds the same page from the same (even broken) markup, added the features listed in 1.5, and replaced Flash for video and games. Its doctype is simply `<!DOCTYPE html>`.

1.3.4 The HTML Living Standard maintained by WHATWG

Since 2019 there is a single official HTML specification: the **HTML Living Standard**, maintained by the WHATWG at `html.spec.whatwg.org`, with the W3C endorsing it. "Living" means it is updated continuously instead of in numbered versions: features are added when at least two browser engines agree to implement them. Recent additions include the `<dialog>` element, the `popover` attribute, exclusive accordions with `<details name>`, the `<search>` element and `inert`, all covered in this book. In practice, "HTML5" today simply means modern HTML. To check whether a feature works in the browsers you need to support, use **MDN Web Docs** and **Can I Use** (16.4.1).

1.4 Why do we need HTML?

1.4.1 Every web page is built on HTML

Whatever technology a website uses (WordPress, React, Angular, a Python or Java back end), what finally arrives in the browser is HTML. Even single-page applications that build their interface with JavaScript create HTML elements in the DOM. You can see this on any site: right-click and choose **View Page Source** or **Inspect**. HTML is the one technology every web developer must know, and it changes slowly: what you learn now will stay valid for years.

1.4.2 Structure and meaning for browsers, search engines and screen readers

Good HTML is not only for the visual reader. **Browsers** use it to build the page and apply default behaviour (a `<button>` is focusable and clickable with the keyboard, a `<form>` submits data). **Search engines** such as Google read headings, links, titles and meta tags to understand what a page is about and how to rank it. **Screen readers** used by blind and low-vision people (NVDA, JAWS, VoiceOver, TalkBack) announce elements by their meaning: "heading level 1", "navigation", "list, 3 items", "link, our website", and let users jump between headings and landmarks. A page built only from meaningless `<div>` boxes may look fine but is hard or impossible to use for them (Modules 9 and 12).



MANY DIFFERENT READERS DEPEND ON THE SAME HTML

1.4.3 Foundation for CSS, JavaScript and all front-end frameworks

CSS selects HTML elements to style them, and JavaScript finds HTML elements to make them interactive, so both depend on well-structured HTML. Frameworks such as React, Angular and Vue (later courses in this series) let you write components, but the components still produce HTML elements, and their quality depends on choosing the right elements (a real `<button>`, a `<label>` for every input). Strong HTML skills make everything you learn later easier.

1.5 What HTML5 added

HTML5 was a large step forward. The most important additions, each covered in detail later in this book, are summarised below.

1.5.1 Semantic elements (header, nav, main, article, section, footer)

Instead of building every page from generic `<div>` boxes with class names, HTML5 introduced elements that describe the **role** of each region of a page: `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`, `<figure>` and more (Module 9).



A TYPICAL PAGE DESCRIBED WITH HTML5 SEMANTIC ELEMENTS

1.5.2 Native audio and video

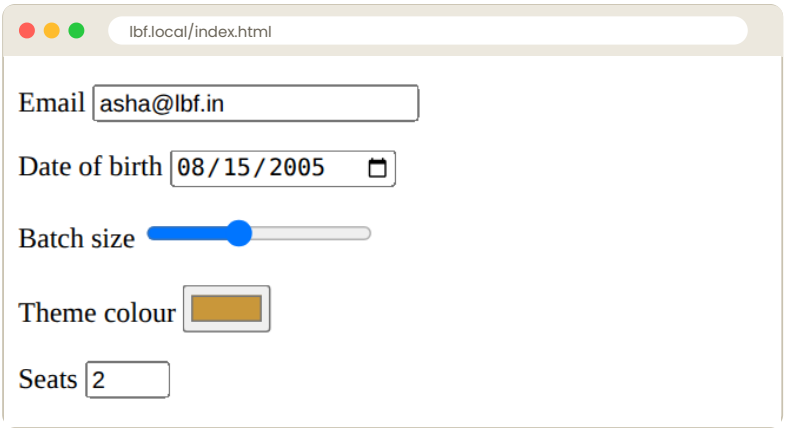
`<audio>` and `<video>` play media directly in the browser with built-in controls, captions through `<track>`, and a JavaScript API, replacing the Flash and QuickTime plug-ins that were insecure, battery-hungry and unavailable on phones (Module 10).

1.5.3 New form input types and built-in validation

New input types (`email`, `tel`, `url`, `number`, `date`, `time`, `range`, `color`, `search`) give users suitable controls and mobile keyboards, and attributes such as `required`, `min`, `max` and `pattern` let the browser validate input without JavaScript (Modules 7 and 8).

SOME HTML5 INPUT TYPES HTML

```
<form>
  <p><label>Email <input type="email" value="asha@lbf.in"></label></p>
  <p><label>Date of birth <input type="date" value="2005-08-15"></label></p>
  <p><label>Batch size <input type="range" min="10" max="60" value="30"></label></p>
  <p><label>Theme colour <input type="color" value="#c9973a"></label></p>
  <p><label>Seats <input type="number" min="1" max="5" value="2"></label></p>
</form>
```



HTML5 INPUTS RENDERED BY THE BROWSER, WITH NO CSS OR JAVASCRIPT

1.5.4 Canvas and SVG graphics

`<canvas>` is a drawing surface for pixel graphics controlled by JavaScript (charts, games, image editing), and inline `<svg>` embeds scalable vector graphics (icons, logos, diagrams) directly in the page (Modules 10 and 15).

INLINE SVG: SHAPES AND TEXT DESCRIBED IN MARKUP HTML

```
<svg width="300" height="120" viewBox="0 0 300 120">
  <rect x="10" y="10" width="120" height="100" rx="14" fill="#1E2A4A" />
  <circle cx="70" cy="60" r="30" fill="#C9973A" />
  <text x="150" y="55" font-size="20" font-family="Arial" fill="#1E2A4A">Vector</text>
  <text x="150" y="82" font-size="20" font-family="Arial" fill="#C9973A">graphics</text>
</svg>
```



HOW THE BROWSER DISPLAYS IT

1.5.5 Browser APIs (storage, geolocation, drag and drop)

Alongside HTML5, browsers gained JavaScript **APIs** that turn pages into applications: Web Storage (`localStorage`) to save data in the browser, Geolocation, Drag and Drop, the History API for single-page navigation, Web Workers, Notifications, offline support with service workers, and more (Module 15).

1.6 Where is HTML used?

1.6.1 Websites and web apps

The obvious use: every website (company sites, blogs, news, e-commerce, government portals) and every web application (Gmail, Google Docs, online banking, LMS platforms, admin dashboards) is delivered as HTML in the browser.

1.6.2 Emails (HTML email)

Most marketing and transactional emails (newsletters, order confirmations, OTP mails) are written in HTML. Email clients such as Gmail and Outlook support only an old subset of HTML and CSS, so HTML emails are still built with tables and inline styles; tools such as MJML help generate them.

1.6.3 Mobile and desktop apps built with web technology

Many apps are web technology inside a native shell: desktop apps with **Electron** or **Tauri** (VS Code, Slack, Discord, Microsoft Teams), mobile apps with **Capacitor** or **Ionic**, and **Progressive Web Apps** (PWAs) that install from the browser and work offline. React Native and Flutter use their own components, but their teams still use HTML for websites and documentation.

1.6.4 E-books (EPUB) and documentation

The **EPUB** e-book format is a zipped package of XHTML and CSS files, so most e-books are HTML inside. Software documentation (MDN, Python docs, the docs of every framework), help centres and even these course notes were produced as HTML and then printed to PDF. Smart TVs, car dashboards and kiosks also often display HTML interfaces.

Area	Examples
Websites	company and college sites, blogs, news, e-commerce, government portals
Web applications	Gmail, Google Docs, banking, learning platforms, dashboards
Email	newsletters, receipts, OTP and notification emails
Desktop apps (Electron, Tauri)	VS Code, Slack, Discord, Teams, Figma desktop
Mobile (PWA, Capacitor, Ionic)	installable web apps, hybrid mobile apps
Publishing and documentation	EPUB e-books, docs sites, help centres, PDF generation
Devices	smart TV apps, kiosks, in-car infotainment screens

The Internet is the global network; the World Wide Web is a service on it made of linked pages. Browsers (clients) request resources from servers using HTTP or, securely, HTTPS; responses carry a status code, headers and the HTML. URLs have a scheme, host, optional port, path, query string and fragment, and DNS translates domain names into IP addresses. Typing a URL triggers DNS lookup, connection, TLS, an HTTP request and response, parsing into the DOM, fetching CSS, JavaScript and images, then layout and painting. HTML (HyperText Markup Language) describes the structure and meaning of content with tags; it is a markup language, not a programming language. HTML gives structure, CSS presentation and JavaScript behaviour. HTML began with Tim Berners-Lee at CERN (1991), went through HTML 2.0 to 4.01 and XHTML, became HTML5 (2014) and is now the WHATWG Living Standard. HTML5 added semantic elements, native audio and video, new input types with validation, canvas and SVG, and browser APIs. HTML is used in websites, web apps, email, desktop and mobile apps, e-books and documentation.

PRACTICE CHECKPOINT

Explain in your own words **what happens between typing a URL and seeing a page**, and **what HTML, CSS and JavaScript each contribute**.

Sample answer. When I type `learnbeingforward.in`, the browser turns it into the URL `https://learnbeingforward.in/` and asks DNS for the IP address of that domain. It connects to that server, sets up an encrypted HTTPS connection, and sends an HTTP GET request. The server replies with status 200 and an HTML document. The browser reads the HTML from top to bottom and builds the DOM tree; when it finds links to CSS files, scripts and images, it requests those as well. It then applies the CSS, works out the layout, paints the page and runs the JavaScript. HTML contributes the structure and meaning of the content (headings, paragraphs, links, images, forms); CSS contributes the appearance (colours, fonts, spacing, layout); and JavaScript contributes behaviour (responding to clicks, validating forms, loading data and updating the page without a reload).

MODULE 02 · BEGINNER

Setup and HTML document structure

Minimum duration **1 day** | Level **Beginner**

By the end of this module you have a working editor and understand the skeleton of every HTML page. It covers VS Code and its essential extensions, browsers and DevTools, online editors, elements, tags and attributes (opening and closing tags, void elements, nesting rules, global attributes), the basic page structure (doctype, html, head, body, charset, viewport and title), creating and opening your first page, viewing source and inspecting elements, writing clean HTML with indentation, comments, conventions and Emmet, and the difference between block and inline elements.

IN THIS MODULE

2.1 Tools
2.2 Elements, tags and attributes
2.3 Basic page structure
2.4 Your first web page
2.5 Writing clean HTML
2.6 Block vs inline elements

2.1 Tools

HTML needs no compiler or special software: any text editor can write it and any browser can display it. A good setup, however, makes you far more productive by completing tags, catching typos, formatting code and refreshing the browser automatically.

2.1.1 Code editor: VS Code

Visual Studio Code (VS Code), free from Microsoft for Windows, macOS and Linux, is the most widely used editor for web development. It understands HTML out of the box: syntax colouring, automatic closing tags, IntelliSense suggestions for elements and attributes with short descriptions on hover, and Emmet abbreviations (2.5.4). Open a whole project folder (**File → Open Folder**) rather than single files, so that links between pages and the integrated terminal work. Useful shortcuts: **Ctrl+S** save, **Shift+Alt+F** format document, **Ctrl+/** toggle comment, **Alt+↑/↓** move a line, **Ctrl+D** select the next occurrence, **Ctrl+P** open a file by name. Turn on **Auto Save** (File menu) and **Format On Save** (Settings) once you install Prettier.

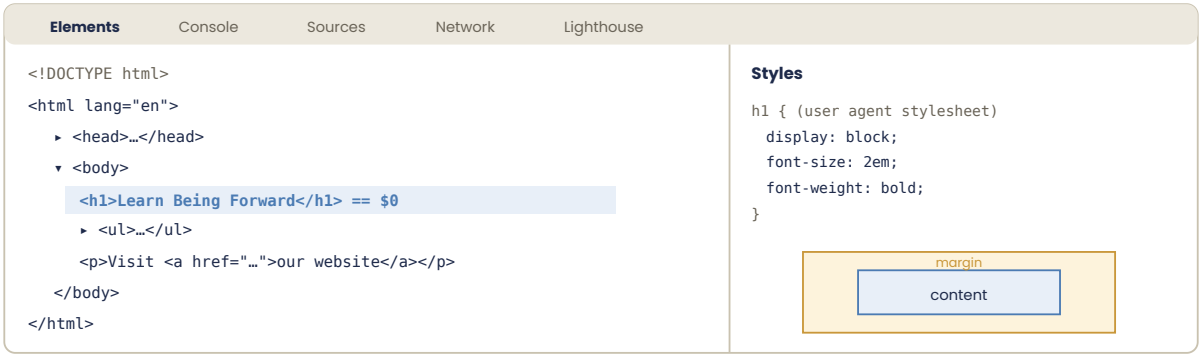
2.1.2 Extensions: Live Server, Prettier, Auto Rename Tag

Extension	What it does	Why it helps
Live Server	runs a small local web server and opens your page; the browser reloads automatically every time you save	instant feedback; pages are served over `http://127.0.0.1:5500`, like a real site
Prettier	formats HTML, CSS and JavaScript consistently (indentation, line breaks, quotes)	clean, readable code with no effort; the same style for the whole team

Extension	What it does	Why it helps
Auto Rename Tag	when you rename an opening tag, the matching closing tag changes too	avoids mismatched tags such as `<h2>...</h3>`
HTML CSS Support, IntelliSense for CSS class names	suggests class names and ids defined in your CSS	fewer typos between HTML and CSS
Error Lens, Code Spell Checker	shows problems inline; catches spelling mistakes in text	catches mistakes as you type

2.1.3 Browsers and DevTools (Elements panel)

Test your pages in at least two modern browsers, for example **Chrome** (or Edge, which uses the same Blink engine) and **Firefox**; Safari matters for iPhone users. Every browser includes **Developer Tools** (DevTools), opened with **F12**, **Ctrl+Shift+I** or right-click → **Inspect**. The **Elements** panel shows the live DOM tree: click the arrow icon (or press Ctrl+Shift+C) and point at anything on the page to jump to its HTML; you can edit text and attributes there to experiment (changes are temporary). The side panel shows CSS styles and the box model. Other panels you will use: **Console** (JavaScript messages and errors), **Network** (every request, 1.1.4), **Lighthouse** (performance, accessibility and SEO audits) and the **device toolbar** (Ctrl+Shift+M) to preview phone and tablet sizes.



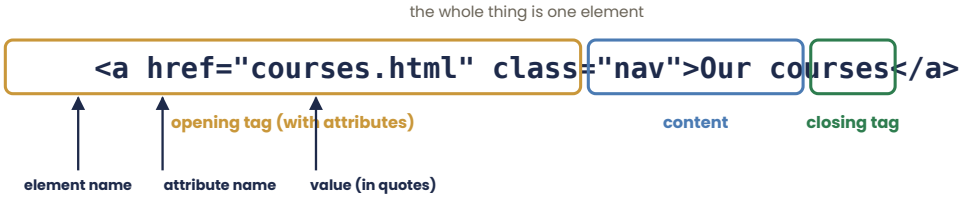
THE ELEMENTS PANEL: THE DOM TREE ON THE LEFT, STYLES AND THE BOX MODEL ON THE RIGHT

2.1.4 Online editors: CodePen

Online editors run in the browser with no installation and show the result as you type: **CodePen** (separate HTML, CSS and JS panels; great for small experiments and sharing demos), **JSFiddle**, **StackBlitz** and **CodeSandbox** (full projects, including frameworks), and **Replit**. They are perfect for practice and for asking questions with a working example, but real projects belong in a local folder under version control (16.5).

2.2 Elements, tags and attributes

An HTML document is made of **elements**. Most elements consist of an **opening tag**, some **content**, and a **closing tag**. The opening tag may carry **attributes** that give extra information about the element.



ANATOMY OF AN HTML ELEMENT

2.2.1 Opening and closing tags

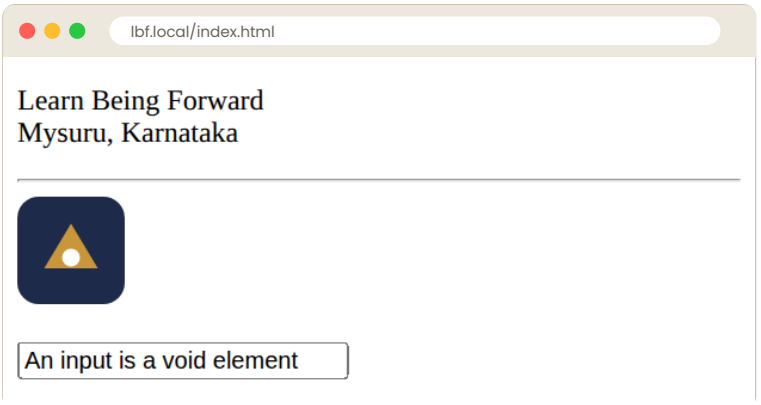
A tag is the element name in angle brackets: `<p>`. The closing tag repeats the name with a slash: `</p>`. Everything between them is the element's **content**, which can be text, other elements, or both. Element names are not case-sensitive, but lowercase is the universal convention. Some closing tags are technically optional (`</p>`, `</li>`, `</td>`), because the browser can infer where they end, but always writing them makes your code clearer and avoids surprises.

2.2.2 Void (self-closing) elements

A few elements cannot have content, so they have no closing tag: they are called **void elements**. The common ones are `<br>` (line break), `<hr>` (thematic break), `<img>` (image), `<input>` (form control), `<meta>`, `<link>`, `<source>`, `<track>`, `<area>`, `<col>`, `<embed>` and `<wbr>`. In HTML5 you may write them as `<br>` or, in the XHTML style, `<br />`; both are valid, and Prettier normalises them. Never write a closing tag such as `</br>` or `</img>`.

VOID ELEMENTS: BR, HR, IMG AND INPUT HAVE NO CLOSING TAG HTML

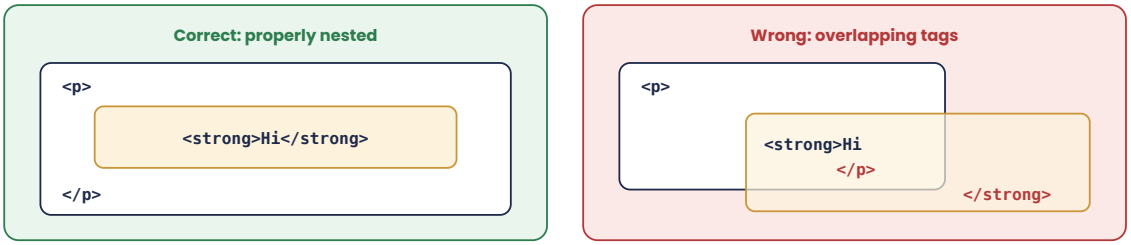
```html
<p>Learn Being Forward<br>Mysuru, Karnataka</p>
<hr>
<img src="logo.svg" alt="LBF logo" width="60" height="60">
<p><input type="text" value="An input is a void element"></p>
```



HOW THE BROWSER DISPLAYS IT

2.2.3 Nesting rules

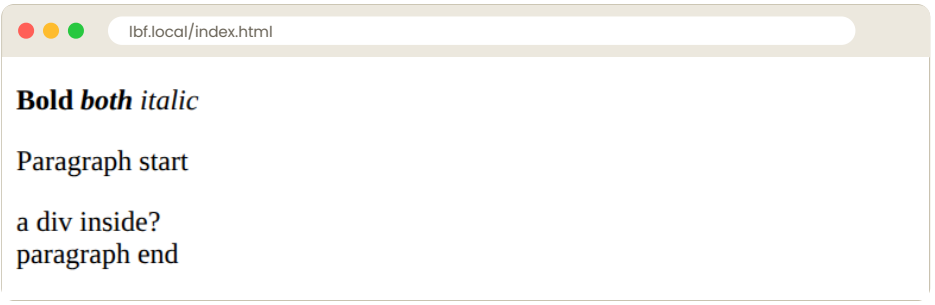
Elements can contain other elements: this is **nesting**, and it creates the parent-child tree that you see in DevTools. The rule is simple: **close elements in the reverse order you opened them**, like boxes inside boxes. `<p><strong>Hi</strong></p>` is correct; `<p><strong>Hi</p></strong>` overlaps and is invalid. Some elements also restrict what they can contain: a `<ul>` may contain only `<li>` elements, a `<p>` cannot contain block elements such as a `<div>` or another `<p>`, and an `<a>` cannot contain another link. The browser silently "fixes" invalid nesting, often in ways you do not expect, which is why the validator (16.1) is useful.



NESTED ELEMENTS MUST CLOSE IN REVERSE ORDER

WHAT THE BROWSER DOES WITH INVALID NESTING HTML

```html
<!-- Written with overlapping tags: <p><b>Bold <i>both</b> italic</i></p> -->
<p><b>Bold <i>both</b> italic</i></p>
<!-- A <p> cannot contain a <div>, so the browser closes the paragraph early -->
<p>Paragraph start <div>a div inside?</div> paragraph end</p>
```



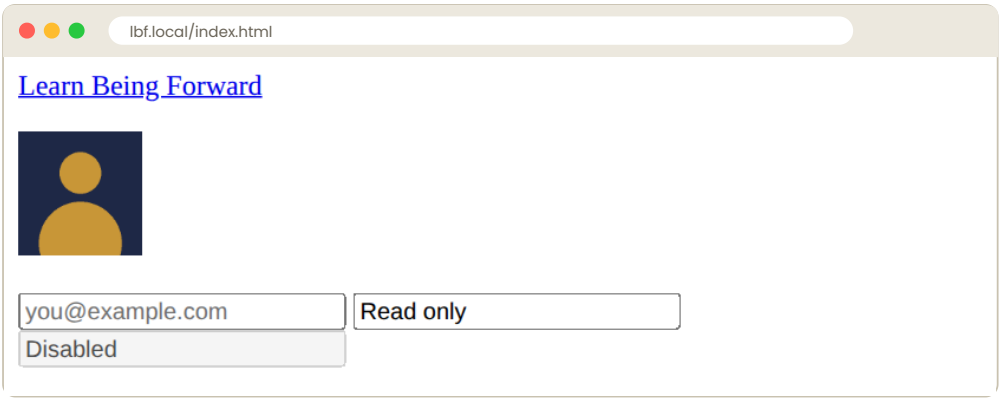
THE BROWSER GUESSES AND REPAIRS THE STRUCTURE; INSPECT IT IN DEVTOOLS TO SEE THE RESULT

2.2.4 Attributes and values

Attributes appear only in the opening tag, as `name="value"` pairs separated by spaces, in any order. They configure the element: `href` gives a link's destination, `src` and `alt` an image's file and description, `type` an input's kind. Always put values in double quotes (quotes are required when the value contains spaces). **Boolean attributes** are true just by being present and false when absent: `<input disabled>`, `<video controls autoplay muted>`; writing `disabled="false"` still disables the input, because the attribute is present. Each element accepts its own set of attributes, documented on MDN.

ELEMENTS CONFIGURED WITH ATTRIBUTES, INCLUDING BOOLEAN ATTRIBUTES HTML

```html
<a href="https://learnbeingforward.in" target="_blank" title="Opens in a new tab">Learn Being
Forward</a>
<p><img src="avatar.png" alt="Profile picture" width="70" height="70"></p>
<p>
  <input type="email" placeholder="you@example.com" required>
  <input type="text" value="Read only" readonly>
  <input type="text" value="Disabled" disabled>
</p>
```



HOW THE BROWSER DISPLAYS IT

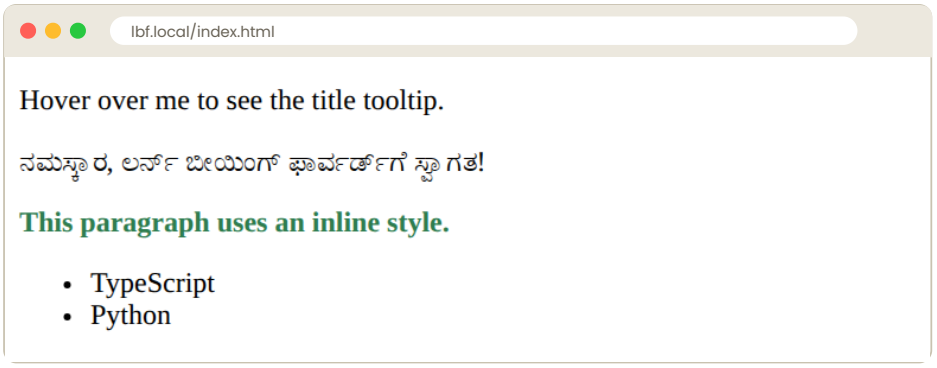
2.2.5 Global attributes: id, class, style, title, lang, hidden, data-*

Global attributes can be used on every element. The most important ones:

Attribute	Purpose	Example
`id`	a **unique** identifier on the page; used for links to sections, labels, CSS and JavaScript	`<section id="pricing">`
`class`	one or more space-separated names that group elements for CSS and JavaScript; may repeat	`<div class="card featured">`
`style`	inline CSS for this element only (avoid in real projects, 16.2.4)	`<p style="color: green">`
`title`	advisory text, usually shown as a tooltip on hover	`<abbr title="Cascading Style Sheets">CSS</abbr>`
`lang`	the language of the content (pronunciation, spellcheck, translation)	`<p lang="kn">ನಮಸ್ಕಾರ</p>`
`hidden`	the element is not relevant now, so it is not displayed	`<p hidden>Shown after payment</p>`
`data-*`	custom data for your own scripts and CSS; any name after data-	`<li data-course-id="TS101">`
`tabindex`, `dir`, `translate`, `contenteditable`, `draggable`, `spellcheck`, `inert`, `popover`	focus order, direction, translation, editing, drag and drop and more	covered in Modules 3, 12 and 14

GLOBAL ATTRIBUTES ON ORDINARY ELEMENTS HTML

```html
<p id="intro" class="lead highlight" title="I have a tooltip">Hover over me to see the title tooltip.
</p>
<p lang="kn">ನಮಸ್ಕಾರ, ಲರ್ನ್ ಬೀಯಿಂಗ್ ಫಾರ್ವರ್ಡ್‌ಗೆ ಸ್ವಾಗತ!</p>
<p hidden>This paragraph has the hidden attribute, so it is not displayed.</p>
<p style="color: #2E7D4F; font-weight: bold;">This paragraph uses an inline style.</p>
<ul>
  <li data-course="TS101" data-fee="9999">TypeScript</li>
  <li data-course="PY201" data-fee="14999">Python</li>
</ul>
```



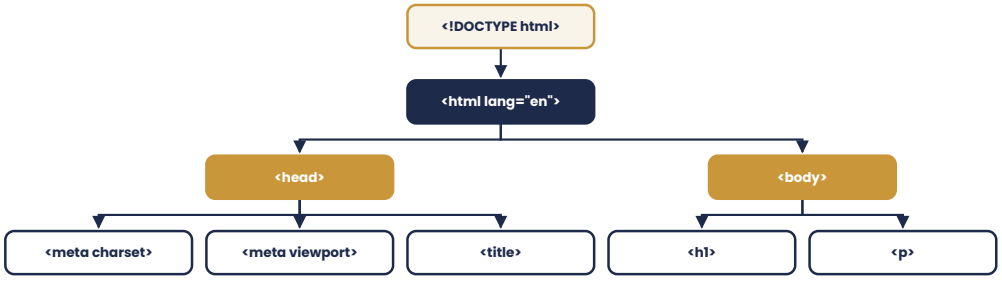
HOW THE BROWSER DISPLAYS IT

2.3 Basic page structure

Every HTML document follows the same skeleton. Learn it by heart: you will type it (or generate it with Emmet) at the start of every page.

INDEX.HTML: THE HTML5 PAGE SKELETON HTML

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Home | Learn Being Forward</title>
  </head>
  <body>
    <h1>Welcome to Learn Being Forward</h1>
    <p>Training for academics, placements and careers.</p>
  </body>
</html>
```



THE DOCUMENT AS A TREE: HTML CONTAINS HEAD AND BODY

2.3.1 <!DOCTYPE html>

The **doctype declaration** must be the very first line. It is not an HTML element; it tells the browser to render the page in **standards mode**. Without it, browsers switch to **quirks mode**, emulating the bugs of 1990s browsers, and your CSS layout behaves unpredictably. Older doctypes were long and complicated (HTML 4.01 and XHTML required a URL to a DTD); HTML5 reduced it to `<!DOCTYPE html>`, which is case-insensitive.

2.3.2 <html lang="en">

`<html>` is the **root element** that contains everything else. Its `lang` attribute declares the primary language of the page using a language code: `en` (English), `en-IN` (English as used in India), `hi` (Hindi), `kn` (Kannada), `ta` (Tamil). Screen readers use it to choose pronunciation, browsers to offer translation

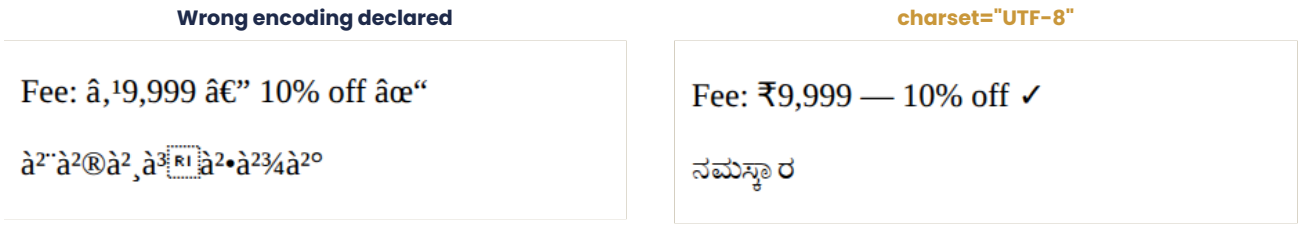
and hyphenation, and search engines to serve the page to the right users. It is required for accessibility (12.2.5).

2.3.3 <head> and <body>

The `<html>` element has exactly two children. The `<head>` contains **metadata**: information about the page that is not displayed in the page itself, such as the character encoding, the title, the description for search engines, links to CSS files and icons, and scripts (Module 11). The `<body>` contains everything the user sees and interacts with: text, images, links, forms, video. There is only one head and one body per document.

2.3.4 <meta charset="UTF-8">

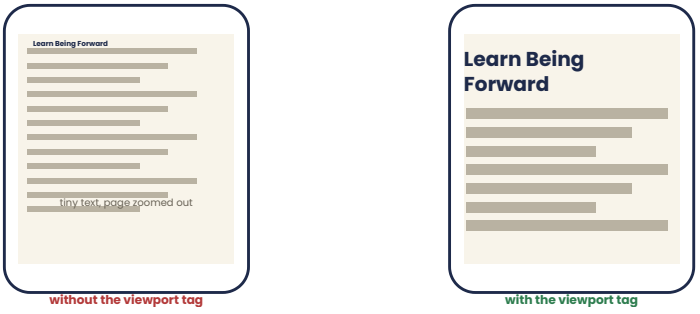
This line declares the **character encoding**: how the bytes of the file map to characters. **UTF-8** can represent every character in Unicode: all languages (including Hindi, Kannada, Tamil and Arabic), symbols such as ₹ and ©, and emojis. It should be the first element in the head (it must appear within the first 1024 bytes). Without it, browsers may guess wrongly and show garbled text such as `â,¹` instead of ₹. Also save your files as UTF-8 in the editor (VS Code does this by default).



THE SAME UTF-8 FILE DISPLAYED WITH A WRONG AND A CORRECT CHARSET DECLARATION

2.3.5 Viewport meta tag

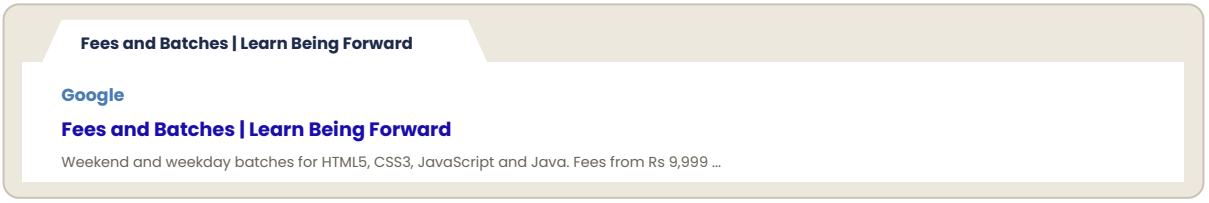
`<meta name="viewport" content="width=device-width, initial-scale=1.0">` tells mobile browsers to make the page as wide as the device screen and not zoomed. Without it, phones pretend to be about 980 pixels wide and shrink the whole page, so text becomes tiny and users must pinch to zoom. Every modern page needs this line; it is the foundation of responsive design (CSS Module 12). Do not add `user-scalable=no` or `maximum-scale=1`, which prevent users from zooming and harm accessibility.



ON A PHONE, THE VIEWPORT META TAG MAKES TEXT READABLE WITHOUT ZOOMING

2.3.6 <title>

The `<title>` element sets the page title, shown in the **browser tab**, in bookmarks and history, and as the clickable headline in **search results**. It is required in every document and should be unique and descriptive: put the page topic first and the site name after, for example `Fees and Batches | Learn Being Forward`, and keep it under about 60 characters so search engines do not cut it off (11.4.1).



THE TITLE APPEARS IN THE BROWSER TAB AND AS THE HEADLINE OF A SEARCH RESULT

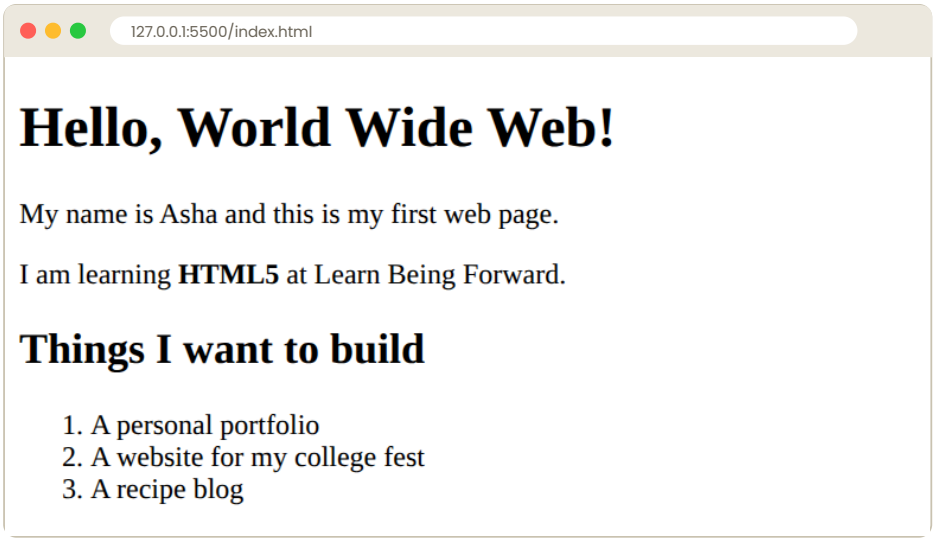
2.4 Your first web page

2.4.1 Creating index.html

Create a folder for the project, for example `my-first-site`, open it in VS Code, and create a file named **`index.html`**. The name `index.html` is special: web servers send it automatically when someone visits a folder's address without a file name (`https://example.com/` serves `/index.html`). Use lowercase file names without spaces (use hyphens: `about-us.html`), because URLs are case-sensitive on most servers and spaces become `%20`.

INDEX.HTML: A COMPLETE FIRST PAGE HTML

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello, World Wide Web!</h1>
    <p>My name is Asha and this is my first web page.</p>
    <p>I am learning <strong>HTML5</strong> at Learn Being Forward.</p>
    <h2>Things I want to build</h2>
    <ol>
      <li>A personal portfolio</li>
      <li>A website for my college fest</li>
      <li>A recipe blog</li>
    </ol>
  </body>
</html>
```



THE FIRST PAGE OPENED IN A BROWSER

2.4.2 Opening it in a browser and with Live Server

There are two ways to view the page. **Double-click** the file (or drag it into the browser): the address bar shows a `file:///C:/Users/.../index.html` URL, and you must refresh manually after each change. Or, with the Live Server extension, right-click the file and choose **Open with Live Server** (or click **Go Live** in the status bar): the page opens at `http://127.0.0.1:5500/index.html` and reloads automatically on every save. Prefer Live Server: it behaves like a real web server, and some features (modules, fetch, some APIs) do not work from `file://` URLs.

2.4.3 Viewing page source and inspecting elements

View Page Source (right-click → View page source, or **Ctrl+U**) shows the HTML exactly as the server sent it. **Inspect** (right-click → Inspect, or F12) shows the **live DOM**: the browser's parsed and possibly corrected version, including changes made by JavaScript. Use both on websites you admire to learn how they are built: look at how professional sites structure headers, navigation, forms and footers.

	View Page Source (Ctrl+U)	Inspect / Elements panel (F12)
Shows	the original HTML file from the server	the live DOM tree as the browser built it
Includes JavaScript changes	no	yes
Shows browser corrections	no (raw text)	yes (e.g. invalid nesting repaired)
Editable	no	yes, temporarily, for experiments
Use it to	see what the server actually sent	debug layout, styles and structure

2.5 Writing clean HTML

Browsers display messy HTML too, but people have to read and change your code. Clean, consistent HTML is faster to debug, easier for teammates to understand, and is what interviewers and code reviewers expect.

2.5.1 Indentation and formatting

Indent each nested level by the same amount (2 spaces is the common convention in web projects; 4 is also fine) so the tree structure is visible at a glance. Put block elements on their own lines, keep short inline content on one line, and break long tags with many attributes over several lines. Let **Prettier** do it for you with **Shift+Alt+F** or Format On Save. Indentation and extra spaces between tags do not change how the page looks (3.2.3).

HARD TO READ: EVERYTHING ON A FEW LINES HTML

```
<body><header><h1>LBF</h1><nav><ul><li><a href="/">Home</a></li><li><a
href="/courses.html">Courses</a></li></ul></nav></header>
<main><h2>Courses</h2><p>We teach   HTML,CSS and
JavaScript.</p></main></body>
```

THE SAME CODE, INDENTED: THE STRUCTURE IS OBVIOUS

HTML

```
<body>
  <header>
    <h1>LBF</h1>
    <nav>
      <ul>
        <li><a href="/">Home</a></li>
        <li><a href="/courses.html">Courses</a></li>
      </ul>
    </nav>
  </header>
  <main>
    <h2>Courses</h2>
    <p>We teach HTML, CSS and JavaScript.</p>
  </main>
</body>
```

2.5.2 Comments <!-- -->

Comments start with `<!--` and end with `-->`; the browser ignores them. Use them to label sections of long pages, to explain non-obvious decisions, and to temporarily disable markup while testing (**Ctrl+/** in VS Code). Remember that comments are **visible to anyone** who views the source, so never put passwords, internal notes or personal data in them. Comments cannot be nested.

COMMENTS ARE VISIBLE IN THE SOURCE BUT NOT ON THE PAGE

HTML

```
<!-- ===== Hero section ===== -->
<h2>Weekend batches now open</h2>
<!-- TODO: replace with the real start date from the office -->
<p>Classes start soon.</p>
<!--
<p>This old offer is disabled by commenting it out.</p>
-->
```



HOW THE BROWSER DISPLAYS IT

2.5.3 Case and quoting conventions

- Write element and attribute names in **lowercase**: `<p class="note">`, not `<P CLASS="note">`.
- Always quote attribute values, preferably with **double quotes**: `href="about.html"`.
- Close every non-void element, even where closing tags are optional.
- Use lowercase, hyphenated file names and class names: `contact-us.html`, `class="course-card"`.
- Write boolean attributes without values: `required`, not `required="required"` (both are valid).
- One element per line for block elements, with consistent indentation.

2.5.4 Emmet shortcuts

Emmet, built into VS Code, expands short CSS-like abbreviations into full HTML when you press **Tab** or Enter. It is the fastest way to write markup and is worth learning thoroughly. Type `!` and press Tab in an

empty `.html` file to generate the whole page skeleton.

Abbreviation	Expands to	Meaning
`!`	the full HTML5 skeleton	doctype, html, head with meta tags and title, body
`p`	`<p></p>`	an element by its name
`div.card`	`<div class="card"></div>`	`.` adds a class
`section#pricing`	`<section id="pricing"></section>`	`#` adds an id
`.box`	`<div class="box"></div>`	a div is assumed
`ul>li*3`	a ul containing three li elements	`>` child, `*` repeat
`h2+p`	an h2 followed by a p	`+` sibling
`a[href=about.html]{About us}`	`<a href="about.html">About us</a>`	`[ ]` attributes, `{ }` text
`ul>li.item$*3`	li elements with classes item1, item2, item3	`$` numbering
`lorem20`	20 words of placeholder text	dummy content
`nav>ul>li*4>a`	a navigation menu with four links	abbreviations combine

ONE EMMET ABBREVIATION AND ITS EXPANSION HTML

```
<!-- Typed: nav>ul>li*3>a[href=#]{Link $}   then Tab -->
<nav>
  <ul>
    <li><a href="#">Link 1</a></li>
    <li><a href="#">Link 2</a></li>
    <li><a href="#">Link 3</a></li>
  </ul>
</nav>
```

2.6 Block vs inline elements

By default, the browser displays elements in one of two main ways. Understanding the difference is essential for both HTML structure and CSS layout. (Technically, HTML5 replaced these categories with content models such as "flow" and "phrasing" content, and CSS controls the actual display; but "block" and "inline" remain the practical way to think about default behaviour.)

2.6.1 Block-level elements

A **block-level element** starts on a **new line** and takes the **full width** available, stacking vertically like paragraphs in a book. Examples: `<div>`, `<p>`, `<h1>`–`<h6>`, `<ul>`, `<ol>`, `<li>`, `<table>`, `<form>`, `<header>`, `<section>`, `<blockquote>`, `<hr>`. Block elements can contain other blocks and inline elements (with exceptions such as `<p>`, which can contain only inline content).

2.6.2 Inline elements

An **inline element** flows **within a line of text**, takes only as much width as its content, and does not start a new line. Examples: `<span>`, `<a>`, `<strong>`, `<em>`, `<code>`, `<img>`, `<br>`, `<input>`, `<label>`, `<button>`.

Inline elements should contain only text and other inline elements (an `<a>` may wrap block content in HTML5, an exception). Width and height set with CSS have no effect on pure inline elements such as span; images and inputs are "inline-block" replaced elements.

COLOURED BACKGROUNDS REVEAL THE BOXES OF BLOCK AND INLINE ELEMENTS HTML

```
<div style="background:#E8EFF8">div (block)</div>
<p style="background:#FDF3DC">p (block) with <span style="background:#C9973A;color:#fff">span (inline)
</span>, <a href="#" style="background:#EAF5EE">a link (inline)</a> and <strong
style="background:#FBE9E7">strong (inline)</strong> flowing in the line.</p>
<h3 style="background:#E8EFF8">h3 (block)</h3>
<span style="background:#C9973A;color:#fff">span 1</span> <span
style="background:#1E2A4A;color:#fff">span 2</span> <em style="background:#EAF5EE">em</em>
<ul style="background:#FDF3DC"><li>li (block)</li></ul>
```



HOW THE BROWSER DISPLAYS IT



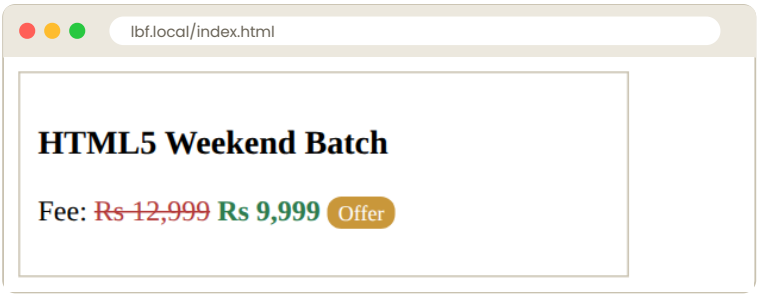
BLOCK ELEMENTS STACK VERTICALLY; INLINE ELEMENTS FLOW WITHIN THE LINE

2.6.3 and as generic containers

`<div>` (division) and `<span>` are **generic containers** with no meaning of their own. `<div>` is a block-level box used to group content for layout and styling (a card, a row, a wrapper); `<span>` is an inline box used to style or script part of a line (a price, a highlighted word, a badge). Use them only when no semantic element fits: a heading should be `<h2>`, not a styled div; a page region may be `<section>` or `<nav>` (Module 9); a clickable control must be a `<button>`, not a div with a click handler.

A DIV GROUPS A CARD; SPANS STYLE PARTS OF A LINE HTML

```
<div class="course-card" style="border:1px solid #C9C2B0; padding:10px; max-width:320px">
  <h3>HTML5 Weekend Batch</h3>
  <p>Fee: <span style="color:#B33A3A; text-decoration:line-through">Rs 12,999</span>
     <span style="color:#2E7D4F; font-weight:bold">Rs 9,999</span>
     <span style="background:#C9973A; color:#fff; padding:2px 6px; border-radius:8px; font-
size:12px">Offer</span></p>
</div>
```



HOW THE BROWSER DISPLAYS IT

VS Code with Live Server, Prettier and Auto Rename Tag, a modern browser with DevTools (Elements, Console, Network, Lighthouse) and online editors such as CodePen make up the toolkit. Elements have an opening tag with attributes, content and a closing tag; void elements (br, hr, img, input, meta, link) have no closing tag; nested elements close in reverse order and follow content rules. Attributes are name="value" pairs; boolean attributes are true when present; global attributes (id, class, style, title, lang, hidden, data-*) work on every element. Every page starts with <!DOCTYPE html>, then <html lang> containing <head> (meta charset UTF-8, viewport meta, title and other metadata) and <body> (visible content). Name the home page index.html, view it with Live Server, and compare View Source with Inspect. Write clean HTML: consistent indentation, comments (never secrets), lowercase names, double-quoted values, Emmet abbreviations. Block elements start on new lines and take full width; inline elements flow within text; div and span are generic block and inline containers to use only when no semantic element fits.

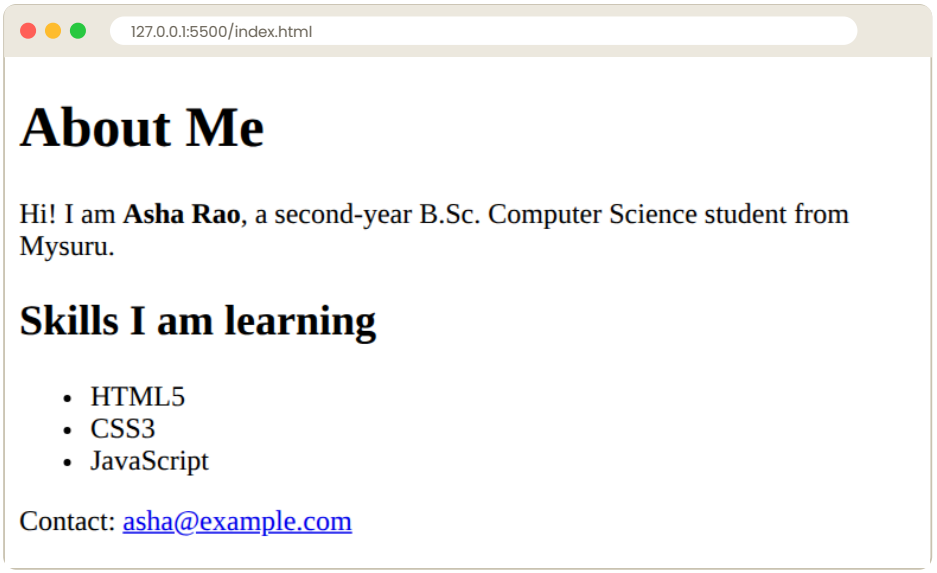
PRACTICE CHECKPOINT

Build a **valid HTML5 page skeleton from memory using Emmet**, and **explain the purpose of each line in the head**.

SAMPLE SOLUTION: TYPED AS ! + TAB, THEN FILLED IN

HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>About Me | Asha Rao</title>
</head>
<body>
  <h1>About Me</h1>
  <p>Hi! I am <strong>Asha Rao</strong>, a second-year B.Sc. Computer Science student from Mysuru.</p>
  <h2>Skills I am learning</h2>
  <ul>
    <li>HTML5</li>
    <li>CSS3</li>
    <li>JavaScript</li>
  </ul>
  <p>Contact: <a href="mailto:asha@example.com">asha@example.com</a></p>
</body>
</html>
```



THE FINISHED PAGE

Line	Purpose
`<!DOCTYPE html>`	puts the browser in standards mode (not quirks mode)
`<html lang="en">`	root element; declares English as the page language for screen readers, translation and search engines
`<head>`	container for metadata that is not displayed in the page
`<meta charset="UTF-8">`	declares the UTF-8 encoding so every character (₹, ನಮಸ್ಕಾರ, emojis) displays correctly
`<meta name="viewport" ...>`	makes the page fit the device width on phones instead of a zoomed-out desktop view
`<title>`	text in the browser tab, bookmarks and search results; should be unique and descriptive

MODULE 03 · BEGINNER

Text content and formatting

Minimum duration **1 day** Level **Beginner**

Text is the core of most web pages, and this module covers every element used to structure and mark up text: headings and their hierarchy, paragraphs, line breaks, thematic breaks, whitespace collapsing and preformatted text, text-level semantics (strong and b, em and i, mark, small, del, ins, s, u, sub, sup, abbr, cite, dfn, time and data), quotations and code, contact information with address, character entities, emojis and Unicode, and bidirectional text with lang, dir, bdi and bdo.

IN THIS MODULE

3.1 Headings
3.2 Paragraphs and breaks
3.3 Text-level semantics
3.4 Quotes and code
3.5 Contact and addresses
3.6 Character entities
3.7 Bidirectional text and language

3.1 Headings

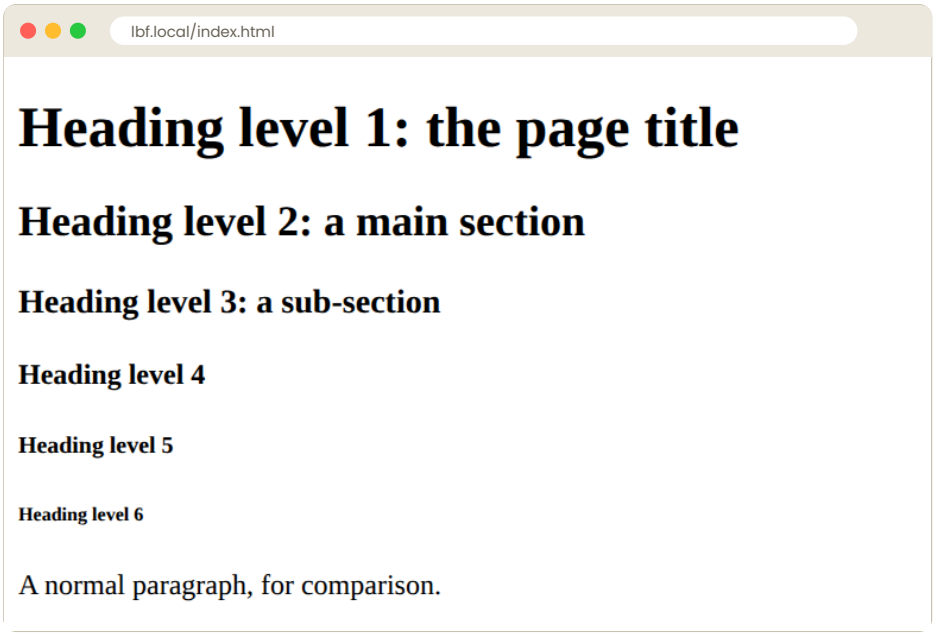
3.1.1 <h1> to <h6>

HTML has six levels of headings, `<h1>` (most important) to `<h6>` (least important). Browsers display them in decreasing sizes by default, but their real purpose is **structure**, not size: headings form the outline of the page, like the chapter, section and sub-section titles of a book. Screen-reader users jump from heading to heading to skim a page (most do this first), and search engines give heading text extra weight. If you want bigger or smaller text, use CSS, not a different heading level.

THE SIX HEADING LEVELS WITH THE BROWSER'S DEFAULT STYLES

HTML

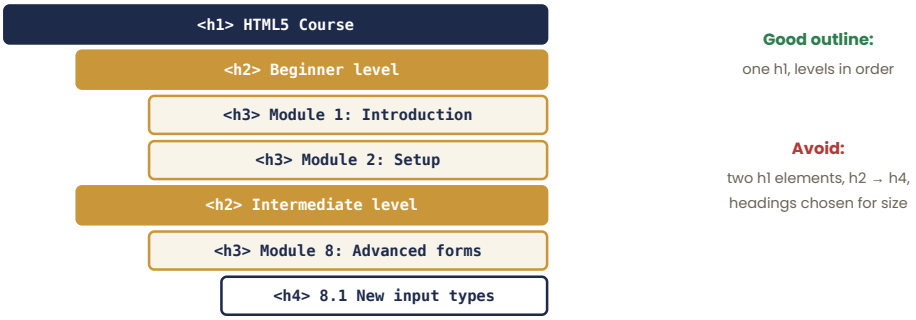
```
<h1>Heading level 1: the page title</h1>
<h2>Heading level 2: a main section</h2>
<h3>Heading level 3: a sub-section</h3>
<h4>Heading level 4</h4>
<h5>Heading level 5</h5>
<h6>Heading level 6</h6>
<p>A normal paragraph, for comparison.</p>
```



HOW THE BROWSER DISPLAYS IT

3.1.2 Heading hierarchy and one <h1> per page

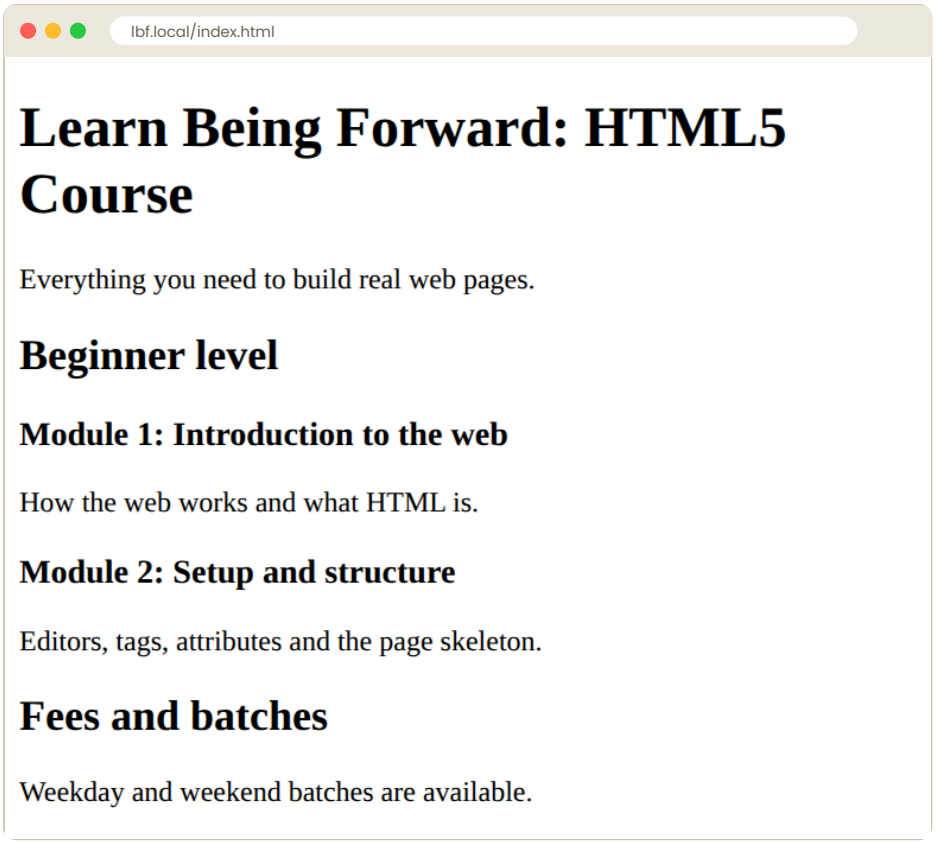
Use **one `<h1>` per page** for its main title (it usually matches the `<title>`), then `<h2>` for the main sections, `<h3>` for sub-sections inside an h2, and so on. **Do not skip levels** going down (an h2 followed directly by an h4 confuses the outline), although you may jump back up to any higher level when a new section starts. Never use headings just to make text bold or big, and never use bold paragraphs where a heading belongs.



HEADINGS FORM A NESTED OUTLINE OF THE PAGE

A CORRECT HEADING HIERARCHY HTML

```
<h1>Learn Being Forward: HTML5 Course</h1>
<p>Everything you need to build real web pages.</p>
<h2>Beginner level</h2>
<h3>Module 1: Introduction to the web</h3>
<p>How the web works and what HTML is.</p>
<h3>Module 2: Setup and structure</h3>
<p>Editors, tags, attributes and the page skeleton.</p>
<h2>Fees and batches</h2>
<p>Weekday and weekend batches are available.</p>
```



HOW THE BROWSER DISPLAYS IT

3.2 Paragraphs and breaks

3.2.1 <p>

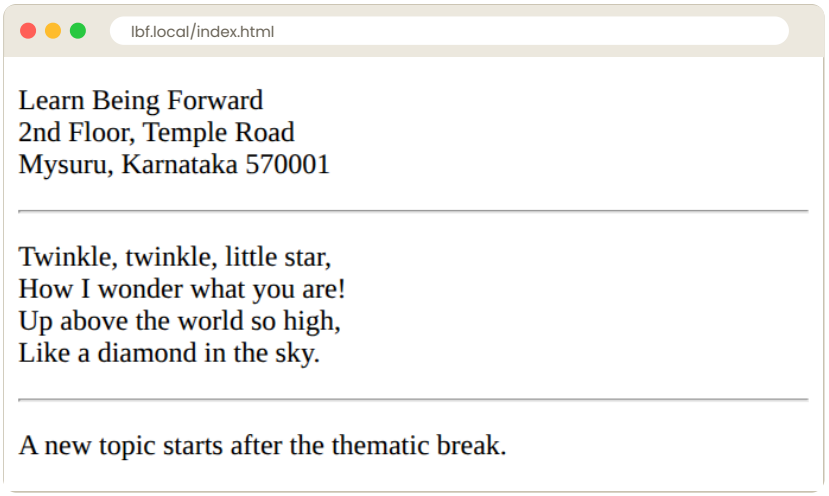
`<p>` marks a **paragraph**: a block of related sentences. Browsers add vertical space (a margin) before and after each paragraph. A paragraph may contain only text and inline elements: you cannot put headings, lists, tables or another paragraph inside it. Do not use empty paragraphs (`<p></p>`) or several `<br>` elements to create space; spacing is a job for CSS margins.

3.2.2
 and <hr>

`<br>` is a **line break** inside the same paragraph, for content where the line breaks are part of the meaning: postal addresses, poems, song lyrics. It is not for spacing or layout. `<hr>` is a **thematic break**: a change of topic or scene between paragraphs, displayed by default as a horizontal line. Both are void elements.

LINE BREAKS INSIDE PARAGRAPHS, AND THEMATIC BREAKS BETWEEN TOPICS — HTML

```
<p>Learn Being Forward<br>
2nd Floor, Temple Road<br>
Mysuru, Karnataka 570001</p>
<hr>
<p>Twinkle, twinkle, little star,<br>How I wonder what you are!<br>Up above the world so high,<br>Like
a diamond in the sky.</p>
<hr>
<p>A new topic starts after the thematic break.</p>
```



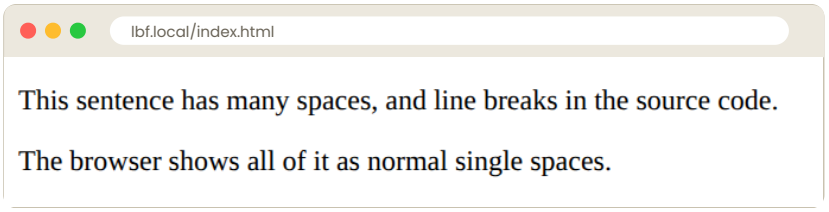
HOW THE BROWSER DISPLAYS IT

3.2.3 Whitespace collapsing

HTML **collapses whitespace**: any sequence of spaces, tabs and line breaks in the source becomes a **single space** on the page, and leading and trailing whitespace in a block is removed. That is why you can indent code freely. It also means you cannot create extra spaces or new lines just by typing them: use separate paragraphs, `<br>` for meaningful line breaks, CSS for spacing, the entity ` ` for a space that must not break (3.6), or `<pre>` for text whose spacing must be kept exactly.

SEVERAL SPACES AND LINE BREAKS COLLAPSE INTO ONE SPACE HTML

```
<p>This     sentence   has    many     spaces,
and line breaks
      in the source code.</p>
<p>The browser shows all of it as normal single spaces.</p>
```



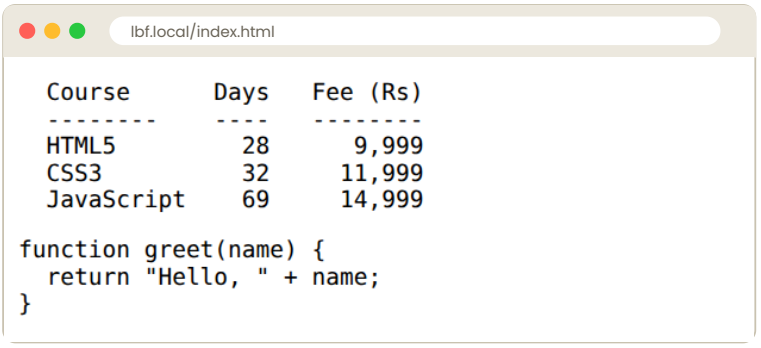
HOW THE BROWSER DISPLAYS IT

3.2.4 <pre> for preformatted text

`<pre>` displays **preformatted text** exactly as written, keeping every space and line break, in a monospace font by default. Use it for ASCII art, text-based tables and, combined with `<code>`, for blocks of program code. Characters such as `<` must still be written as entities inside `<pre>`.

PRE KEEPS SPACES AND LINE BREAKS EXACTLY HTML

```
<pre>
  Course      Days   Fee (Rs)
  --------    ----   --------
  HTML5         28      9,999
  CSS3          32     11,999
  JavaScript    69     14,999
</pre>
<pre><code>function greet(name) {
  return "Hello, " + name;
}</code></pre>
```



HOW THE BROWSER DISPLAYS IT

3.3 Text-level semantics

These inline elements give **meaning** to words and phrases inside a paragraph. Each has a default appearance (bold, italic, underline), but choose the element by meaning, not by look; the look can always be changed with CSS.

3.3.1 vs <b>

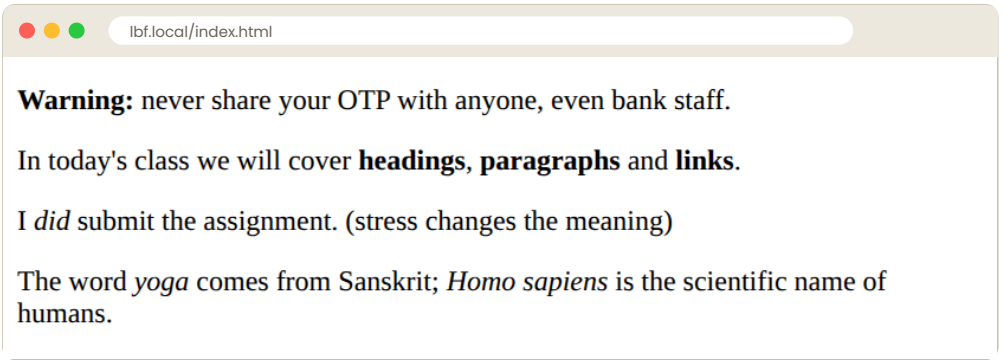
`<strong>` marks text of **strong importance, seriousness or urgency** ("**Warning:** do not share your OTP"); screen readers may emphasise it. `<b>` ("bring attention") marks text that is **stylistically offset** without extra importance: keywords in a summary, product names in a review, the lead words of an article. Both are bold by default. If in doubt, ask: would the sentence lose meaning if the bold were removed? If yes, use `<strong>`.

3.3.2 vs <i>

`<em>` marks **stress emphasis** that changes the meaning of a sentence, like stressing a word when speaking: "I *did* submit the form" versus "*I* did submit the form". `<i>` marks text in an **alternate voice or mood** without emphasis: technical terms, foreign words (with `lang`), thoughts, ship names, taxonomic names. Both are italic by default.

STRONG, B, EM AND I: SAME LOOK, DIFFERENT MEANINGS HTML

```
<p><strong>Warning:</strong> never share your OTP with anyone, even bank staff.</p>
<p>In today's class we will cover <b>headings</b>, <b>paragraphs</b> and <b>links</b>.</p>
<p>I <em>did</em> submit the assignment. (stress changes the meaning)</p>
<p>The word <i lang="sa">yoga</i> comes from Sanskrit; <i>Homo sapiens</i> is the scientific name of humans.</p>
```



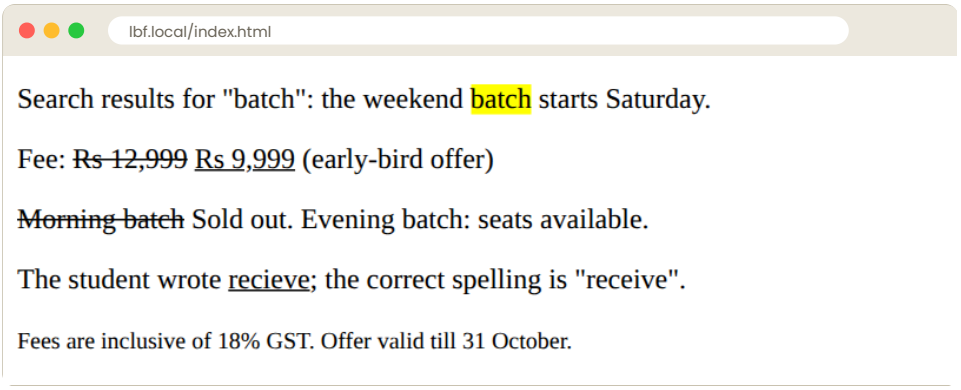
HOW THE BROWSER DISPLAYS IT

3.3.3 <mark>, <small>, <del>, <ins>, <s>, <u>

Element	Meaning	Default look	Example use
`<mark>`	highlighted because relevant in the current context	yellow background	search terms highlighted in results
`<small>`	side comments, fine print	smaller text	terms and conditions, copyright line
`<del>`	text removed from the document (an edit)	strikethrough	old price, changed requirement
`<ins>`	text added to the document (an edit)	underline	the new price or correction
`<s>`	no longer accurate or relevant (not an edit)	strikethrough	"Sold out" items, outdated info
`<u>`	unarticulated annotation, such as a spelling error	underline	marking misspelt words; avoid elsewhere (looks like a link)

MARK, DEL, INS, S, U AND SMALL HTML

```
<p>Search results for "batch": the weekend <mark>batch</mark> starts Saturday.</p>
<p>Fee: <del>Rs 12,999</del> <ins>Rs 9,999</ins> (early-bird offer)</p>
<p><s>Morning batch</s> Sold out. Evening batch: seats available.</p>
<p>The student wrote <u>recieve</u>; the correct spelling is "receive".</p>
<p><small>Fees are inclusive of 18% GST. Offer valid till 31 October.</small></p>
```



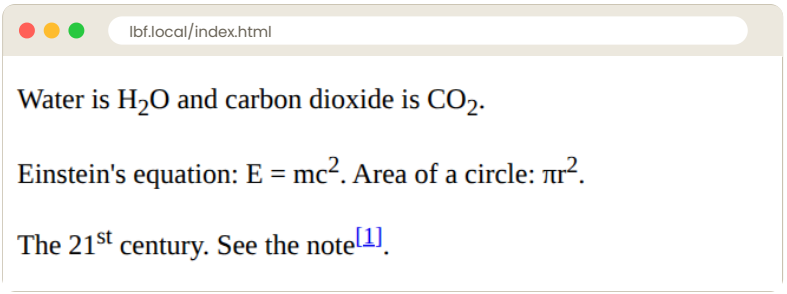
HOW THE BROWSER DISPLAYS IT

3.3.4 <sub> and <sup>

`<sub>` displays **subscript** text, lowered and smaller, required by typographical convention: chemical formulas (H_2O), mathematical indices (x_1). `<sup>` displays **superscript**: exponents (x^2), ordinal suffixes in some languages (1st), and footnote markers.

SUBSCRIPT AND SUPERSCRIPT HTML

```
<p>Water is H<sub>2</sub>O and carbon dioxide is CO<sub>2</sub>.</p>
<p>Einstein's equation: E = mc<sup>2</sup>. Area of a circle: &pi;r<sup>2</sup>.</p>
<p>The 21<sup>st</sup> century. See the note<sup><a href="#note1">[1]</a></sup>.</p>
```



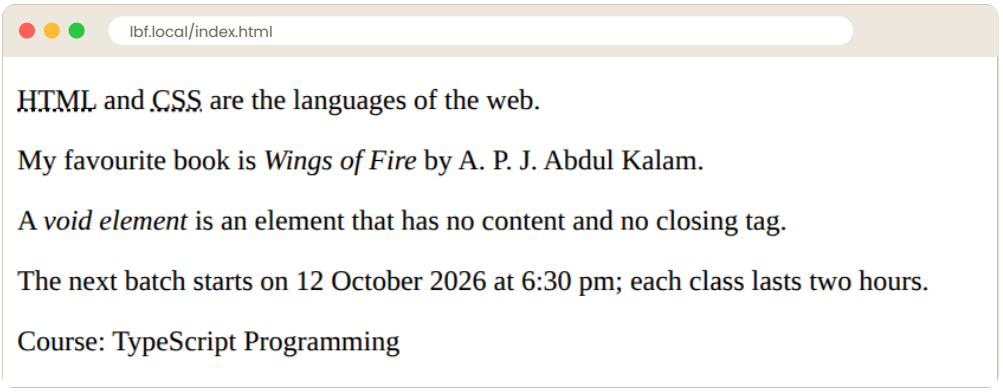
HOW THE BROWSER DISPLAYS IT

3.3.5 <abbr>, <cite>, <dfn>, <time>, <data>

`<abbr>` marks an **abbreviation or acronym**; its `title` gives the full form (shown as a tooltip). `<cite>` marks the **title of a creative work** (book, film, song, article, website). `<dfn>` marks the **defining instance of a term**, the place where it is defined. `<time>` marks a date or time, with a machine-readable `datetime` attribute (formats such as `2026-10-12`, `2026-10-12T18:30+05:30`, `18:30`, `PT2H30M` for durations), so browsers, search engines and calendars can understand it. `<data>` links content to a machine-readable `value`, such as a product code.

ABBR, CITE, DFN, TIME AND DATA HTML

```
<p><abbr title="HyperText Markup Language">HTML</abbr> and <abbr title="Cascading Style
Sheets">CSS</abbr> are the languages of the web.</p>
<p>My favourite book is <cite>Wings of Fire</cite> by A. P. J. Abdul Kalam.</p>
<p>A <dfn>void element</dfn> is an element that has no content and no closing tag.</p>
<p>The next batch starts on <time datetime="2026-10-12">12 October 2026</time> at <time
datetime="18:30">6:30 pm</time>; each class lasts <time datetime="PT2H">two hours</time>.</p>
<p>Course: <data value="TS101">TypeScript Programming</data></p>
```



HOW THE BROWSER DISPLAYS IT

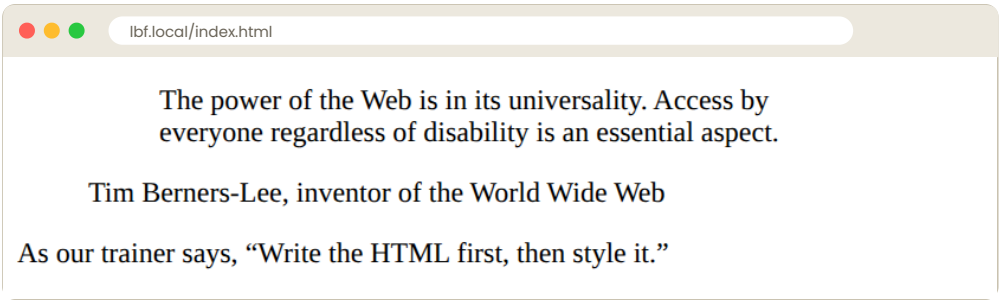
3.4 Quotes and code

3.4.1 <blockquote> and <q>

`<blockquote>` is a **block quotation**: a longer quote displayed as its own indented block, with an optional `cite` attribute holding the source URL; the visible attribution (author, work) goes outside the quote, often in a `<figure>` with `<figcaption>`. `<q>` is a short **inline quotation**; the browser adds the quotation marks automatically, appropriate to the language, so do not type them yourself.

A BLOCK QUOTATION WITH ATTRIBUTION, AND AN INLINE QUOTATION HTML

```
<figure>
  <blockquote cite="https://www.w3.org/People/Berners-Lee/">
    <p>The power of the Web is in its universality. Access by everyone regardless of disability is an
essential aspect.</p>
  </blockquote>
  <figcaption>Tim Berners-Lee, inventor of the World Wide Web</figcaption>
</figure>
<p>As our trainer says, <q>Write the HTML first, then style it.</q></p>
```



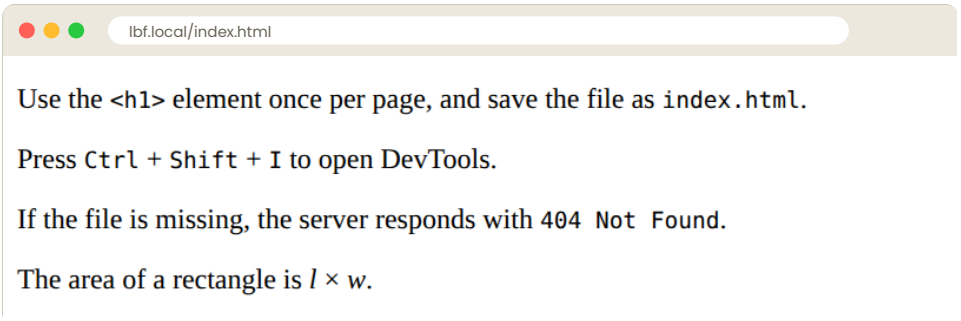
HOW THE BROWSER DISPLAYS IT

3.4.2 <code>, <kbd>, <samp>, <var>

Four elements mark up computer-related text, all shown in a monospace font by default. `<code>` is a fragment of **computer code** (an element name, a function, a file name). `<kbd>` is **keyboard input** the user should type or press (<kbd>Ctrl</kbd>+<kbd>S</kbd>). `<samp>` is **sample output** from a program. `<var>` is a **variable** in mathematics or programming (italic by default).

CODE, KBD, SAMP AND VAR HTML

```
<p>Use the <code>&lt;h1&gt;</code> element once per page, and save the file as
<code>index.html</code>.</p>
<p>Press <kbd>Ctrl</kbd> + <kbd>Shift</kbd> + <kbd>I</kbd> to open DevTools.</p>
<p>If the file is missing, the server responds with <samp>404 Not Found</samp>.</p>
<p>The area of a rectangle is <var>l</var> &times; <var>w</var>.</p>
```



HOW THE BROWSER DISPLAYS IT

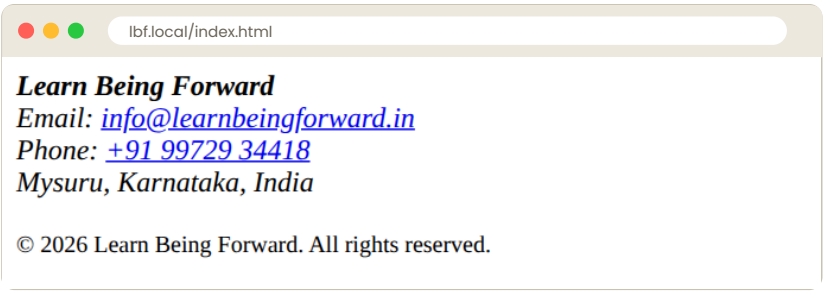
3.5 Contact and addresses

3.5.1 <address>

`<address>` provides **contact information** for the nearest `<article>` or for the whole page (when placed in the page footer): a name, email, phone, social links or a postal address. It is displayed in italics by default. It is not for every postal address on a page (an address mentioned in a story is just a paragraph), only for contact details of the author or owner.

CONTACT DETAILS IN AN ADDRESS ELEMENT HTML

```
<footer>
  <address>
    <strong>Learn Being Forward</strong><br>
    Email: <a href="mailto:info@learnbeingforward.in">info@learnbeingforward.in</a><br>
    Phone: <a href="tel:+919972934418">+91 99729 34418</a><br>
    Mysuru, Karnataka, India
  </address>
  <p><small>&copy; 2026 Learn Being Forward. All rights reserved.</small></p>
</footer>
```



HOW THE BROWSER DISPLAYS IT

3.6 Character entities

3.6.1 <, >, &, , ©, ₹

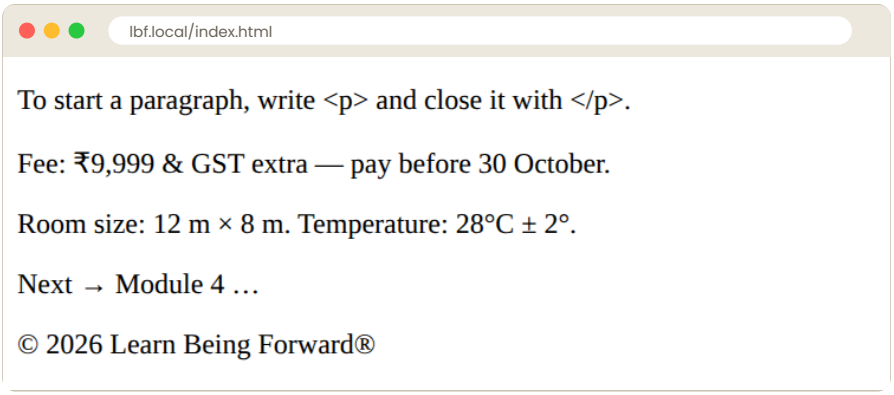
Some characters have special meaning in HTML: `<` starts a tag and `&` starts an entity. To display them as text, write a **character entity**: an ampersand, a name or number, and a semicolon. Named entities include `<` (<), `>` (>), `&` (&), `"` ("), ` ` (a non-breaking space that keeps two words on the same line and does not collapse), `©` (©), `®` (®), `™` (™), `×` (×) and `→` (→). Any Unicode character can be written as a **numeric** entity, decimal (`₹` for the rupee sign ₹) or hexadecimal (`₹`). With UTF-8 you can also type most characters directly; entities remain necessary for `<` and `&`, and useful for invisible characters such as non-breaking spaces.

Character	Named entity	Numeric entity	Use
<	`<`	`<`	show a tag as text
>	`>`	`>`	show a tag as text
&	`&`	`&`	ampersands in text and URLs
" and '	`"` `'`	`"` `'`	quotes inside attribute values
(non-breaking space)	` `	` `	"Rs 500", "10 km"
© ® ™	`©` `®` `™`	`©` …	legal notices
₹	none	`₹` or `₹`	Indian rupee sign
× ÷ ° ±	`×` `÷` `°` `±`		maths and measurements

Character	Named entity	Numeric entity	Use
← → … —	`←` `→` `…` `—`		arrows, ellipsis, dash

CHARACTER ENTITIES HTML

```
<p>To start a paragraph, write &lt;p&gt; and close it with &lt;/p&gt;.</p>
<p>Fee: &#8377;9,999 &amp; GST extra &mdash; pay before 30 October.</p>
<p>Room size: 12 m &times; 8 m. Temperature: 28&deg;C &plusmn; 2&deg;.</p>
<p>Next &rarr; Module 4 &hellip;</p>
<p>&copy; 2026 Learn Being Forward&reg;</p>
```



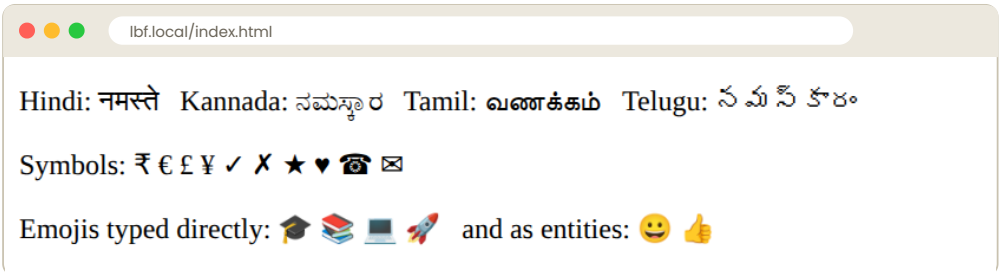
HOW THE BROWSER DISPLAYS IT

3.6.2 Emojis and Unicode

Unicode is the universal character set that gives every character in every writing system a number (a code point, written U+20B9 for ₹). **UTF-8** is the encoding that stores those numbers as bytes, and with `<meta charset="UTF-8">` you can type any of them directly: Indian scripts, accented letters, symbols and **emojis**, which are simply Unicode characters (😀 is U+1F600, entity `😀`). Emojis look different on each operating system and some screen readers read their names aloud, so use them sparingly and never as the only way to convey important information.

UTF-8 TEXT IN SEVERAL SCRIPTS, SYMBOLS AND EMOJIS HTML

```
<p>Hindi: नमस्ते   Kannada: ನಮಸ್ಕಾರ   Tamil: வணக்கம்   Telugu: నమస్కారం</p>
<p>Symbols: ₹ € £ ¥ ✓ ✗ ★ ♥ ☎ ✉</p>
<p>Emojis typed directly: 🎓 📚 💻 🚀   and as entities: &#x1F600; &#x1F44D;</p>
```



HOW THE BROWSER DISPLAYS IT

3.7 Bidirectional text and language

3.7.1 lang and dir attributes, <bdi>, <bdo>

Some languages, such as Arabic, Hebrew and Urdu, are written **right to left** (RTL). The global `dir` attribute sets the base direction of an element: `ltr`, `rtl` or `auto` (let the browser decide from the content); for a whole RTL page put `dir="rtl"` on `<html>`. Mark text in another language with `lang` so it is pronounced and hyphenated correctly. `<bdi>` (bidirectional isolate) isolates text whose direction is unknown, typically user-generated names in a list or comment, so it does not scramble the surrounding text. `<bdo dir="rtl">` (bidirectional override) forces a direction, reversing the order of characters.

RIGHT-TO-LEFT TEXT, ISOLATED NAMES AND A DIRECTION OVERRIDE HTML

```
<p dir="rtl" lang="ar">مرحبا بكم في دورة HTML</p>
<p dir="rtl" lang="ur">ہمارے کورس میں خوش آمدید</p>
<ul>
  <li>User <bdi>إيمان</bdi>: 3 posts</li>
  <li>User <bdi>Asha</bdi>: 5 posts</li>
</ul>
<p>Normal: HTML5   Overridden: <bdo dir="rtl">HTML5</bdo></p>
```



HOW THE BROWSER DISPLAYS IT

Headings h1 to h6 form the outline: one h1 per page, levels in order, chosen for structure not size. Paragraphs (p) hold inline content; br is for meaningful line breaks, hr for a change of topic; whitespace collapses to single spaces except in pre. Text-level elements carry meaning: strong (importance) vs b (offset), em (stress) vs i (alternate voice), mark (highlight), small (fine print), del and ins (edits), s (no longer accurate), u (annotation), sub and sup, abbr (with title), cite (work titles), dfn (definitions), time (with datetime) and data (with value). blockquote and q mark quotations (with cite and figcaption for attribution); code, kbd, samp and var mark computer text. address holds contact details for the page or article. Entities such as < > & © and numeric ₹ display special characters; with UTF-8 any Unicode character and emoji can be typed directly. lang declares languages, dir sets text direction, bdi isolates unknown-direction text and bdo overrides direction.

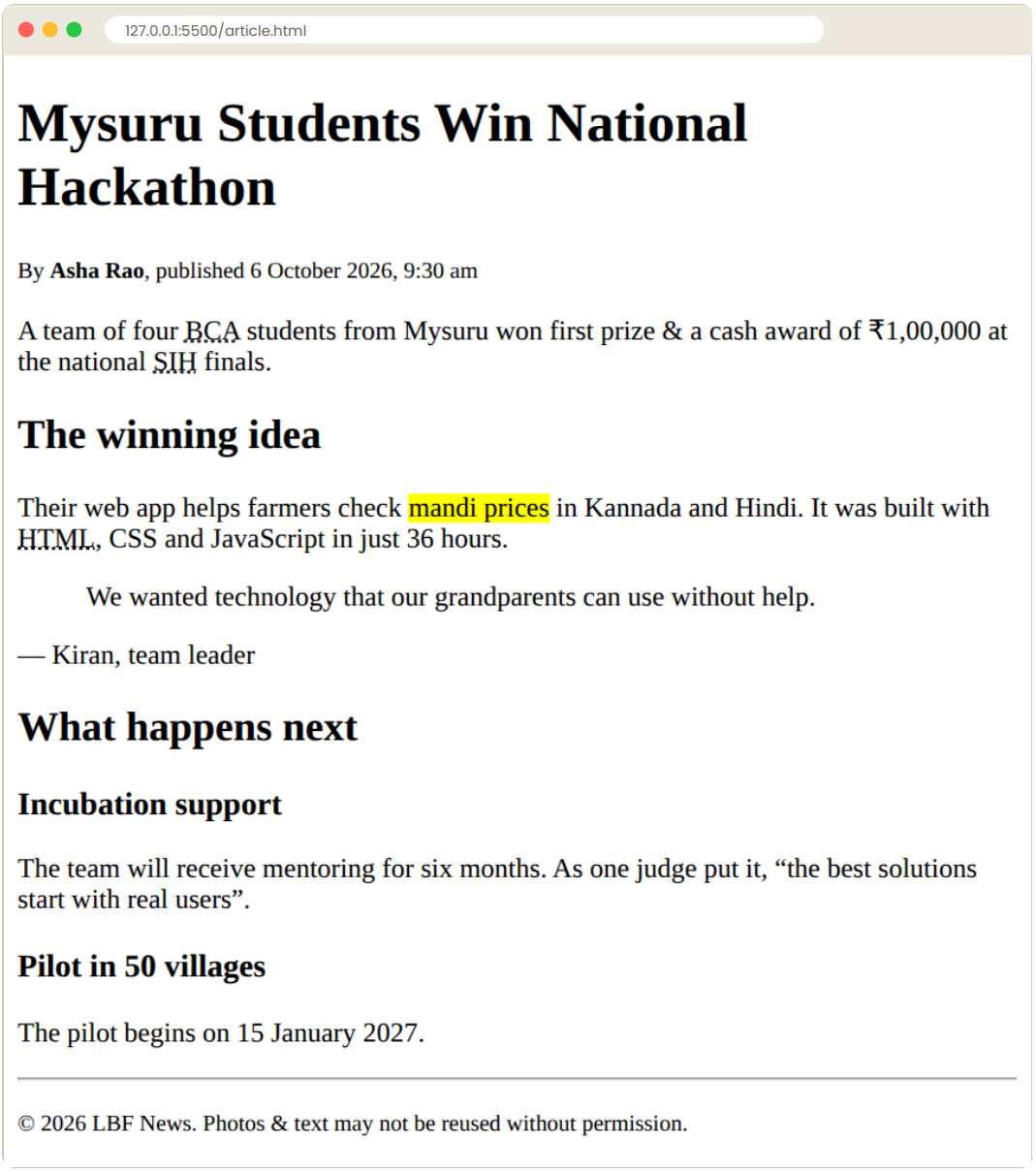
PRACTICE CHECKPOINT

Recreate a **news article page** with a proper **heading hierarchy**, **quotes**, **abbreviations**, a **dated byline using <time>**, and **special characters**.

SAMPLE SOLUTION: A NEWS ARTICLE WITH SEMANTIC TEXT MARKUP

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mysuru students win national hackathon | LBF News</title>
</head>
<body>
  <h1>Mysuru Students Win National Hackathon</h1>
  <p><small>By <strong>Asha Rao</strong>, published <time datetime="2026-10-06T09:30+05:30">6 October 2026, 9:30 am</time></small></p>
  <p>A team of four <abbr title="Bachelor of Computer Applications">BCA</abbr> students from Mysuru won first prize &amp;amp; a cash award of &amp;#8377;1,00,000 at the national <abbr title="Smart India Hackathon">SIH</abbr> finals.</p>
  <h2>The winning idea</h2>
  <p>Their web app helps farmers check <mark>mandi prices</mark> in Kannada and Hindi. It was built with <abbr title="HyperText Markup Language">HTML</abbr>, CSS and JavaScript in just 36&amp;nbsp;hours.</p>
  <blockquote>
    <p>We wanted technology that our grandparents can use without help.</p>
  </blockquote>
  <p>&amp;mdash; Kiran, team leader</p>
  <h2>What happens next</h2>
  <h3>Incubation support</h3>
  <p>The team will receive mentoring for six months. As one judge put it, <q>the best solutions start with real users</q>.</p>
  <h3>Pilot in 50 villages</h3>
  <p>The pilot begins on <time datetime="2027-01-15">15 January 2027</time>.</p>
  <hr>
  <p><small>&amp;copy; 2026 LBF News. Photos &amp;amp; text may not be reused without permission.</small></p>
</body>
</html>
```



THE NEWS ARTICLE PAGE

MODULE 04 · BEGINNER

Links and navigation

Minimum duration **1 day** | Level **Beginner**

Links connect pages and are what make the web a "web". This module covers the anchor element, absolute and relative URLs and file paths, the target, rel, title and download attributes, links to other pages, to sections within a page, to email, phone and WhatsApp, navigation menus built from lists of links, multi-page websites and folder structure, and best practices for link text and link states.

IN THIS MODULE

4.1 The anchor element
4.2 Link attributes
4.3 Types of links
4.4 Navigation menus
4.5 Link best practices

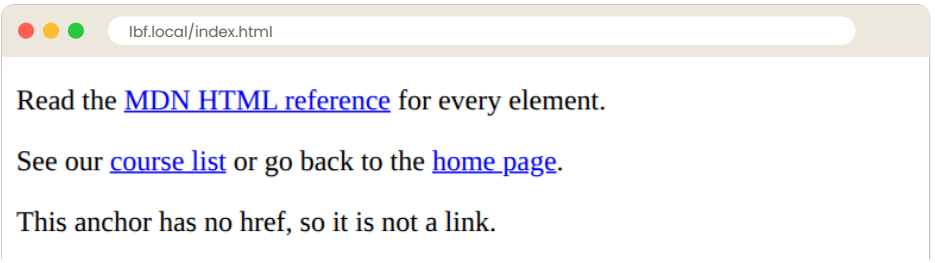
4.1 The anchor element

4.1.1 <a href="">

The **anchor** element `<a>` creates a **hyperlink**. Its `href` (hypertext reference) attribute holds the destination URL, and its content is the clickable part, usually text, but it may also be an image or even a whole card. Browsers display unvisited links blue and underlined, visited links purple, and show the destination in the status bar on hover. A link is focusable with the Tab key and activated with Enter, so links are accessible by default. An `<a>` without `href` is just a placeholder, not a link.

LINKS TO ANOTHER WEBSITE AND TO PAGES OF THE SAME SITE — HTML

```
<p>Read the <a href="https://developer.mozilla.org/en-US/docs/Web/HTML">MDN HTML reference</a> for
every element.</p>
<p>See our <a href="courses.html">course list</a> or go back to the <a href="index.html">home
page</a>.</p>
<p><a>This anchor has no href, so it is not a link.</a></p>
```



HOW THE BROWSER DISPLAYS IT

4.1.2 Absolute vs relative URLs

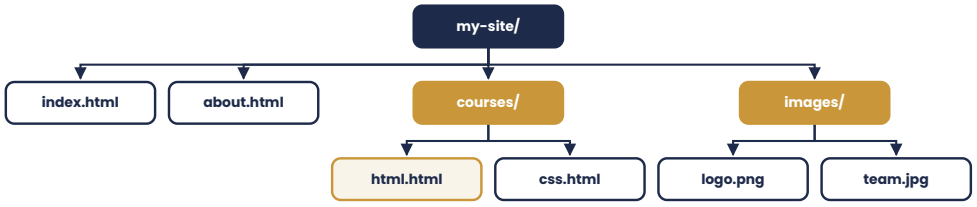
An **absolute URL** contains the full address, including the scheme and domain: `https://learnbeingforward.in/courses/html.html`. Use absolute URLs to link to **other websites**. A **relative URL** gives a location relative to the current page: `about.html`, `images/logo.png`, `../index.html`. Use

relative URLs for links **within your own site**: they are shorter and keep working when the whole site moves to another domain or is opened locally. A URL starting with `/` is **root-relative**: `/courses/html.html` always starts from the site's root folder, whatever the current page (on a real server; with `file://` it points to the root of your disk).

Type	Example	Points to	Use for
absolute	`https://mdn.dev/`	exactly that address	other websites
protocol-relative (old)	`//cdn.example.com/a.js`	same scheme as the page	avoid; use https://
root-relative	`/images/logo.png`	from the site root	shared assets on a hosted site
document-relative	`images/logo.png`	from the current page's folder	most internal links
parent folder	`../index.html`	one folder up	links from sub-folders back up
fragment only	`#contact`	a section of the same page	in-page navigation

4.1.3 File paths: same folder, sub-folder, parent folder ../

Relative paths describe how to walk from the folder of the **current file** to the target: a plain name means the **same folder**; `folder/name` goes **down** into a sub-folder; `../` goes **up** one level (and `../../` two levels). Paths use forward slashes `/`, even on Windows. File names in URLs are case-sensitive on most web servers: `About.html` and `about.html` are different files, a common cause of links that work locally on Windows but break once deployed.



A SMALL SITE: WHICH PATH LEADS FROM ONE FILE TO ANOTHER?

From the file	To	Relative path
`index.html`	`about.html`	`about.html` (same folder)
`index.html`	`courses/html.html`	`courses/html.html` (down into a sub-folder)
`index.html`	`images/logo.png`	`images/logo.png`
`courses/html.html`	`courses/css.html`	`css.html` (same folder)
`courses/html.html`	`index.html`	`../index.html` (up one level)
`courses/html.html`	`images/team.jpg`	`../images/team.jpg` (up, then down)

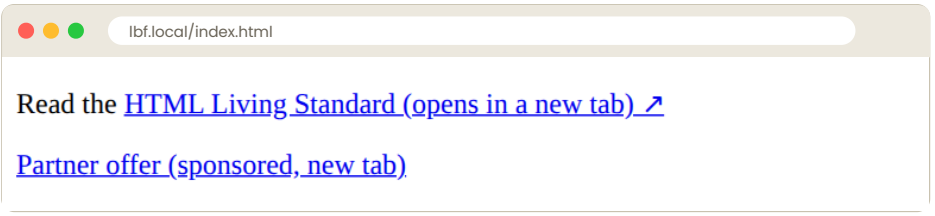
4.2 Link attributes

4.2.1 target="_blank" and rel="noopener noreferrer"

By default a link opens in the same tab. `target="_blank"` opens it in a **new tab**. Use it sparingly, mainly for external links when the user is in the middle of a task (filling a form, watching a lesson), because unexpected new tabs confuse many users, especially screen-reader users; if you do open a new tab, say so in the link text or with an icon. For security, add `rel="noopener"` to external links that open in a new tab: it stops the new page from controlling the original tab through `window.opener` (a phishing technique called "tabnabbing"). Modern browsers apply noopener automatically for `_blank`, but writing it is still recommended. `noreferrer` additionally hides the referring page's URL from the destination. Other `rel` values include `nofollow` and `sponsored` (tell search engines not to pass ranking credit, for user-generated and paid links).

EXTERNAL LINKS THAT OPEN IN A NEW TAB SAFELY HTML

```html
<p>Read the <a href="https://html.spec.whatwg.org/" target="_blank" rel="noopener noreferrer">HTML Living Standard (opens in a new tab) &#8599;</a></p>
<p><a href="https://example.com/partner-offer" target="_blank" rel="noopener sponsored">Partner offer (sponsored, new tab)</a></p>
```



HOW THE BROWSER DISPLAYS IT

4.2.2 title

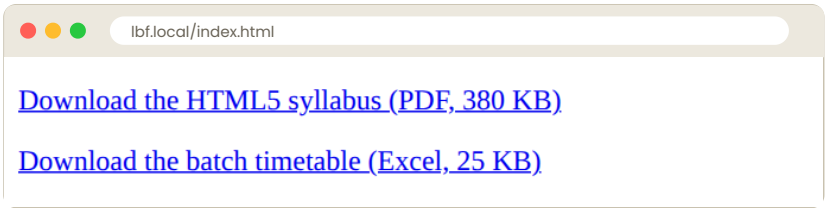
The `title` attribute adds advisory text shown as a tooltip on hover. It is **not** a substitute for clear link text: tooltips do not appear on touch screens or for keyboard users, and screen readers treat them inconsistently. Use it only for supplementary information.

4.2.3 download

The `download` attribute tells the browser to **download** the linked file instead of opening it; an optional value suggests a file name: `<a href="files/syllabus-2026.pdf" download="LBF-HTML5-Syllabus.pdf">`. It works for same-origin files (files from your own site); for files on other domains the browser usually ignores it. Tell users the file type and size in the link text.

DOWNLOAD LINKS THAT STATE THE FILE TYPE AND SIZE HTML

```html
<p><a href="files/html5-syllabus.pdf" download="LBF-HTML5-Syllabus.pdf">Download the HTML5 syllabus (PDF, 380 KB)</a></p>
<p><a href="files/timetable.xlsx" download>Download the batch timetable (Excel, 25 KB)</a></p>
```



HOW THE BROWSER DISPLAYS IT

4.3 Types of links

4.3.1 Links to other pages and sites

Most links go to another page of the same site (relative URL) or to another website (absolute URL). A link can wrap more than text: since HTML5 an `<a>` may contain block content such as a heading, image and paragraph, making a whole card clickable (as long as it contains no other interactive elements such as buttons or links).

A WHOLE CARD AS ONE LINK (BLOCK CONTENT INSIDE A) HTML

```
<a href="courses/html.html" style="display:block; max-width:300px; border:1px solid #C9C2B0;
padding:10px; color:inherit; text-decoration:none">
  <img src="logo.svg" alt="" width="40" height="40">
  <h3>HTML5 Course</h3>
  <p>17 modules &middot; 28 days &middot; from &#8377;9,999</p>
</a>
```



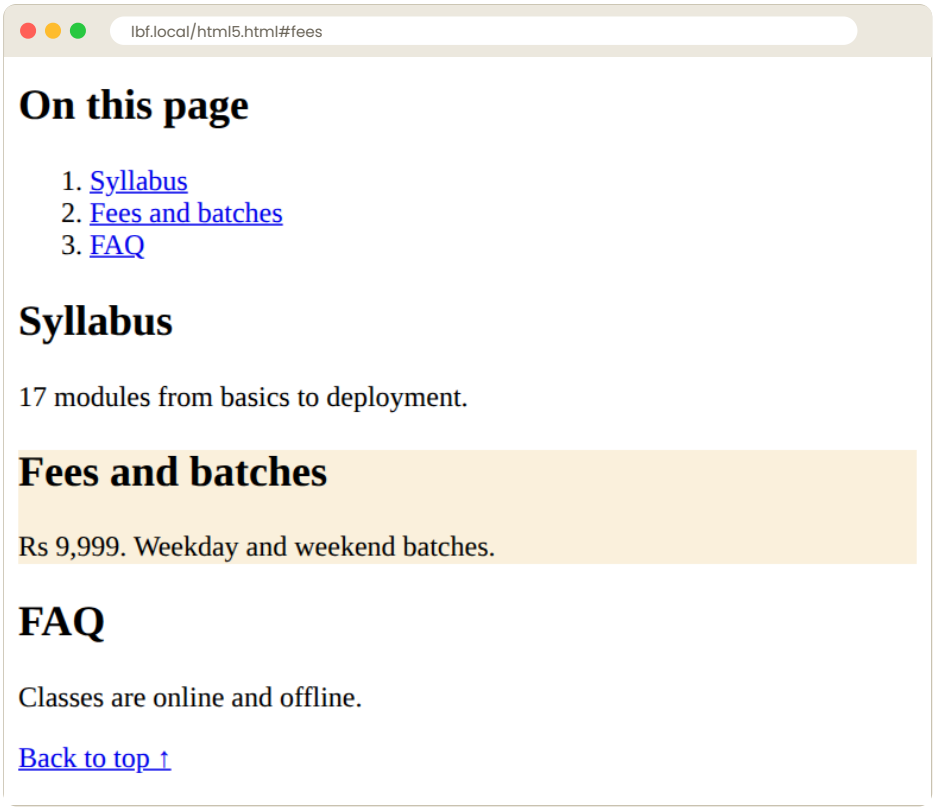
HOW THE BROWSER DISPLAYS IT

4.3.2 Page sections with id anchors (#section)

A **fragment** link jumps to an element on a page: give the target an `id` (`<section id="fees">`) and link to `#fees` on the same page, or to `courses.html#fees` on another page. This builds tables of contents, "back to top" links (`href="#top"`, which browsers support even without an element with that id), and skip links (12.3.2). The browser scrolls the target into view and updates the URL, so the section can be bookmarked and shared. The CSS property `scroll-behavior: smooth` animates the jump.

A TABLE OF CONTENTS WITH FRAGMENT LINKS; THE BROWSER SCROLLED TO #FEES AFTER THE CLICK HTML

```
<nav>
  <h2>On this page</h2>
  <ol>
    <li><a href="#syllabus">Syllabus</a></li>
    <li><a href="#fees">Fees and batches</a></li>
    <li><a href="#faq">FAQ</a></li>
  </ol>
</nav>
<section id="syllabus"><h2>Syllabus</h2><p>17 modules from basics to deployment.</p></section>
<section id="fees" style="background:#FDF3DC"><h2>Fees and batches</h2><p>Rs 9,999. Weekday and
weekend batches.</p></section>
<section id="faq"><h2>FAQ</h2><p>Classes are online and offline.</p><p><a href="#top">Back to top
&uarr;</a></p></section>
```



HOW THE BROWSER DISPLAYS IT

4.3.3 Email mailto: and phone tel: links

`mailto:` links open the user's email app with a new message; you can pre-fill the subject, body, cc and bcc as query parameters, with spaces encoded as `%20`. `tel:` links dial a number on phones (and offer to call through apps on computers); write the number in international format with the country code and no spaces: `tel:+919972934418`. `sms:` links open the messaging app. Always show the address or number in the visible text too, for users without a configured email app.

MAILTO:, TEL: AND SMS: LINKS HTML

```
<p>Email: <a href="mailto:info@learnbeingforward.in">info@learnbeingforward.in</a></p>
<p><a href="mailto:info@learnbeingforward.in?
subject=Enquiry%20about%20HTML5%20batch&amp;body=Hello%2C%20I%20would%20like%20to%20know%20the%20batch
%20timings.">Send an enquiry email</a> (subject and message pre-filled)</p>
<p>Call us: <a href="tel:+919972934418">+91 99729 34418</a></p>
<p><a href="sms:+919972934418?body=Please%20call%20me%20back">Send us an SMS</a></p>
```



HOW THE BROWSER DISPLAYS IT

4.3.4 WhatsApp and other app links

Many apps provide **web links** that open the app on phones and the web version on computers. WhatsApp's click-to-chat link is `https://wa.me/919972934418` (country code and number, without + or spaces), with an optional pre-filled message: `https://wa.me/919972934418?text=Hi%2C%20I%20want%20to%20join`. Similar links exist for Telegram (`https://t.me/username`), maps (`https://maps.google.com/?q=...` or `geo:`), app stores and video calls. Prefer these https links over custom schemes such as `whatsapp://`, which fail when the app is not installed.

LINKS THAT OPEN APPS HTML

```
<p><a href="https://wa.me/919972934418?text=Hi%2C%20I%20want%20to%20join%20the%20HTML5%20batch"
target="_blank" rel="noopener">Chat with us on WhatsApp</a></p>
<p><a href="https://maps.google.com/?q=Mysuru+Palace" target="_blank" rel="noopener">Find us on Google
Maps</a></p>
<p><a href="https://t.me/learnbeingforward" target="_blank" rel="noopener">Join our Telegram
channel</a></p>
```



HOW THE BROWSER DISPLAYS IT

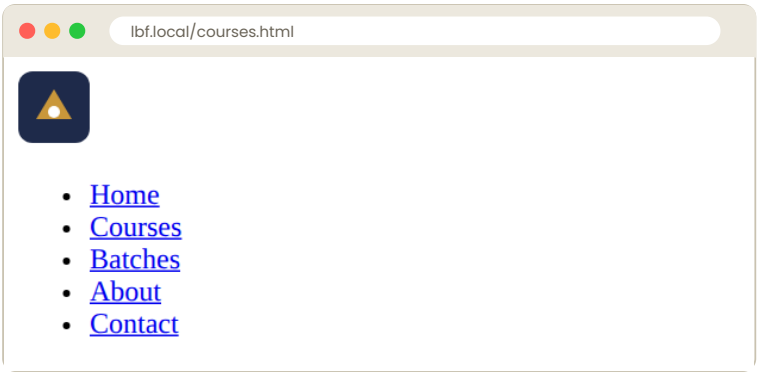
4.4 Navigation menus

4.4.1 Building a navbar with a list of links

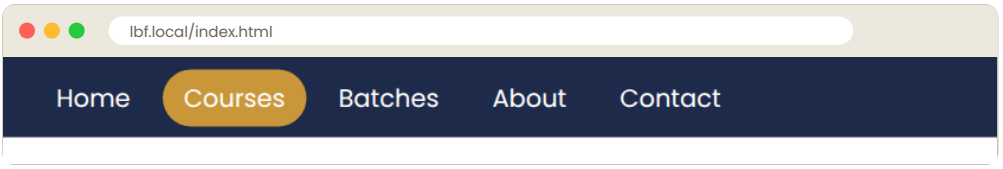
A site's main navigation is a **list of links** inside a `<nav>` element: the list tells screen readers how many items there are ("list, 5 items"), and `<nav>` marks the region as navigation so users can jump to it (9.2.2). Mark the current page with `aria-current="page"` so assistive technology announces it, and style it differently with CSS. Without CSS the menu is a vertical bulleted list; the CSS course turns it into a horizontal bar.

A NAVIGATION MENU: A LIST OF LINKS INSIDE NAV HTML

```
<header>
  <a href="index.html"><img src="logo.svg" alt="Learn Being Forward home" width="40" height="40"></a>
  <nav aria-label="Main">
    <ul>
      <li><a href="index.html">Home</a></li>
      <li><a href="courses.html" aria-current="page">Courses</a></li>
      <li><a href="batches.html">Batches</a></li>
      <li><a href="about.html">About</a></li>
      <li><a href="contact.html">Contact</a></li>
    </ul>
  </nav>
</header>
```



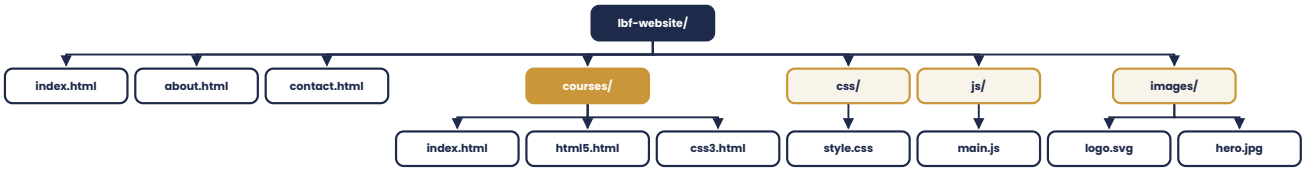
HOW THE BROWSER DISPLAYS IT



WITH CSS THE SAME HTML BECOMES A HORIZONTAL NAVIGATION BAR

4.4.2 Multi-page websites and folder structure

A real site has many pages that share the same header, navigation and footer. In plain HTML each page contains its own copy of these parts (later, tools and frameworks generate them from one template). Organise files in folders by type, keep the home page as `index.html` in the root, and use consistent, lowercase, hyphenated names. Every page should link back to the home page (usually through the logo) and include the same navigation in the same place, so users always know where they are.



A TYPICAL FOLDER STRUCTURE FOR A MULTI-PAGE WEBSITE

4.5 Link best practices

4.5.1 Descriptive link text (not "click here")

Link text should make sense **on its own**, because many users scan links without reading the surrounding sentence: screen-reader users often pull up a list of all links on the page, and sighted users skim. "Click here", "read more" and "this link" are meaningless out of context and also bad for search engines, which use link text to understand the target page. Describe the destination or action: "Download the HTML5 syllabus (PDF)", "Read the fees and batch timings". Avoid making raw URLs the link text, and do not link the same text to different destinations on one page.

Vague link text	Descriptive link text
To see the batch timings, click here.	See the weekday and weekend batch timings.
Our new syllabus is ready. Read more	Read the new HTML5 syllabus.
Download the brochure here.	Download the course brochure (PDF, 1.2 MB).

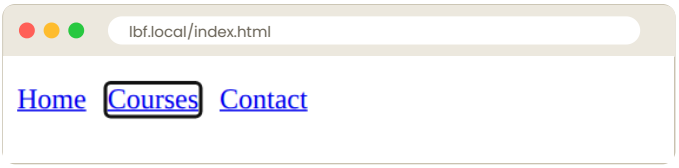
THE SAME LINKS, BEFORE AND AFTER: ONLY THE RIGHT-HAND LINKS MAKE SENSE WHEN READ ALONE

4.5.2 Visited and focus states (styled with CSS)

Links have **states** that CSS can style: `:link` (unvisited), `:visited`, `:hover` (mouse over), `:focus` / `:focus-visible` (selected with the keyboard) and `:active` (being clicked). Keep visited links distinguishable on content-heavy sites, and **never remove the focus outline** without a clear replacement: keyboard users rely on it to see where they are (12.3.3). In the screenshot below, the second link has keyboard focus (after pressing Tab twice), shown by the browser's default focus ring.

DEFAULT LINK STYLES; THE SECOND LINK HAS KEYBOARD FOCUS HTML

```
<p><a href="index.html">Home</a>   <a href="courses.html">Courses</a>   <a
href="contact.html">Contact</a></p>
```



HOW THE BROWSER DISPLAYS IT

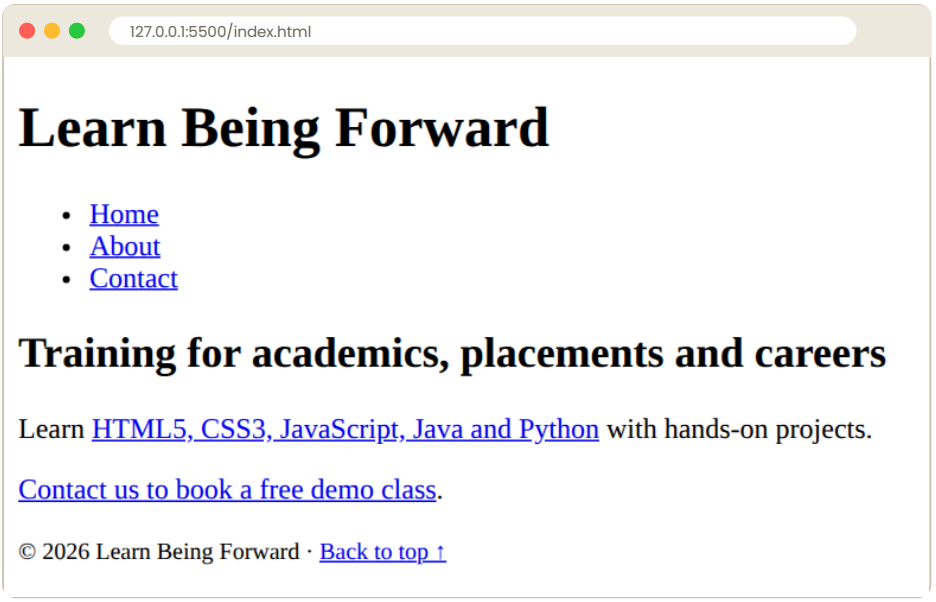
The anchor element a with an href creates a link; its content (text, image or a block such as a card) is clickable. Absolute URLs (https://...) are for other sites; relative URLs (page.html, folder/page.html, ../page.html) for your own pages; root-relative URLs start with /. target="_blank" opens a new tab and should be paired with rel="noopener" (and noreferrer if needed); rel="nofollow" and "sponsored" inform search engines; title gives a tooltip; download downloads a same-origin file. Fragment links (#id) jump to sections, enable tables of contents and back-to-top links. mailto:, tel: and sms: open email, phone and messaging, and https links such as wa.me open apps. Main navigation is a list of links inside nav, with aria-current="page" on the current item; multi-page sites repeat the header, nav and footer and use clear folders. Write descriptive link text, never "click here", and keep visible focus styles.

PRACTICE CHECKPOINT

A **three-page website (Home, About, Contact)** with a **shared navbar**, in-page **"back to top" links**, and **email and phone links**.

INDEX.HTML HTML

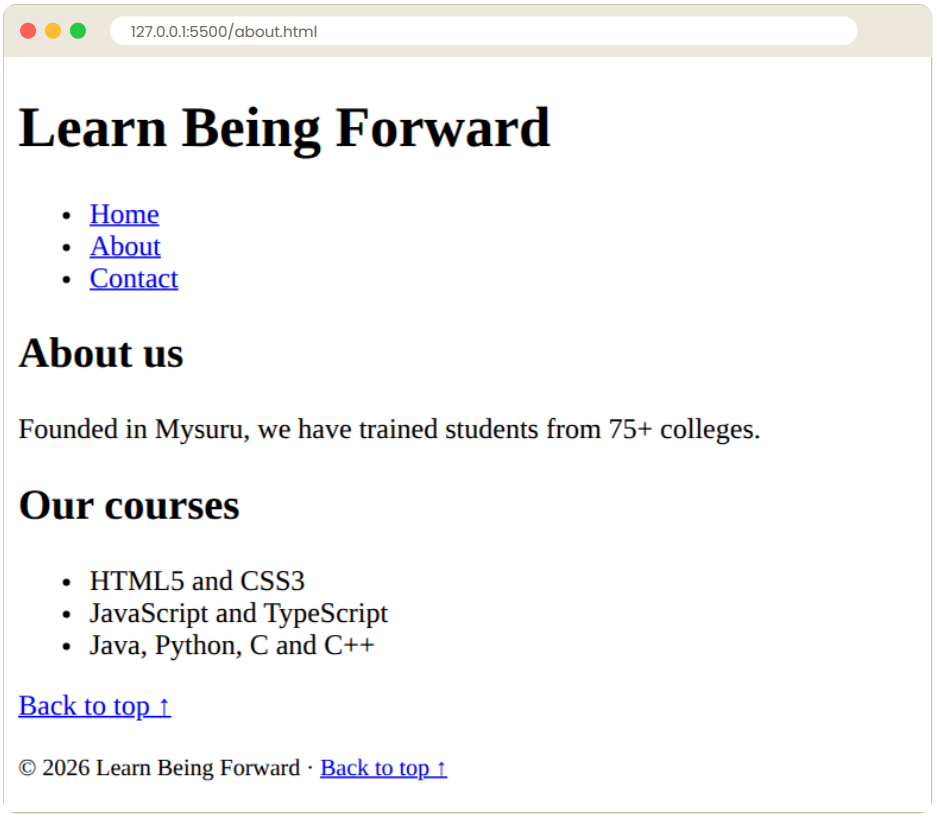
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Home | Learn Being Forward</title>
</head>
<body>
  <header id="top">
    <h1>Learn Being Forward</h1>
    <nav aria-label="Main">
      <ul>
        <li><a href="index.html" aria-current="page">Home</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="contact.html">Contact</a></li>
      </ul>
    </nav>
  </header>
  <main>
    <h2>Training for academics, placements and careers</h2>
    <p>Learn <a href="about.html#courses">HTML5, CSS3, JavaScript, Java and Python</a> with hands-on
projects.</p>
    <p><a href="contact.html">Contact us to book a free demo class</a>.</p>
  </main>
  <footer>
    <p><small>&copy; 2026 Learn Being Forward &middot; <a href="#top">Back to top &uarr;</a></small>
</p>
  </footer>
</body>
</html>
```



HOME PAGE

ABOUT.HTML HTML

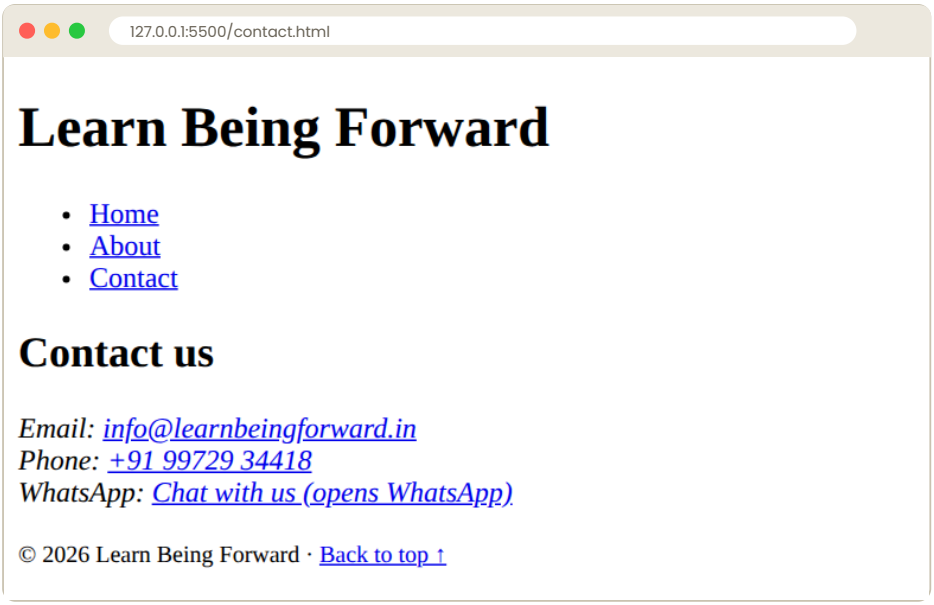
```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>About | Learn Being Forward</title>
</head>
<body>
  <header id="top">
    <h1>Learn Being Forward</h1>
    <nav aria-label="Main">
      <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="about.html" aria-current="page">About</a></li>
        <li><a href="contact.html">Contact</a></li>
      </ul>
    </nav>
  </header>
  <main>
    <h2>About us</h2>
    <p>Founded in Mysuru, we have trained students from 75+ colleges.</p>
    <h2 id="courses">Our courses</h2>
    <ul>
      <li>HTML5 and CSS3</li>
      <li>JavaScript and TypeScript</li>
      <li>Java, Python, C and C++</li>
    </ul>
    <p><a href="#top">Back to top &uarr;</a></p>
  </main>
  <footer>
    <p><small>&copy; 2026 Learn Being Forward &middot; <a href="#top">Back to top &uarr;</a></small>
</p>
  </footer>
</body>
</html>
```


127.0.0.1:5500/about.html
Learn Being Forward
Home
About
Contact
About us
Founded in Mysuru, we have trained students from 75+ colleges.
Our courses
HTML5 and CSS3
JavaScript and TypeScript
Java, Python, C and C++
Back to top ↑
© 2026 Learn Being Forward · Back to top ↑

ABOUT PAGE

CONTACT.HTML HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Contact | Learn Being Forward</title>
</head>
<body>
  <header id="top">
    <h1>Learn Being Forward</h1>
    <nav aria-label="Main">
      <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="contact.html" aria-current="page">Contact</a></li>
      </ul>
    </nav>
  </header>
  <main>
    <h2>Contact us</h2>
    <address>
      Email: <a href="mailto:info@learnbeingforward.in?
subject=Course%20enquiry">info@learnbeingforward.in</a><br>
      Phone: <a href="tel:+919972934418">+91 99729 34418</a><br>
      WhatsApp: <a href="https://wa.me/919972934418" target="_blank" rel="noopener">Chat with us
(opens WhatsApp)</a>
    </address>
  </main>
  <footer>
    <p><small>&copy; 2026 Learn Being Forward &middot; <a href="#top">Back to top &uarr;</a></small>
</p>
  </footer>
</body>
</html>
```



CONTACT PAGE

MODULE 05 · BEGINNER

Images and basic media

Minimum duration **1 day** | Level **Beginner**

Images make pages engaging, and this module covers adding them correctly and accessibly: the img element with src, alt, width and height, why explicit dimensions prevent layout shift, writing good alt text and marking decorative images, the image formats JPEG, PNG, GIF, SVG, WebP and AVIF and how to choose and optimise them, figures with captions, clickable images and image maps, and favicons.

IN THIS MODULE

5.1 The img element
5.2 Image formats
5.3 Figures and captions
5.4 Image links and image maps
5.5 Favicons

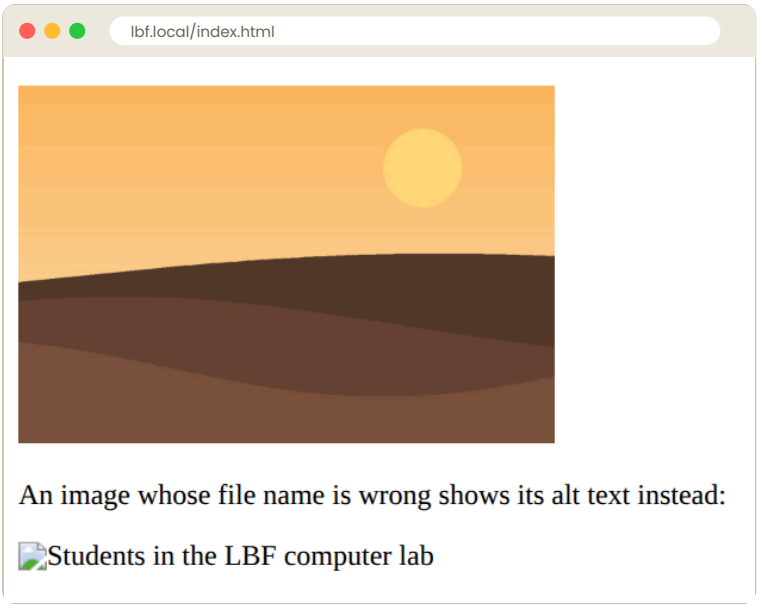
5.1 The img element

5.1.1 src and alt

`<img>` embeds an image. It is a void element with two essential attributes: `src` (source), the URL of the image file (relative or absolute, 4.1.2), and `alt` (alternative text), a text replacement for the image. The alt text is read by screen readers, shown when the image fails to load, used by search engines to understand the image, and displayed while images are blocked or switched off on slow connections. Images are inline-level replaced elements, so they sit in the line of text unless CSS changes that.

AN IMAGE, AND A BROKEN IMAGE SHOWING ITS ALT TEXT — HTML

```
<p><img src="mysuru-palace.jpg" alt="Mysuru Palace lit by the evening sun" width="300" height="200">
</p>
<p>An image whose file name is wrong shows its alt text instead:</p>
<p><img src="missing-photo.jpg" alt="Students in the LBF computer lab" width="300" height="80"></p>
```



HOW THE BROWSER DISPLAYS IT

5.1.2 width and height and why they prevent layout shift

Always set the `width` and `height` attributes to the image's **intrinsic** size in pixels (numbers only, no "px"). Images download after the HTML, and without dimensions the browser does not know how much space to reserve, so text jumps down when each image arrives: **Cumulative Layout Shift** (CLS), annoying for users (who click the wrong thing) and penalised in Google's Core Web Vitals (13.3.1). With both attributes, the browser computes the **aspect ratio** and reserves the right space immediately, even when CSS makes the image responsive (`max-width: 100%; height: auto;`). To display an image at a different size, change the CSS size, not the file's pixels, and for large photos provide smaller versions (Module 13).



EXPLICIT DIMENSIONS STOP THE PAGE FROM JUMPING WHILE IMAGES LOAD

5.1.3 Writing good alt text and decorative images (alt="")

Good alt text conveys the **purpose and content** of the image in context, briefly (usually under about 125 characters), as if describing it to someone on the phone. Do not start with "image of" or "picture of" (screen readers already announce "image"), do not repeat the surrounding text, and do not stuff keywords. The right text depends on the **role** of the image:

Image role	Good alt text	Example
informative photo	describe what matters in context	`alt="Students presenting their project to judges"`
image link or button	describe the destination or action, not the picture	`alt="Learn Being Forward home"` on a logo link
image of text	the same text	`alt="50% off all weekend batches"`

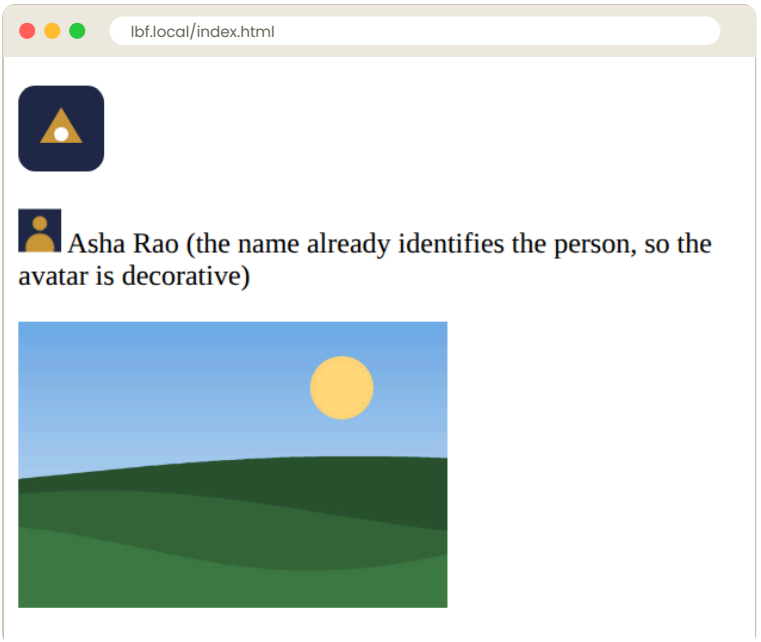
Image role	Good alt text	Example
chart or diagram	the key message; give details nearby	`alt="Placements rose from 120 in 2024 to 310 in 2026"`
decorative (adds nothing)	empty alt: `alt=""`, so screen readers skip it	dividers, background flourishes, icons next to text that says the same
complex image	short alt plus a longer description in the page or a figcaption	maps, infographics

WATCH OUT

Never omit the `alt` attribute entirely: without it, many screen readers read the file name ("IMG underscore 2045 dot JPG"). `alt=""` (empty) is the correct way to say "decorative, skip me", and it is different from a missing alt.

ALT TEXT FOR A LOGO LINK, A DECORATIVE AVATAR AND A PHOTO HTML

```
<p><a href="index.html"><img src="logo.svg" alt="Learn Being Forward home" width="48" height="48"></a>
</p>
<p><img src="avatar.png" alt="" width="24" height="24"> Asha Rao (the name already identifies the
person, so the avatar is decorative)</p>
<p><img src="hills.jpg" alt="Green Chamundi hills under a clear blue sky" width="240" height="160">
</p>
```



HOW THE BROWSER DISPLAYS IT

5.2 Image formats

5.2.1 JPEG, PNG, GIF, SVG, WebP, AVIF

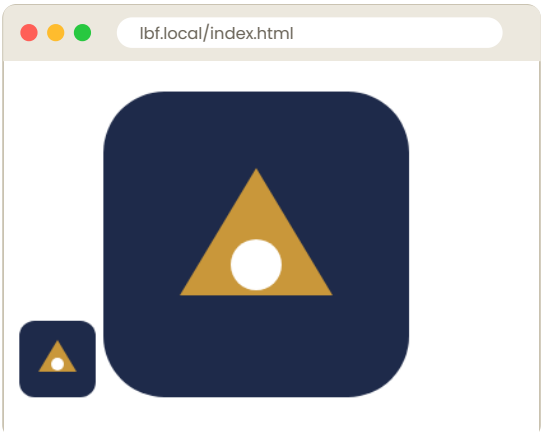
Format	Type	Transparency	Animation	Best for
JPEG (.jpg)	raster, lossy compression	no	no	photographs; universally supported

Format	Type	Transparency	Animation	Best for
PNG (.png)	raster, lossless	yes (alpha)	APNG (rare)	screenshots, graphics with text, sharp edges, transparency
GIF (.gif)	raster, 256 colours	yes (1-bit)	yes	old simple animations; prefer video or WebP today
SVG (.svg)	vector (XML text)	yes	yes (CSS/SMIL)	logos, icons, illustrations, diagrams: sharp at any size
WebP (.webp)	raster, lossy or lossless	yes	yes	modern replacement for JPEG, PNG and GIF; ~25-35% smaller
AVIF (.avif)	raster, lossy or lossless	yes	yes	best compression (~50% smaller than JPEG); supported by all modern browsers

Raster (bitmap) formats store a grid of pixels: they suit photos but become blurry or pixelated when enlarged. **Vector** formats (SVG) store shapes as mathematical descriptions: they stay perfectly sharp at any size, are usually tiny for simple graphics, and can be styled with CSS. **Lossy** compression discards detail the eye barely notices to make files much smaller; **lossless** keeps every pixel exactly.

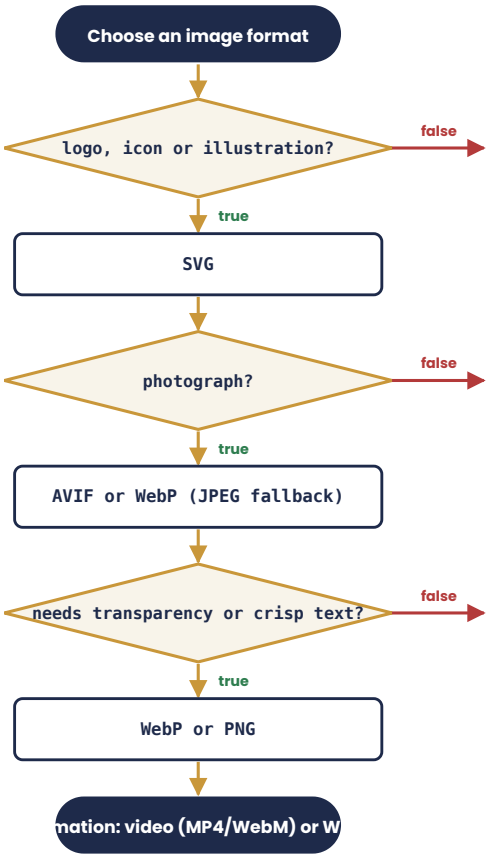
ONE SVG FILE AT TWO SIZES: VECTOR GRAPHICS STAY SHARP HTML

```html
<p>
  <img src="logo.svg" alt="SVG logo shown small" width="40" height="40">
  <img src="logo.svg" alt="The same SVG logo shown large and still sharp" width="160" height="160">
</p>
```



HOW THE BROWSER DISPLAYS IT

5.2.2 Choosing the right format



A QUICK DECISION GUIDE FOR IMAGE FORMATS

In practice: use **SVG** for logos and icons; **AVIF or WebP** for photos, with a JPEG fallback for very old browsers using `<picture>` (13.1.4); **PNG** or lossless WebP for screenshots and graphics that need transparency; and short muted **videos** instead of large animated GIFs, which are often ten times bigger.

5.2.3 Optimising image size

Images are usually the heaviest part of a page. Before publishing: **resize** to the largest size the image is displayed at (a 4000-pixel camera photo shown 800 pixels wide wastes more than 90% of its bytes; allow 2x for high-density screens); **compress** with a sensible quality (JPEG/WebP quality 70-85 is usually indistinguishable); **convert** to a modern format; and **strip metadata** (EXIF data can include the GPS location where the photo was taken). Free tools include **Squoosh** (squoosh.app, in the browser), TinyPNG, ImageOptim, and SVGOMG for SVG files. Aim for hero images under about 200 KB and thumbnails under 50 KB.

Version of the same photo	Dimensions	Approx. size
original from a phone camera (JPEG)	4032 × 3024	3.8 MB
resized JPEG, quality 80	1200 × 900	210 KB
resized WebP, quality 75	1200 × 900	140 KB
resized AVIF, quality 60	1200 × 900	85 KB
thumbnail WebP	400 × 300	22 KB

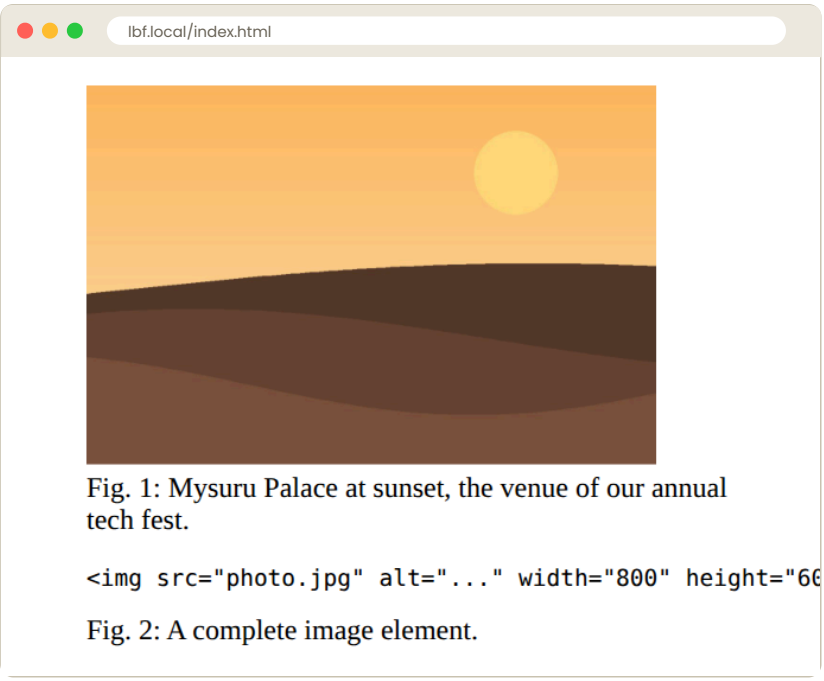
5.3 Figures and captions

5.3.1 <figure> and <figcaption>

`<figure>` marks self-contained content, typically an image, diagram, chart, code listing or quotation, that is referenced from the main text but could be moved elsewhere (to an appendix or a sidebar) without breaking the flow. `<figcaption>`, as the first or last child, gives it a visible **caption**, which is associated with the figure for assistive technology. The caption and the alt text have different jobs: the alt replaces the image for those who cannot see it; the caption is extra information for everyone. Browsers indent figures by default.

FIGURES WITH CAPTIONS: AN IMAGE AND A CODE LISTING HTML

```
<figure>
  <img src="mysuru-palace.jpg" alt="Mysuru Palace with its domes glowing in orange evening light"
width="320" height="213">
  <figcaption>Fig. 1: Mysuru Palace at sunset, the venue of our annual tech fest.</figcaption>
</figure>
<figure>
  <pre><code>&lt;img src="photo.jpg" alt="..." width="800" height="600"&gt;</code></pre>
  <figcaption>Fig. 2: A complete image element.</figcaption>
</figure>
```



HOW THE BROWSER DISPLAYS IT

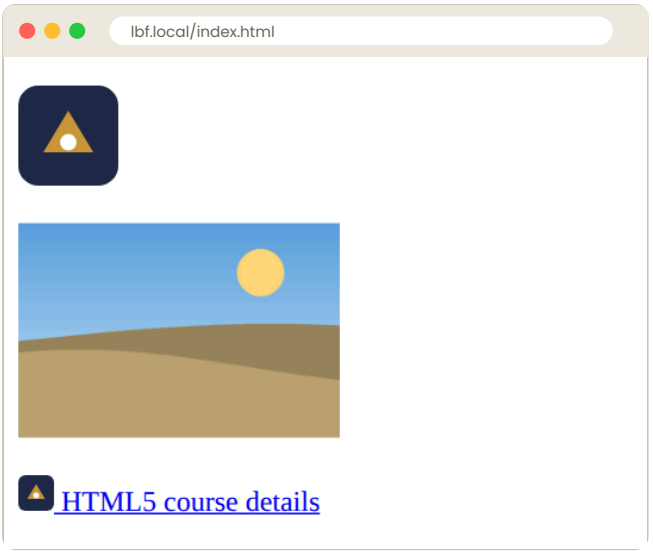
5.4 Image links and image maps

5.4.1 Clickable images

Wrap an image in `<a>` to make it a link. The image's alt text then becomes the **link text**, so it must describe the destination ("Learn Being Forward home", "View the full-size photo"), not the picture. If the link also contains visible text, give the image `alt=""` to avoid repeating it.

IMAGES USED AS LINKS HTML

```
<p><a href="index.html"><img src="logo.svg" alt="Learn Being Forward home" width="56" height="56"></a>
</p>
<p><a href="gallery/beach-large.jpg"><img src="beach.jpg" alt="View the full-size beach photo"
width="180" height="120"></a></p>
<p><a href="courses/html5.html"><img src="logo.svg" alt="" width="20" height="20"> HTML5 course
details</a></p>
```



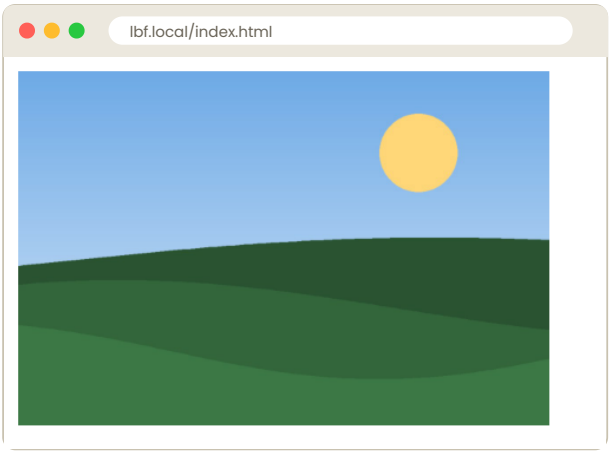
HOW THE BROWSER DISPLAYS IT

5.4.2 <map> and <area> (overview)

An **image map** defines several clickable regions on one image. The `<img>` refers to a `<map name="...">` with `usemap="#name"`, and each `<area>` inside the map gives a `shape` (`rect`, `circle` or `poly`), pixel `coords`, an `href` and an `alt`. Image maps are rarely used today because the coordinates do not scale with responsive images; for clickable diagrams, inline SVG with links (10.4.1) or positioned HTML elements work better.

AN IMAGE MAP WITH THREE CLICKABLE AREAS (THE AREAS ARE INVISIBLE) HTML

```
<img src="hills.jpg" alt="Campus map" width="300" height="200" usemap="#campus">
<map name="campus">
  <area shape="rect" coords="0,0,150,100" href="#library" alt="Library">
  <area shape="circle" coords="225,60,40" href="#lab" alt="Computer lab">
  <area shape="poly" coords="0,200,150,120,300,200" href="#ground" alt="Sports ground">
</map>
```



HOW THE BROWSER DISPLAYS IT

5.5 Favicons

5.5.1 <link rel="icon">

A **favicon** is the small icon shown in browser tabs, bookmarks, history and search results. Declare it in the `<head>` with `<link rel="icon">`. A modern minimal set is an SVG icon (sharp at every size, and it can even adapt to dark mode), a 32×32 PNG or `favicon.ico` fallback, and a 180×180 `apple-touch-icon` for iPhone home screens; installable web apps also list larger icons in a web app manifest (11.2.3). If no icon is declared, browsers request `/favicon.ico` from the site root automatically, so keeping one there avoids 404 errors in the server logs.

DECLARING FAVICONS IN THE HEAD HTML

```html
<head>
  <meta charset="UTF-8">
  <title>Learn Being Forward</title>
  <link rel="icon" href="/favicon.ico" sizes="32x32">
  <link rel="icon" href="/icon.svg" type="image/svg+xml">
  <link rel="apple-touch-icon" href="/apple-touch-icon.png">
</head>
```



FAVICONS IDENTIFY YOUR SITE AMONG OPEN TABS

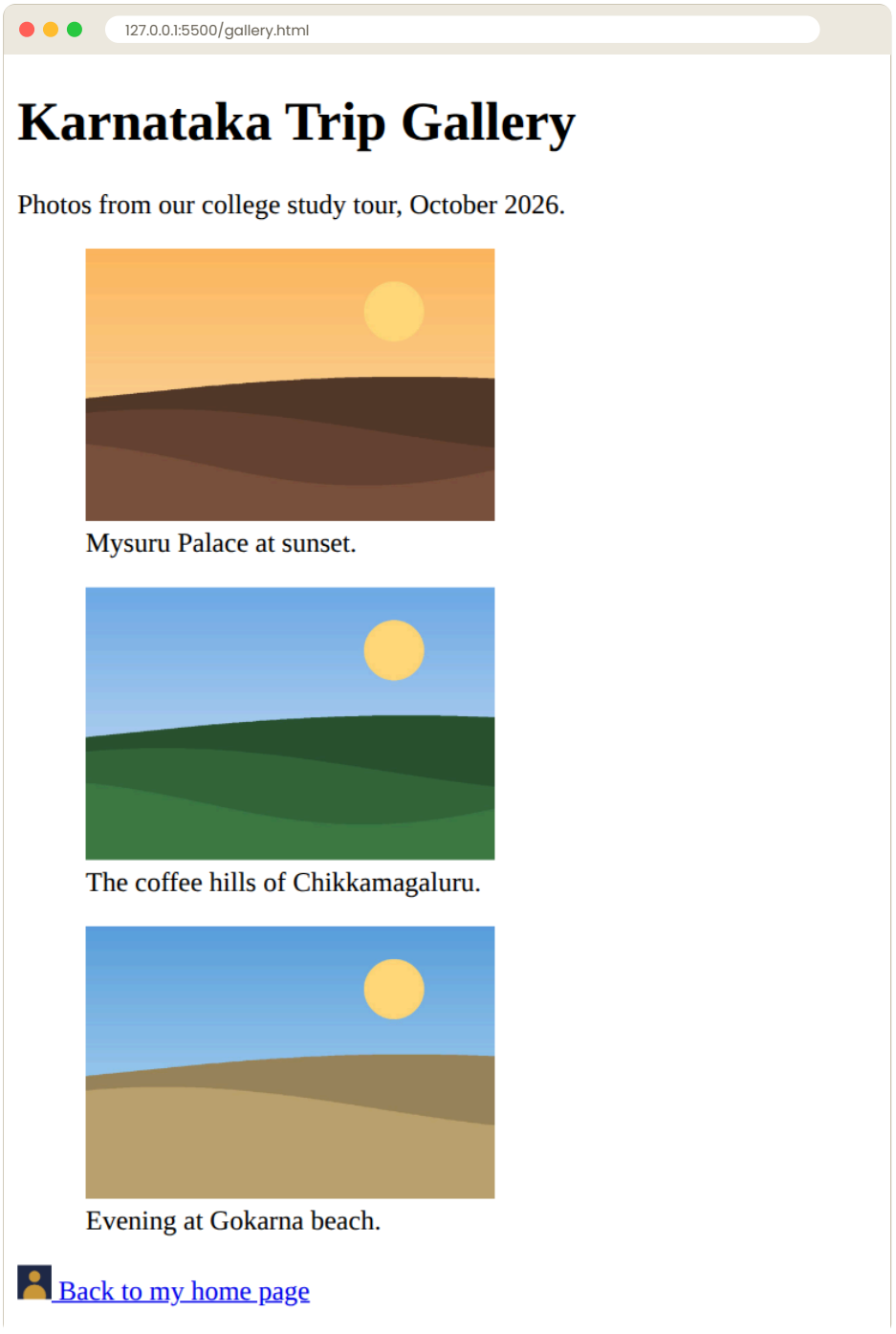
img embeds an image with src (its URL) and alt (a text replacement read by screen readers and shown on failure). Always set width and height to the intrinsic size so the browser reserves space and avoids layout shift. Alt text describes the image's purpose in context; links describe their destination; decorative images use alt="" (never omit alt). Formats: JPEG for photos, PNG for lossless graphics with transparency, GIF only for legacy animations, SVG for logos and icons (vector, sharp at any size), and WebP and AVIF as smaller modern formats. Resize, compress and convert images before publishing. figure with figcaption groups an image or listing with a visible caption. Images inside a are links whose alt is the link text; image maps (map, area) define clickable regions but are rarely used. Favicons are declared with link rel="icon" (SVG, ICO/PNG) and apple-touch-icon.

PRACTICE CHECKPOINT

A **photo gallery page** with **captioned figures**, correct **alt text**, **explicit dimensions** and a **favicon**.

GALLERY.HTML: SAMPLE SOLUTION HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Karnataka Trip Gallery | Asha Rao</title>
  <link rel="icon" href="logo.svg" type="image/svg+xml">
</head>
<body>
  <h1>Karnataka Trip Gallery</h1>
  <p>Photos from our college study tour, October 2026.</p>
  <figure>
    <img src="mysuru-palace.jpg" alt="Mysuru Palace glowing orange at sunset" width="240"
height="160">
    <figcaption>Mysuru Palace at sunset.</figcaption>
  </figure>
  <figure>
    <img src="hills.jpg" alt="Rolling green hills under a blue sky near Chikkamagaluru" width="240"
height="160">
    <figcaption>The coffee hills of Chikkamagaluru.</figcaption>
  </figure>
  <figure>
    <img src="beach.jpg" alt="Golden beach and calm sea at Gokarna" width="240" height="160">
    <figcaption>Evening at Gokarna beach.</figcaption>
  </figure>
  <p><a href="index.html"><img src="avatar.png" alt="" width="20" height="20"> Back to my home
page</a></p>
</body>
</html>
```



THE GALLERY PAGE (FIGURES STACK VERTICALLY UNTIL CSS LAYS THEM OUT IN A GRID)

MODULE 06 · BEGINNER

Lists and tables

Minimum duration **2 days** | Level **Beginner**

Lists group related items and tables present tabular data; both are essential for structured content. This module covers unordered, ordered (with type, start and reversed) and description lists, nested lists and lists for navigation, then tables: rows, cells and header cells, captions, thead, tbody and tfoot, column groups, merging cells with colspan and rowspan, accessible tables with scope and with headers and id, and the rule that tables are for data, never for page layout.

IN THIS MODULE

6.1 Lists
6.2 Table basics
6.3 Table structure
6.4 Merging cells
6.5 Accessible tables
6.6 When to use tables

6.1 Lists

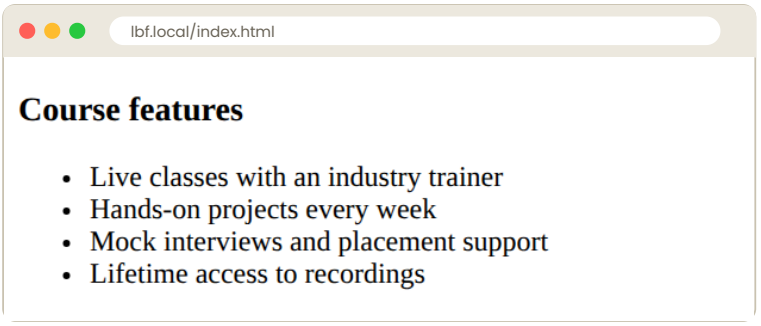
Lists are everywhere on the web: navigation menus, feature lists, steps in a tutorial, search results, product grids, comment threads. Marking such content as a list tells browsers and screen readers how many items there are and that they belong together ("list, 5 items").

6.1.1 Unordered lists

`<ul>` is an **unordered list**, for items whose order does not matter; each item is an `<li>` (list item). Browsers show bullets by default; the bullet style is changed with the CSS property `list-style-type` (disc, circle, square, none) rather than the obsolete `type` attribute. A `<ul>` may contain only `<li>` elements (plus script-supporting elements); any other content goes inside the list items.

AN UNORDERED LIST — HTML

```
<h3>Course features</h3>
<ul>
  <li>Live classes with an industry trainer</li>
  <li>Hands-on projects every week</li>
  <li>Mock interviews and placement support</li>
  <li>Lifetime access to recordings</li>
</ul>
```



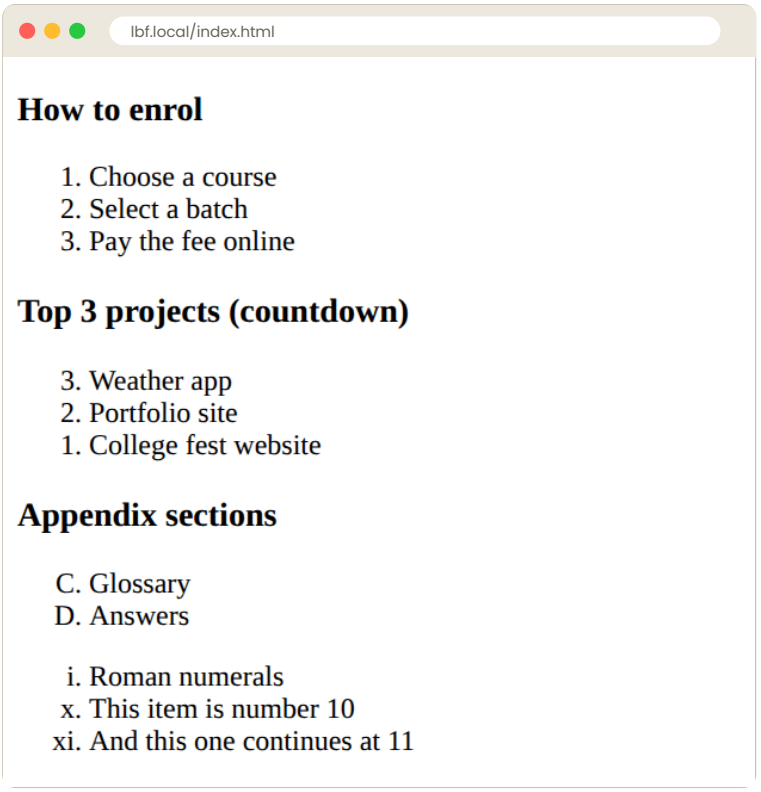
HOW THE BROWSER DISPLAYS IT

6.1.2 Ordered lists <ol> with type, start and reversed

`<ol>` is an **ordered list**, for items whose sequence matters: steps, rankings, instructions. Items are numbered automatically, so you can insert or remove steps without renumbering. Attributes: `type` chooses the numbering (`1` default, `a`, `A`, `i`, `I`); `start` sets the first number (always given as an integer, even with letters); `reversed` counts down; and `value` on an individual `<li>` sets its number.

ORDERED LISTS WITH TYPE, START, REVERSED AND VALUE HTML

```
<h3>How to enrol</h3>
<ol>
  <li>Choose a course</li>
  <li>Select a batch</li>
  <li>Pay the fee online</li>
</ol>
<h3>Top 3 projects (countdown)</h3>
<ol reversed>
  <li>Weather app</li>
  <li>Portfolio site</li>
  <li>College fest website</li>
</ol>
<h3>Appendix sections</h3>
<ol type="A" start="3">
  <li>Glossary</li>
  <li>Answers</li>
</ol>
<ol type="i">
  <li>Roman numerals</li>
  <li value="10">This item is number 10</li>
  <li>And this one continues at 11</li>
</ol>
```



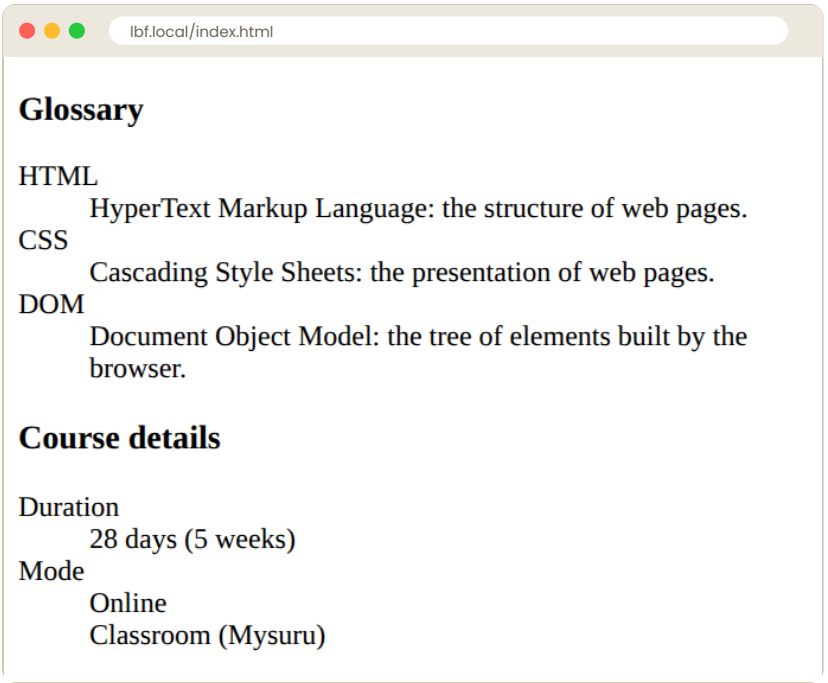
HOW THE BROWSER DISPLAYS IT

6.1.3 Description lists <dl>, <dt>, <dd>

`<dl>` is a **description list** of name-value groups: each `<dt>` (term) is followed by one or more `<dd>` (description) elements. Use it for glossaries, FAQs, metadata ("Duration: 28 days"), specifications and contact details. Several terms may share one description and one term may have several descriptions. Groups may be wrapped in a `<div>` for styling.

DESCRIPTION LISTS: A GLOSSARY AND KEY-VALUE DETAILS HTML

```
<h3>Glossary</h3>
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language: the structure of web pages.</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets: the presentation of web pages.</dd>
  <dt>DOM</dt>
  <dd>Document Object Model: the tree of elements built by the browser.</dd>
</dl>
<h3>Course details</h3>
<dl>
  <dt>Duration</dt><dd>28 days (5 weeks)</dd>
  <dt>Mode</dt><dd>Online</dd><dd>Classroom (Mysuru)</dd>
</dl>
```



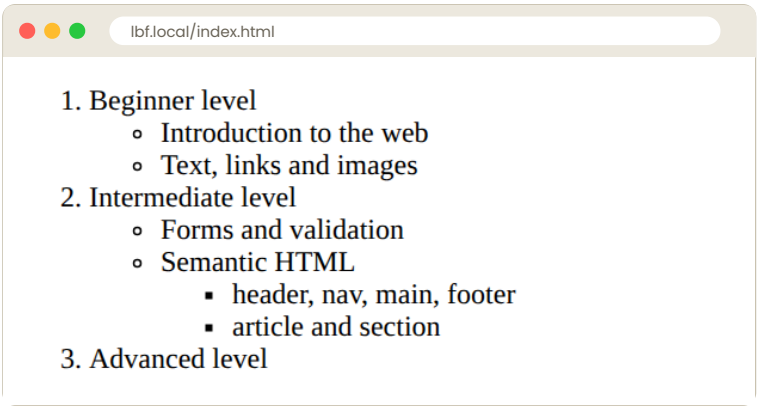
HOW THE BROWSER DISPLAYS IT

6.1.4 Nested lists

A list item can contain another complete list, creating a **nested** (multi-level) list: course syllabi, outlines, sub-menus, folder trees. The inner `<ul>` or `<ol>` goes **inside** the `<li>`, after its text, never directly inside the outer list. Browsers change the bullet style and indent each level. Ordered and unordered lists can be mixed.

A NESTED LIST: THE INNER LISTS SIT INSIDE LIST ITEMS HTML

```html
<ol>
  <li>Beginner level
    <ul>
      <li>Introduction to the web</li>
      <li>Text, links and images</li>
    </ul>
  </li>
  <li>Intermediate level
    <ul>
      <li>Forms and validation</li>
      <li>Semantic HTML
        <ul>
          <li>header, nav, main, footer</li>
          <li>article and section</li>
        </ul>
      </li>
    </ul>
  </li>
  <li>Advanced level</li>
</ol>
```



HOW THE BROWSER DISPLAYS IT

6.1.5 Lists for navigation menus

As shown in 4.4.1, menus are lists of links inside `<nav>`, and drop-down menus are nested lists. CSS removes the bullets and arranges the items horizontally; the list semantics remain, which helps screen-reader users. Other "lists in disguise" include breadcrumb trails (an ordered list, because the order is a path), tag lists, card grids of products and image galleries.

A BREADCRUMB (ORDERED LIST) AND A MENU WITH A SUB-MENU (NESTED LIST) HTML

```
<nav aria-label="Breadcrumb">
  <ol>
    <li><a href="/">Home</a></li>
    <li><a href="/courses/">Courses</a></li>
    <li><a href="/courses/html5.html" aria-current="page">HTML5</a></li>
  </ol>
</nav>
<nav aria-label="Main">
  <ul>
    <li><a href="/">Home</a></li>
    <li><a href="/courses/">Courses</a>
      <ul>
        <li><a href="/courses/html5.html">HTML5</a></li>
        <li><a href="/courses/css3.html">CSS3</a></li>
      </ul>
    </li>
    <li><a href="/contact.html">Contact</a></li>
  </ul>
</nav>
```



HOW THE BROWSER DISPLAYS IT

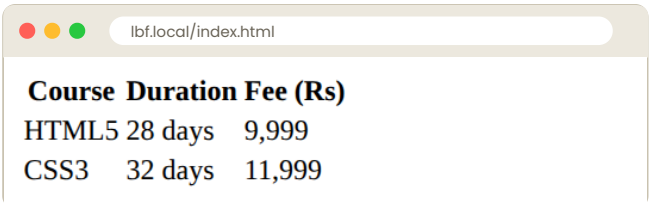
6.2 Table basics

6.2.1 <table>, <tr>, <td>, <th>

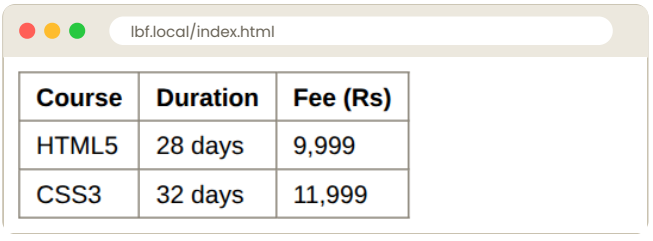
A table is built **row by row**. `<table>` contains `<tr>` (table row) elements, and each row contains cells: `<td>` (table data) for ordinary cells and `<th>` (table header) for header cells, which browsers show bold and centred and screen readers announce as headings for the cells they label. Every row should have the same number of columns (counting merged cells, 6.4). Without CSS, tables have no borders; the CSS course adds borders, spacing and zebra stripes (a few lines of CSS are used in some examples below to make the cells visible).

A BASIC TABLE: ROWS OF HEADER AND DATA CELLS (NO CSS) HTML

```
<table>
  <tr>
    <th>Course</th>
    <th>Duration</th>
    <th>Fee (Rs)</th>
  </tr>
  <tr>
    <td>HTML5</td>
    <td>28 days</td>
    <td>9,999</td>
  </tr>
  <tr>
    <td>CSS3</td>
    <td>32 days</td>
    <td>11,999</td>
  </tr>
</table>
```



WITHOUT CSS A TABLE HAS NO BORDERS



THE SAME TABLE WITH BORDERS ADDED BY CSS

6.2.2 <caption>

`<caption>`, which must be the **first child** of the table, gives the table a visible title that describes its contents. Screen readers announce it when users reach the table, helping them decide whether to read it. It is displayed above the table by default (CSS `caption-side` can move it below).

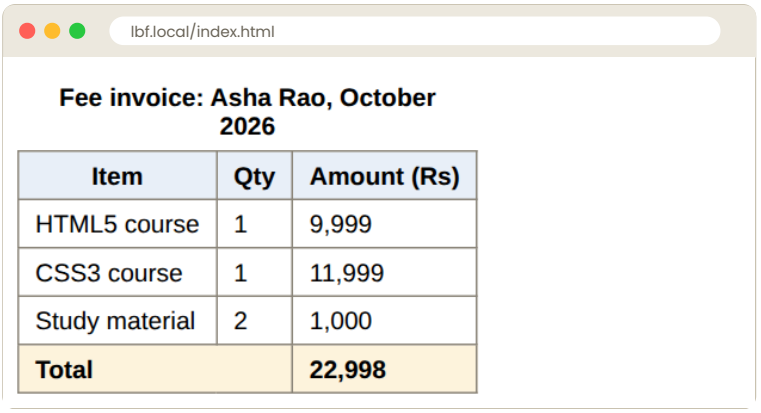
6.3 Table structure

6.3.1 <thead>, <tbody>, <tfoot>

Rows can be grouped into three sections: `<thead>` for header rows, `<tbody>` for the main data (a table can have several tbody groups), and `<tfoot>` for summary rows such as totals. The grouping makes the structure explicit for assistive technology, lets CSS style each part, allows long tables to scroll their body while keeping headers visible, and makes browsers repeat the header and footer on every printed page. If you omit `<tbody>`, the browser adds it automatically in the DOM.

A TABLE WITH CAPTION, THEAD, TBODY AND TFOOT HTML

```
<table>
  <caption>Fee invoice: Asha Rao, October 2026</caption>
  <thead>
    <tr><th scope="col">Item</th><th scope="col">Qty</th><th scope="col">Amount (Rs)</th></tr>
  </thead>
  <tbody>
    <tr><td>HTML5 course</td><td>1</td><td>9,999</td></tr>
    <tr><td>CSS3 course</td><td>1</td><td>11,999</td></tr>
    <tr><td>Study material</td><td>2</td><td>1,000</td></tr>
  </tbody>
  <tfoot>
    <tr><td colspan="2">Total</td><td>22,998</td></tr>
  </tfoot>
</table>
```



HOW THE BROWSER DISPLAYS IT

6.3.2 <colgroup> and <col>

`<colgroup>` (placed after the caption, before thead) contains `<col>` elements that represent **columns**, so you can style whole columns at once; `span` makes one col cover several columns. Only a few CSS properties apply to columns (background, width, border, visibility), but it is handy for highlighting a column such as "Today" in a timetable.

COLGROUP AND COL HIGHLIGHT A WHOLE COLUMN HTML

```html
<table>
  <caption>Weekly class hours</caption>
  <colgroup>
    <col>
    <col span="5">
    <col style="background:#FDF3DC">
  </colgroup>
  <tr><th>Batch</th><th>Mon</th><th>Tue</th><th>Wed</th><th>Thu</th><th>Fri</th><th>Sat</th></tr>
  <tr><td>Morning</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>3</td></tr>
  <tr><td>Evening</td><td>2</td><td>2</td><td>-</td><td>2</td><td>2</td><td>3</td></tr>
</table>
```

lbf.local/index.html

Weekly class hours

Batch	Mon	Tue	Wed	Thu	Fri	Sat
Morning	2	2	2	2	2	3
Evening	2	2	-	2	2	3

HOW THE BROWSER DISPLAYS IT

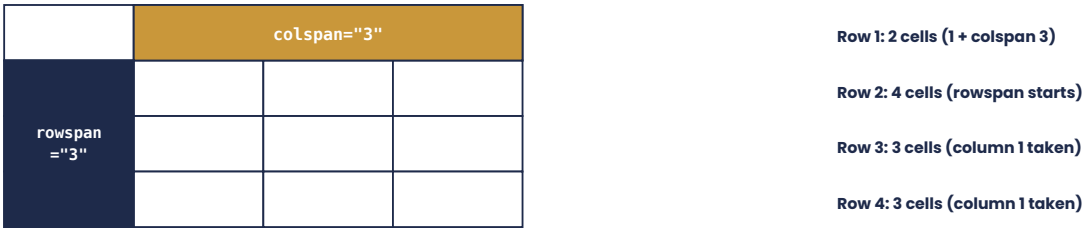
6.4 Merging cells

6.4.1 colspan

`colspan="n"` makes a cell **span n columns** horizontally. The row then contains fewer cell elements: each merged cell counts as n. Typical uses: a heading over several columns, a total label across columns, a "Lunch break" across a timetable row.

6.4.2 rowspan

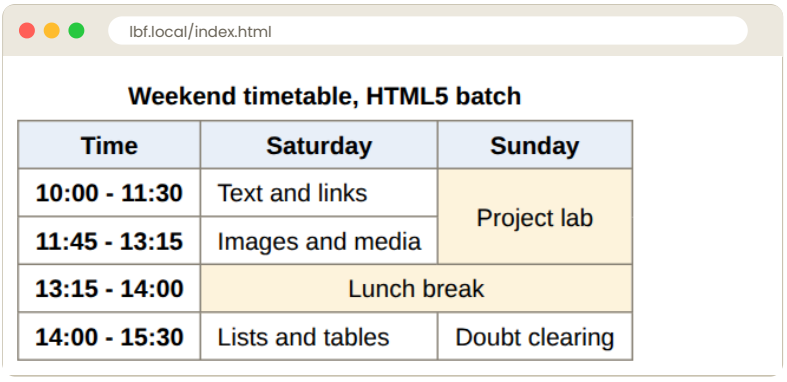
`rowspan="n"` makes a cell **span n rows** vertically. The rows below then **omit** a cell in that column, because it is already occupied. Plan merged tables on paper (or in a spreadsheet) first: draw the grid, mark the merged areas, then write each row skipping occupied cells.



MERGED CELLS REDUCE THE NUMBER OF CELL ELEMENTS IN THE AFFECTED ROWS

A TIMETABLE WITH ROWSPAN AND COLSPAN HTML

```
<table>
  <caption>Weekend timetable, HTML5 batch</caption>
  <thead>
    <tr><th scope="col">Time</th><th scope="col">Saturday</th><th scope="col">Sunday</th></tr>
  </thead>
  <tbody>
    <tr><th scope="row">10:00 - 11:30</th><td>Text and links</td><td rowspan="2">Project lab</td></tr>
    <tr><th scope="row">11:45 - 13:15</th><td>Images and media</td></tr>
    <tr><th scope="row">13:15 - 14:00</th><td colspan="2">Lunch break</td></tr>
    <tr><th scope="row">14:00 - 15:30</th><td>Lists and tables</td><td>Doubt clearing</td></tr>
  </tbody>
</table>
```



HOW THE BROWSER DISPLAYS IT

6.5 Accessible tables

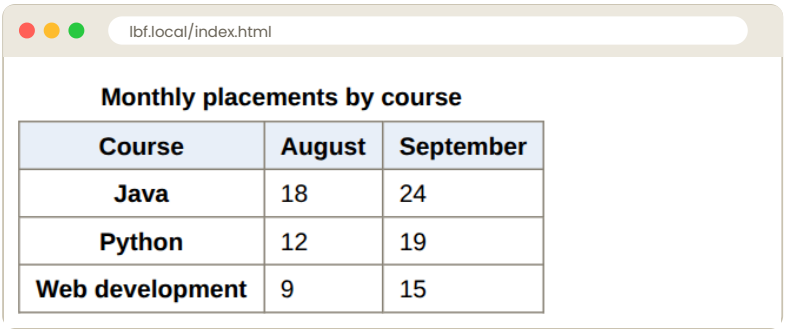
Sighted users understand a table by glancing at the headers above and to the left of a cell. Screen-reader users hear one cell at a time, so the table must tell the screen reader which headers belong to each cell. With proper markup, moving to the cell "9,999" announces "HTML5, Fee, 9,999".

6.5.1 scope attribute

The `scope` attribute on a `<th>` says which cells it labels: `scope="col"` for a column header, `scope="row"` for a row header, and `colgroup` or `rowgroup` for headers spanning groups. For simple tables browsers can often guess, but adding scope makes the relationships explicit and reliable. Use `<th>` (not bold `<td>`) for every header cell, including row headers in the first column.

COLUMN AND ROW HEADERS MARKED WITH SCOPE HTML

```
<table>
  <caption>Monthly placements by course</caption>
  <thead>
    <tr><th scope="col">Course</th><th scope="col">August</th><th scope="col">September</th></tr>
  </thead>
  <tbody>
    <tr><th scope="row">Java</th><td>18</td><td>24</td></tr>
    <tr><th scope="row">Python</th><td>12</td><td>19</td></tr>
    <tr><th scope="row">Web development</th><td>9</td><td>15</td></tr>
  </tbody>
</table>
```



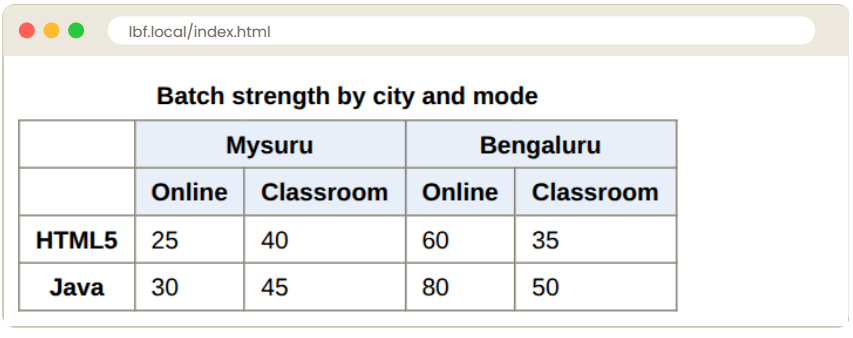
HOW THE BROWSER DISPLAYS IT

6.5.2 headers and id for complex tables

When a table has **several levels of headers** (merged header cells, sub-headings inside the body), scope is not enough. Give each header cell an `id`, and give each data cell a `headers` attribute listing the ids of all headers that apply, separated by spaces. Because this is laborious and error-prone, consider whether the data can be split into several simpler tables instead.

A TWO-LEVEL HEADER TABLE USING ID AND HEADERS HTML

```
<table>
  <caption>Batch strength by city and mode</caption>
  <thead>
    <tr>
      <td></td>
      <th id="mys" colspan="2">Mysuru</th>
      <th id="blr" colspan="2">Bengaluru</th>
    </tr>
    <tr>
      <td></td>
      <th id="mys-on" headers="mys">Online</th><th id="mys-off" headers="mys">Classroom</th>
      <th id="blr-on" headers="blr">Online</th><th id="blr-off" headers="blr">Classroom</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th id="html">HTML5</th>
      <td headers="mys mys-on html">25</td><td headers="mys mys-off html">40</td>
      <td headers="blr blr-on html">60</td><td headers="blr blr-off html">35</td>
    </tr>
    <tr>
      <th id="java">Java</th>
      <td headers="mys mys-on java">30</td><td headers="mys mys-off java">45</td>
      <td headers="blr blr-on java">80</td><td headers="blr blr-off java">50</td>
    </tr>
  </tbody>
</table>
```



HOW THE BROWSER DISPLAYS IT

6.6 When to use tables

6.6.1 Tables for data, never for page layout

In the 1990s, before CSS layout existed, whole websites were laid out with nested invisible tables. This is now considered a serious mistake: screen readers announce rows and columns that mean nothing, the source order is confusing, the layout cannot adapt to small screens, and the markup is huge and hard to maintain. Use tables **only for tabular data**: information that is naturally organised in rows and columns, where comparing across rows and down columns makes sense (timetables, price comparisons, results, invoices, statistics). For page layout, cards and grids, use semantic elements with CSS Flexbox and Grid. (HTML emails remain the exception, 1.6.2.)

Use a table for	Do not use a table for
exam results, mark sheets, timetables	placing a sidebar next to the main content
price and feature comparisons	a grid of product or course cards
invoices, bank statements, order summaries	aligning form labels and inputs
statistics and data reports	creating columns of text
sports scores and fixtures	centring things or adding spacing

ul is an unordered list and ol an ordered list (type, start, reversed, value); both contain only li items. dl holds term-description groups (dt, dd) for glossaries and details. Nested lists go inside an li; menus and breadcrumbs are lists of links in nav. Tables are built from tr rows of td data cells and th header cells; caption (first child) titles the table; thead, tbody and tfoot group header, body and summary rows; colgroup and col target columns. colspan and rowspan merge cells; rows then contain fewer cell elements. For accessibility use th with scope="col" or "row", and id with headers for multi-level headers. Tables are for tabular data only, never for page layout.

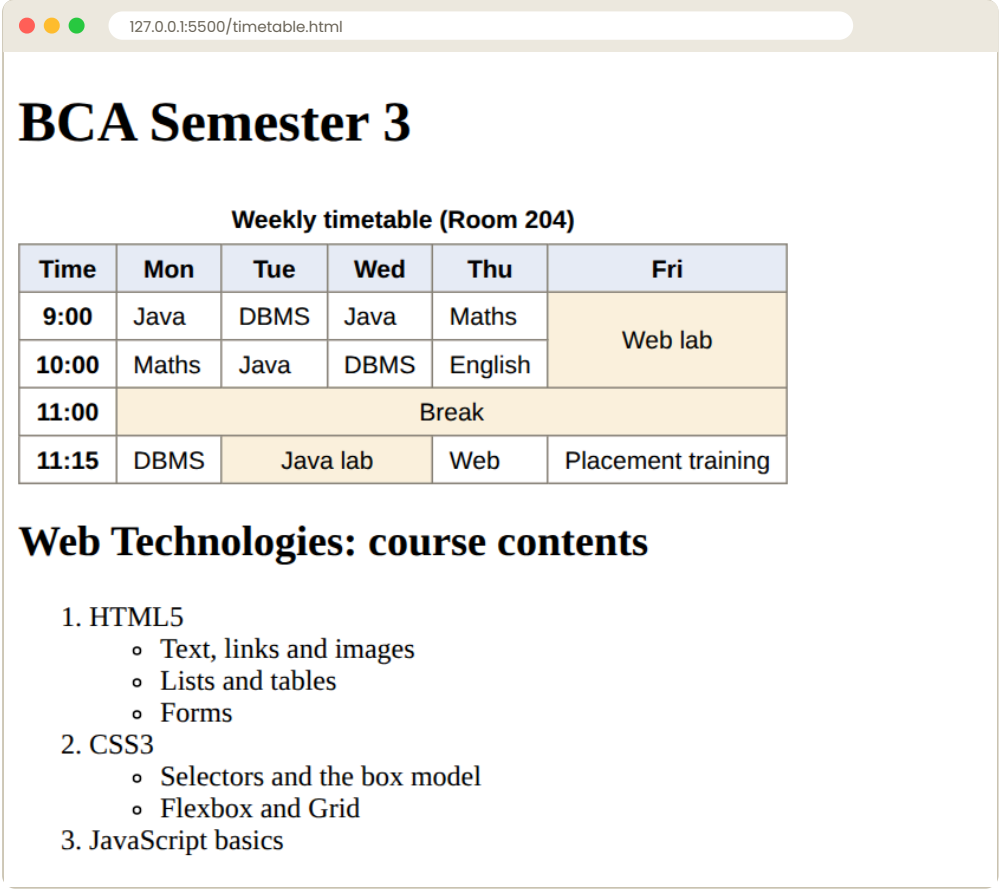
PRACTICE CHECKPOINT

A **college timetable** using a table with **merged cells**, **header scopes** and a **caption**, plus a **nested course-contents list**.

TIMETABLE.HTML: SAMPLE SOLUTION HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>BCA Semester 3 Timetable | Learn Being Forward</title>
</head>
<body>
  <h1>BCA Semester 3</h1>
  <table>
    <caption>Weekly timetable (Room 204)</caption>
    <thead>
      <tr>
        <th scope="col">Time</th><th scope="col">Mon</th><th scope="col">Tue</th>
        <th scope="col">Wed</th><th scope="col">Thu</th><th scope="col">Fri</th>
      </tr>
```

```
    </thead>
    <tbody>
      <tr><th scope="row">9:00</th><td>Java</td><td>DBMS</td><td>Java</td><td>Maths</td><td
rowspan="2">Web lab</td></tr>
      <tr><th scope="row">10:00</th><td>Maths</td><td>Java</td><td>DBMS</td><td>English</td></tr>
      <tr><th scope="row">11:00</th><td colspan="5">Break</td></tr>
      <tr><th scope="row">11:15</th><td>DBMS</td><td colspan="2">Java lab</td><td>Web</td>
<td>Placement training</td></tr>
    </tbody>
  </table>
  <h2>Web Technologies: course contents</h2>
  <ol>
    <li>HTML5
      <ul>
        <li>Text, links and images</li>
        <li>Lists and tables</li>
        <li>Forms</li>
      </ul>
    </li>
    <li>CSS3
      <ul>
        <li>Selectors and the box model</li>
        <li>Flexbox and Grid</li>
      </ul>
    </li>
    <li>JavaScript basics</li>
  </ol>
</body>
</html>
```



THE TIMETABLE PAGE (WITH A LITTLE CSS FOR BORDERS)

MODULE 07 · BEGINNER

Forms basics

Minimum duration **2 days** | Level **Beginner**

Forms collect user input and are the main way users interact with websites: login, sign-up, search, enquiry, checkout and feedback. This module covers the form element with action and method (GET versus POST) and how form data is sent as name-value pairs, labels and inputs with the name, value and placeholder attributes, the common input types (text, password, email, number, tel, url, checkbox, radio, file, hidden, submit, reset and button), textarea, select with option and optgroup, button versus input type="submit", and grouping controls with fieldset and legend.

IN THIS MODULE

7.1 The form element
7.2 Labels and inputs
7.3 Common input types
7.4 Other form controls
7.5 Grouping

7.1 The form element

7.1.1 <form>, action and method (GET vs POST)

`<form>` wraps a set of controls that are submitted together. Its `action` attribute is the URL that receives the data (a server-side script or API endpoint written in PHP, Node.js, Java, Python…); if omitted, the form submits to the current page. Its `method` attribute chooses how the data travels: **GET** (the default) appends the data to the URL as a query string, while **POST** sends it in the body of the HTTP request.

	GET	POST
Data travels in	the URL: `/search?q=html&level=beginner`	the request body (not visible in the URL)
Visible, bookmarkable, shareable	yes	no
Size limit	URLs are limited (a few KB in practice)	large data, including file uploads
Stored in history and server logs	yes	no
Use for	searches, filters, anything that only **reads** data	logins, sign-ups, payments, anything that **changes** data or is sensitive

WATCH OUT

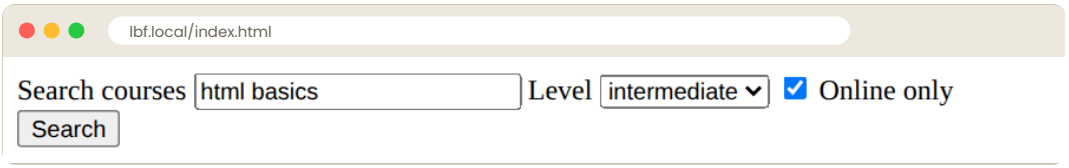
Never send passwords or personal data with GET: they would appear in the address bar, browser history, server logs and shared links. And remember that POST is **not** encryption: only HTTPS protects data in transit.

7.1.2 How form data is sent (name-value pairs)

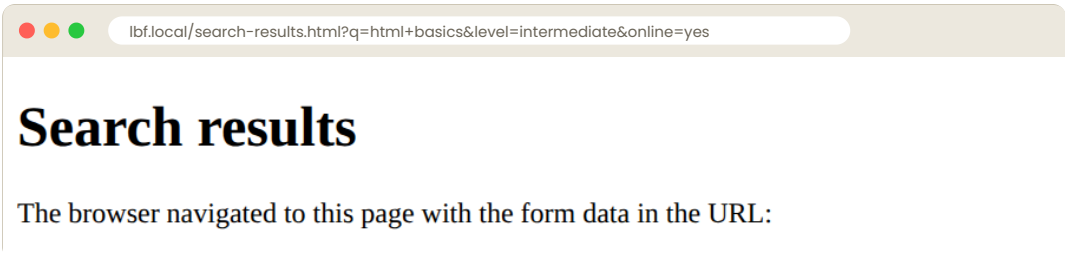
When a form is submitted, the browser collects every **successful control** (an enabled control with a `name`; checked checkboxes and radios only) and builds a list of **name=value pairs**, joining them with `&` and encoding special characters (spaces become `+` or `%20`, `@` becomes `%40`). A control without a `name` attribute is **not sent at all**, a common beginner bug. The example below submits a GET form; the address bar shows the resulting query string.

A GET FORM HTML

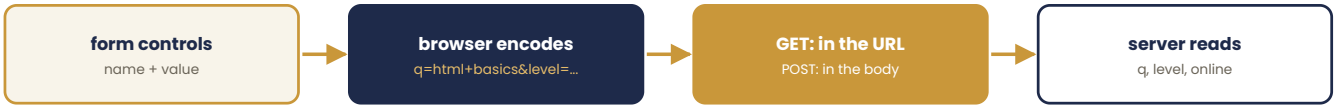
```html
<form action="search-results.html" method="get">
  <label>Search courses <input type="text" name="q" value="html basics"></label>
  <label>Level
    <select name="level">
      <option>beginner</option>
      <option selected>intermediate</option>
    </select>
  </label>
  <label><input type="checkbox" name="online" value="yes" checked> Online only</label>
  <button>Search</button>
</form>
```



BEFORE SUBMITTING



AFTER CLICKING SEARCH: THE DATA ARRIVES AS NAME=VALUE PAIRS IN THE QUERY STRING



FROM FORM CONTROLS TO NAME-VALUE PAIRS ON THE SERVER

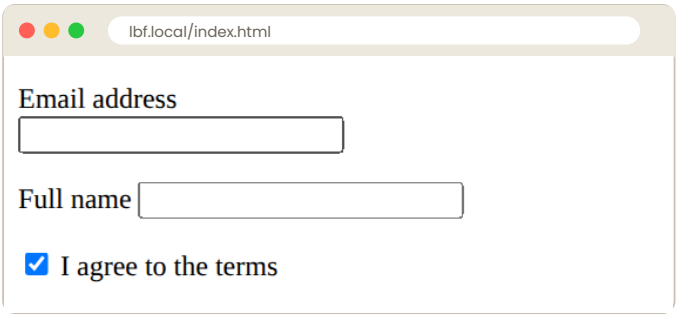
7.2 Labels and inputs

7.2.1 <label> with for and wrapping labels

Every form control needs a **label**: visible text that says what to enter. The `<label>` element connects that text to the control in one of two ways: **explicitly**, with `for` set to the control's `id`, or **implicitly**, by wrapping the control inside the label. The connection does three things: screen readers read the label when the control receives focus; clicking the label focuses the field (or toggles a checkbox), giving a bigger target, especially on phones; and voice-control users can say "click Email". A **placeholder is not a label**: it disappears when typing starts, often has low contrast, and is not reliably read by assistive technology.

EXPLICIT LABELS (FOR + ID) AND AN IMPLICIT (WRAPPING) LABEL HTML

```html
<form>
  <p>
    <label for="email">Email address</label><br>
    <input type="email" id="email" name="email">
  </p>
  <p>
    <label>Full name <input type="text" name="fullname"></label>
  </p>
  <p>
    <input type="checkbox" id="terms" name="terms">
    <label for="terms">I agree to the terms</label>
  </p>
</form>
```



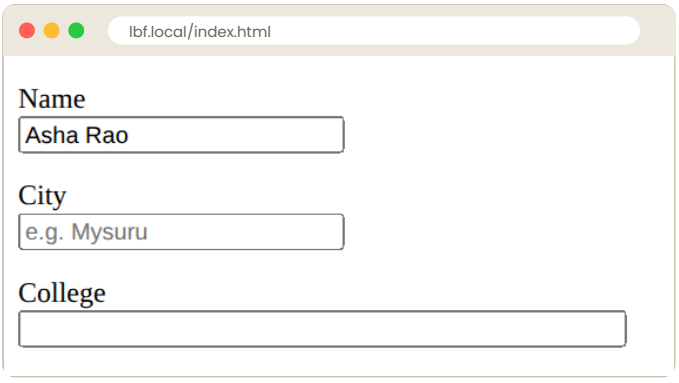
CLICKING THE LABEL "I AGREE TO THE TERMS" TOGGLED ITS CHECKBOX

7.2.2 <input> and the name, value, placeholder attributes

`<input>` is the most versatile control: a void element whose `type` attribute decides what it is (7.3). Key attributes: `name` is the key under which the value is sent; `value` is the initial value (for text-like inputs) or the value sent when checked (checkboxes and radios) or the button text (submit buttons); `placeholder` shows a short **example** of the expected format inside an empty field ("e.g. 98450 12345"); `id` connects it to its label. More attributes (`required`, `min`, `maxlength`, `pattern`, `autocomplete`…) follow in Module 8.

NAME, VALUE AND PLACEHOLDER HTML

```html
<form>
  <p><label for="name">Name</label><br>
     <input type="text" id="name" name="name" value="Asha Rao"></p>
  <p><label for="city">City</label><br>
     <input type="text" id="city" name="city" placeholder="e.g. Mysuru"></p>
  <p><label for="college">College</label><br>
     <input type="text" id="college" name="college" size="40"></p>
</form>
```



HOW THE BROWSER DISPLAYS IT

7.3 Common input types

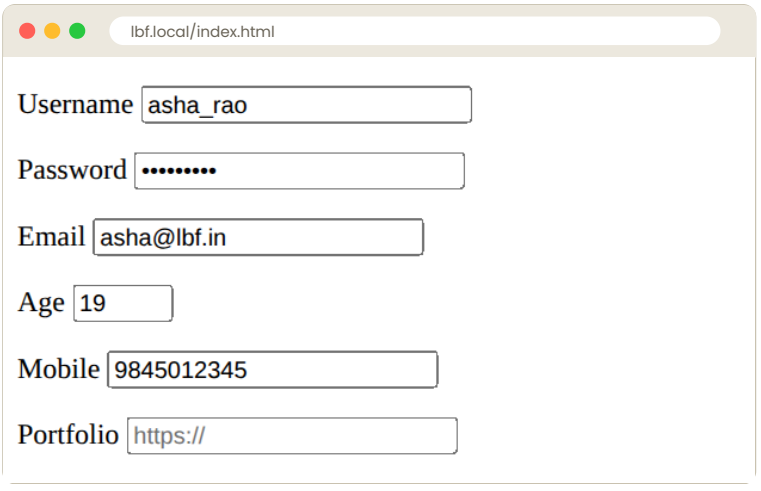
Choosing the right `type` gives users a suitable control, the right on-screen keyboard on phones, and built-in validation. Browsers that do not know a type fall back to a text field, so new types are safe to use.

7.3.1 text, password, email, number, tel, url

Type	Shows	Mobile keyboard	Built-in check
`text`	single-line text field (default)	normal	none
`password`	characters masked as dots	normal	none (never sent with GET)
`email`	text field	with @ and .	must look like an email address; `multiple` allows a list
`number`	field with up/down spinner	numeric	must be a number; `min`, `max`, `step`
`tel`	text field	phone keypad	none (formats vary; use `pattern`)
`url`	text field	with / and .com	must be an absolute URL (https://…)

TEXT-LIKE INPUT TYPES HTML

```
<form>
  <p><label>Username <input type="text" name="user" value="asha_rao"></label></p>
  <p><label>Password <input type="password" name="pass" value="secret123"></label></p>
  <p><label>Email <input type="email" name="email" value="asha@lbf.in"></label></p>
  <p><label>Age <input type="number" name="age" min="15" max="60" value="19"></label></p>
  <p><label>Mobile <input type="tel" name="mobile" value="9845012345"></label></p>
  <p><label>Portfolio <input type="url" name="site" placeholder="https://"></label></p>
</form>
```



HOW THE BROWSER DISPLAYS IT

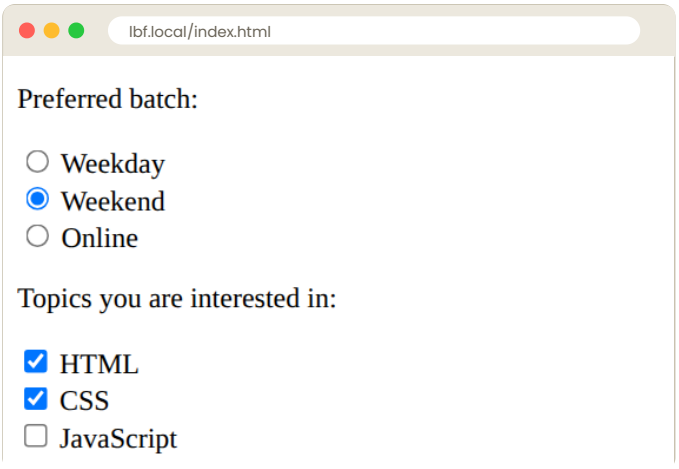
Use `number` only for real quantities (age, seats, marks). For numbers that are really identifiers, such as phone numbers, PIN codes, card numbers or OTPs, use `text` or `tel` with `inputmode="numeric"` (8.2.7): spinners, scroll-wheel changes and removal of leading zeros make no sense for them.

7.3.2 checkbox and radio (grouping by name)

Checkboxes are independent on/off choices: the user can tick none, one or many. **Radio buttons** are mutually exclusive choices: only one in a group can be selected. Radios form a group by sharing the same `name`; each has a different `value`, which is what gets sent. The `checked` attribute pre-selects an option. An unchecked checkbox sends nothing; a checked one sends its value (or `on` if it has no value). Several checkboxes may share a name to send a list of values (`topics=html&topics=css`).

A RADIO GROUP (ONE CHOICE) AND CHECKBOXES (MANY CHOICES) HTML

```
<form>
  <p>Preferred batch:</p>
  <label><input type="radio" name="batch" value="weekday" checked> Weekday</label><br>
  <label><input type="radio" name="batch" value="weekend"> Weekend</label><br>
  <label><input type="radio" name="batch" value="online"> Online</label>
  <p>Topics you are interested in:</p>
  <label><input type="checkbox" name="topics" value="html" checked> HTML</label><br>
  <label><input type="checkbox" name="topics" value="css" checked> CSS</label><br>
  <label><input type="checkbox" name="topics" value="js"> JavaScript</label>
</form>
```



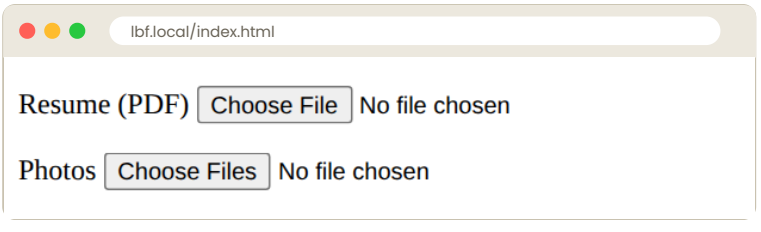
AFTER CLICKING "WEEKEND", THE WEEKDAY RADIO WAS AUTOMATICALLY UNSELECTED

7.3.3 file

`type="file"` lets the user choose files from their device (or take a photo on phones). `accept` limits the selectable types (`accept=".pdf"`, `accept="image/*"`), `multiple` allows several files, and `capture` asks for the camera. To upload files, the form must use `method="post"` and `enctype="multipart/form-data"` (8.5.1). The browser shows only the file name; the server must check type and size again.

FILE INPUTS HTML

```
<form action="/upload" method="post" enctype="multipart/form-data">
  <p><label>Resume (PDF) <input type="file" name="resume" accept=".pdf,application/pdf"></label></p>
  <p><label>Photos <input type="file" name="photos" accept="image/*" multiple></label></p>
</form>
```



HOW THE BROWSER DISPLAYS IT

7.3.4 hidden

`type="hidden"` sends a value that the user does not see or edit: a record id, the source page, a campaign code, a security (CSRF) token generated by the server. It is not secret: anyone can see it in the page source and change it with DevTools, so the server must never trust it blindly.

HIDDEN INPUTS TRAVEL WITH THE FORM BUT ARE NOT DISPLAYED HTML

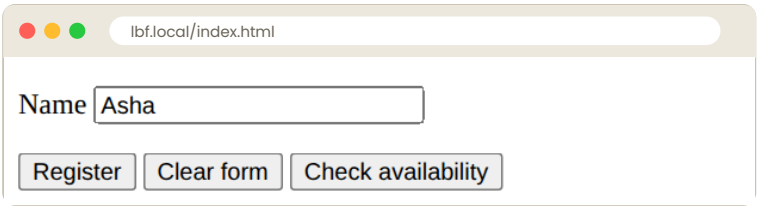
```html
<form action="/enquiry" method="post">
  <input type="hidden" name="course_id" value="HTML5-2026-WKND">
  <input type="hidden" name="source" value="landing-page">
  <label>Phone <input type="tel" name="phone"></label>
  <button>Request a call back</button>
</form>
```

7.3.5 submit, reset, button

`type="submit"` submits the form (its `value` is the button text). `type="reset"` resets every control to its initial value; it is rarely useful and easily clicked by mistake, so most forms leave it out. `type="button"` does nothing by itself and is used with JavaScript. An `image` input (`type="image"`) is a graphical submit button. Pressing **Enter** in a text field submits the form using the first submit button.

SUBMIT, RESET AND BUTTON INPUTS HTML

```html
<form>
  <p><label>Name <input type="text" name="name" value="Asha"></label></p>
  <input type="submit" value="Register">
  <input type="reset" value="Clear form">
  <input type="button" value="Check availability">
</form>
```



HOW THE BROWSER DISPLAYS IT

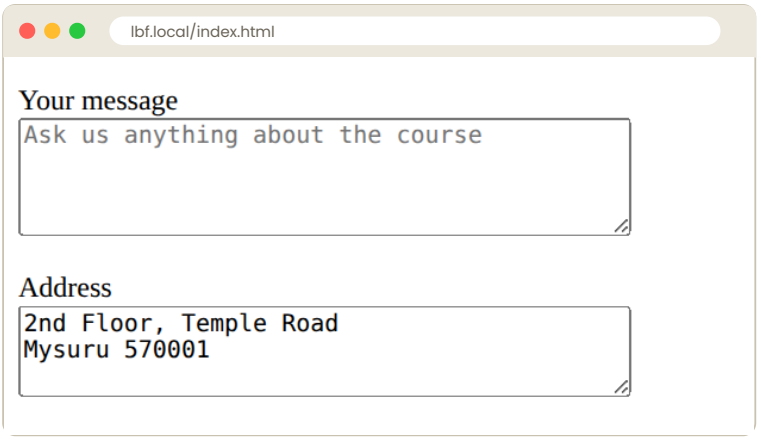
7.4 Other form controls

7.4.1 <textarea>

`<textarea>` is a **multi-line** text field for comments, messages, addresses and feedback. Unlike input, it has a closing tag, and its initial value goes **between** the tags (whitespace inside is kept, so do not indent it). `rows` and `cols` set its default visible size; users can resize it (CSS `resize` controls this); `maxlength` limits the length.

TEXTAREA WITH A PLACEHOLDER, AND WITH INITIAL TEXT HTML

```
<form>
  <p><label for="msg">Your message</label><br>
  <textarea id="msg" name="message" rows="4" cols="40" maxlength="500" placeholder="Ask us anything
about the course"></textarea></p>
  <p><label for="addr">Address</label><br>
  <textarea id="addr" name="address" rows="3" cols="40">2nd Floor, Temple Road
Mysuru 570001</textarea></p>
</form>
```



HOW THE BROWSER DISPLAYS IT

7.4.2 <select>, <option>, <optgroup>

`<select>` is a **drop-down list**. Each `<option>` has a `value` (sent to the server) and visible text (which is sent if there is no value). `selected` pre-selects an option; a first option with an empty value and `disabled` (or `hidden`) works as a "Please choose" prompt. `<optgroup label="...">` groups related options under a non-selectable heading. With `multiple`, users can choose several options (with Ctrl/Cmd) and the list is displayed as a box, whose height `size` controls; checkboxes are usually friendlier for multiple choice.

A SELECT WITH A PROMPT AND OPTION GROUPS, AND A MULTIPLE SELECT HTML

```
<form>
  <p><label for="course">Course</label><br>
  <select id="course" name="course">
    <option value="" disabled selected>Please choose a course</option>
    <optgroup label="Web development">
      <option value="html5">HTML5</option>
      <option value="css3">CSS3</option>
      <option value="js">JavaScript</option>
    </optgroup>
    <optgroup label="Programming">
      <option value="java">Java</option>
      <option value="python">Python</option>
    </optgroup>
  </select></p>
  <p><label for="langs">Languages you know (Ctrl+click for several)</label><br>
  <select id="langs" name="langs" multiple size="4">
    <option selected>English</option><option>Kannada</option><option selected>Hindi</option>
<option>Tamil</option>
  </select></p>
</form>
```



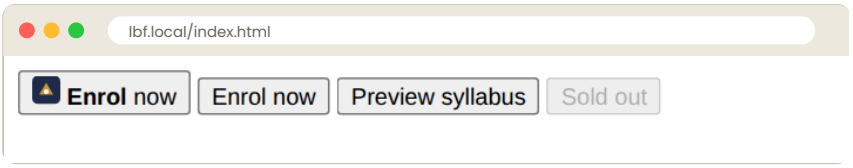
HOW THE BROWSER DISPLAYS IT

7.4.3 <button> vs <input type="submit">

`<button>` has a closing tag and can therefore contain **rich content**: text with formatting, an icon or image. Its `type` is `submit` (the default inside a form), `reset` or `button`. Because the default is submit, always write `type="button"` for buttons that only run JavaScript, or they will submit the form unexpectedly. `<input type="submit">` can only display plain text from its `value`. Modern code generally prefers `<button>`. Never use a `<div>` or `<a>` as a button: they are not focusable and do not respond to Enter and Space without extra code.

BUTTON WITH RICH CONTENT VERSUS INPUT TYPE="SUBMIT" HTML

```
<form>
  <button type="submit"><img src="logo.svg" alt="" width="16" height="16"> <strong>Enrol</strong>
now</button>
  <input type="submit" value="Enrol now">
  <button type="button">Preview syllabus</button>
  <button type="submit" disabled>Sold out</button>
</form>
```



HOW THE BROWSER DISPLAYS IT

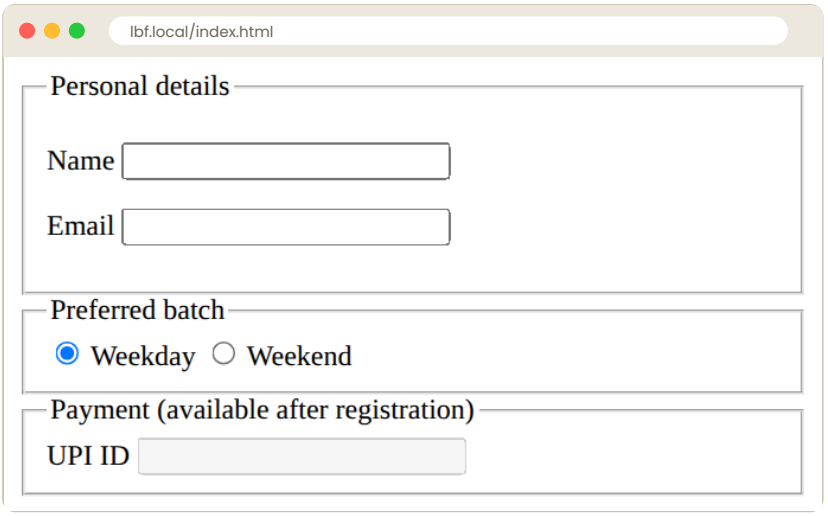
7.5 Grouping

7.5.1 <fieldset> and <legend>

`<fieldset>` groups related controls (personal details, address, payment), and its first child `<legend>` captions the group. Browsers draw a border with the legend in it. Screen readers announce the legend when users enter the group, which is **essential for radio and checkbox groups**: the legend is the question ("Preferred batch") and each label an answer. `disabled` on a fieldset disables every control inside it at once.

FIELDSETS WITH LEGENDS, INCLUDING A DISABLED GROUP HTML

```html
<form>
  <fieldset>
    <legend>Personal details</legend>
    <p><label>Name <input type="text" name="name"></label></p>
    <p><label>Email <input type="email" name="email"></label></p>
  </fieldset>
  <fieldset>
    <legend>Preferred batch</legend>
    <label><input type="radio" name="batch" value="weekday" checked> Weekday</label>
    <label><input type="radio" name="batch" value="weekend"> Weekend</label>
  </fieldset>
  <fieldset disabled>
    <legend>Payment (available after registration)</legend>
    <label>UPI ID <input type="text" name="upi"></label>
  </fieldset>
</form>
```



HOW THE BROWSER DISPLAYS IT

form wraps controls; action is where the data goes and method is GET (data in the URL, for reading and searching) or POST (data in the body, for changes and sensitive data). Only enabled controls with a name are sent, as name=value pairs. Every control needs a label, connected with for and id or by wrapping; placeholders are examples, not labels. input types include text, password, email, number, tel, url, checkbox (independent choices), radio (one choice per shared name), file (with accept and multiple, POST and multipart encoding), hidden (not secret), submit, reset and button. textarea is multi-line with its value between the tags; select with option and optgroup is a drop-down (multiple for several choices); button can contain rich content and defaults to type="submit" inside forms. fieldset and legend group related controls and give radio and checkbox groups their question.

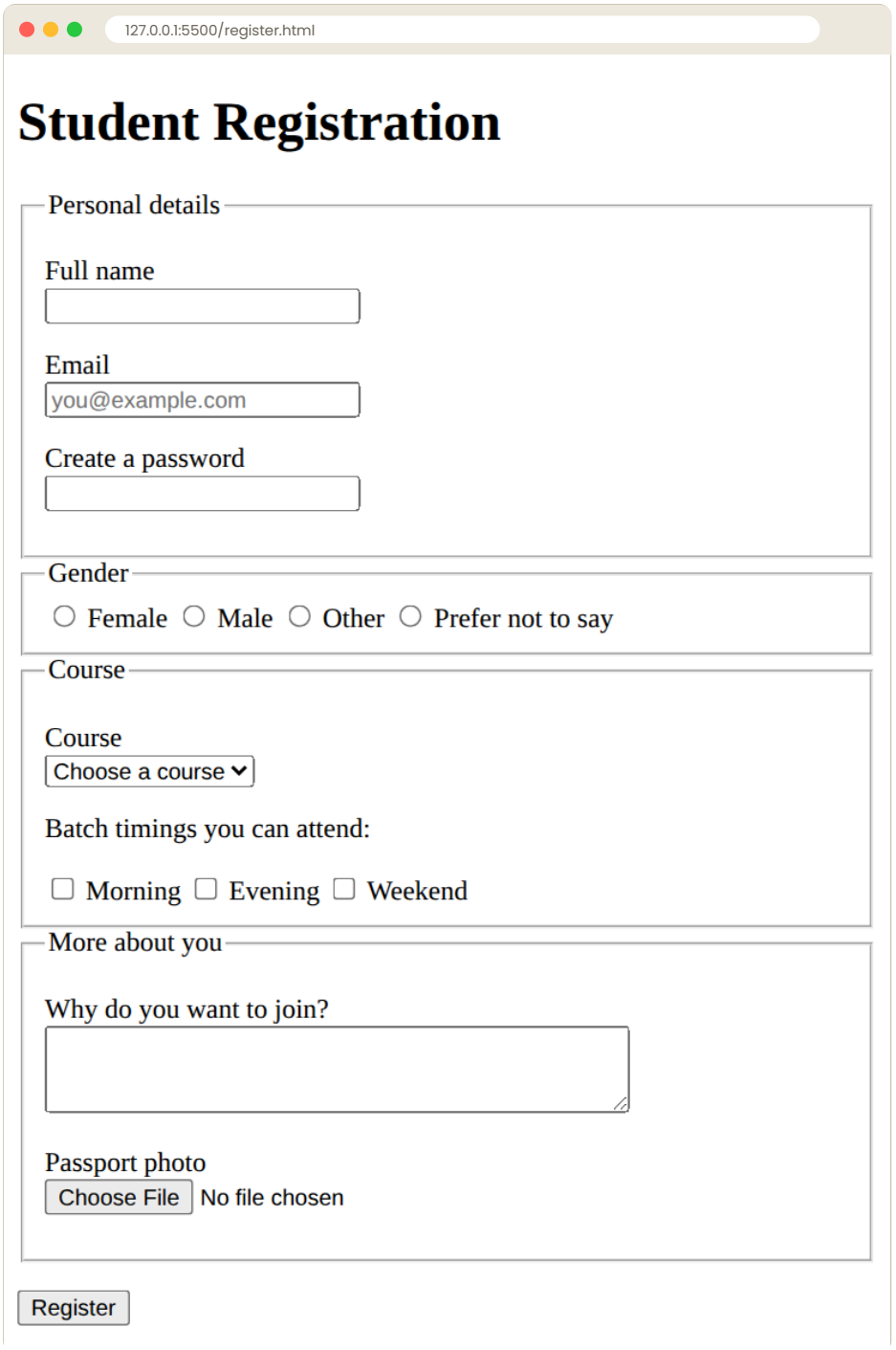
PRACTICE CHECKPOINT

A **student registration form** with **text, email, password, radio, checkbox, select, textarea and file** inputs, all properly **labelled and grouped**.

REGISTER.HTML: SAMPLE SOLUTION HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
```

```
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Registration | Learn Being Forward</title>
</head>
<body>
  <h1>Student Registration</h1>
  <form action="/register" method="post" enctype="multipart/form-data">
    <fieldset>
      <legend>Personal details</legend>
      <p><label for="fname">Full name</label><br><input type="text" id="fname" name="fullname"></p>
      <p><label for="email">Email</label><br><input type="email" id="email" name="email"
placeholder="you@example.com"></p>
      <p><label for="pwd">Create a password</label><br><input type="password" id="pwd"
name="password"></p>
    </fieldset>
    <fieldset>
      <legend>Gender</legend>
      <label><input type="radio" name="gender" value="female"> Female</label>
      <label><input type="radio" name="gender" value="male"> Male</label>
      <label><input type="radio" name="gender" value="other"> Other</label>
      <label><input type="radio" name="gender" value="na"> Prefer not to say</label>
    </fieldset>
    <fieldset>
      <legend>Course</legend>
      <p><label for="course">Course</label><br>
        <select id="course" name="course">
          <option value="" disabled selected>Choose a course</option>
          <option value="html5">HTML5</option>
          <option value="java">Java</option>
          <option value="python">Python</option>
        </select></p>
      <p>Batch timings you can attend:</p>
      <label><input type="checkbox" name="slots" value="morning"> Morning</label>
      <label><input type="checkbox" name="slots" value="evening"> Evening</label>
      <label><input type="checkbox" name="slots" value="weekend"> Weekend</label>
    </fieldset>
    <fieldset>
      <legend>More about you</legend>
      <p><label for="about">Why do you want to join?</label><br><textarea id="about" name="about"
rows="3" cols="40"></textarea></p>
      <p><label for="photo">Passport photo</label><br><input type="file" id="photo" name="photo"
accept="image/*"></p>
    </fieldset>
    <p><button type="submit">Register</button></p>
  </form>
</body>
</html>
```



THE REGISTRATION FORM

LEVEL 2

Intermediate level

Outcome: create validated forms, semantic layouts, multimedia pages, SEO-ready metadata and accessible markup.

5 MODULES · 9 DAYS MINIMUM

Module 08 · Advanced forms and validation	**2 days**
Module 09 · Semantic HTML5 and page structure	**2 days**
Module 10 · Multimedia and embedded content	**2 days**
Module 11 · The head, metadata and SEO	**1 day**
Module 12 · Accessibility (a11y)	**2 days**

MODULE 08 · INTERMEDIATE

Advanced forms and validation

Minimum duration **2 days** | Level **Intermediate**

HTML5 added many input types and built-in validation that reduce the JavaScript needed for good forms. This module covers the date and time inputs, range, color and search, the input attributes that constrain values (required, disabled, readonly, min, max, step, minlength, maxlength, pattern, autofocus, autocomplete, multiple, accept, inputmode, enterkeyhint), suggestions and output elements (datalist, output, progress, meter), how the browser validates on submit, novalidate, the :valid, :invalid and :user-invalid pseudo-classes, a preview of the Constraint Validation API, form attributes such as enctype, form, formaction and formmethod, and the usability and security rules that every real form must follow.

IN THIS MODULE

8.1 HTML5 input types
8.2 Input attributes
8.3 Suggestions and output
8.4 Built-in validation
8.5 Form attributes
8.6 Form usability and security

8.1 HTML5 input types

Before HTML5, every text-like field was `type="text"` and developers needed JavaScript date pickers, sliders and colour pickers. HTML5 added types that give the browser enough information to show the right widget, the right mobile keyboard and to validate the value. A browser that does not understand a type simply falls back to a plain text box, so using them is always safe.

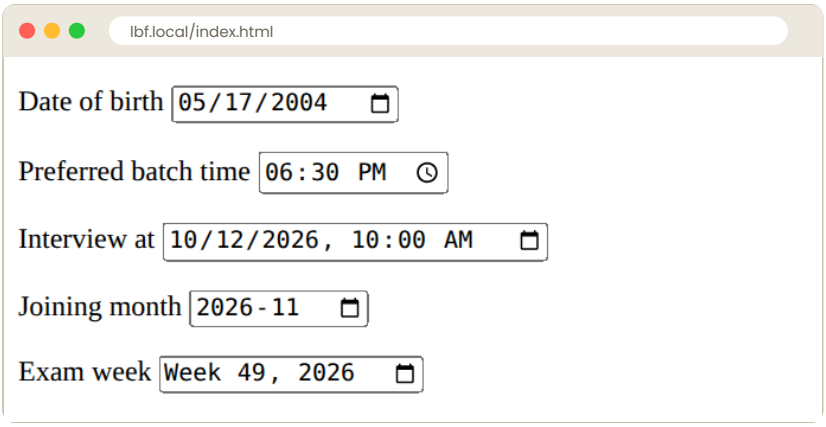
8.1.1 date, time, datetime-local, month, week

These types show a native picker. Whatever format the user sees (which follows their locale, such as 08/10/2026 in India or 10/08/2026 in the USA), the **value submitted is always in a fixed ISO format**, which makes server code simple.

Type	Picks	Submitted value format	Example value
`date`	a calendar day	YYYY-MM-DD	`2026-10-08`
`time`	a time of day	HH:MM (24-hour)	`14:30`
`datetime-local`	date and time, no time zone	YYYY-MM-DDTHH:MM	`2026-10-08T14:30`
`month`	a month of a year	YYYY-MM	`2026-10`
`week`	an ISO week of a year	YYYY-Www	`2026-W41`

DATE AND TIME INPUT TYPES HTML

```
<form>
  <p><label>Date of birth <input type="date" name="dob" value="2004-05-17"></label></p>
  <p><label>Preferred batch time <input type="time" name="slot" value="18:30"></label></p>
  <p><label>Interview at <input type="datetime-local" name="interview" value="2026-10-12T10:00">
</label></p>
  <p><label>Joining month <input type="month" name="join" value="2026-11"></label></p>
  <p><label>Exam week <input type="week" name="exam" value="2026-W49"></label></p>
</form>
```



NATIVE PICKERS IN CHROMIUM; THE DISPLAYED FORMAT FOLLOWS THE BROWSER LOCALE

TIP

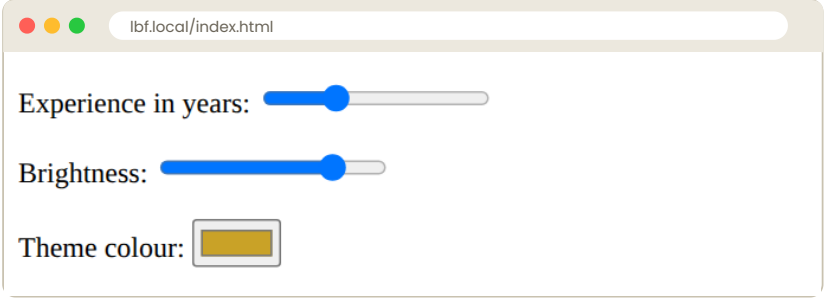
`datetime-local` has no time zone. If users in different countries book the same event, convert the value to UTC on the server (or with JavaScript) before storing it.

8.1.2 range and color

`type="range"` shows a slider for choosing an approximate number between `min` (default 0) and `max` (default 100) in steps of `step` (default 1). Use it when the exact number matters less than the position, such as volume or a rating, because sliders are hard to set precisely. `type="color"` opens a colour picker and always submits a lower-case 7-character hex value such as `#c9a227`.

RANGE AND COLOR HTML

```
<form>
  <p><label>Experience in years: <input type="range" name="exp" min="0" max="10" step="1" value="3">
</label></p>
  <p><label>Brightness: <input type="range" name="b" min="0" max="100" value="80"></label></p>
  <p><label>Theme colour: <input type="color" name="theme" value="#c9a227"></label></p>
</form>
```



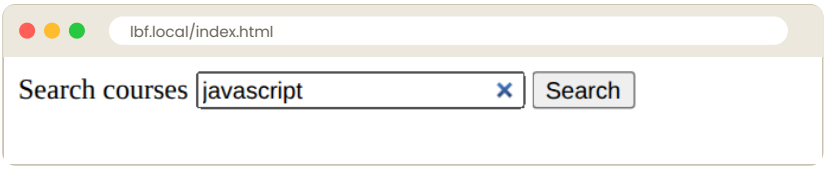
HOW THE BROWSER DISPLAYS IT

8.1.3 search

`type="search"` behaves like a text box but tells the browser it is a search field: most browsers add a clear (x) button, mobile keyboards show a "Search" key, and assistive technology announces it as a search box. Put it inside the `<search>` element (Module 9) for a complete search landmark.

A SEARCH FIELD HTML

```html
<search>
  <form action="/courses" role="search">
    <label for="q">Search courses</label>
    <input type="search" id="q" name="q" value="javascript" placeholder="e.g. Python">
    <button>Search</button>
  </form>
</search>
```



THE CLEAR BUTTON APPEARS WHEN THE FIELD HAS TEXT AND IS HOVERED OR FOCUSED

All the new types together, with what each one gives you for free:

Type	Widget on desktop	Mobile keyboard	Built-in validation
email	text box	with @ and .	must look like name@domain
url	text box	with / and .com	must be an absolute URL
tel	text box	phone keypad	none (formats vary by country)
number	spinner	numeric	number, min, max, step
date / time / …	picker	picker	valid date, min, max, step
range	slider	slider	always valid (clamped)
color	colour picker	colour picker	always valid
search	text box with clear	Search key	none

8.2 Input attributes

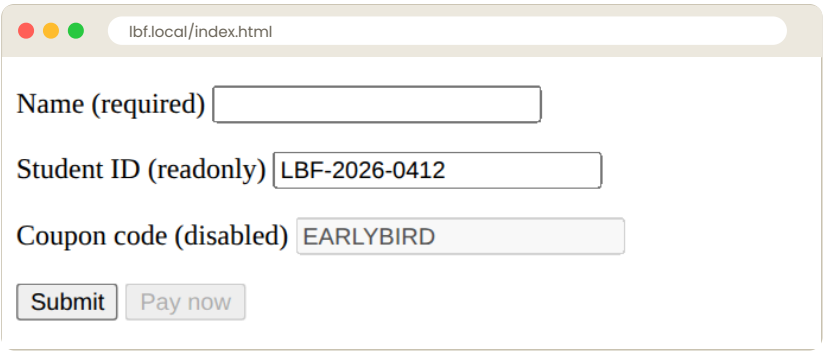
8.2.1 required, disabled, readonly

`required` means the field must have a value before the form can be submitted (for checkboxes: must be checked; for a radio group: one must be chosen). `disabled` makes a control unusable, greyed out, skipped by Tab **and not submitted**. `readonly` lets the user see, select and copy a text value but not change it, and the value **is** submitted. All three are boolean attributes: their presence is what matters.

REQUIRED, READONLY AND DISABLED

HTML

```
<form>
  <p><label>Name (required) <input name="name" required></label></p>
  <p><label>Student ID (readonly) <input name="sid" value="LBF-2026-0412" readonly></label></p>
  <p><label>Coupon code (disabled) <input name="coupon" value="EARLYBIRD" disabled></label></p>
  <p><button>Submit</button> <button disabled>Pay now</button></p>
</form>
```



HOW THE BROWSER DISPLAYS IT

	Editable	Focusable	Submitted	Validated
normal	yes	yes	yes	yes
readonly	no	yes	yes	no (constraints are skipped)
disabled	no	no	no	no

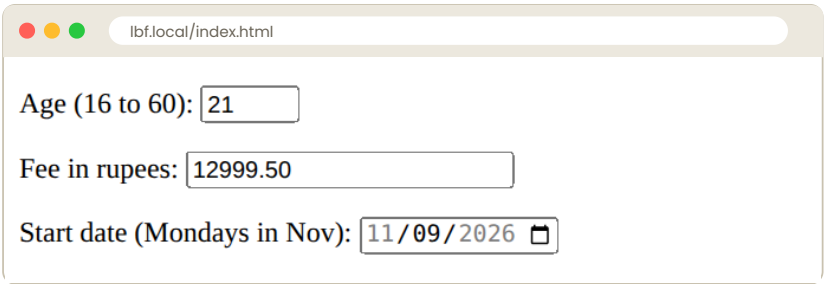
8.2.2 min, max, step

For number, range and the date/time types, `min` and `max` set the allowed range and `step` the allowed increments, counted from `min`. `step="0.01"` allows money values; `step="any"` allows any decimal. For dates, `step` is in days (date), seconds (time) or weeks (week): `step="7"` on a date with `min` on a Monday allows Mondays only.

MIN, MAX AND STEP

HTML

```
<form>
  <p><label>Age (16 to 60): <input type="number" name="age" min="16" max="60" value="21"></label></p>
  <p><label>Fee in rupees: <input type="number" name="fee" min="0" step="0.01" value="12999.50">
</label></p>
  <p><label>Start date (Mondays in Nov): <input type="date" name="start" min="2026-11-02" max="2026-
11-30" step="7" value="2026-11-09"></label></p>
</form>
```



HOW THE BROWSER DISPLAYS IT

8.2.3 minlength, maxlength

For text-like fields, `maxlength` stops the user typing more characters than allowed (the browser simply refuses extra keystrokes), and `minlength` marks a shorter non-empty value as invalid. An **empty** optional field is always valid; add `required` if it must be filled.

LENGTH LIMITS HTML

```
<label>Username <input name="user" minlength="4" maxlength="16" required></label>
<label>Short bio <textarea name="bio" maxlength="160"></textarea></label>
```

8.2.4 pattern with regular expressions

`pattern` holds a JavaScript regular expression that the **whole** value must match (the browser anchors it with `^(?:` and `)$` automatically). It works on text, tel, email, url, password and search. Always describe the expected format in visible text or a `title`, because the default error message only says "Please match the requested format."

Need	pattern	Matches
Indian mobile number	`[6-9][0-9]{9}`	9845012345
6-digit PIN code	`[1-9][0-9]{5}`	570001
Roll number like 21BCA045	`[0-9]{2}[A-Z]{3}[0-9]{3}`	21BCA045
Username: letters, digits, underscore	`[A-Za-z0-9_]{4,16}`	asha_rao
PAN card format	`[A-Z]{5}[0-9]{4}[A-Z]`	ABCDE1234F

The browser applies these rules itself; we can verify them by asking each field whether it is valid with the Constraint Validation API (8.4.4). This script runs in Chromium and checks several values against the phone pattern:

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```
<input id="phone" type="tel" name="phone" pattern="[6-9][0-9]{9}" title="10-digit Indian mobile
number">
```

TESTING A PATTERN IN THE BROWSER main.js

```
const phone = document.querySelector("#phone");
for (const v of ["9845012345", "98450 12345", "5845012345", "984501234", ""]) {
  phone.value = v;
  console.log(JSON.stringify(v).padEnd(14), phone.validity.valid ? "valid" : "invalid",
              phone.validity.patternMismatch ? "(patternMismatch)" : "");
}
```

BROWSER CONSOLE

```
"9845012345"   valid
"98450 12345"  invalid (patternMismatch)
"5845012345"   invalid (patternMismatch)
"984501234"    invalid (patternMismatch)
""             valid
```

NOTE

The empty string is **valid**: pattern is only checked when there is a value. Combine `pattern` with `required` for mandatory fields.

8.2.5 autofocus, autocomplete values

`autofocus` puts the cursor in a field when the page loads. Use it at most once per page and only when the form is the page's main purpose (a login or search page), because it scrolls the page and moves screen-reader users past the content above. `autocomplete` tells the browser and password managers what a field contains so they can fill it in correctly: `autocomplete="off"` asks the browser not to suggest values, and specific **tokens** enable accurate autofill.

Token	Use for	Token	Use for
name	full name	email	email address
given-name / family-name	first / last name	tel	phone number
username	login name	street-address	full address
current-password	login password	postal-code	PIN code
new-password	sign-up / change password	one-time-code	OTP from SMS
bday	date of birth	cc-number	card number

A LOGIN FORM WITH CORRECT AUTOCOMPLETE TOKENS — HTML

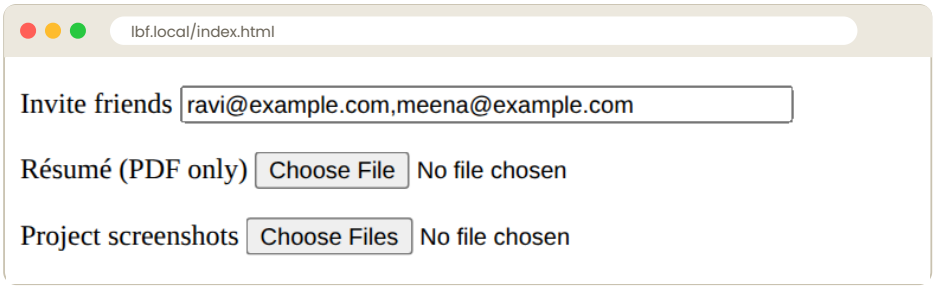
```
<form action="/login" method="post">
  <label>Email <input type="email" name="email" autocomplete="username" autofocus required></label>
  <label>Password <input type="password" name="pw" autocomplete="current-password" required></label>
  <label>OTP <input name="otp" inputmode="numeric" autocomplete="one-time-code" maxlength="6"></label>
  <button>Log in</button>
</form>
```

8.2.6 multiple, accept

`multiple` lets an `email` input accept a comma-separated list of addresses, a `file` input accept several files and a `select` allow several options. `accept` on a file input filters the file chooser by MIME type or extension: `accept="application/pdf"`, `accept=".pdf,.docx"`, `accept="image/*"`. It is a **hint** for the chooser only; users can still choose "All files", so the server must check the file type again.

MULTIPLE AND ACCEPT — HTML

```
<form>
  <p><label>Invite friends <input type="email" name="invite" multiple size="40"
value="ravi@example.com, meena@example.com"></label></p>
  <p><label>Résumé (PDF only) <input type="file" name="cv" accept="application/pdf,.pdf"></label></p>
  <p><label>Project screenshots <input type="file" name="shots" accept="image/png,image/jpeg"
multiple></label></p>
</form>
```



HOW THE BROWSER DISPLAYS IT

8.2.7 inputmode and enterkeyhint for mobile keyboards

`inputmode` chooses the on-screen keyboard **without** changing the field's type or validation. It is the right choice for things that are digits but not numbers, such as OTPs, PIN codes and card numbers, where `type="number"` would add spinners, drop leading zeros and accept "1e5". `enterkeyhint` changes the label on the Enter key.

inputmode	Keyboard	enterkeyhint	Enter key shows
text	standard	enter	Enter / return
numeric	digits 0-9	done	Done
decimal	digits with decimal point	go	Go
tel	phone keypad	next	Next
email	with @	previous	Previous
url	with / and .com	search	Search (magnifier)
search	with Search key	send	Send

MOBILE-FRIENDLY FIELDS

HTML

```
<input name="otp" inputmode="numeric" pattern="[0-9]{6}" autocomplete="one-time-code"
enterkeyhint="done">
<input name="pincode" inputmode="numeric" pattern="[1-9][0-9]{5}" enterkeyhint="next">
<input type="search" name="q" enterkeyhint="search">
```

8.3 Suggestions and output

8.3.1 <datalist>

`<datalist>` provides a list of **suggestions** for an input, connected with the input's `list` attribute. Unlike `<select>`, the user can still type any value; the suggestions filter as they type. It also works with range (tick marks), color, date and number inputs.

DATALIST SUGGESTIONS HTML

```html
<label for="city">City</label>
<input id="city" name="city" list="cities" placeholder="Start typing...">
<datalist id="cities">
  <option value="Bengaluru"></option>
  <option value="Belagavi"></option>
  <option value="Ballari"></option>
  <option value="Mysuru"></option>
  <option value="Mangaluru"></option>
</datalist>
```



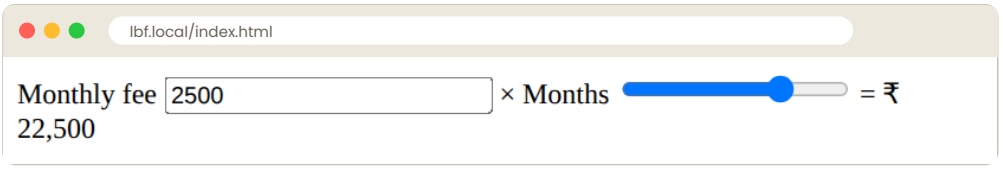
THE FIELD SHOWS A DROP-DOWN ARROW; AS THE USER TYPES "B", A NATIVE POP-UP LISTS BENGALURU, BELAGAVI AND BALLARI

8.3.2 <output>

`<output>` holds the **result of a calculation** or user action. Its `for` attribute lists the ids of the inputs that contributed, which assistive technology can use; many screen readers announce changes to it. A tiny inline `oninput` handler on the form keeps it updated.

A LIVE FEE CALCULATOR WITH OUTPUT HTML

```html
<form oninput="total.value = (Number(fee.value) * Number(months.value)).toLocaleString('en-IN')">
  <label>Monthly fee <input type="number" id="fee" value="2500" min="0"></label>
  &times;
  <label>Months <input type="range" id="months" min="1" max="12" value="6"></label>
  = &#8377; <output name="total" for="fee months">15,000</output>
</form>
```



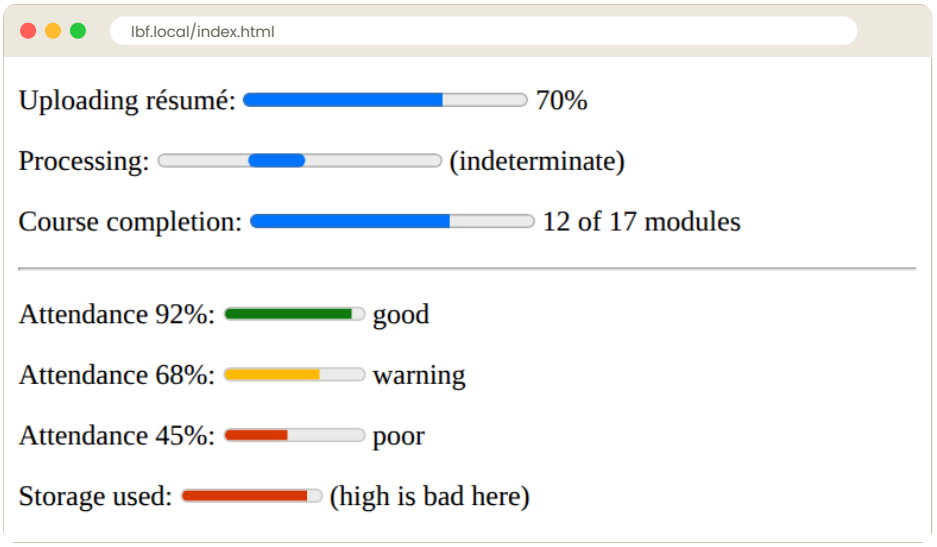
AFTER DRAGGING THE SLIDER TO 9 MONTHS

8.3.3 <progress> and <meter>

Both draw a bar but mean different things. `<progress>` shows **how far a task has got** (upload, course completion): `value` out of `max`; without `value` it is **indeterminate** (an animated "working" bar). `<meter>` shows a **measurement within a known range** (disk usage, score, battery), and its `low`, `high` and `optimum` attributes make the browser colour it green, yellow or red.

PROGRESS VS METER HTML

```
<p>Uploading résumé: <progress value="70" max="100">70%</progress> 70%</p>
<p>Processing: <progress>working...</progress> (indeterminate)</p>
<p>Course completion: <progress value="12" max="17"></progress> 12 of 17 modules</p>
<hr>
<p>Attendance 92%: <meter value="92" min="0" max="100" low="60" high="75" optimum="100">92%</meter>
good</p>
<p>Attendance 68%: <meter value="68" min="0" max="100" low="60" high="75" optimum="100">68%</meter>
warning</p>
<p>Attendance 45%: <meter value="45" min="0" max="100" low="60" high="75" optimum="100">45%</meter>
poor</p>
<p>Storage used: <meter value="0.9" low="0.6" high="0.8" optimum="0.2">90%</meter> (high is bad here)
</p>
```



METER COLOURS DEPEND ON WHICH REGION THE VALUE FALLS IN RELATIVE TO OPTIMUM



CHOOSING BETWEEN PROGRESS AND METER

8.4 Built-in validation

8.4.1 How browsers validate on submit

When a form is submitted, the browser checks every control against its **constraints** (required, type, min/max, step, minlength/maxlength, pattern). If any control fails, submission is cancelled, the first invalid control is focused and a message bubble appears next to it. Each failure sets a flag on the control's `validity` object, which JavaScript can read:

validity flag	Set when	Default Chrome message (English)
valueMissing	required field is empty	Please fill out this field.
typeMismatch	email/url value is malformed	Please include an '@' in the email address...

validity flag	Set when	Default Chrome message (English)
patternMismatch	value does not match pattern	Please match the requested format.
tooShort / tooLong	shorter than minlength / longer than maxlength	Please lengthen this text to 4 characters or more…
rangeUnderflow / rangeOverflow	below min / above max	Value must be greater than or equal to 16.
stepMismatch	not on a valid step	Please enter a valid value. The two nearest…
badInput	browser cannot convert input (e.g. "abc" in number)	Please enter a number.
customError	setCustomValidity() was called with a message	your message

The following script fills a form with bad values and asks the browser to validate it; the messages printed are exactly what the user would see in the bubbles:

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```html
<form>
  <input name="name" required>
  <input name="email" type="email" required>
  <input name="age" type="number" min="16" max="60">
  <input name="pin" pattern="[1-9][0-9]{5}" title="6-digit PIN code">
  <button>Send</button>
</form>
```

WHAT THE BROWSER CHECKS ON SUBMIT main.js

```javascript
const form = document.querySelector("form");
form.email.value = "asha.example.com";
form.age.value = "14";
form.pin.value = "57001";
console.log("form valid?", form.checkValidity());
for (const el of form.elements) {
  if (el.name && !el.validity.valid) console.log(el.name.padEnd(6), "->", el.validationMessage);
}
```

BROWSER CONSOLE

```
form valid? false
name   -> Please fill out this field.
email  -> Please include an '@' in the email address. 'asha.example.com' is missing an '@'.
age    -> Value must be greater than or equal to 16.
pin    -> Please match the requested format.
```

8.4.2 novalidate and formnovalidate

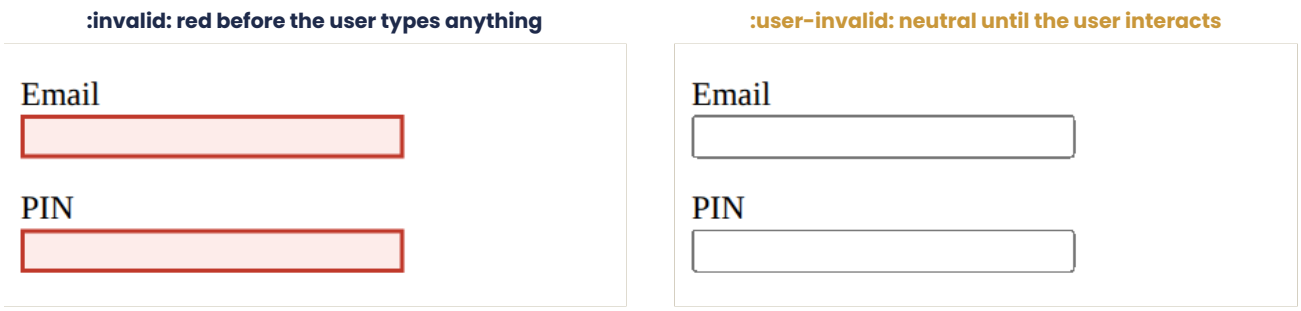
`novalidate` on the form turns off the browser's validation for the whole form, usually because JavaScript will validate and show custom messages instead (the constraint attributes still work and the `:invalid` state is still set). `formnovalidate` on one submit button skips validation only when that button is used, perfect for "Save draft" next to "Submit".

SAVE A DRAFT WITHOUT VALIDATING HTML

```
<form action="/application" method="post">
  <input name="name" required>
  <textarea name="statement" required minlength="200"></textarea>
  <button type="submit">Submit application</button>
  <button type="submit" formnovalidate name="action" value="draft">Save draft</button>
</form>
```

8.4.3 Styling valid and invalid fields with :valid, :invalid, :user-invalid

CSS can style fields according to their validity. `:valid` and `:invalid` apply **immediately**, so an empty required field is red before the user has touched it, which feels like being told off. `:user-invalid` (supported in all modern browsers since 2023) only applies **after the user has interacted** with the field (changed it and left, or tried to submit), which is the behaviour users expect.



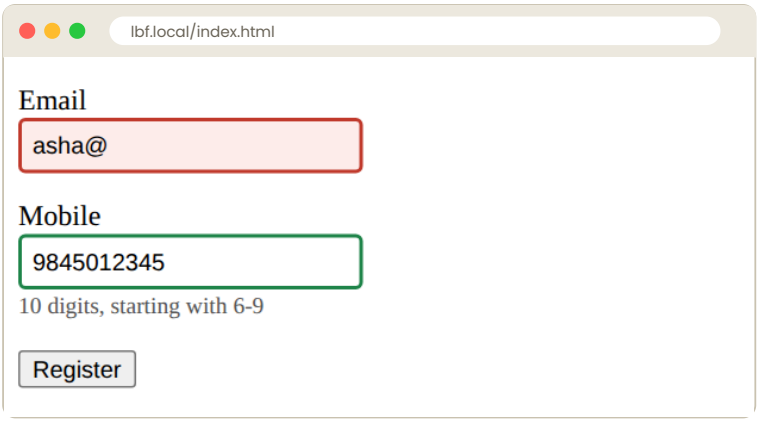
THE SAME UNTOUCHED FORM STYLED WITH :INVALID (LEFT) AND :USER-INVALID (RIGHT)

VALIDATION PSEUDO-CLASSES text

```
input:invalid      { border: 2px solid #c0392b; }   /* immediately */
input:user-invalid { border: 2px solid #c0392b; }   /* only after interaction: preferred */
input:user-valid   { border: 2px solid #1e8449; }
input:required     { border-left-width: 4px; }      /* mark required fields */
input:out-of-range { background: #fdecea; }         /* number/date outside min-max */
```

:USER-INVALID AFTER A SUBMIT ATTEMPT HTML

```
<style>
  input { padding: 6px; border: 1px solid #999; border-radius: 4px; }
  input:user-invalid { border: 2px solid #c0392b; background: #fdecea; }
  input:user-valid { border: 2px solid #1e8449; }
  .hint { font-size: 13px; color: #555; }
</style>
<form>
  <p><label for="e">Email</label><br><input id="e" type="email" required value="asha@"></p>
  <p><label for="m">Mobile</label><br><input id="m" type="tel" pattern="[6-9][0-9]{9}" required
value="9845012345">
     <br><span class="hint">10 digits, starting with 6-9</span></p>
  <p><button>Register</button></p>
</form>
```



AFTER CLICKING REGISTER: THE BAD EMAIL TURNS RED, THE CORRECT MOBILE NUMBER TURNS GREEN

8.4.4 Custom messages with the Constraint Validation API (preview)

JavaScript can replace the default message with `setCustomValidity(message)`; a non-empty message makes the field invalid, and an empty string makes it valid again. `reportValidity()` shows the bubbles immediately. The common use is a rule HTML attributes cannot express, such as "confirm password must match password". Module 15 introduces the DOM in more detail.

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```html
<form>
  <label>Password <input type="password" name="pw" minlength="8" required></label>
  <label>Confirm <input type="password" name="confirm" required></label>
  <button>Create account</button>
</form>
```

CONFIRM-PASSWORD CHECK WITH SETCUSTOMVALIDITY main.js

```javascript
const form = document.querySelector("form");
function checkMatch() {
  form.confirm.setCustomValidity(
    form.confirm.value === form.pw.value ? "" : "Passwords do not match.");
}
form.pw.addEventListener("input", checkMatch);
form.confirm.addEventListener("input", checkMatch);

form.pw.value = "Lbf@2026"; form.confirm.value = "Lbf@2025";
checkMatch();
console.log("1:", form.checkValidity(), form.confirm.validationMessage);
form.confirm.value = "Lbf@2026";
checkMatch();
console.log("2:", form.checkValidity(), JSON.stringify(form.confirm.validationMessage));
```

BROWSER CONSOLE

```
1: false Passwords do not match.
2: true ""
```

8.5 Form attributes

8.5.1 enctype for file uploads

`enctype` sets how POST data is encoded. The default, `application/x-www-form-urlencoded`, sends name=value pairs like a query string and **cannot carry file contents**: only the file name would be sent.

For any form with a file input, use `method="post"` and `enctype="multipart/form-data"`, which sends each field as a separate part with its own headers.

enctype	Body looks like	Use for
application/x-www-form-urlencoded (default)	`name=Asha+Rao&course=html5`	ordinary forms
multipart/form-data	parts separated by a boundary, each with headers and raw bytes	any form with `type="file"`
text/plain	`name=Asha Rao` lines	debugging only; not reliable

A FILE UPLOAD FORM index.html

```
<form action="/apply" method="post" enctype="multipart/form-data">
  <input name="name" value="Asha Rao">
  <input type="file" name="cv" accept=".pdf">
  <button>Apply</button>
</form>
```

REQUEST SENT (DevTools › Network › Payload, view source)

```
POST /apply HTTP/1.1
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4YWxk

------WebKitFormBoundary7MA4YWxk
Content-Disposition: form-data; name="name"

Asha Rao
------WebKitFormBoundary7MA4YWxk
Content-Disposition: form-data; name="cv"; filename="asha-cv.pdf"
Content-Type: application/pdf

%PDF-1.7 ... (file bytes) ...
------WebKitFormBoundary7MA4YWxk--
```

8.5.2 form attribute for controls outside a form

A control normally belongs to the form that contains it. The `form` attribute, set to a form's `id`, attaches a control anywhere in the page to that form. It is useful when the layout puts a button in a page header or a field in a table row that cannot be inside the form element (a form cannot wrap only part of a table).

CONTROLS ATTACHED WITH FORM="..." HTML

```
<form id="filters" action="/courses"></form>

<header>
  <button type="submit" form="filters">Apply filters</button>   <!-- outside the form -->
</header>
<aside>
  <label>Level <select name="level" form="filters">...</select></label>
</aside>
```

8.5.3 formaction and formmethod

Submit buttons can override the form's own settings for that submission only: `formaction`, `formmethod`, `formenctype`, `formtarget` and `formnovalidate`. One form can therefore have "Preview" and "Publish" buttons that go to different URLs.

PER-BUTTON OVERRIDES HTML

```
<form action="/posts" method="post">
  <input name="title" required>
  <textarea name="body"></textarea>
  <button>Publish</button>                                    <!-- POST /posts -->
  <button formaction="/posts/preview" formtarget="_blank">Preview</button>
  <button formaction="/posts/search" formmethod="get" formnovalidate>Find similar</button>
</form>
```

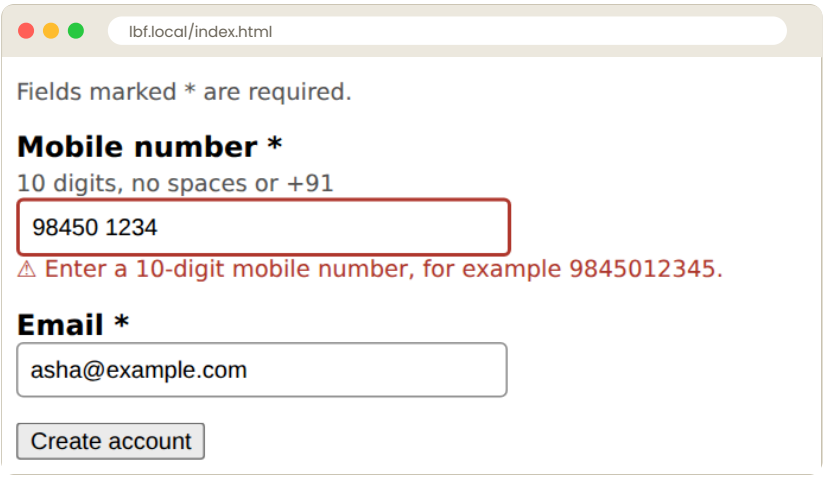
8.6 Form usability and security

8.6.1 Clear labels, hints and error messages

- **Visible label for every field**, above or beside it, never only a placeholder.
- **Say the format up front** in hint text ("DD/MM/YYYY", "10 digits"), linked with `aria-describedby` (Module 12) so screen readers read it too.
- **Mark required (or optional) fields** consistently and explain the marker once ("* required").
- **Ask only what you need**: every extra field reduces completions.
- **Use the right type, inputmode and autocomplete** so phones show the right keyboard and autofill works.
- **Error messages say what is wrong and how to fix it**: "Enter a 10-digit mobile number starting with 6-9", not "Invalid input". Show them next to the field, in text (not colour alone), and keep the user's data.
- **One column** forms are completed faster than multi-column ones; group related fields with fieldset.
- **Clear button text**: "Create account" beats "Submit".

A HELPFUL ERROR MESSAGE HTML

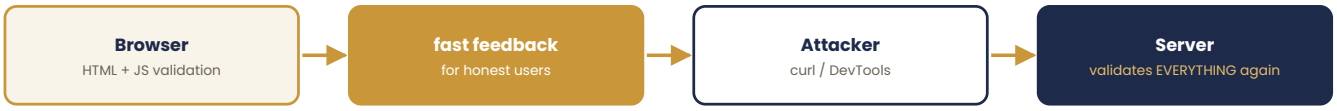
```
<style>
  body { font-family: system-ui, sans-serif; }
  .field { margin-bottom: 14px; }
  label { font-weight: 600; display: block; }
  .hint { color: #555; font-size: 13px; }
  input { padding: 7px; width: 260px; border: 1px solid #888; border-radius: 4px; }
  .err input { border: 2px solid #b03a2e; }
  .msg { color: #b03a2e; font-size: 13px; }
</style>
<form>
  <p class="hint">Fields marked * are required.</p>
  <div class="field err">
    <label for="mob">Mobile number *</label>
    <span class="hint" id="mob-hint">10 digits, no spaces or +91</span><br>
    <input id="mob" name="mobile" inputmode="numeric" value="98450 1234" aria-describedby="mob-hint
mob-err" aria-invalid="true">
    <div class="msg" id="mob-err">&#9888; Enter a 10-digit mobile number, for example 9845012345.
</div>
  </div>
  <div class="field">
    <label for="mail">Email *</label>
    <input id="mail" type="email" name="email" autocomplete="email" value="asha@example.com">
  </div>
  <button>Create account</button>
</form>
```



ERROR SHOWN BESIDE THE FIELD, WITH AN ICON AND TEXT, NOT COLOUR ALONE

8.6.2 Why client-side validation never replaces server-side validation

HTML validation is a **convenience for honest users**, not security. Anyone can bypass it by removing attributes in DevTools, adding `novalidate`, disabling JavaScript, or sending the request directly with curl or Postman. The server must validate every field again: types, lengths, ranges, allowed values, file type and size, and authorisation.



CLIENT-SIDE CHECKS HELP USERS; SERVER-SIDE CHECKS PROTECT THE SYSTEM

BYPASSING THE BROWSER ENTIRELY terminal

```
curl -X POST https://lbf.local/register \
     -d "name=" -d "age=-5" -d "email=not-an-email"
```

TERMINAL

```
HTTP/1.1 200 OK
{"saved": {"name": "", "age": -5, "email": "not-an-email"}}

(A server that trusted the HTML attributes just stored invalid data:
 required, min="16" and type="email" were never checked.)
```

WATCH OUT

Also on the server: escape output to prevent XSS, use parameterised queries to prevent SQL injection, protect state-changing forms with CSRF tokens, rate-limit login forms, and never trust a file's name or extension.

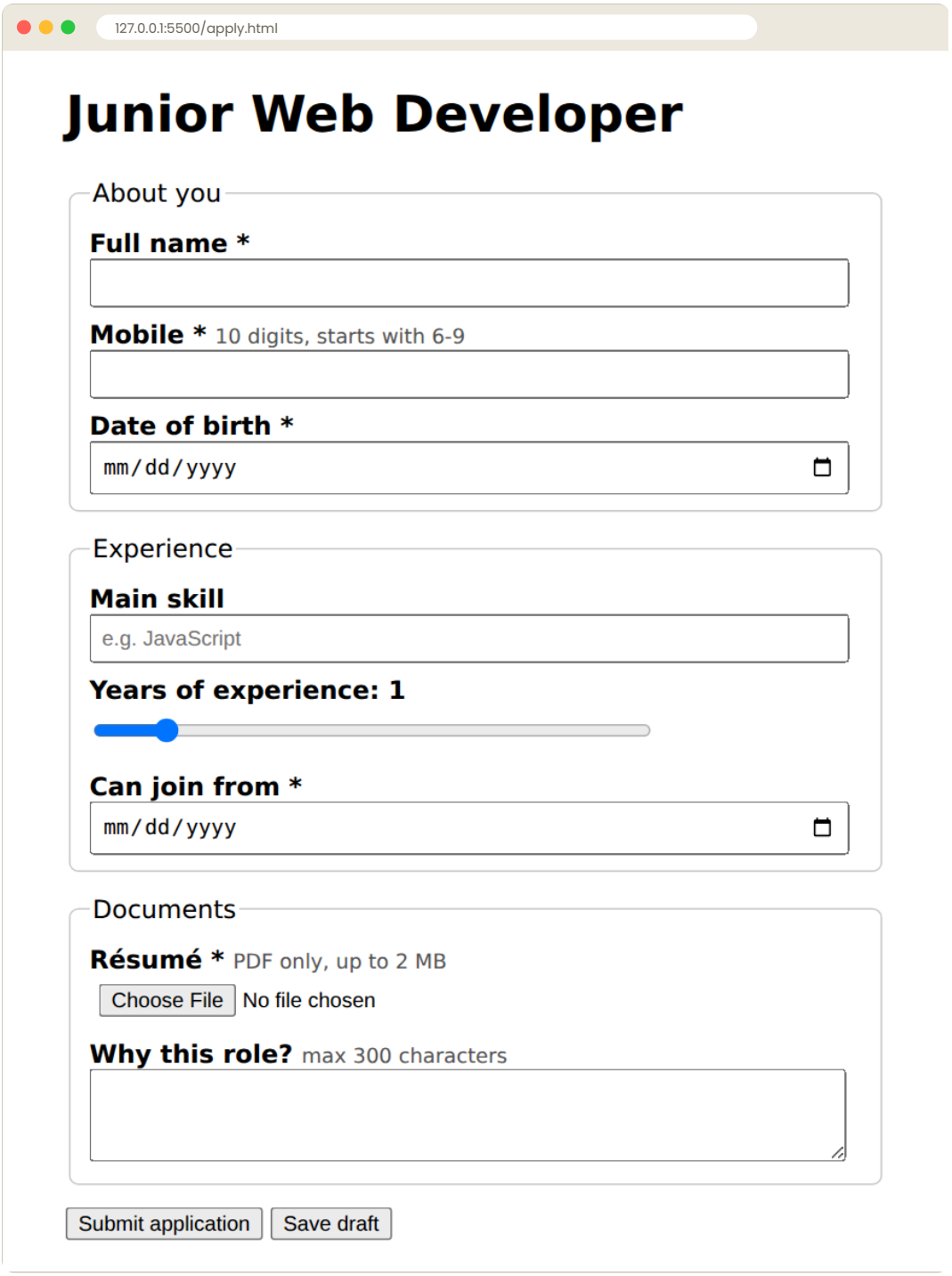
PRACTICE CHECKPOINT

A **job application form** using date, range, datalist, a pattern-validated phone number, required fields and a file upload limited to PDFs.

APPLY.HTML: SAMPLE SOLUTION HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
  <meta charset="UTF-8">
  <title>Apply: Junior Web Developer</title>
  <style>
    body { font-family: system-ui, sans-serif; max-width: 520px; margin: 20px auto; }
    fieldset { border: 1px solid #ccc; border-radius: 6px; margin-bottom: 14px; }
    label { display: block; margin-top: 8px; font-weight: 600; }
    .hint { font-weight: 400; color: #555; font-size: 13px; }
    input, select, textarea { padding: 6px; width: 95%; }
    input[type=range] { width: 70%; }
    input:user-invalid { border: 2px solid #c0392b; }
  </style>
</head>
<body>
  <h1>Junior Web Developer</h1>
  <form action="/careers/apply" method="post" enctype="multipart/form-data">
    <fieldset>
      <legend>About you</legend>
      <label for="name">Full name *</label>
      <input id="name" name="name" autocomplete="name" required>
      <label for="phone">Mobile * <span class="hint" id="ph">10 digits, starts with 6-9</span></label>
      <input id="phone" name="phone" type="tel" inputmode="numeric" pattern="[6-9][0-9]{9}"
             aria-describedby="ph" autocomplete="tel" required>
      <label for="dob">Date of birth *</label>
      <input id="dob" name="dob" type="date" min="1990-01-01" max="2008-12-31" required>
    </fieldset>
    <fieldset>
      <legend>Experience</legend>
      <label for="skill">Main skill</label>
      <input id="skill" name="skill" list="skills" placeholder="e.g. JavaScript">
      <datalist id="skills">
        <option value="HTML and CSS"></option><option value="JavaScript"></option>
        <option value="React"></option><option value="Java"></option><option value="Python"></option>
      </datalist>
      <label for="years">Years of experience: <output id="yo" for="years">1</output></label>
      <input id="years" name="years" type="range" min="0" max="10" value="1"
             oninput="yo.value = this.value">
      <label for="start">Can join from *</label>
      <input id="start" name="start" type="date" min="2026-10-15" required>
    </fieldset>
    <fieldset>
      <legend>Documents</legend>
      <label for="cv">Résumé * <span class="hint">PDF only, up to 2 MB</span></label>
      <input id="cv" name="cv" type="file" accept="application/pdf,.pdf" required>
      <label for="why">Why this role? <span class="hint">max 300 characters</span></label>
      <textarea id="why" name="why" rows="3" maxlength="300"></textarea>
    </fieldset>
    <button type="submit">Submit application</button>
    <button type="submit" formnovalidate name="draft" value="1">Save draft</button>
  </form>
</body>
</html>
```



THE JOB APPLICATION FORM

MODULE 09 · INTERMEDIATE

Semantic HTML5 and page structure

Minimum duration **2 days** | Level **Intermediate**

Semantic elements describe the meaning of content, which helps accessibility, SEO and maintainability. This module explains meaning versus appearance, who benefits from semantics, and the difference between "div soup" and semantic markup (shown with the real accessibility tree that Chromium builds). It then covers every sectioning element (header, footer, nav, main, article, section, aside and the new search element), how to choose between article, section and div, headings inside sections, three common page layouts (blog, landing page, dashboard) and structured data with Schema.org JSON-LD.

IN THIS MODULE

9.1 Why semantics matter
9.2 Sectioning elements
9.3 Choosing the right element
9.4 Common page layouts
9.5 Microdata and structured data

9.1 Why semantics matter

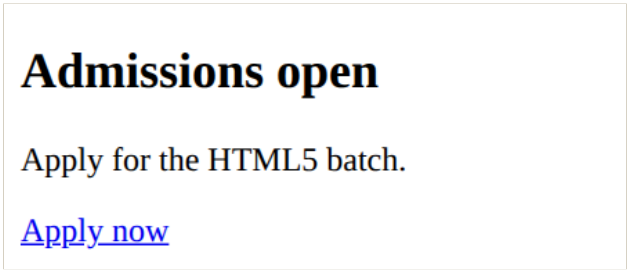
9.1.1 Meaning vs appearance

HTML describes **what content is**; CSS decides **how it looks**. A `<div>` and a `<nav>` look identical without CSS (both are plain blocks), but only `<nav>` tells the browser, search engines and assistive technology "this is the main navigation". Choosing elements by meaning rather than by default appearance is called **semantic markup**. A big bold line that is a heading must be an `<h2>`, not a `<div class="big-bold">`; a list of links in the header is a `<nav>` with a `<ul>`; a clickable thing that does something is a `<button>`, not a `<div onclick>`.

Divs styled to look like a heading and a link



A real heading, paragraph and link



THEY LOOK ALMOST THE SAME, BUT ONLY THE RIGHT-HAND VERSION IS A HEADING AND A WORKING, FOCUSABLE LINK

9.1.2 Benefits for screen readers, search engines and developers

Who	What semantics give them
Screen-reader users	Jump between **landmarks** (banner, navigation, main, complementary, contentinfo) and headings with one key; hear "navigation, list, 5 items"; skip repeated menus.

Who	What semantics give them
Keyboard users	Native links, buttons and form controls are focusable and work with Enter/Space without extra code.
Search engines	Understand the main content (`<main>` , `<article>`), headings and dates (`<time>`), and ignore boilerplate in nav and footer.
Browsers and tools	Reader mode, translation, voice assistants and "save to read later" extract the article correctly.
Developers	Readable code: `</article>` says what closes; CSS selectors like `main > article` are self-documenting.

9.1.3 "Div soup" vs semantic markup

"Div soup" is a page built entirely from `<div>` and `<span>` with class names. It can look perfect, but the browser sees only anonymous boxes. The two blocks below build the same page; underneath each is the **accessibility tree** that Chromium actually builds from it, captured with Playwright. This tree is what screen readers read.

DIV SOUP AND ITS ACCESSIBILITY TREE index.html

```
<div class="header">
  <div class="logo">Learn Being Forward</div>
  <div class="menu">
    <div><span onclick="go('/')">Home</span></div>
    <div><span onclick="go('/courses')">Courses</span></div>
  </div>
</div>
<div class="content">
  <div class="title">Why learn HTML5?</div>
  <div class="text">Every website starts with HTML.</div>
</div>
<div class="footer">&copy; 2026 LBF</div>
```

ACCESSIBILITY TREE (CHROMIUM)

```
- text: Learn Being Forward Home Courses Why learn HTML5? Every website starts with HTML. © 2026 LBF
```

SEMANTIC MARKUP AND ITS ACCESSIBILITY TREE

index.html

```
<header>
  <p>Learn Being Forward</p>
  <nav aria-label="Main">
    <ul>
      <li><a href="/">Home</a></li>
      <li><a href="/courses">Courses</a></li>
    </ul>
  </nav>
</header>
<main>
  <h1>Why learn HTML5?</h1>
  <p>Every website starts with HTML.</p>
</main>
<footer>&copy; 2026 LBF</footer>
```

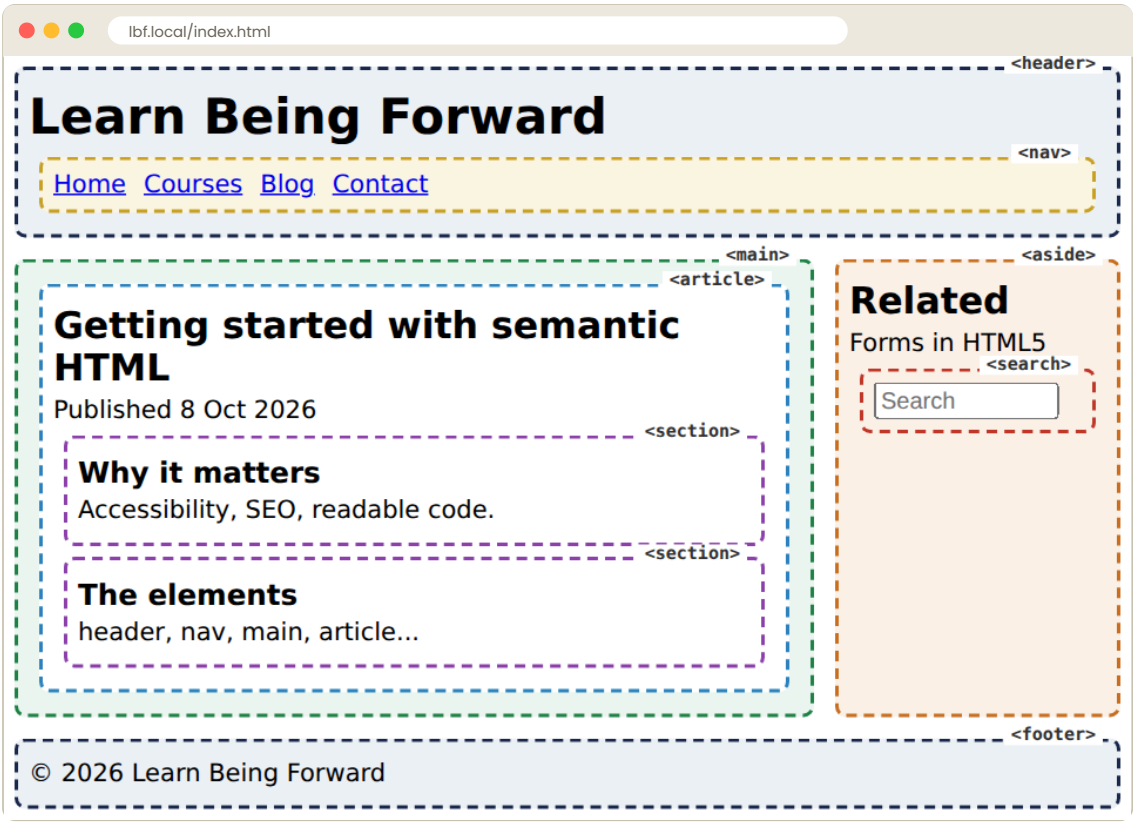
ACCESSIBILITY TREE (CHROMIUM)

```
- banner:
  - paragraph: Learn Being Forward
  - navigation "Main":
    - list:
      - listitem:
        - link "Home":
          - /url: /
      - listitem:
        - link "Courses":
          - /url: /courses
- main:
  - heading "Why learn HTML5?" [level=1]
  - paragraph: Every website starts with HTML.
- contentinfo: © 2026 LBF
```

The div version gives only text: no heading to jump to, no navigation landmark, and the "links" cannot be reached by Tab or announced as links. The semantic version produces a **banner**, a **navigation** with a **list** of real **links**, a **main** landmark with a **heading level 1** and a **contentinfo** footer, all without a single ARIA attribute.

9.2 Sectioning elements

HTML5 added a family of elements that describe the regions of a page. The figure shows a typical page with each element outlined and labelled; the sections below explain each one.



A TYPICAL PAGE WITH EVERY SECTIONING ELEMENT OUTLINED AND LABELLED

9.2.1 <header> and <footer>

`<header>` holds **introductory content**: a logo, site name, heading, navigation or search. `<footer>` holds **information about its section**: copyright, contact details, related links, author or date. Both can appear **many times**: once for the whole page and again inside each article (an article's header with title and byline, its footer with tags). Only the page-level ones (direct children of body, not inside article, section, main, aside or nav) become the **banner** and **contentinfo** landmarks.

PAGE-LEVEL AND ARTICLE-LEVEL HEADER/FOOTER HTML

```
<body>
  <header>                        <!-- page header: banner landmark -->
    <a href="/"><img src="logo.svg" alt="Learn Being Forward home"></a>
    <nav>...</nav>
  </header>
  <main>
    <article>
      <header>                    <!-- article header: not a landmark -->
        <h1>Semantic HTML in 10 minutes</h1>
        <p>By Asha Rao, <time datetime="2026-10-08">8 October 2026</time></p>
      </header>
      <p>...</p>
      <footer>Tags: <a href="/tag/html">html</a></footer>
    </article>
  </main>
  <footer>&copy; 2026 Learn Being Forward &middot; <a href="/privacy">Privacy</a></footer>
</body>
```

WATCH OUT

A `<header>` cannot contain another header or footer, and neither can be placed inside an `<address>`. And `<header>` is not the same as `<head>`: head holds metadata and is never displayed.

9.2.2 <nav>

`<nav>` marks a block of **major navigation links**: the main menu, a table of contents, breadcrumbs, pagination. Not every group of links needs it; a few links in the footer usually do not. When a page has more than one nav, give each an accessible name with `aria-label` so a screen-reader user hears "Main navigation" and "Breadcrumb navigation".

TWO NAMED NAVIGATION LANDMARKS HTML

```
<nav aria-label="Main">
  <ul>
    <li><a href="/" aria-current="page">Home</a></li>
    <li><a href="/courses">Courses</a></li>
    <li><a href="/contact">Contact</a></li>
  </ul>
</nav>

<nav aria-label="Breadcrumb">
  <ol>
    <li><a href="/">Home</a></li>
    <li><a href="/courses">Courses</a></li>
    <li><a href="/courses/html5" aria-current="page">HTML5</a></li>
  </ol>
</nav>
```

9.2.3 <main>

`<main>` wraps the **dominant, unique content** of the page: everything except repeated site-wide parts (header, navigation, sidebar, footer). There must be **only one visible main** per page, and it must not be inside article, aside, footer, header or nav. Skip links (Module 12) jump to it, and "reader mode" uses it.

9.2.4 <article>

`<article>` is a **self-contained** composition that would make sense on its own if syndicated or shared: a blog post, news story, forum post, product card, comment, review or course card. The test: *could this be published elsewhere (in an RSS feed, on another site) and still make sense?* Articles can nest: comments are articles inside a blog-post article.

9.2.5 <section>

`<section>` is a **thematic grouping** of content, normally with a heading: the chapters of an article, the "Features", "Pricing" and "FAQ" parts of a landing page, the tabs of a settings page. If you cannot think of a heading for it, it is probably not a section; use a div. A section becomes a **region** landmark only when it has an accessible name (`aria-labelledby` pointing to its heading, or `aria-label`).

9.2.6 <aside>

`<aside>` holds content **tangentially related** to the content around it: a sidebar of related posts, a glossary box, a pull quote, an advert, author bio. If removing it leaves the main content complete, it is an aside. Page-level asides become the **complementary** landmark.

9.2.7 <search>

`<search>` (added to HTML in 2023, supported in all major browsers) wraps the form or controls used to **search or filter** the site or page. It produces the **search** landmark that previously needed `<form role="search">`. It is not for showing search results.

THE SEARCH ELEMENT

HTML

```
<search>
  <form action="/courses">
    <label for="q">Search courses</label>
    <input type="search" id="q" name="q">
    <button>Search</button>
  </form>
</search>
```

A summary of the landmarks these elements create, as seen in the accessibility tree:

Element	Landmark role	Condition
`<header>`	banner	not inside article/aside/main/nav/section
`<nav>`	navigation	always
`<main>`	main	always (one per page)
`<aside>`	complementary	not inside article/aside/main/nav/section (or when named)
`<footer>`	contentinfo	not inside article/aside/main/nav/section
`<section>`	region	only with an accessible name
`<form>`	form	only with an accessible name
`<search>`	search	always
`<article>`	article (not a landmark)	always

EVERY LANDMARK IN ONE PAGE

index.html

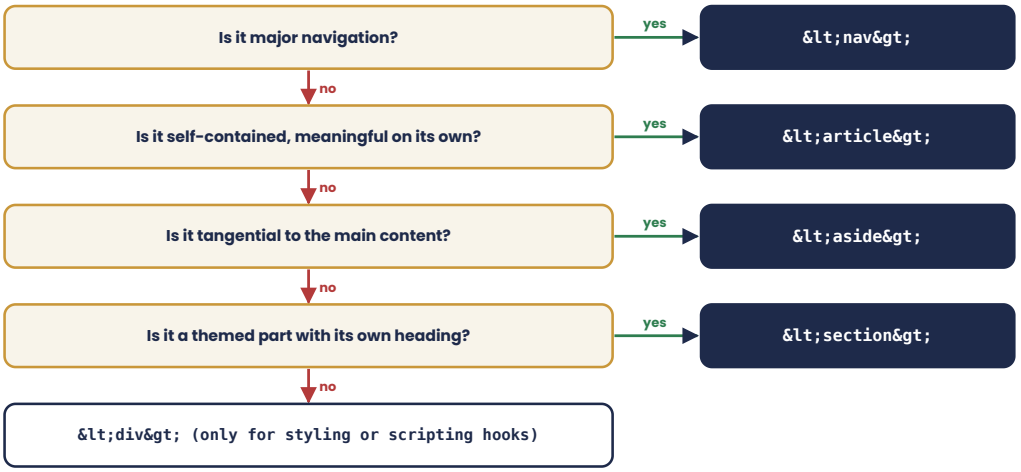
```
<header><p>LBF</p></header>
<nav aria-label="Main"><a href="/">Home</a></nav>
<search><input type="search" aria-label="Search courses"></search>
<main>
  <article><h1>Post title</h1><p>Body</p></article>
  <section aria-labelledby="faq"><h2 id="faq">FAQ</h2></section>
  <section><h2>Unnamed section</h2></section>
</main>
<aside><p>Related links</p></aside>
<footer><p>&copy; 2026</p></footer>
```

ACCESSIBILITY TREE (CHROMIUM)

```
- banner:
  - paragraph: LBF
- navigation "Main":
  - link "Home":
    - /url: /
- search:
  - searchbox "Search courses"
- main:
  - article:
    - heading "Post title" [level=1]
    - paragraph: Body
  - region "FAQ":
    - heading "FAQ" [level=2]
  - heading "Unnamed section" [level=2]
- complementary:
  - paragraph: Related links
- contentinfo:
  - paragraph: © 2026
```

9.3 Choosing the right element

9.3.1 article vs section vs div



CHOOSING A SECTIONING ELEMENT

Content	Best element	Why
Blog post on the home page	article	could appear in an RSS feed by itself
"Our Courses" block on a landing page	section	a themed part of the page with a heading

Content	Best element	Why
Each course card inside that block	article	a self-contained card with title, price and link
Wrapper used only to make a 3-column grid	div	no meaning, only layout
User comment	article (nested)	self-contained and has its own author/date
"Related posts" sidebar	aside	related but not part of the main content
Chapter of a long tutorial	section	a part of the article with its own heading

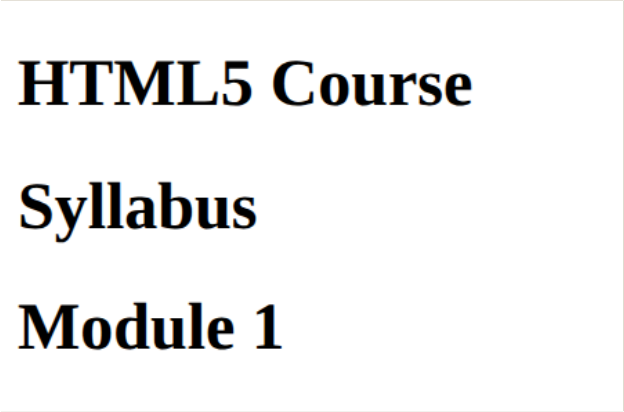
NOTE

`<div>` is not bad: it is the right choice for pure layout wrappers. The rule is "use a semantic element when one fits, a div when none does", not "never use div".

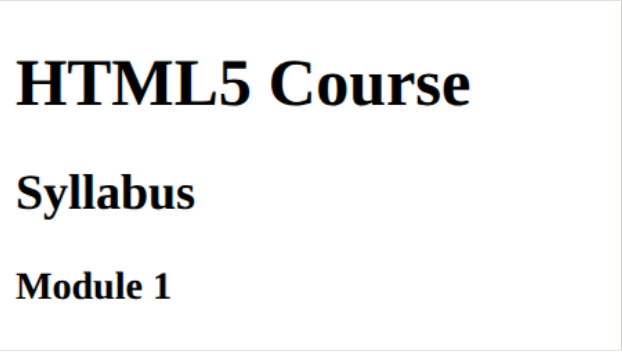
9.3.2 Headings inside sections

Every section and article should normally begin with a heading. HTML once proposed an "outline algorithm" in which each section could restart at `<h1>`, but **no browser or screen reader ever implemented it** and it was removed from the standard in 2022. Heading levels therefore mean exactly what they say: use **one h1** for the page topic and nest h2, h3… according to the real structure, without skipping levels.

Restarting at h1 in every section (avoid)



Real levels: h1 > h2 > h3



SCREEN READERS BUILD THEIR HEADING LIST FROM THE LEVELS WRITTEN, NOT FROM NESTING

9.4 Common page layouts

Semantic structure is independent of visual layout: the same elements can be arranged in columns, stacked on a phone, or reordered with CSS (CSS Flexbox and Grid are covered in the CSS3 course). Three structures cover most sites.

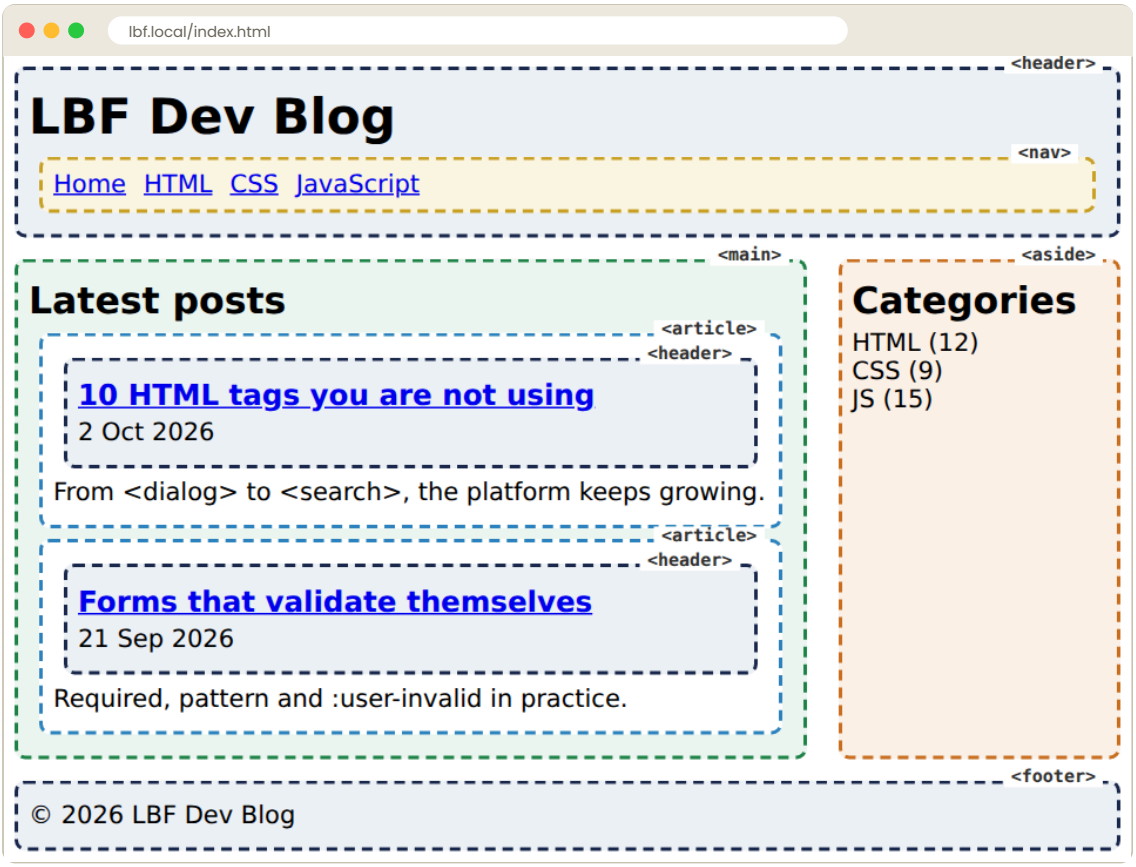
9.4.1 Blog layout

header	logo + nav (banner, navigation)
main	h1 "Latest posts" + list of article cards
article x n	header (h2, time) + summary + footer (tags, read more)
aside	about the author, categories, search
footer	copyright, social links (contentinfo)

BLOG LAYOUT STRUCTURE

BLOG.HTML HTML

```
<header>
  <h1>LBF Dev Blog</h1>
  <nav aria-label="Main"><a href="#">Home</a><a href="#">HTML</a><a href="#">CSS</a><a
href="#">JavaScript</a></nav>
</header>
<div style="display:flex;gap:6px">
  <main style="flex:3">
    <h2>Latest posts</h2>
    <article>
      <header><h3><a href="#">10 HTML tags you are not using</a></h3><p><time datetime="2026-10-02">2
Oct 2026</time></p></header>
      <p>From &lt;dialog&gt; to &lt;search&gt;, the platform keeps growing.</p>
    </article>
    <article>
      <header><h3><a href="#">Forms that validate themselves</a></h3><p><time datetime="2026-09-21">21
Sep 2026</time></p></header>
      <p>Required, pattern and :user-invalid in practice.</p>
    </article>
  </main>
  <aside style="flex:1"><h2>Categories</h2><p>HTML (12)<br>CSS (9)<br>JS (15)</p></aside>
</div>
<footer><p>&copy; 2026 LBF Dev Blog</p></footer>
```



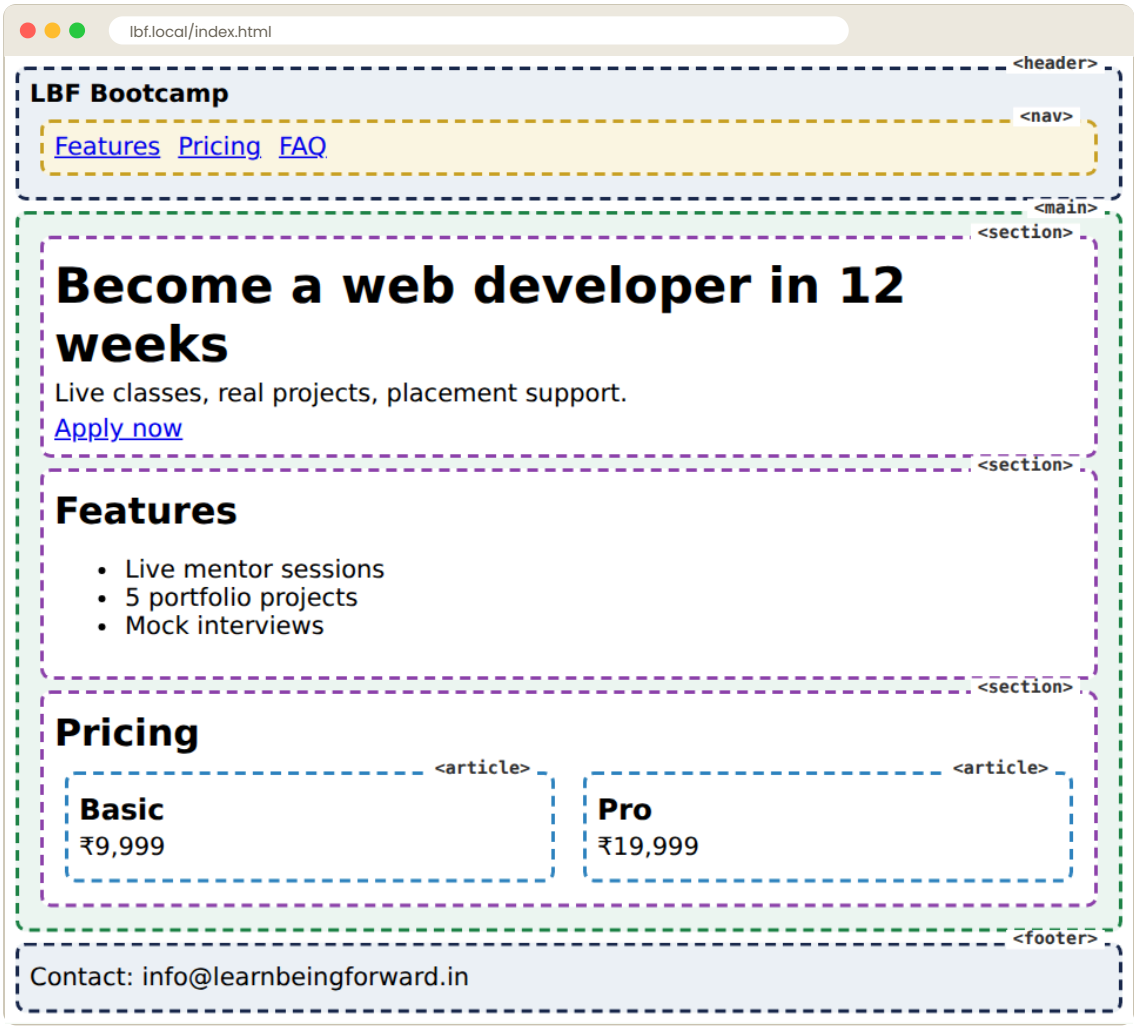
A SEMANTIC BLOG HOME PAGE (OUTLINES ADDED WITH CSS TO SHOW THE ELEMENTS)

9.4.2 Landing page layout

A landing page sells one thing: a header, a **hero** section with a headline and call to action, then themed sections (features, testimonials, pricing, FAQ) and a footer. Each themed part is a `<section>` with an h2; repeated cards inside (features, plans) are articles or list items.

LANDING.HTML (STRUCTURE) HTML

```
<header><strong>LBF Bootcamp</strong> <nav><a href="#features">Features</a><a
href="#pricing">Pricing</a><a href="#faq">FAQ</a></nav></header>
<main>
  <section aria-labelledby="hero"><h1 id="hero">Become a web developer in 12 weeks</h1><p>Live
classes, real projects, placement support.</p><a href="#">Apply now</a></section>
  <section id="features" aria-labelledby="f"><h2 id="f">Features</h2>
    <ul><li>Live mentor sessions</li><li>5 portfolio projects</li><li>Mock interviews</li></ul>
</section>
  <section id="pricing" aria-labelledby="pr"><h2 id="pr">Pricing</h2>
    <div style="display:flex;gap:4px"><article style="flex:1"><h3>Basic</h3><p>&#8377;9,999</p>
</article><article style="flex:1"><h3>Pro</h3><p>&#8377;19,999</p></article></div></section>
</main>
<footer><p>Contact: info@learnbeingforward.in</p></footer>
```



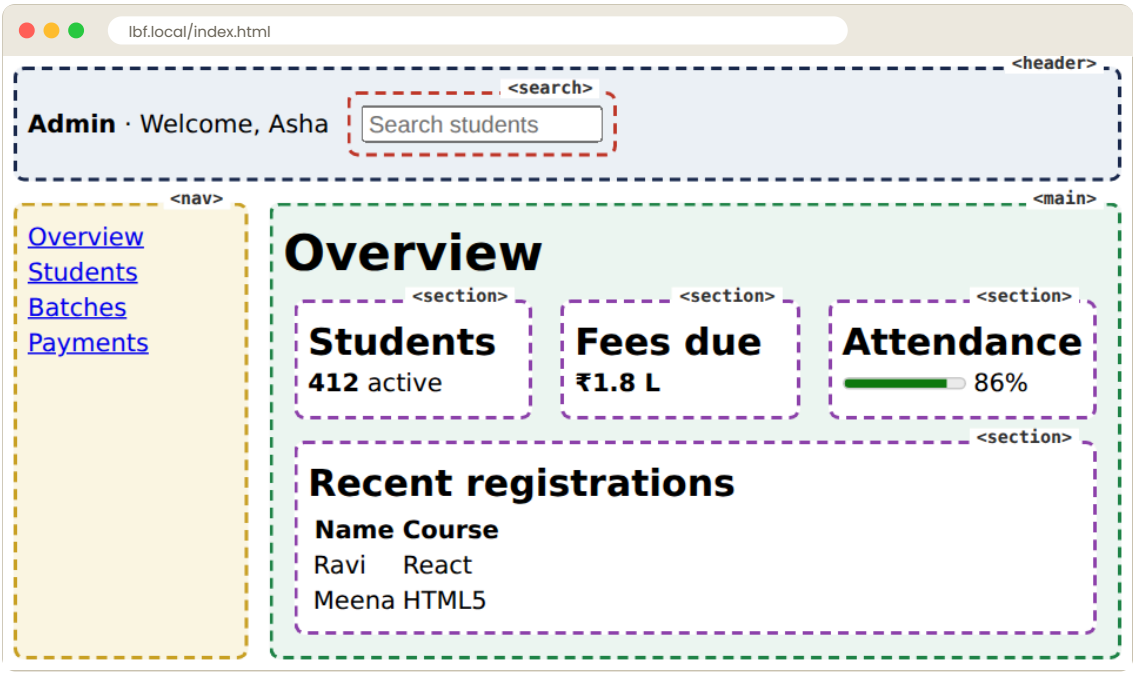
LANDING PAGE: HERO, FEATURES AND PRICING SECTIONS

9.4.3 Dashboard layout

A web app dashboard usually has a top bar (header), a side navigation (nav), and a main area containing **sections** of widgets (cards, charts, tables). Each widget is a section with a heading so screen-reader users can jump between them.

DASHBOARD.HTML (STRUCTURE) HTML

```
<header><strong>Admin</strong> &middot; Welcome, Asha <search style="display:inline-block"><input
type="search" placeholder="Search students" size="14"></search></header>
<div style="display:flex">
  <nav aria-label="Dashboard" style="flex:1"><p><a href="#">Overview</a></p><p><a
href="#">Students</a></p><p><a href="#">Batches</a></p><p><a href="#">Payments</a></p></nav>
  <main style="flex:4"><h1>Overview</h1>
    <div style="display:flex;gap:4px">
      <section style="flex:1"><h2>Students</h2><p><b>412</b> active</p></section>
      <section style="flex:1"><h2>Fees due</h2><p><b>&#8377;1.8 L</b></p></section>
      <section style="flex:1"><h2>Attendance</h2><p><meter value="0.86">86%</meter> 86%</p></section>
    </div>
    <section><h2>Recent registrations</h2>
      <table><tr><th>Name</th><th>Course</th></tr><tr><td>Ravi</td><td>React</td></tr><tr>
<td>Meena</td><td>HTML5</td></tr></table></section>
  </main>
</div>
```



DASHBOARD: HEADER, SIDE NAV AND MAIN WITH WIDGET SECTIONS

9.5 Microdata and structured data

9.5.1 Schema.org with JSON-LD (overview)

Semantic elements say "this is an article"; **structured data** says much more: "this is a Course called HTML5, offered by Learn Being Forward, 28 days long, costing 9,999 rupees". Search engines use it for **rich results** (star ratings, prices, event dates, FAQs, breadcrumbs in search listings). The shared vocabulary is **Schema.org** (types like Person, Organization, Course, Event, Product, Article, FAQPage, BreadcrumbList). There are three syntaxes: **JSON-LD** (a script block, recommended by Google), **Microdata** (`itemscope`, `itemtype`, `itemprop` attributes on the visible HTML) and RDFa.

JSON-LD FOR A COURSE (IN THE HEAD OR BODY) HTML

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Course",
  "name": "HTML5 Web Foundations",
  "description": "Complete HTML5 course from basics to APIs and accessibility.",
  "provider": {
    "@type": "Organization",
    "name": "Learn Being Forward",
    "sameAs": "https://learnbeingforward.in"
  },
  "offers": { "@type": "Offer", "price": "9999", "priceCurrency": "INR", "category": "Paid" },
  "hasCourseInstance": { "@type": "CourseInstance", "courseMode": "Blended", "courseWorkload": "P28D"
}
}
</script>
```

The same information in **Microdata**, attached to the visible content:

MICRODATA VERSION HTML

```html
<article itemscope itemtype="https://schema.org/Course">
  <h2 itemprop="name">HTML5 Web Foundations</h2>
  <p itemprop="description">Complete HTML5 course from basics to APIs.</p>
  <p>By <span itemprop="provider" itemscope itemtype="https://schema.org/Organization">
       <span itemprop="name">Learn Being Forward</span></span></p>
</article>
```

A browser ignores JSON-LD for display but it is part of the DOM, so a script (or a search engine crawler) can read it. This script really parses the JSON-LD block in the page, the same way a crawler would:

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```html
<script type="application/ld+json">
{"@context":"https://schema.org","@type":"Course","name":"HTML5 Web Foundations",
 "provider":{"@type":"Organization","name":"Learn Being Forward"},
 "offers":{"@type":"Offer","price":"9999","priceCurrency":"INR"}}
</script>
<h1>HTML5 Web Foundations</h1>
```

READING STRUCTURED DATA LIKE A CRAWLER main.js

```javascript
const block = document.querySelector('script[type="application/ld+json"]');
const data = JSON.parse(block.textContent);
console.log("Type:    ", data["@type"]);
console.log("Name:    ", data.name);
console.log("Provider:", data.provider.name);
console.log("Price:   ", data.offers.priceCurrency, data.offers.price);
```

BROWSER CONSOLE

```
Type:     Course
Name:     HTML5 Web Foundations
Provider: Learn Being Forward
Price:    INR 9999
```

TIP

Test structured data with Google's **Rich Results Test** (search.google.com/test/rich-results) and the **Schema Markup Validator** (validator.schema.org). Structured data must describe content that is actually visible on the page.

PRACTICE CHECKPOINT

Rebuild a **blog homepage using only semantic elements**, then compare its outline with a div-only version in the browser's accessibility tree (DevTools > Elements > Accessibility pane, or the "Full accessibility tree" toggle).

SAMPLE SOLUTION: THE SEMANTIC BLOG AND THE TREE CHROMIUM BUILDS

index.html

```
<header>
  <a href="/">LBF Dev Blog</a>
  <nav aria-label="Main">
    <ul><li><a href="/">Home</a></li><li><a href="/about">About</a></li></ul>
  </nav>
  <search><form><label>Search posts <input type="search" name="q"></label></form></search>
</header>
<main>
  <h1>Latest posts</h1>
  <article>
    <h2><a href="/posts/semantic-html">Semantic HTML in 10 minutes</a></h2>
    <p><time datetime="2026-10-08">8 Oct 2026</time></p>
    <p>Why header, nav and main matter.</p>
  </article>
  <article>
    <h2><a href="/posts/forms">Forms that validate themselves</a></h2>
    <p><time datetime="2026-09-21">21 Sep 2026</time></p>
  </article>
</main>
<aside aria-label="About the author"><h2>About</h2><p>Asha writes about the web.</p></aside>
<footer><p>&copy; 2026 LBF Dev Blog</p></footer>
```

ACCESSIBILITY TREE (CHROMIUM)

```
- banner:
  - link "LBF Dev Blog":
    - /url: /
  - navigation "Main":
    - list:
      - listitem:
        - link "Home":
          - /url: /
      - listitem:
        - link "About":
          - /url: /about
  - search:
    - text: Search posts
    - searchbox "Search posts"
- main:
  - heading "Latest posts" [level=1]
  - article:
    - heading "Semantic HTML in 10 minutes" [level=2]:
      - link "Semantic HTML in 10 minutes":
        - /url: /posts/semantic-html
    - paragraph:
      - time: 8 Oct 2026
    - paragraph: Why header, nav and main matter.
  - article:
    - heading "Forms that validate themselves" [level=2]:
      - link "Forms that validate themselves":
        - /url: /posts/forms
    - paragraph:
      - time: 21 Sep 2026
- complementary "About the author":
  - heading "About" [level=2]
  - paragraph: Asha writes about the web.
- contentinfo:
  - paragraph: © 2026 LBF Dev Blog
```

Compare it with the div-soup tree in 9.1.3: the semantic page exposes banner, navigation, search, main, two articles with h2 headings, a named complementary region and contentinfo. A screen-reader user

can now press **D** (NVDA) or use the rotor (VoiceOver) to jump between these landmarks, and **H** to jump between headings.

MODULE 10 · INTERMEDIATE

Multimedia and embedded content

Minimum duration **2 days** | Level **Intermediate**

HTML5 made audio, video and graphics native to the browser, without plug-ins such as Flash. This module covers the audio element and its attributes, multiple source formats, the video element with poster and playsinline, captions and subtitles with track and WebVTT, embedding external content with iframe (YouTube, Google Maps) and its security and performance attributes, embed and object, inline SVG (shapes, paths, text, viewBox), SVG as an image versus inline SVG, the canvas element, and media accessibility: captions, transcripts and audio descriptions. Every player shown here is a real screenshot of Chromium playing a real media file.

IN THIS MODULE

10.1 Audio
10.2 Video
10.3 Embedding external content
10.4 Graphics
10.5 Media accessibility

10.1 Audio

10.1.1 <audio> with controls, autoplay, loop, muted, preload

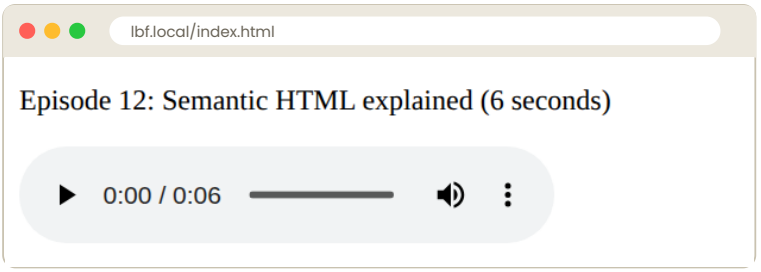
`<audio>` plays sound. Without the `controls` attribute it is **invisible** (useful only when JavaScript drives it); with `controls` the browser draws its own player with play/pause, a timeline, volume and a menu. All the attributes are boolean except `preload`.

Attribute	Effect
`controls`	show the browser's player UI
`autoplay`	start playing as soon as possible. Browsers **block** autoplay with sound until the user has interacted with the site, so do not rely on it.
`loop`	restart from the beginning when it ends
`muted`	start with the sound off
`preload`	`none` (download nothing until play), `metadata` (only duration and size; the usual choice), `auto` (the browser may download the whole file)
`src`	the file URL (or use child `<source>` elements)

A PODCAST PLAYER

HTML

```
<p>Episode 12: Semantic HTML explained (6 seconds)</p>
<audio controls preload="metadata" src="podcast.mp3">
  Your browser does not support audio. <a href="podcast.mp3">Download the MP3</a>.
</audio>
```



THE DEFAULT CHROMIUM AUDIO PLAYER AFTER LOADING THE FILE'S METADATA

The text inside `<audio>` is **fallback content**, shown only by browsers that do not support the element. Today a download link is still a good fallback.

10.1.2 Multiple <source> formats

Different browsers once supported different codecs, so a media element can list several `<source>` children. The browser picks the **first one it can play**, using the `type` attribute to skip unsupported formats without downloading them. Put the most efficient format first.

TWO SOURCES WITH A FALLBACK HTML

```html
<audio controls preload="metadata">
  <source src="podcast.ogg" type="audio/ogg; codecs=opus">   <!-- small, high quality -->
  <source src="podcast.mp3" type="audio/mpeg">               <!-- plays everywhere -->
  <p>Download the episode as <a href="podcast.mp3">MP3</a>.</p>
</audio>
```

JavaScript can ask the browser the same question it asks itself. `canPlayType()` answers "probably", "maybe" or "" (no); this script ran in the Chromium used to build this book and also reports which source the audio element chose:

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```html
<audio id="p" preload="metadata">
  <source src="podcast.ogg" type="audio/ogg; codecs=opus">
  <source src="podcast.mp3" type="audio/mpeg">
</audio>
```

WHICH AUDIO FORMATS CAN THIS BROWSER PLAY? main.js

```js
const a = document.createElement("audio");
for (const t of ["audio/mpeg", "audio/ogg; codecs=opus", "audio/wav", "audio/aac", "audio/flac",
"audio/x-ms-wma"]) {
  console.log(t.padEnd(24), JSON.stringify(a.canPlayType(t)));
}
const player = document.querySelector("#p");
player.addEventListener("loadedmetadata", () => {
  console.log("chosen source:", player.currentSrc.split("/").pop());
});
```

BROWSER CONSOLE

```
audio/mpeg               "probably"
audio/ogg; codecs=opus   "probably"
audio/wav                "maybe"
audio/aac                ""
audio/flac               "probably"
audio/x-ms-wma           ""
chosen source: podcast.ogg
```

Format	Container / codec	Notes
MP3	audio/mpeg	universal; good default
AAC (M4A)	audio/mp4	universal; better quality than MP3 at the same size
Opus	audio/ogg or audio/webm	best quality per byte; all modern browsers
WAV	audio/wav	uncompressed, very large; short sound effects only
FLAC	audio/flac	lossless; large

10.2 Video

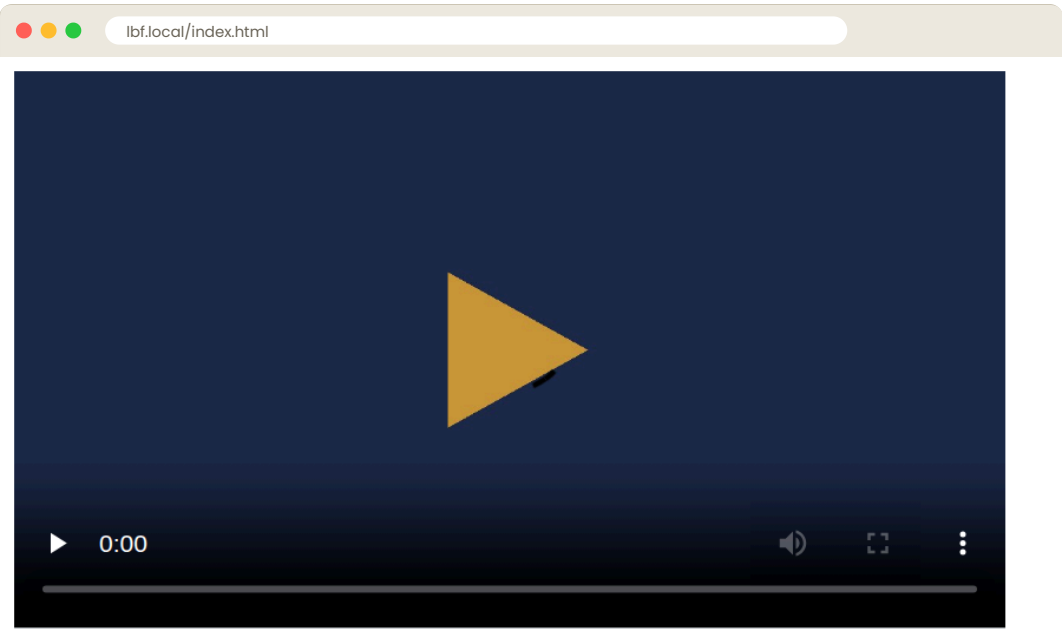
10.2.1 <video> attributes including poster and playsinline

`<video>` shares all the audio attributes and adds a few of its own. Always set `width` and `height` (or CSS aspect-ratio) so the page does not jump when the video loads.

Attribute	Effect
`poster`	image shown before playback starts (otherwise the first frame or a blank box)
`width`, `height`	display size in CSS pixels; reserves space and sets the aspect ratio
`playsinline`	on iPhones, play inside the page instead of jumping to full screen
`muted autoplay loop playsinline`	the combination allowed to autoplay everywhere: silent background or hero videos
`controlslist`	hide some controls, e.g. `controlslist="nodownload"` (Chromium)
`disablepictureinpicture`	hide the picture-in-picture option

A VIDEO WITH A POSTER IMAGE HTML

```
<video controls width="560" height="315" poster="poster.jpg" preload="none">
  <source src="intro.webm" type="video/webm">
  <p>Your browser cannot play this video. <a href="intro.webm">Download it</a>.</p>
</video>
```



BEFORE PLAYBACK: THE POSTER IS SHOWN WITH THE PLAY CONTROLS

BACKGROUND VIDEO PATTERN HTML

```html
<!-- A silent looping background video for a hero section -->
<video autoplay muted loop playsinline poster="hero.jpg" width="1280" height="720">
  <source src="hero.webm" type="video/webm">
  <source src="hero.mp4" type="video/mp4">
</video>
```

10.2.2 Multiple sources and formats (MP4, WebM)

MP4 with H.264 video and AAC audio plays in every browser and is the safe baseline. **WebM** with VP9 (or AV1) is open, royalty-free and usually smaller at the same quality. Offer WebM/AV1 first and MP4 as the fallback. Tools such as **FFmpeg** or HandBrake convert between them:

CONVERTING A RECORDING FOR THE WEB terminal

```terminal
ffmpeg -i lecture.mov -c:v libx264 -crf 23 -c:a aac -movflags +faststart lecture.mp4
ffmpeg -i lecture.mov -c:v libvpx-vp9 -crf 32 -b:v 0 -c:a libopus lecture.webm
```

TERMINAL

```
lecture.mp4    18.4 MB   (H.264 + AAC, starts playing before fully downloaded: +faststart)
lecture.webm   12.1 MB   (VP9 + Opus)
```

TIP

Do not host long videos yourself unless you must: YouTube or Vimeo handle adaptive streaming (lower quality on slow networks), global CDNs and captions for you. Self-host short clips, hero loops and private content.

10.2.3 Captions and subtitles with <track> and WebVTT

`<track>` adds timed text to a video from a **WebVTT** (.vtt) file. `kind` says what the text is: `captions` (dialogue **plus** important sounds, for deaf and hard-of-hearing viewers), `subtitles` (translation of dialogue), `descriptions` (text descriptions of visual content), `chapters` (navigation titles). `srclang` gives the language, `label` the name shown in the menu, and `default` turns one track on automatically.

CAPTIONS-EN.VTT text

```
WEBVTT

00:00:00.000 --> 00:00:03.000
Welcome to the HTML5 course at Learn Being Forward.

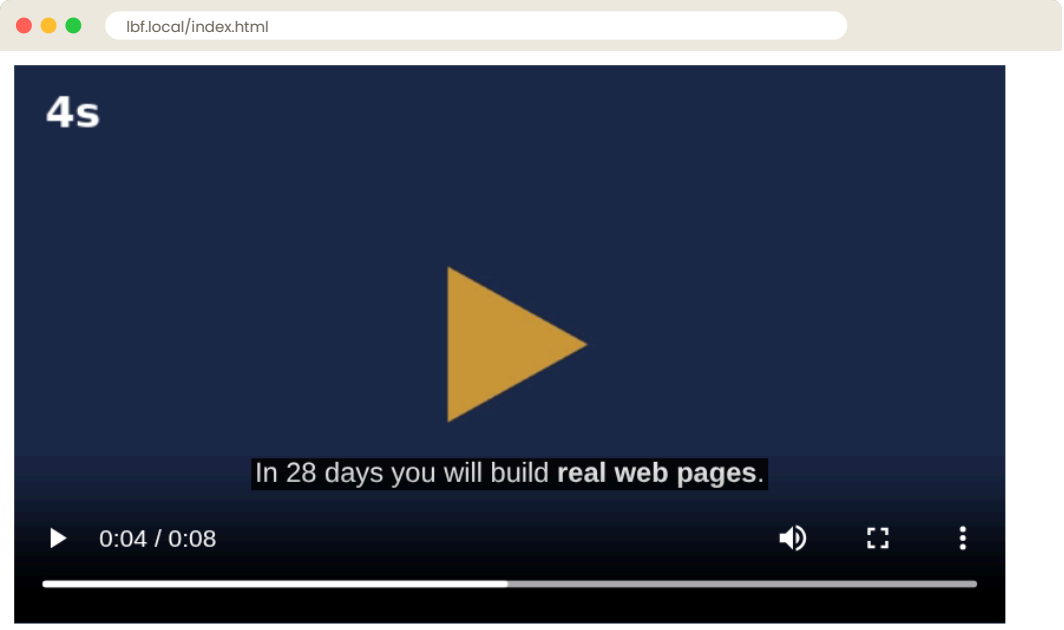
00:00:03.000 --> 00:00:06.000
In 28 days you will build <b>real web pages</b>.

00:00:06.000 --> 00:00:08.000
Let's begin!
```

A WebVTT file starts with the line `WEBVTT`, followed by **cues**: a time range `hh:mm:ss.mmm --> hh:mm:ss.mmm` and the text, separated by blank lines. Cues may contain simple tags such as `<b>`, `<i>` and `<v Speaker>`.

VIDEO WITH ENGLISH CAPTIONS AND KANNADA SUBTITLES HTML

```
<video controls width="560" height="315" poster="poster.jpg">
  <source src="intro.webm" type="video/webm">
  <track kind="captions" src="captions-en.vtt" srclang="en" label="English" default>
  <track kind="subtitles" src="subtitles-kn.vtt" srclang="kn" label="Kannada">
</video>
```



PAUSED AT 4 SECONDS: THE SECOND CAPTION CUE IS DISPLAYED OVER THE VIDEO

WATCH OUT

Tracks are fetched like any resource: a page opened from `file://` may refuse to load them in some browsers. Use a local server (Live Server) while developing. Track files on another domain also need CORS and the `crossorigin` attribute on the video.

10.3 Embedding external content

10.3.1 <iframe> for YouTube videos and Google Maps

`<iframe>` (inline frame) embeds **another complete web page** inside yours, in its own isolated window with its own document, scripts and styles. Services give you ready-made embed codes: on YouTube, **Share > Embed**; on Google Maps, **Share > Embed a map**. Always add a `title` and lazy loading.

TYPICAL EMBED CODES HTML

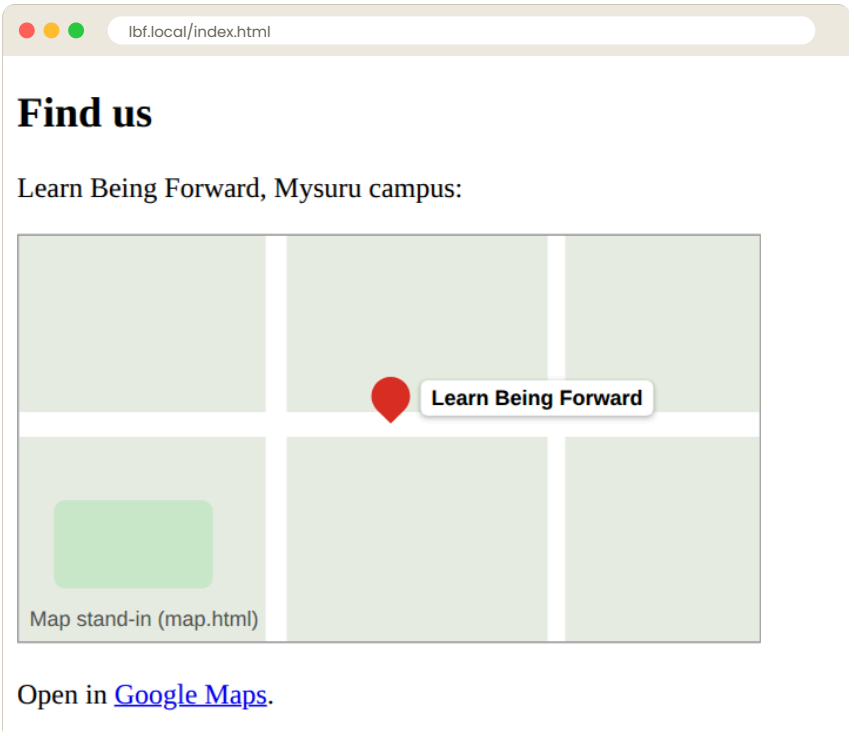
```
<!-- YouTube: use youtube-nocookie.com for privacy-enhanced mode -->
<iframe width="560" height="315"
        src="https://www.youtube-nocookie.com/embed/VIDEO_ID?start=30"
        title="HTML5 in 100 seconds (YouTube video)"
        loading="lazy"
        allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-
picture; web-share"
        referrerpolicy="strict-origin-when-cross-origin"
        allowfullscreen></iframe>

<!-- Google Maps -->
<iframe width="600" height="400"
        src="https://www.google.com/maps/embed?pb=!1m18!1m12!...(long code from Google)..."
        title="Map: Learn Being Forward, Mysuru"
        loading="lazy" style="border:0"
        referrerpolicy="no-referrer-when-downgrade"
        allowfullscreen></iframe>
```

The screenshot below shows the mechanics with a page from our own site standing in for the third-party content (the book is built offline): the outer page and the framed page are separate documents, each with its own styles.

AN IFRAME EMBEDDING ANOTHER PAGE HTML

```
<h2>Find us</h2>
<p>Learn Being Forward, Mysuru campus:</p>
<iframe src="map.html" width="420" height="230" title="Map: Learn Being Forward, Mysuru"
loading="lazy" style="border:1px solid #999"></iframe>
<p>Open in <a href="#">Google Maps</a>.</p>
```



THE FRAMED PAGE (A STAND-IN MAP) IS A SEPARATE DOCUMENT INSIDE THE OUTER PAGE

10.3.2 iframe attributes: title, loading, allow, sandbox, referrerpolicy

Attribute	Purpose	Example
`title`	accessible name; screen readers announce it when entering the frame (required for accessibility)	`title="Course intro video"`
`loading`	`lazy` defers loading until the frame is near the viewport; big savings for YouTube and maps	`loading="lazy"`
`allow`	permissions policy: which powerful features the frame may use	`allow="fullscreen; autoplay; geolocation"`
`sandbox`	applies strict restrictions (no scripts, forms, popups, same-origin access…); each token re-enables one thing	`sandbox="allow-scripts allow-forms"`
`referrerpolicy`	how much of your URL is sent to the framed site	`referrerpolicy="no-referrer"`
`srcdoc`	inline HTML for the frame instead of a URL	`srcdoc="<p>Hi</p>"`
`allowfullscreen`	legacy way to allow full screen	

`sandbox` is the most important security attribute. An empty `sandbox` blocks scripts entirely; this live test frames the same script twice, once sandboxed:

SANDBOX IN ACTION HTML

```
<p>Without sandbox:</p>
<iframe srcdoc="<p id=o>Scripts blocked</p><script>o.textContent='Script ran:
'+navigator.userAgent.split(' ')[0]</script>" height="50" width="380" title="Unsandboxed demo">
</iframe>
<p>With <code>sandbox</code> (empty):</p>
<iframe sandbox srcdoc="<p id=o>Scripts blocked</p><script>o.textContent='Script ran:
'+navigator.userAgent.split(' ')[0]</script>" height="50" width="380" title="Sandboxed demo"></iframe>
<p>With <code>sandbox="allow-scripts"</code>:</p>
<iframe sandbox="allow-scripts" srcdoc="<p id=o>Scripts blocked</p><script>o.textContent='Script ran:
'+navigator.userAgent.split(' ')[0]</script>" height="50" width="380" title="Scripts allowed demo">
</iframe>
```



THE EMPTY SANDBOX STOPPED THE SCRIPT; ALLOW-SCRIPTS RE-ENABLED IT

WATCH OUT

Never combine `allow-scripts` and `allow-same-origin` for content from your own origin: the framed script could then remove its own sandbox. And remember that some sites forbid being framed at all (with the `X-Frame-Options` or CSP `frame-ancestors` headers), so an iframe of google.com or your bank shows an error.

10.3.3 <embed> and <object>

`<embed>` and `<object>` were designed for plug-ins (Flash, Java applets), which no longer exist. Today their only common use is **showing a PDF** or an SVG inside the page with the browser's built-in viewer. `<object>` has fallback content between its tags; `<embed>` is a void element with none. For PDFs, an iframe or simply a link ("Download the syllabus (PDF, 240 KB)") works better on phones.

OBJECT AND EMBED FOR A PDF HTML

```
<object data="syllabus.pdf" type="application/pdf" width="600" height="500">
  <p>Your browser cannot show PDFs. <a href="syllabus.pdf">Download the syllabus (PDF, 240 KB)</a>.
</p>
</object>

<embed src="syllabus.pdf" type="application/pdf" width="600" height="500">
```

10.4 Graphics

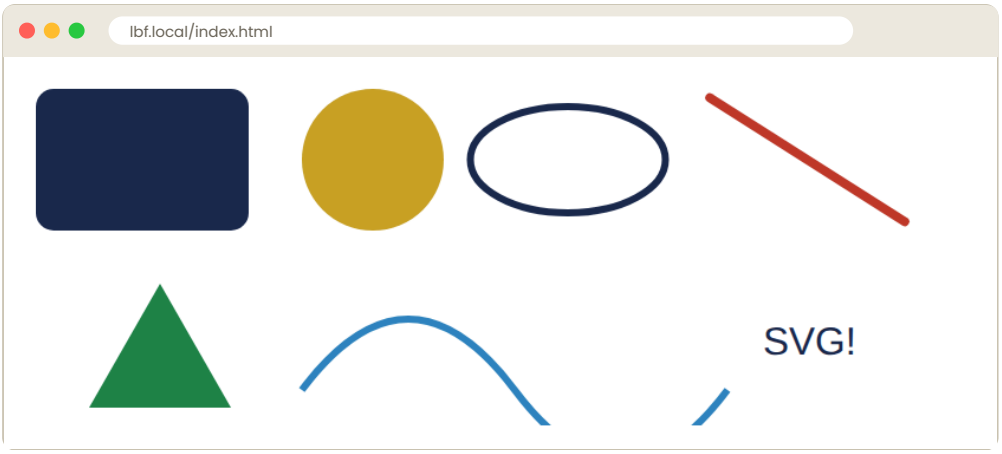
10.4.1 Inline SVG basics: shapes, paths, text, viewBox

SVG (Scalable Vector Graphics) describes images with XML elements: shapes, lines and text defined by coordinates. Being vectors, SVGs stay sharp at any size, are usually tiny for icons, logos and diagrams, and can be styled with CSS and changed with JavaScript. Every diagram in this book is SVG.

Element	Draws	Key attributes
`<rect>`	rectangle	x, y, width, height, rx (rounded corners)
`<circle>`	circle	cx, cy, r
`<ellipse>`	ellipse	cx, cy, rx, ry
`<line>`	straight line	x1, y1, x2, y2
`<polyline>` / `<polygon>`	connected lines / closed shape	points="x,y x,y ..."
`<path>`	any shape	d: M (move), L (line), H/V, C/Q (curves), A (arc), Z (close)
`<text>`	text	x, y, font-size, text-anchor
`<g>`	group	transform, shared fill/stroke
common	painting	fill, stroke, stroke-width, opacity

BASIC SVG SHAPES HTML

```html
<svg width="520" height="200" viewBox="0 0 520 200" xmlns="http://www.w3.org/2000/svg">
  <rect x="10" y="10" width="120" height="80" rx="10" fill="#1a2a4f"/>
  <circle cx="200" cy="50" r="40" fill="#c9a227"/>
  <ellipse cx="310" cy="50" rx="55" ry="30" fill="none" stroke="#1a2a4f" stroke-width="4"/>
  <line x1="390" y1="15" x2="500" y2="85" stroke="#c0392b" stroke-width="5" stroke-linecap="round"/>
  <polygon points="40,190 80,120 120,190" fill="#1e8449"/>
  <path d="M160 180 Q 220 100 280 180 T 400 180" fill="none" stroke="#2e86c1" stroke-width="4"/>
  <text x="420" y="160" font-family="sans-serif" font-size="22" fill="#1a2a4f">SVG!</text>
</svg>
```



RECT, CIRCLE, ELLIPSE, LINE, POLYGON, A CURVED PATH AND TEXT

The `viewBox="minX minY width height"` attribute defines the SVG's own **coordinate system**; `width` and `height` set the size on the page. The browser scales the viewBox to fit, so the same drawing can be shown at any size. Here one logo source is displayed at three sizes:

ONE VIEWBOX, MANY SIZES HTML

```
<svg viewBox="0 0 100 100" width="40"><rect width="100" height="100" rx="18" fill="#1a2a4f"/><path
d="M20 72 L50 22 L80 72 Z" fill="#c9a227"/><circle cx="50" cy="58" r="8" fill="#1a2a4f"/></svg>
<svg viewBox="0 0 100 100" width="80"><rect width="100" height="100" rx="18" fill="#1a2a4f"/><path
d="M20 72 L50 22 L80 72 Z" fill="#c9a227"/><circle cx="50" cy="58" r="8" fill="#1a2a4f"/></svg>
<svg viewBox="0 0 100 100" width="160"><rect width="100" height="100" rx="18" fill="#1a2a4f"/><path
d="M20 72 L50 22 L80 72 Z" fill="#c9a227"/><circle cx="50" cy="58" r="8" fill="#1a2a4f"/></svg>
<svg viewBox="25 15 50 50" width="160"><rect width="100" height="100" rx="18" fill="#1a2a4f"/><path
d="M20 72 L50 22 L80 72 Z" fill="#c9a227"/><circle cx="50" cy="58" r="8" fill="#1a2a4f"/></svg>
```



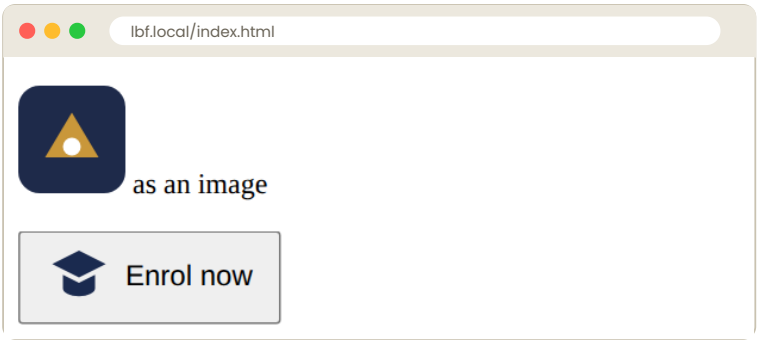
SIZES 40, 80 AND 160 PX ARE EQUALLY SHARP; THE LAST USES VIEWBOX="25 15 50 50" TO ZOOM INTO THE CENTRE

10.4.2 SVG as image vs inline SVG

	`<img src="logo.svg">`	Inline `<svg>...</svg>`
Caching	cached separately, reused across pages	part of the HTML; downloaded with each page
Style with page CSS	no	yes (fill, stroke, hover, dark mode, currentColor)
Script / animate parts	no	yes
Accessibility	`alt` text	`role="img"` plus `<title>` or `aria-label`
Best for	logos, illustrations used as images	icons that change colour, interactive charts, animated graphics

INLINE SVG ICON THAT FOLLOWS CSS COLOUR HTML

```
<style>
  .icon { width: 36px; height: 36px; color: #1a2a4f; }
  button:hover .icon { color: #c9a227; }
  button { font: 16px sans-serif; display: inline-flex; gap: 8px; align-items: center; padding: 6px
14px; }
</style>
<p><img src="logo.svg" width="60" alt="Learn Being Forward"> as an image</p>
<button>
  <svg class="icon" viewBox="0 0 24 24" role="img" aria-hidden="true"><path fill="currentColor" d="M12
2 2 7l10 5 10-5-10-5Zm-6 9v5c0 1.7 2.7 3 6 3s6-1.3 6-3v-5l-6 3-6-3Z"/></svg>
  Enrol now
</button>
```



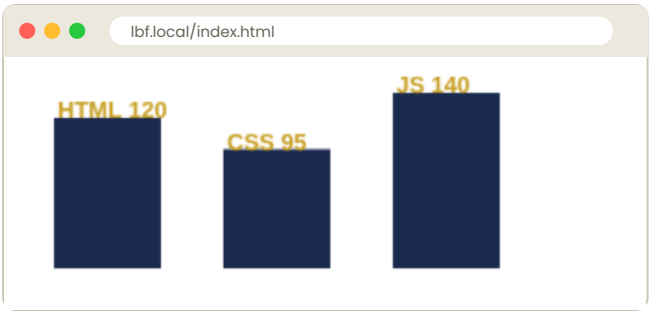
HOVERING THE BUTTON TURNS THE INLINE ICON GOLD THROUGH CURRENTCOLOR; THE IMG VERSION CANNOT BE RESTYLED

10.4.3 <canvas> element (drawing covered in Module 15)

`<canvas>` is a **bitmap drawing surface**: an empty rectangle of pixels that JavaScript paints on. Unlike SVG, the result is just pixels, with no elements to style or click, so it suits games, image editing, charts with thousands of points and visual effects. The markup is only the element; always provide fallback text and set its size with attributes.

A CANVAS PREVIEW HTML

```
<canvas id="c" width="300" height="120">A bar chart of enrolments: HTML 120, CSS 95, JS 140.</canvas>
<script>
  const ctx = document.getElementById("c").getContext("2d");
  [["HTML", 120], ["CSS", 95], ["JS", 140]].forEach(([label, n], i) => {
    ctx.fillStyle = "#1a2a4f"; ctx.fillRect(20 + i * 95, 110 - n * 0.7, 60, n * 0.7);
    ctx.fillStyle = "#c9a227"; ctx.font = "bold 13px sans-serif"; ctx.fillText(label + " " + n, 22 + i
* 95, 12 + (140 - n) * 0.7);
  });
</script>
```



PIXELS DRAWN BY JAVASCRIPT; MODULE 15 EXPLAINS THE CANVAS API

10.5 Media accessibility

10.5.1 Captions, transcripts and audio descriptions

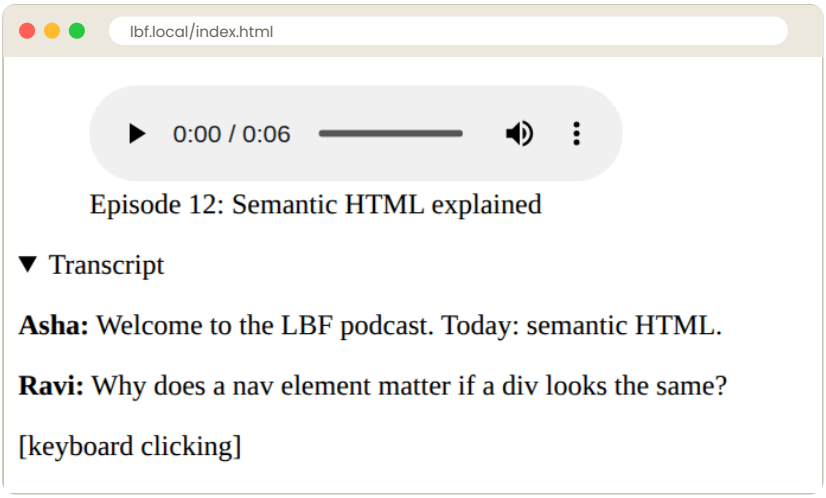
Alternative	For	What it contains	WCAG
Captions	deaf and hard-of-hearing users, people in noisy or quiet places	all dialogue plus speaker names and important sounds ([applause], [phone rings])	required for recorded video with audio (A)
Transcript	deaf-blind users (braille displays), skimmers, search engines	all speech and important visual information as text on the page	required for audio-only content such as podcasts (A)

Alternative	For	What it contains	WCAG
Audio description	blind and low-vision users	narration of important visual content during pauses	required for recorded video (AA)
Sign language	Deaf users whose first language is a sign language	interpreter video	AAA

- Never autoplay media with sound; if anything plays for more than 3 seconds, give a way to pause it.
- Use the native `controls` or fully keyboard-accessible custom controls.
- Put the transcript right below the player (a `<details>` element keeps it tidy) and link to it.
- Auto-generated captions (YouTube) are a start, but correct them: names and technical terms are often wrong.

A PODCAST WITH AN ACCESSIBLE TRANSCRIPT HTML

```
<figure>
  <audio controls src="podcast.mp3" preload="metadata"></audio>
  <figcaption>Episode 12: Semantic HTML explained</figcaption>
</figure>
<details open>
  <summary>Transcript</summary>
  <p><b>Asha:</b> Welcome to the LBF podcast. Today: semantic HTML.</p>
  <p><b>Ravi:</b> Why does a nav element matter if a div looks the same?</p>
  <p>[keyboard clicking]</p>
</details>
```



HOW THE BROWSER DISPLAYS IT

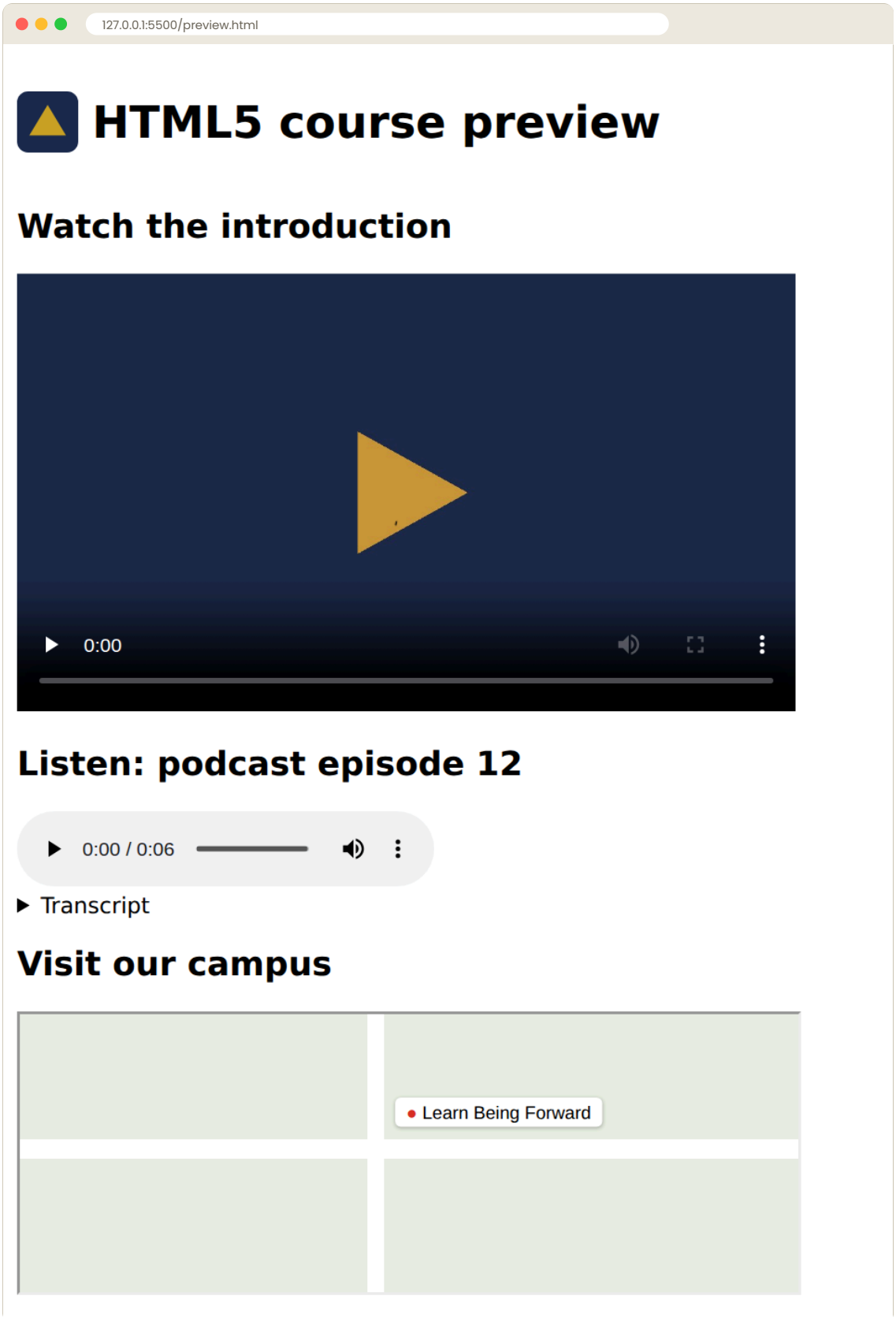
PRACTICE CHECKPOINT

A **course-preview page** with a captioned video, a podcast audio player, an embedded map and an inline SVG logo.

PREVIEW.HTML: SAMPLE SOLUTION

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>HTML5 Course Preview | Learn Being Forward</title>
  <style>body{font-family:system-ui,sans-serif;max-width:620px;margin:16px auto}
header{display:flex;gap:10px;align-items:center}</style>
</head>
<body>
  <header>
    <svg viewBox="0 0 100 100" width="44" role="img" aria-labelledby="lt">
      <title id="lt">Learn Being Forward logo</title>
      <rect width="100" height="100" rx="18" fill="#1a2a4f"/><path d="M20 72 L50 22 L80 72 Z"
fill="#c9a227"/>
    </svg>
    <h1>HTML5 course preview</h1>
  </header>
  <main>
    <section aria-labelledby="v"><h2 id="v">Watch the introduction</h2>
      <video controls width="560" height="315" poster="poster.jpg" preload="none" playsinline>
        <source src="intro.webm" type="video/webm">
        <track kind="captions" src="captions-en.vtt" srclang="en" label="English" default>
        <a href="intro.webm">Download the video</a>
      </video>
    </section>
    <section aria-labelledby="pod"><h2 id="pod">Listen: podcast episode 12</h2>
      <audio controls preload="metadata"><source src="podcast.ogg" type="audio/ogg"><source
src="podcast.mp3" type="audio/mpeg"></audio>
      <details><summary>Transcript</summary><p>Welcome to the LBF podcast...</p></details>
    </section>
    <section aria-labelledby="map"><h2 id="map">Visit our campus</h2>
      <iframe src="map.html" width="560" height="200" title="Map: Learn Being Forward, Mysuru"
loading="lazy"></iframe>
    </section>
  </main>
</body>
</html>
```



THE COURSE-PREVIEW PAGE (THE MAP FRAME USES THE LOCAL STAND-IN PAGE)

MODULE 11 · INTERMEDIATE

The head, metadata and SEO

Minimum duration **1 day** | Level **Intermediate**

The head section controls how a page is read by browsers, search engines and social networks. This module covers the essential meta tags (charset, viewport, description, robots, theme-color), linking stylesheets, scripts (defer, async, type="module"), icons, the web app manifest and canonical URLs, the base element, Open Graph and Twitter/X card tags for social sharing, and the fundamentals of SEO: titles and descriptions that rank, heading structure and keywords, semantic markup and structured data, sitemaps and robots.txt, and clean URLs with internal linking.

IN THIS MODULE

11.1 Essential meta tags
11.2 Linking resources
11.3 Social sharing
11.4 SEO fundamentals

11.1 Essential meta tags

Nothing in `<head>` is displayed in the page itself, yet it decides how the page is decoded, scaled on phones, titled in tabs and search results, shared on WhatsApp and LinkedIn, and which CSS and JavaScript load. A complete head for a production page looks like this; the sections below explain every line.

A COMPLETE PRODUCTION HEAD

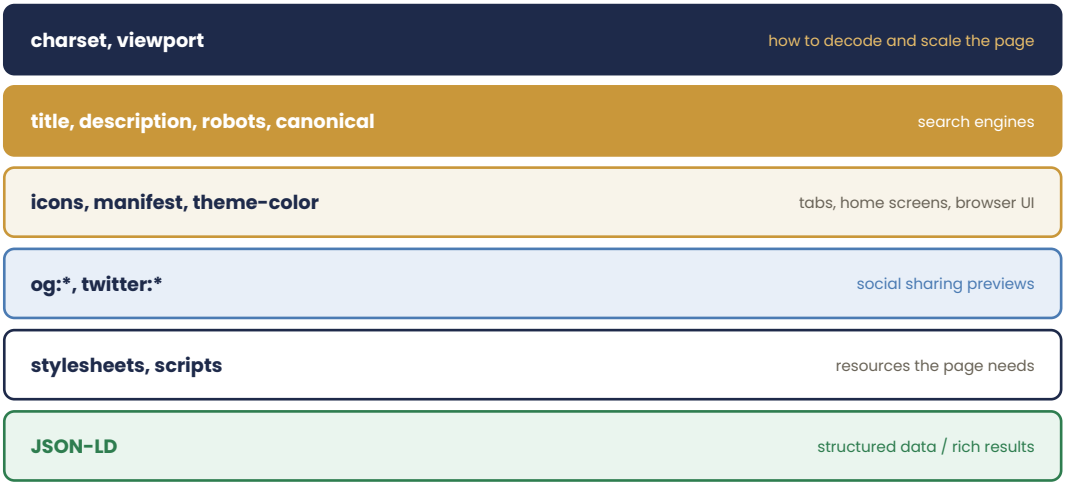
HTML

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>HTML5 Course in Mysuru | Learn Being Forward</title>
  <meta name="description" content="Learn HTML5 from scratch in 28 days: semantic pages, forms,
accessibility and APIs. Live classes in Mysuru and online.">
  <meta name="robots" content="index, follow">
  <meta name="theme-color" content="#1a2a4f">
  <link rel="canonical" href="https://learnbeingforward.in/courses/html5">

  <link rel="icon" href="/favicon.ico" sizes="32x32">
  <link rel="icon" href="/icon.svg" type="image/svg+xml">
  <link rel="apple-touch-icon" href="/apple-touch-icon.png">
  <link rel="manifest" href="/site.webmanifest">

  <meta property="og:type" content="website">
  <meta property="og:title" content="HTML5 Web Foundations course">
  <meta property="og:description" content="28 days from zero to professional HTML5.">
  <meta property="og:image" content="https://learnbeingforward.in/og/html5.jpg">
  <meta property="og:url" content="https://learnbeingforward.in/courses/html5">
  <meta name="twitter:card" content="summary_large_image">

  <link rel="stylesheet" href="/css/styles.css">
  <script src="/js/app.js" defer></script>
  <script type="application/ld+json">{ "@context": "https://schema.org", "@type": "Course", ... }
</script>
</head>
```

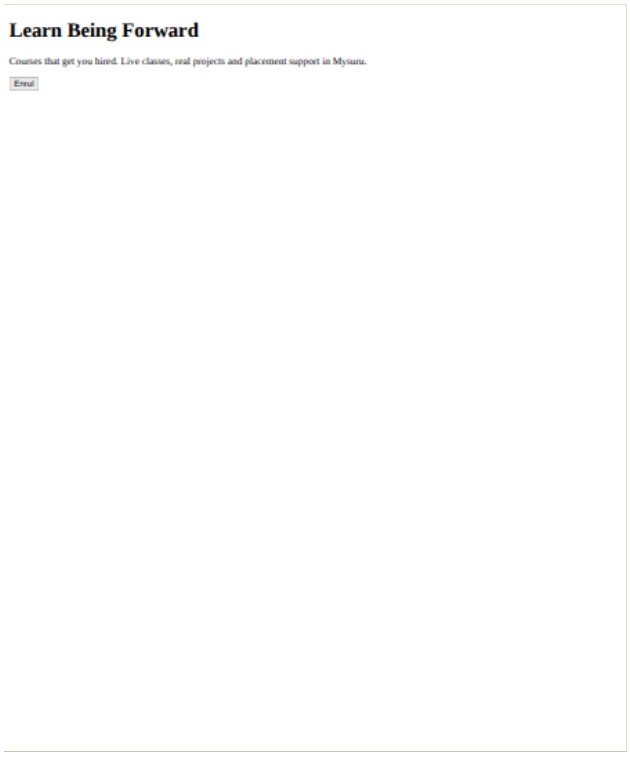


WHAT EACH GROUP OF HEAD ELEMENTS IS FOR

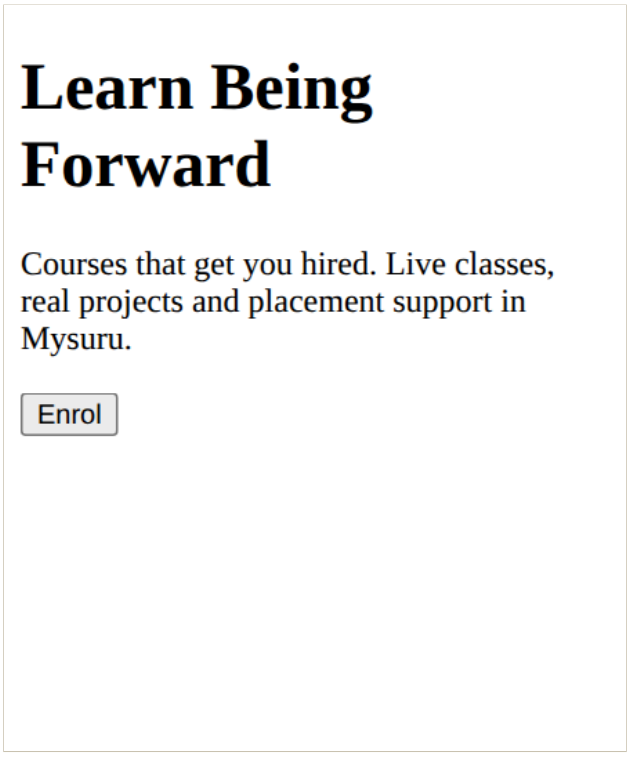
11.1.1 charset and viewport

`<meta charset="UTF-8">` tells the browser how bytes become characters. It must be within the **first 1024 bytes**, so put it first. Without it, characters such as ₹, é and Kannada text may appear as garbage (mojibake). The viewport meta tag tells phones to use the device width instead of pretending to be a 980-pixel desktop and zooming out:

Without the viewport meta tag



With width=device-width



THE SAME PAGE IN CHROMIUM EMULATING A 300PX-WIDE PHONE. NEVER ADD MAXIMUM-SCALE=1 OR USER-SCALABLE=NO: USERS MUST BE ABLE TO ZOOM

11.1.2 description

`<meta name="description">` summarises the page in one or two sentences. Search engines often show it as the **snippet** under the blue title, and social networks use it when no og:description exists. It does not directly affect ranking, but a clear, specific description increases the click-through rate. Aim for about 140-160 characters, unique on every page, containing the main keyword naturally and a reason to click.

11.1.3 robots

`<meta name="robots">` gives instructions to search-engine crawlers for **this page**. The default is `index, follow`, so the tag is only needed to restrict.

Value	Meaning	Typical use
`noindex`	do not show this page in results	thank-you pages, internal search results, staging sites
`nofollow`	do not follow the links on this page	pages with untrusted user links
`noarchive`	do not show a cached copy	frequently changing prices
`nosnippet` / `max-snippet:50`	no / shorter text snippet	paywalled content
`max-image-preview:large`	allow large image previews	blogs, news

11.1.4 theme-color

`<meta name="theme-color">` suggests a colour for the browser interface around the page: the address bar on Android Chrome, the title bar of an installed web app, the tab bar in Safari. A `media` attribute gives

different colours for light and dark mode.

THEME-COLOR HTML

```
<meta name="theme-color" content="#1a2a4f">
<meta name="theme-color" content="#ffffff" media="(prefers-color-scheme: light)">
<meta name="theme-color" content="#0d1528" media="(prefers-color-scheme: dark)">
```

11.2 Linking resources

11.2.1 Stylesheets with <link rel="stylesheet">

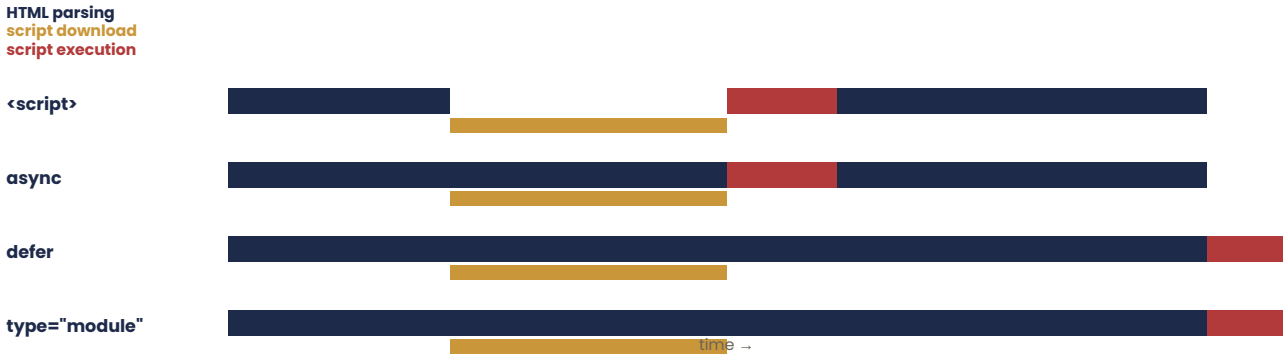
`<link rel="stylesheet" href="...">` loads an external CSS file. It is **render-blocking**: the browser waits for it before painting, so the page never flashes unstyled. The `media` attribute loads a stylesheet that only applies in some conditions (the browser still downloads it but at low priority and without blocking). Styles can also be placed in a `<style>` element, useful for small critical CSS.

STYLESHEETS HTML

```
<link rel="stylesheet" href="/css/styles.css">
<link rel="stylesheet" href="/css/print.css" media="print">
<link rel="stylesheet" href="/css/wide.css" media="(min-width: 1024px)">
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="stylesheet" href="https://fonts.googleapis.com/css2?
family=Poppins:wght@400;600&display=swap">
```

11.2.2 Scripts with <script>, defer, async, type="module"

A plain `<script src>` in the head **stops HTML parsing** while the script downloads and runs, delaying everything below it. Two attributes fix that, and modules behave like defer by default.



HOW EACH SCRIPT KIND AFFECTS PARSING: PLAIN SCRIPTS BLOCK, ASYNC RUNS AS SOON AS DOWNLOADED, DEFER AND MODULES RUN AFTER PARSING

	Blocks parsing	Runs when	Order kept	Use for
`<script src>` in head	yes	immediately when reached	yes	avoid (or place before </body>)
`async`	only while executing	as soon as downloaded	no	independent scripts: analytics, ads
`defer`	no	after parsing, before DOMContentLoaded	yes	your app scripts that use the DOM

	Blocks parsing	Runs when	Order kept	Use for
`type="module"`	no	like defer (or async if marked async)	yes	modern JS with import/export

This script really runs in Chromium and logs the order in which a deferred script, a module and an inline script execute, compared with the moment parsing finishes:

EXECUTION ORDER: INLINE, DEFER, MODULE main.js

```
console.log("1. inline script at end of body");
document.addEventListener("DOMContentLoaded", () => console.log("4. DOMContentLoaded"));
```

BROWSER CONSOLE

```
1. inline script at end of body
2. defer script (h1 exists: true)
3. module script (deferred by default)
4. DOMContentLoaded
```

11.2.3 Icons, the web app manifest and canonical URLs

Icons: a `favicon.ico` (32×32) for old browsers, an SVG icon for modern ones (it can even adapt to dark mode), and a 180×180 `apple-touch-icon` for iPhone home screens (Module 5 covered favicons). The **web app manifest** is a JSON file that lets the site be installed like an app, with name, icons, colours and start URL:

SITE.WEBMANIFEST text

```
{
  "name": "Learn Being Forward",
  "short_name": "LBF",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#1a2a4f",
  "icons": [
    { "src": "/icons/192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icons/512.png", "sizes": "512x512", "type": "image/png" },
    { "src": "/icons/maskable.png", "sizes": "512x512", "type": "image/png", "purpose": "maskable" }
  ]
}
```

A **canonical URL** tells search engines which address is the "real" one when the same content is reachable at several URLs (with and without www, with tracking parameters, sorted views). Without it, search engines split ranking signals between duplicates.

ONE CANONICAL URL FOR DUPLICATES HTML

```
<!-- On all of these:
     https://learnbeingforward.in/courses/html5
     https://www.learnbeingforward.in/courses/html5
     https://learnbeingforward.in/courses/html5?utm_source=whatsapp
     https://learnbeingforward.in/courses/html5/?ref=footer          -->
<link rel="canonical" href="https://learnbeingforward.in/courses/html5">
```

11.2.4 <base> (overview)

`<base href="...">` sets the base URL for **every relative URL** in the page, and `<base target="_blank">` a default link target. Only one is allowed, and it must come before any element that uses URLs. It is rarely needed and easy to misuse (it also changes in-page `#anchor` links), but single-page applications such as Angular use it to make routing work from a subfolder.

HOW BASE CHANGES RELATIVE LINKS HTML

```
<base href="https://learnbeingforward.in/courses/">
<p><a id="a" href="html5">html5</a></p>
<p><a id="b" href="../about">../about</a></p>
<p><a id="c" href="/contact">/contact</a></p>
<pre id="out"></pre>
<script>
  out.textContent = ["a", "b", "c"].map(id => id + " -> " +
document.getElementById(id).href).join("\n");
</script>
```



EACH LINK RESOLVED BY THE BROWSER AGAINST THE BASE URL

11.3 Social sharing

11.3.1 Open Graph tags

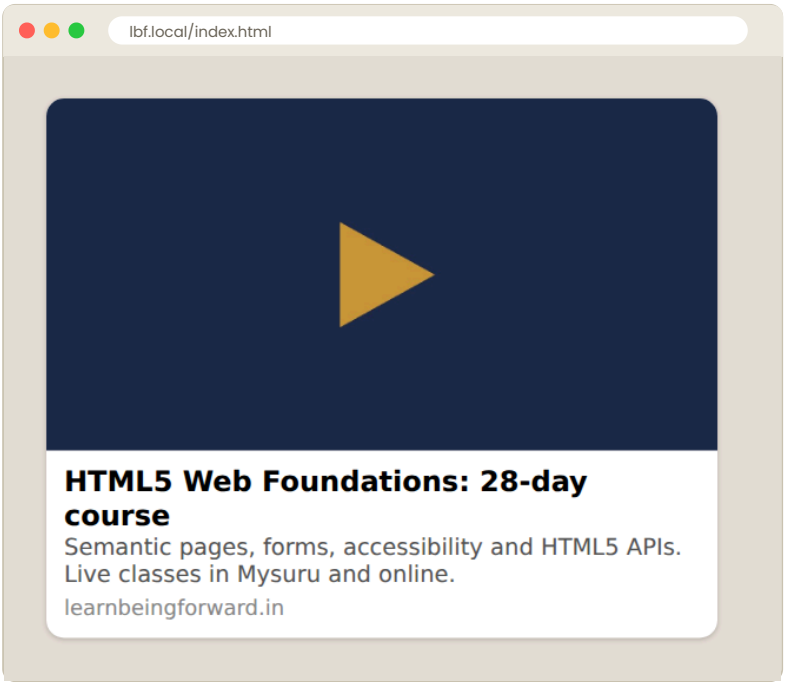
When a link is pasted into WhatsApp, LinkedIn, Facebook, Slack, Discord or Telegram, the platform fetches the page and builds a **preview card** from **Open Graph** (OG) meta tags, a protocol created by Facebook. Note that OG tags use `property`, not `name`.

Tag	Content	Notes
`og:title`	card title	can differ from <title>; under about 60 characters
`og:description`	one or two sentences	about 100–200 characters
`og:image`	**absolute** URL of the image	1200×630 px (1.91:1), under about 5 MB, JPG/PNG
`og:image:alt`	text alternative for the image	accessibility
`og:url`	canonical URL	same as rel=canonical
`og:type`	website, article, profile, video.movie…	article adds article:published_time etc.
`og:site_name`	site name	"Learn Being Forward"
`og:locale`	language_TERRITORY	`en_IN`

Platforms render the card themselves, so the way to check tags locally is to read them exactly as a platform would and draw the card from them. This page contains the OG tags in its head and a small script that builds a card preview from them; the screenshot is that preview:

OG-PREVIEW.HTML: A CARD DRAWN FROM THE PAGE'S OWN OG TAGS HTML

```
<!DOCTYPE html>
<html lang="en"><head>
<meta charset="UTF-8">
<title>HTML5 Course | Learn Being Forward</title>
<meta property="og:title" content="HTML5 Web Foundations: 28-day course">
<meta property="og:description" content="Semantic pages, forms, accessibility and HTML5 APIs. Live
classes in Mysuru and online.">
<meta property="og:image" content="poster.jpg">
<meta property="og:url" content="https://learnbeingforward.in/courses/html5">
<meta property="og:site_name" content="Learn Being Forward">
<style>
  body{font-family:system-ui,sans-serif;background:#e5ddd5;padding:16px}
  .card{width:380px;background:#fff;border-radius:10px;overflow:hidden;box-shadow:0 1px 3px #0003}
  .card img{width:100%;aspect-ratio:1.91;object-fit:cover;display:block}
  .t{font-weight:700;padding:8px 10px 2px}.d{font-size:13px;color:#555;padding:0 10px}.u{font-
size:12px;color:#888;padding:4px 10px 10px}
</style></head>
<body>
<div class="card" id="card"></div>
<script>
  const og = p => document.querySelector(`meta[property="og:${p}"]`).content;
  card.innerHTML = `<img src="${og("image")}" alt=""><div class="t">${og("title")}</div>
    <div class="d">${og("description")}</div><div class="u">${new URL(og("url")).hostname}</div>`;
</script>
</body></html>
```



A CHAT-APP STYLE LINK PREVIEW GENERATED FROM THE OG:TITLE, OG:DESCRIPTION, OG:IMAGE AND OG:URL TAGS

TIP

Online checkers fetch your live page and show exactly what each platform sees: the **LinkedIn Post Inspector**, the **Facebook Sharing Debugger** (which also clears Facebook's cache after you change tags) and opengraph.xyz. WhatsApp caches previews, so test with a new URL parameter.

11.3.2 Twitter/X card tags

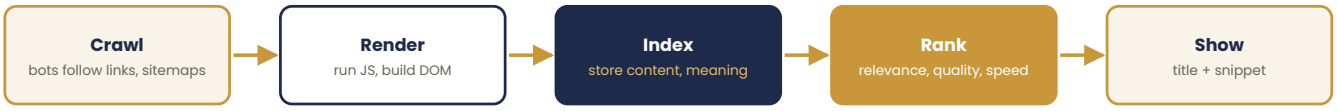
X (formerly Twitter) reads its own `twitter:` tags and **falls back to Open Graph** for anything missing, so in practice only the card type is required. `summary` shows a small square thumbnail; `summary_large_image` shows a large image above the title.

X CARD TAGS HTML

```
<meta name="twitter:card" content="summary_large_image">
<meta name="twitter:site" content="@learnbeingfwd">          <!-- the site's account -->
<meta name="twitter:creator" content="@asha_dev">            <!-- the author's account -->
<!-- twitter:title, twitter:description and twitter:image fall back to og:title, og:description,
og:image -->
```

11.4 SEO fundamentals

Search Engine Optimisation is making pages easy for search engines to find, understand and rank, and attractive for people to click. HTML authors control a large part of it.



HOW A SEARCH ENGINE PROCESSES A PAGE

11.4.1 Titles and descriptions that rank

The `<title>` is the single most important on-page SEO element: it is the clickable blue headline in results and a strong relevance signal. Write a **unique** title per page, put the main keyword near the start, keep it under about 60 characters (longer titles are cut off), and add the brand at the end.

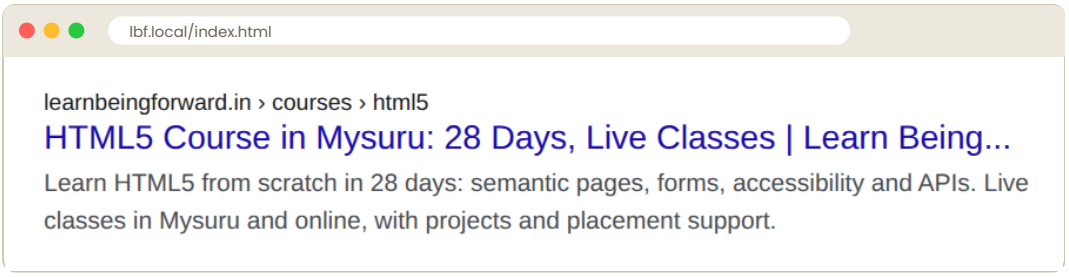
Weak	Strong
Home	HTML5 & Web Development Courses in Mysuru | LBF
Course	HTML5 Course: 28 Days, Live Classes | Learn Being Forward
Learn Being Forward - Learn Being Forward - courses courses	Placement-Ready Java Course with Projects | LBF

The preview below is built from a page's own title, URL and description, the way a results page lays them out (search engines may rewrite the snippet, so this is an approximation):

SERP-PREVIEW.HTML

HTML

```
<!DOCTYPE html><html lang="en"><head><meta charset="UTF-8">
<title>HTML5 Course in Mysuru: 28 Days, Live Classes | Learn Being Forward</title>
<meta name="description" content="Learn HTML5 from scratch in 28 days: semantic pages, forms,
accessibility and APIs. Live classes in Mysuru and online, with projects and placement support.">
<link rel="canonical" href="https://learnbeingforward.in/courses/html5">
<style>body{font-family:arial,sans-serif;padding:10px 14px}.u{font-size:13px;color:#202124}.t{font-
size:19px;color:#1a0dab;margin:3px 0}
.d{font-size:14px;color:#4d5156;line-height:1.5}.w{width:560px}</style></head>
<body><div class="w" id="r"></div>
<script>
  const cut = (s, n) => s.length > n ? s.slice(0, n - 1).trimEnd() + "..." : s;
  const url = new URL(document.querySelector("link[rel=canonical]").href);
  r.innerHTML = `<div class="u">${url.hostname} &rsaquo;
${url.pathname.split("/").filter(Boolean).join(" &rsaquo; ")}</div>
    <div class="t">${cut(document.title, 60)}</div>
    <div class="d">${cut(document.querySelector("meta[name=description]").content, 158)}</div>`;
</script></body></html>
```



AN APPROXIMATE SEARCH-RESULT SNIPPET: THE 66-CHARACTER TITLE IS TRUNCATED AT ABOUT 60 CHARACTERS

11.4.2 Heading structure and keywords

- One `<h1>` that states the page topic and contains the main keyword ("HTML5 Course in Mysuru").
- h2/h3 subheadings that match what people search for ("HTML5 course syllabus", "Course fee and duration", "FAQs").
- Write for people first: use the words your audience uses, naturally. Repeating keywords ("keyword stuffing") is penalised.
- Put the important content in HTML text, not inside images or canvas, and give images descriptive `alt` text and file names (`html5-course-classroom.jpg`, not `IMG_2031.jpg`).

11.4.3 Semantic markup and structured data

Semantic elements (Module 9) help crawlers find the main content and ignore navigation and footers; `<time datetime>`, `<address>` and descriptive link text add meaning. JSON-LD structured data (9.5) can earn **rich results**: course listings, FAQ drop-downs, breadcrumbs, review stars, event dates. Page speed and mobile friendliness (Module 13) are also ranking factors.

11.4.4 Sitemaps and robots.txt (overview)

robots.txt, at the site root, tells crawlers which paths they may crawl and where the sitemap is. It is a request, not security: never use it to hide private pages (they would be listed publicly in the file). An **XML sitemap** lists the URLs you want indexed, with their last-modified dates, so new pages are discovered quickly. Submit it in Google Search Console and Bing Webmaster Tools.

ROBOTS.TXT text

```
User-agent: *
Disallow: /admin/
Disallow: /search
Allow: /

Sitemap: https://learnbeingforward.in/sitemap.xml
```

SITEMAP.XML HTML

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://learnbeingforward.in/</loc>
    <lastmod>2026-10-01</lastmod>
  </url>
  <url>
    <loc>https://learnbeingforward.in/courses/html5</loc>
    <lastmod>2026-10-08</lastmod>
  </url>
</urlset>
```

11.4.5 Clean URLs and internal linking

Avoid	Prefer	Why
`/page.php?id=372&cat=9`	`/courses/html5`	readable, memorable, keywords in the URL
`/Courses/HTML_5_Course.html`	`/courses/html5-course`	lower case, hyphens between words
`/2026/10/08/post-final-v2`	`/blog/semantic-html`	short and stable; dates make URLs age
"click here", "read more"	"read the <a>HTML5 syllabus"	descriptive anchor text tells users and crawlers what the target is

Internal links (links between your own pages) help crawlers discover every page and tell them which pages are most important: link to key pages from the navigation, link related articles to each other, add breadcrumbs, and make sure no page is an "orphan" with no links pointing to it. When a URL must change, add a **301 redirect** from the old URL so links and rankings are kept.

PRACTICE CHECKPOINT

Add complete **metadata, Open Graph tags and JSON-LD** structured data to the three-page website, then preview its social card.

ABOUT.HTML HEAD: SAMPLE SOLUTION

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>About Asha Rao: Web Developer in Mysuru</title>
  <meta name="description" content="Asha Rao is a front-end developer in Mysuru building accessible,
fast websites with HTML5, CSS and JavaScript.">
  <link rel="canonical" href="https://asharao.dev/about.html">
  <meta name="theme-color" content="#1a2a4f">
  <link rel="icon" href="favicon.svg" type="image/svg+xml">

  <meta property="og:type" content="profile">
  <meta property="og:title" content="Asha Rao, Web Developer">
  <meta property="og:description" content="Accessible, fast websites with HTML5, CSS and JavaScript.">
  <meta property="og:image" content="https://asharao.dev/images/og-about.jpg">
  <meta property="og:image:alt" content="Asha Rao at her desk">
  <meta property="og:url" content="https://asharao.dev/about.html">
  <meta property="og:locale" content="en_IN">
  <meta name="twitter:card" content="summary_large_image">

  <script type="application/ld+json">
  {
    "@context": "https://schema.org",
    "@type": "Person",
    "name": "Asha Rao",
    "jobTitle": "Front-end Developer",
    "address": { "@type": "PostalAddress", "addressLocality": "Mysuru", "addressCountry": "IN" },
    "url": "https://asharao.dev",
    "sameAs": ["https://github.com/asharao", "https://www.linkedin.com/in/asharao"]
  }
  </script>
  <link rel="stylesheet" href="css/style.css">
  <script src="js/main.js" defer></script>
</head>
```

Check list for each of the three pages: a unique title and description, canonical URL, OG title, description, absolute image URL and alt, and JSON-LD of the right type (WebSite or Organization on the home page, Person on About, ContactPage on Contact). Preview the card with the LinkedIn Post Inspector or opengraph.xyz after deploying (Module 16), or locally with the og-preview technique shown in 11.3.1.

MODULE 12 · INTERMEDIATE

Accessibility (a11y)

Minimum duration **2 days** Level **Intermediate**

Accessible HTML lets everyone, including people using screen readers or only a keyboard, use a website, and it is a legal requirement in many places. This module covers what accessibility is and who benefits, types of disability and assistive technologies, the four WCAG principles and the A, AA and AAA conformance levels, accessible HTML (semantic elements first, alt text and accessible names, heading order and landmarks, accessible forms, page language), keyboard accessibility (focus order, tabindex, skip links, visible focus), ARIA (the first rule of ARIA, roles, states and properties, aria-label, aria-labelledby, aria-describedby, aria-expanded, aria-hidden, aria-live) and how to test with the keyboard, screen readers, Lighthouse and axe. The accessibility trees shown are the ones Chromium really builds.

IN THIS MODULE

12.1 Accessibility basics
12.2 Accessible HTML
12.3 Keyboard accessibility
12.4 ARIA
12.5 Testing accessibility

12.1 Accessibility basics

12.1.1 What accessibility is and who benefits

Web accessibility (often shortened to **a11y**: "a", 11 letters, "y") means designing and building websites so that people with disabilities can perceive, understand, navigate and interact with them. The World Health Organization estimates that about **1.3 billion people, 1 in 6**, live with a significant disability. Accessibility also helps everyone in a **situational** or **temporary** limitation: bright sunlight on a phone screen, a broken arm, a noisy train, a slow connection, ageing eyes.

Permanent	Temporary	Situational
blind user with a screen reader	eye surgery recovery	phone in bright sunlight
one arm	arm in a cast	holding a baby, using one hand
deaf	ear infection	loud railway station, or a sleeping baby nearby
cognitive disability	concussion, medication	distracted, tired, reading in a second language

There are also strong practical reasons: accessible sites rank better (search engines are, in effect, blind users), work better on phones and voice assistants, and reduce legal risk. In India, the **Rights of Persons with Disabilities Act, 2016** and the government's **GIGW** guidelines require accessible websites for government and many public services; the EU's **European Accessibility Act** (applying from June 2025) and the US **ADA** cases make it a legal requirement for many businesses too.

12.1.2 Types of disabilities and assistive technologies

Disability	Examples	Assistive technology / strategy	What HTML must provide
Visual	blindness, low vision, colour blindness	screen readers (NVDA, JAWS, VoiceOver, TalkBack), screen magnifiers, high contrast, zoom	text alternatives, semantic structure, contrast, resizable text
Hearing	deafness, hard of hearing	captions, transcripts, sign language	captions and transcripts for media
Motor	tremor, paralysis, missing limbs, RSI	keyboard only, switch devices, voice control (Voice Access), eye tracking	full keyboard support, large targets, no time limits
Cognitive and learning	dyslexia, ADHD, memory, autism	reading tools, simplified layouts	clear language, consistent navigation, helpful errors
Speech	difficulty speaking	alternatives to voice input	never require voice only
Seizure / vestibular	photosensitive epilepsy, motion sickness	reduced motion settings	no flashing over 3 per second, respect prefers-reduced-motion

12.1.3 WCAG principles: Perceivable, Operable, Understandable, Robust

The **Web Content Accessibility Guidelines** (WCAG), published by the W3C, are the worldwide standard. The current version is **WCAG 2.2** (October 2023), organised under four principles, remembered as **POUR**, each broken into guidelines and testable **success criteria**.

Perceivable

Users can perceive the content: text alternatives, captions, adaptable structure, enough contrast.

Operable

Users can operate the interface: keyboard access, enough time, no seizures, navigable, input beyond the mouse.

Understandable

Content and operation are understandable: readable language, predictable behaviour, input assistance.

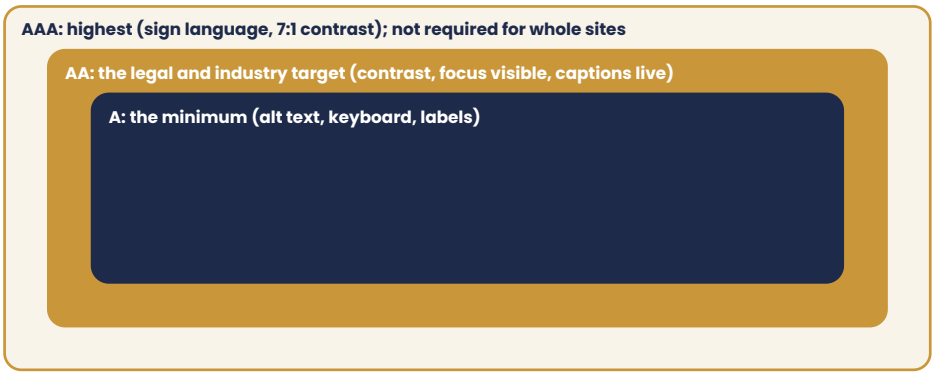
Robust

Content works with current and future technologies, including assistive technology: valid, semantic markup with correct names and roles.

Success criterion (examples)	Level	In HTML terms
1.1.1 Non-text Content	A	every meaningful image has alt text
1.3.1 Info and Relationships	A	headings, lists, tables, labels are real elements
1.4.3 Contrast (Minimum)	AA	text contrast at least 4.5:1 (3:1 for large text)
2.1.1 Keyboard	A	everything works with the keyboard
2.4.1 Bypass Blocks	A	skip link or landmarks
2.4.7 Focus Visible	AA	never remove the focus outline without a replacement
2.5.8 Target Size (Minimum) (new in 2.2)	AA	click targets at least 24×24 CSS px
3.1.1 Language of Page	A	`<html lang="en">`

Success criterion (examples)	Level	In HTML terms
3.3.2 Labels or Instructions	A	every input has a label
4.1.2 Name, Role, Value	A	custom widgets expose name, role and state

12.1.4 Conformance levels A, AA, AAA



EACH LEVEL INCLUDES EVERYTHING IN THE LEVEL BELOW IT

Level A removes the biggest barriers; **Level AA** is what laws, government guidelines (including GIGW in India) and company policies require; **Level AAA** is aspirational and not achievable for all content. "WCAG 2.2 AA" is therefore the usual target.

12.2 Accessible HTML

12.2.1 Semantic elements first

Most accessibility comes free with correct HTML. A `<button>` is focusable, activates with Enter and Space, and is announced as "button". A `<div onclick>` is none of these. Chromium's accessibility tree shows the difference:

FAKE CONTROLS VS REAL ELEMENTS

index.html

```
<div class="btn" onclick="save()">Save</div>
<button onclick="save()">Save</button>
<span class="link" onclick="location='/fees'">Fees</span>
<a href="/fees">Fees</a>
<div class="h">Course details</div>
<h2>Course details</h2>
```

ACCESSIBILITY TREE (CHROMIUM)

```
- text: Save
- button "Save"
- text: Fees
- link "Fees":
  - /url: /fees
- text: Course details
- heading "Course details" [level=2]
```

The div and span become plain **text**: a screen reader never says "button" or "link", Tab skips them, and Enter does nothing. Fixing a fake button would need `role="button"`, `tabindex="0"` and key handlers for Enter and Space: three things that a real `<button>` gives for free.

12.2.2 Alt text and accessible names

Every interactive element and every meaningful image needs an **accessible name**: the text a screen reader announces. The browser computes it from, in order of priority: `aria-labelledby`, `aria-label`, the element's own content or label (`<label>`, `alt`, button text, `<caption>`, `<legend>`), and finally `title`.

ACCESSIBLE NAMES COMPUTED BY CHROMIUM

index.html

```
<img src="mysuru-palace.jpg" alt="Mysuru Palace lit up at night">
<img src="divider.png" alt="">
<a href="/cart"><img src="cart.svg" alt="Cart"></a>
<button><svg aria-hidden="true" width="16" height="16"></svg>Search</button>
<button aria-label="Close dialog">&times;</button>
<a href="/syllabus.pdf">Download the HTML5 syllabus (PDF, 240 KB)</a>
<a href="/syllabus.pdf">Click here</a>
```

ACCESSIBILITY TREE (CHROMIUM)

```
- img "Mysuru Palace lit up at night"
- link "Cart":
  - /url: /cart
  - img "Cart"
- button "Search"
- button "Close dialog": ×
- link "Download the HTML5 syllabus (PDF, 240 KB)":
  - /url: /syllabus.pdf
- link "Click here":
  - /url: /syllabus.pdf
```

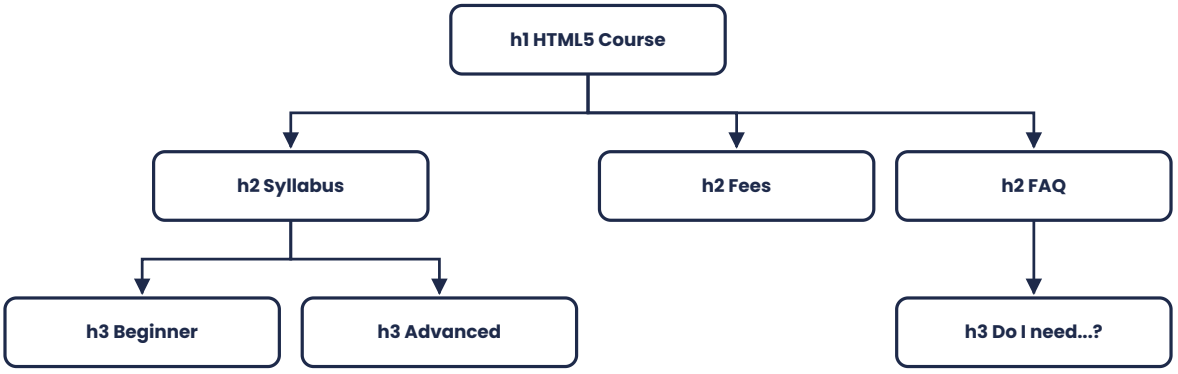
Image type	alt	Example
Informative	describe the meaning, briefly	`alt="Students in the LBF lab building a website"`
Decorative (adds nothing)	empty: `alt=""` (never omit alt)	divider lines, background flourishes
Functional (link or button)	describe the action/destination	logo link: `alt="Learn Being Forward home"`
Image of text	the same text	a banner reading "Admissions open": `alt="Admissions open"`
Complex (chart, diagram)	short alt + full description nearby	`alt="Bar chart of placements, details below"` + a table

WATCH OUT

Do not start alt text with "image of" or "picture of" (screen readers already say "image"), and avoid generic link text like "click here" or "read more": screen-reader users often navigate a list of links out of context.

12.2.3 Heading order and landmarks

Screen-reader users navigate mainly by **headings** (the H key in NVDA and JAWS, the rotor in VoiceOver) and **landmarks** (D in NVDA, the rotor). Surveys of screen-reader users by WebAIM consistently find that around two-thirds use headings first to find information on a long page. So: one h1, no skipped levels, a heading at the start of each major section, and the landmark elements from Module 9.



THE HEADING OUTLINE A SCREEN-READER USER NAVIGATES

12.2.4 Accessible forms: labels, instructions, error messages

Every control needs a programmatically associated label (Module 7). Instructions and errors must be **connected** to the field with `aria-describedby`, so they are read when the field gets focus, and an invalid field should have `aria-invalid="true"`. Required fields use the `required` attribute (announced as "required").

A FIELD WITH A LABEL, HINT AND ERROR index.html

```
<label for="pin">PIN code</label>
<input id="pin" name="pin" inputmode="numeric" required aria-invalid="true"
       aria-describedby="pin-hint pin-err" value="5700">
<p id="pin-hint">6 digits, for example 570001.</p>
<p id="pin-err">Error: PIN code must have 6 digits.</p>
```

ACCESSIBILITY TREE (CHROMIUM)

```
- text: PIN code
- textbox "PIN code": "5700"
- paragraph: 6 digits, for example 570001.
- paragraph: "Error: PIN code must have 6 digits."
```

The tree shows the textbox name ("PIN code") and the state (required, invalid). The description from `aria-describedby` does not appear in Playwright's snapshot format, but this check reads it from the accessibility tree through the DOM, and it is what a screen reader reads after the name:

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```
<label for="pin">PIN code</label>
<input id="pin" required aria-invalid="true" aria-describedby="pin-hint pin-err" value="5700">
<p id="pin-hint">6 digits, for example 570001.</p>
<p id="pin-err">Error: PIN code must have 6 digits.</p>
```

WHAT THE SCREEN READER ANNOUNCES FOR THE FIELD main.js

```
const pin = document.getElementById("pin");
const describedBy = pin.getAttribute("aria-describedby").split(" ")
  .map(id => document.getElementById(id).textContent).join(" ");
console.log("Name:       ", pin.labels[0].textContent);
console.log("Description:", describedBy);
console.log("Required:   ", pin.required, "| Invalid:", pin.getAttribute("aria-invalid"));
```

BROWSER CONSOLE

```
Name:        PIN code
Description: 6 digits, for example 570001. Error: PIN code must have 6 digits.
Required:    true | Invalid: true
```

NOTE

A screen reader such as NVDA typically announces: "PIN code, edit, required, invalid entry, 5700. 6 digits, for example 570001. Error: PIN code must have 6 digits."

12.2.5 Language of the page

`<html lang="en">` tells screen readers which voice and pronunciation rules to use, browsers which dictionary to spell-check with and translation tools what to translate from. Mark passages in another language with `lang` on the element so they are pronounced correctly (WCAG 3.1.2 Language of Parts).

PAGE AND PART LANGUAGES HTML

```
<html lang="en-IN">
...
<p>In French, "welcome" is <span lang="fr">bienvenue</span>.</p>
<blockquote lang="es">Hola, bienvenidos al curso.</blockquote>
```

12.3 Keyboard accessibility

12.3.1 Focus order and tabindex

Keyboard users move with **Tab** (next) and **Shift+Tab** (previous), activate with **Enter** (links, buttons) or **Space** (buttons, checkboxes), move inside radio groups and selects with the **arrow keys** and close things with **Esc**. Focus moves through focusable elements (links with href, buttons, form controls, elements with tabindex) in **source order**, so the DOM order must match the visual reading order.

tabindex	Effect	Use
(none)	native behaviour: interactive elements focusable, others not	almost always
`0`	makes an element focusable, in source order	a custom widget that cannot be a native element
`-1`	focusable by script (`el.focus()`) but not by Tab	headings or containers you move focus to, e.g. after an error summary
`1` or more	jumps ahead of everything else in the tab order	**never**: it creates a confusing order that is hard to maintain

This script presses Tab repeatedly in Chromium and logs each element that receives focus, showing how a positive tabindex hijacks the order:

THE PAGE THIS SCRIPT RUNS IN (HTML)

HTML

```
<a href="#">Home</a>
<a href="#">Courses</a>
<div>Not focusable</div>
<input name="search" placeholder="Search">
<button tabindex="1">Login (tabindex=1)</button>
<div tabindex="0">Custom widget (tabindex=0)</div>
<span tabindex="-1">Script-only (tabindex=-1)</span>
```

MEASURING THE REAL TAB ORDER

main.js

```
const log = [];
document.addEventListener("focusin", e => log.push(e.target.textContent.trim() || e.target.name));
```

BROWSER CONSOLE

```
1. Login (tabindex=1)
2. Home
3. Courses
4. search
5. Custom widget (tabindex=0)
```

Simulated user actions: presses Tab five times, then prints the focus log.

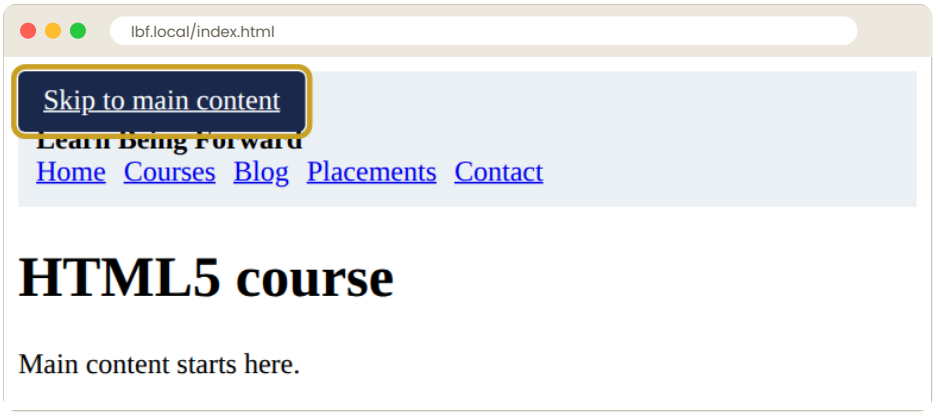
12.3.2 Skip links

Keyboard users would otherwise press Tab through the whole header and menu on every page. A **skip link** is the first focusable element, usually hidden until focused, that jumps to `<main>` (WCAG 2.4.1). It is good practice to give the target `tabindex="-1"` so focus really moves there in every browser.

A SKIP LINK

HTML

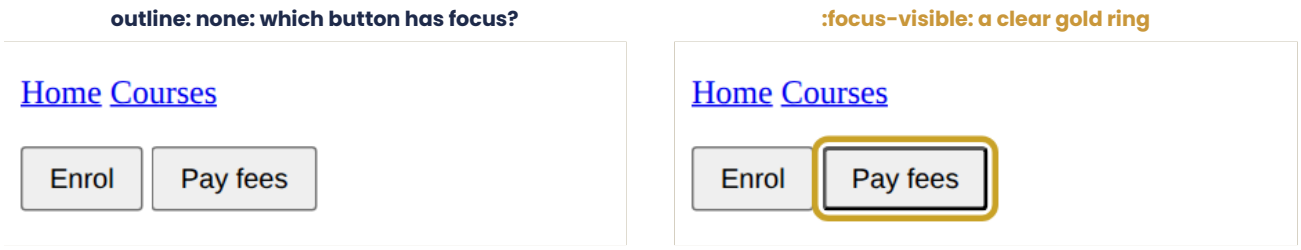
```
<style>
  .skip { position: absolute; left: -9999px; }
  .skip:focus { left: 8px; top: 8px; background: #1a2a4f; color: #fff; padding: 8px 14px; border-
radius: 4px; z-index: 10; outline: 3px solid #c9a227; }
  header { background: #eef1f8; padding: 30px 10px 10px; } nav a { margin-right: 10px; }
</style>
<a class="skip" href="#main">Skip to main content</a>
<header>
  <strong>Learn Being Forward</strong>
  <nav><a href="#">Home</a><a href="#">Courses</a><a href="#">Blog</a><a href="#">Placements</a><a
href="#">Contact</a></nav>
</header>
<main id="main" tabindex="-1"><h1>HTML5 course</h1><p>Main content starts here.</p></main>
```



AFTER PRESSING TAB ONCE: THE SKIP LINK APPEARS; PRESSING ENTER WOULD MOVE FOCUS TO MAIN

12.3.3 Visible focus

Browsers draw a **focus indicator** (outline) around the focused element. Removing it with `outline: none` leaves keyboard users lost, and fails WCAG 2.4.7. Use `:focus-visible` to show a strong custom ring for keyboard users while hiding it after mouse clicks.



THE "PAY FEES" BUTTON IS FOCUSED IN BOTH PAGES

12.4 ARIA

ARIA (Accessible Rich Internet Applications, a W3C specification) is a set of HTML attributes that **change what the accessibility tree says** about an element: its role, name, state and relationships. ARIA changes only semantics; it adds **no behaviour**: `role="button"` does not make a div focusable or clickable by keyboard.

12.4.1 The first rule of ARIA: use native HTML when possible

The W3C's first rule of ARIA use: *"If you can use a native HTML element or attribute with the semantics and behaviour you require already built in, instead of re-purposing an element and adding an ARIA role, state or property to make it accessible, then do so."* WebAIM's yearly analysis of the top one million home pages repeatedly finds that pages **with** ARIA have **more** detected accessibility errors on average than pages without it, largely because ARIA is so often misused. No ARIA is better than bad ARIA.

Instead of	Use
`<div role="button" tabindex="0">` + key handlers	`<button>`
`<div role="navigation">`	`<nav>`
`<span role="checkbox" aria-checked="true">`	`<input type="checkbox" checked>`
`<div role="heading" aria-level="2">`	`<h2>`
custom disclosure with aria-expanded	`<details><summary>` (Module 14)
`aria-required="true"`	`required`

12.4.2 Roles, states and properties

Category	Examples	Meaning
Widget roles	button, link, checkbox, tab, tablist, tabpanel, menu, menuitem, switch, slider, dialog	what the element is and how to interact with it
Landmark roles	banner, navigation, main, complementary, contentinfo, search, region, form	page regions (prefer the HTML elements)
Live region roles	status, alert, log, timer	content that updates

Category	Examples	Meaning
States (change)	aria-expanded, aria-checked, aria-selected, aria-pressed, aria-disabled, aria-invalid, aria-current	the current condition
Properties (stable)	aria-label, aria-labelledby, aria-describedby, aria-controls, aria-haspopup, aria-required	names and relationships

ROLES AND STATES IN THE ACCESSIBILITY TREE

index.html

```
<div role="tablist" aria-label="Course levels">
  <button role="tab" aria-selected="true" aria-controls="p1" id="t1">Beginner</button>
  <button role="tab" aria-selected="false" aria-controls="p2" id="t2" tabindex="-1">Advanced</button>
</div>
<div role="tabpanel" id="p1" aria-labelledby="t1">Modules 1 to 7</div>
<button aria-pressed="true">Dark mode</button>
<nav aria-label="Pagination"><a href="?p=2" aria-current="page">2</a></nav>
```

ACCESSIBILITY TREE (CHROMIUM)

```
- tablist "Course levels":
  - tab "Beginner" [selected]
  - tab "Advanced"
- tabpanel "Beginner": Modules 1 to 7
- button "Dark mode" [pressed]
- navigation "Pagination":
  - link "2":
    - /url: "?p=2"
```

12.4.3 aria-label, aria-labelledby, aria-describedby

Attribute	Value	Use when
`aria-label`	a string	there is no visible text: icon buttons (`aria-label="Close"`), several navs
`aria-labelledby`	id(s) of visible element(s)	a visible heading or text already names it: a section, dialog, or a name built from several pieces
`aria-describedby`	id(s) of element(s)	extra information read after the name: hints, errors, password rules

NAMING WITH ARIA-LABEL AND ARIA-LABELLEDBY

index.html

```
<button aria-label="Search courses"><svg aria-hidden="true" width="16" height="16"></svg></button>
<section aria-labelledby="fees-h"><h2 id="fees-h">Fees</h2><p>Rs 9,999</p></section>
<h3 id="c1">HTML5</h3>
<a href="/html5" id="more1" aria-labelledby="more1 c1">Read more</a>
```

ACCESSIBILITY TREE (CHROMIUM)

```
- button "Search courses"
- region "Fees":
  - heading "Fees" [level=2]
  - paragraph: Rs 9,999
- heading "HTML5" [level=3]
- link "Read more HTML5":
  - /url: /html5
  - text: Read more
```

The last example turns a vague "Read more" into "Read more HTML5" by combining two ids, so a list of links makes sense out of context.

12.4.4 aria-expanded, aria-hidden, aria-live

`aria-expanded="true|false"` on a button that shows and hides something tells users whether it is open (menus, accordions). `aria-hidden="true"` removes an element **and its children** from the accessibility tree while leaving it visible: use it for decorative icons and duplicated content, never on focusable elements. `aria-live` makes a screen reader announce changes to a region without moving focus: `polite` waits until the user is idle (cart updated, results count), `assertive` interrupts (session expiring). `role="status"` is polite and `role="alert"` assertive.

A DISCLOSURE MENU AND A LIVE STATUS MESSAGE HTML

```
<style>button{padding:6px 12px} #menu{border:1px solid #ccc;padding:4px 10px;margin:6px 0;width:160px}
</style>
<button id="mb" aria-expanded="false" aria-controls="menu">&#9776; Courses</button>
<ul id="menu" hidden><li><a href="#">HTML5</a></li><li><a href="#">CSS3</a></li><li><a
href="#">React</a></li></ul>
<p><button id="add">Add HTML5 to cart</button> <span role="status" id="msg"></span></p>
<script>
  mb.onclick = () => {
    const open = mb.getAttribute("aria-expanded") === "true";
    mb.setAttribute("aria-expanded", String(!open));
    menu.hidden = open;
  };
  add.onclick = () => msg.textContent = "HTML5 added. Cart has 1 item.";
</script>
```



AFTER CLICKING BOTH BUTTONS: THE MENU IS OPEN (ARIA-EXPANDED="TRUE") AND THE STATUS TEXT WOULD BE ANNOUNCED POLITELY

THE SAME WIDGETS IN THE ACCESSIBILITY TREE index.html

```
<button aria-expanded="true" aria-controls="menu">Courses</button>
<ul id="menu"><li><a href="#">HTML5</a></li></ul>
<span aria-hidden="true">&#9733;&#9733;&#9733;&#9733;&#9734;</span><span>Rated 4 out of 5</span>
<p role="status">HTML5 added. Cart has 1 item.</p>
```

ACCESSIBILITY TREE (CHROMIUM)

```
- button "Courses" [expanded]
- list:
  - listitem:
    - link "HTML5":
      - /url: "#"
- text: Rated 4 out of 5
- status: HTML5 added. Cart has 1 item.
```

12.5 Testing accessibility

Automated tools find only about **30 to 40 percent** of WCAG issues (missing alt, low contrast, missing labels); whether alt text is *meaningful* or the focus order *makes sense* needs a human. Use all three: automated scans, keyboard testing and a screen reader.

12.5.1 Keyboard-only testing

1. Put the mouse away. Click the address bar, then press Tab.
2. Is the first stop a skip link? Does it work?
3. Can you see where focus is at every step?
4. Does focus move in a logical order matching the layout?
5. Can you reach and operate everything: menus, dialogs, carousels, forms, custom widgets (Enter, Space, arrows, Esc)?
6. Is there any keyboard trap (focus stuck in a widget or iframe)?
7. When a dialog opens, does focus move into it, stay inside, and return to the trigger on close?

12.5.2 Screen readers (NVDA, VoiceOver)

Screen reader	Platform	Start	Key commands
NVDA (free)	Windows	Ctrl+Alt+N	H / Shift+H headings, D landmarks, K links, F form fields, Insert+F7 elements list, Ctrl stop speech
VoiceOver (built in)	macOS / iOS	Cmd+F5 / triple-click side button	VO (Ctrl+Option) + U rotor, VO + arrows to read; swipe on iOS
TalkBack (built in)	Android	hold both volume keys	swipe to move, double-tap to activate
JAWS (paid)	Windows		similar keys to NVDA; widely used in enterprises

12.5.3 Lighthouse and axe DevTools

Lighthouse is built into Chrome DevTools (Lighthouse tab, choose "Accessibility", Analyze page load) and gives a 0-100 score with a list of failed audits; its accessibility checks are powered by Deque's open-source **axe-core** engine. The **axe DevTools** browser extension runs the full axe rule set and explains each issue with the WCAG criterion and how to fix it. WAVE (WebAIM) is another popular extension.

We can write a tiny audit of our own that checks a few of the same rules axe checks, and run it in Chromium against a deliberately broken page:

A MINI ACCESSIBILITY AUDIT (RULE NAMES AS IN AXE-CORE) main.js

```
const issues = [];
document.querySelectorAll("img:not([alt])").forEach(el => issues.push(["image-alt", el.outerHTML]));
document.querySelectorAll("input:not([type=hidden]), select, textarea").forEach(el => {
  if (!el.labels.length && !el.getAttribute("aria-label") && !el.getAttribute("aria-labelledby"))
    issues.push(["label", el.outerHTML]);
});
document.querySelectorAll("button, a[href]").forEach(el => {
  if (!el.textContent.trim() && !el.getAttribute("aria-label") &&
!el.querySelector("img[alt]:not([alt=''])"))
    issues.push([el.tagName === "A" ? "link-name" : "button-name", el.outerHTML]);
});
if (!document.documentElement.lang) issues.push(["html-has-lang", "<html>"]);
const levels = [...document.querySelectorAll("h1,h2,h3,h4,h5,h6")].map(h => +h.tagName[1]);
levels.forEach((l, i) => { if (i && l > levels[i - 1] + 1) issues.push(["heading-order", "h" +
levels[i - 1] + " -> h" + l]); });
console.log(issues.length + " issues found");
issues.forEach(([rule, html]) => console.log(rule.padEnd(14), html));
```

BROWSER CONSOLE

```
6 issues found
image-alt      <img src="banner.jpg">
label          <input type="text" placeholder="Name">
button-name    <button><svg width="16" height="16"></svg></button>
link-name      <a href="/help"></a>
html-has-lang  <html>
heading-order  h1 -> h3
```

Real tools report in the same style but with far more rules (around 90 in axe-core). A typical Lighthouse result for the same broken page:

LIGHTHOUSE > ACCESSIBILITY (SUMMARY) index.html

```
DevTools > Lighthouse > Mode: Navigation, Device: Mobile, Categories: Accessibility > Analyze page
load
```

LIGHTHOUSE REPORT

```
Accessibility   52

  NAMES AND LABELS
  x  Image elements do not have [alt] attributes
  x  Form elements do not have associated labels
  x  Buttons do not have an accessible name
  x  Links do not have a discernible name
  NAVIGATION
  x  Heading elements are not in a sequentially-descending order
  INTERNATIONALIZATION AND LOCALIZATION
  x  <html> element does not have a [lang] attribute

  Additional items to manually check (10)
```

TIP

Add automated checks to your workflow: run Lighthouse on every important page before release, use axe in tests (axe-core with Playwright or Cypress), and keep a short manual checklist (keyboard + screen reader) for every new component.

PRACTICE CHECKPOINT

Audit the registration form and blog page with Lighthouse and axe, fix every issue found, and navigate both pages using only the keyboard.

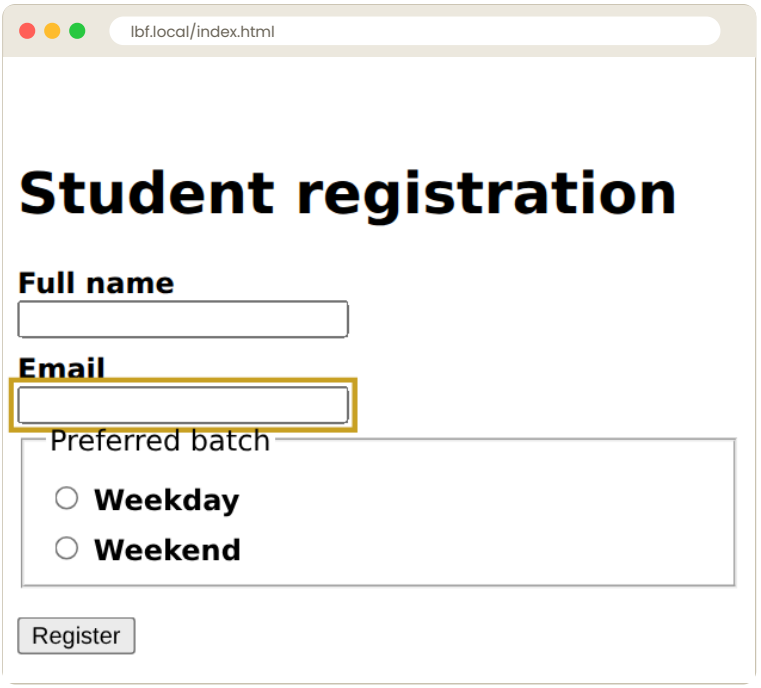
Fixes typically needed for the beginner versions of these pages, and the keyboard check after fixing:

Issue found	Fix
Images without alt / logo link with no name	meaningful alt; alt="" for decorative images; logo alt "LBF home"
Inputs with placeholder only	`<label for>` for every field
Radio group without a group label	wrap in fieldset with legend
Low-contrast grey hint text	darker colour (at least 4.5:1)
Errors shown only in red	text message linked with aria-describedby, aria-invalid
Missing lang, skipped heading level	`<html lang="en">`, correct h1-h2-h3
No skip link, outline removed	skip link to main; :focus-visible style

REGISTER.HTML (ACCESSIBLE VERSION, EXCERPT) HTML

```
<style>

.skip{position:absolute;left:-9999px}.skip:focus{left:8px;top:6px;background:#1a2a4f;color:#fff;paddin
g:6px 10px}
  body{font-family:system-ui,sans-serif;padding-top:28px}
  :focus-visible{outline:3px solid #c9a227;outline-offset:2px}
  label{display:block;margin-top:8px;font-weight:600} .hint{color:#444;font-size:13px}
</style>
<a class="skip" href="#main">Skip to main content</a>
<main id="main" tabindex="-1">
  <h1>Student registration</h1>
  <form>
    <label for="n">Full name</label><input id="n" autocomplete="name" required>
    <label for="e">Email</label><input id="e" type="email" autocomplete="email" required>
    <fieldset><legend>Preferred batch</legend>
      <label><input type="radio" name="b" value="wd"> Weekday</label>
      <label><input type="radio" name="b" value="we"> Weekend</label></fieldset>
    <p><button>Register</button></p>
  </form>
</main>
```



KEYBOARD TEST: AFTER TAB, ENTER ON THE SKIP LINK, THEN TAB TWICE, FOCUS IS ON THE EMAIL FIELD WITH A VISIBLE RING

LEVEL 3

Advanced level

Outcome: optimise performance, use modern interactive elements and HTML5 APIs, follow best practices and deploy a professional portfolio.

5 MODULES · 10 DAYS MINIMUM

Module 13 · Responsive images and performance	**1 day**
Module 14 · Interactive elements and modern HTML	**2 days**
Module 15 · HTML5 APIs with JavaScript	**3 days**
Module 16 · Best practices, validation and tooling	**1 day**
Module 17 · Projects and interview preparation	**3 days**

MODULE 13 · ADVANCED

Responsive images and performance

Minimum duration **1 day** | Level **Advanced**

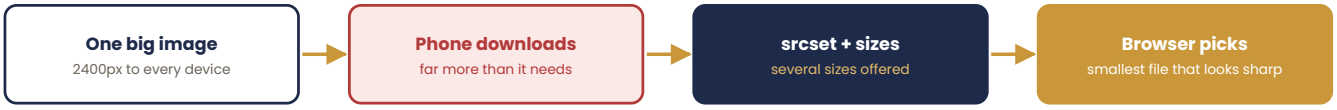
Fast pages keep users. This module covers delivering the right image to each device with srcset (width and pixel-density descriptors), sizes and the picture element for art direction and modern formats (WebP, AVIF), then loading strategies: lazy loading, decoding="async", fetchpriority, resource hints (preload, prefetch, preconnect, dns-prefetch) and script loading with defer and async. It ends with measuring performance through the Core Web Vitals (LCP, INP, CLS), Lighthouse and PageSpeed Insights. Which image the browser picked, and which requests it made, are measured in Chromium, not assumed.

IN THIS MODULE

13.1 Responsive images
13.2 Loading strategies
13.3 Measuring performance

13.1 Responsive images

Images are usually the heaviest part of a page. Sending a 2400-pixel photo to a 360-pixel phone wastes data, battery and time; sending a 400-pixel photo to a 4K laptop looks blurry. Responsive images let the **browser** choose the best file, because only the browser knows the screen size, pixel density, zoom and network conditions at load time.



FROM ONE-SIZE-FITS-ALL TO BROWSER-CHOSEN IMAGES

13.1.1 srcset with width descriptors

`srcset` lists candidate files with their **intrinsic width** in pixels, written with a `w` descriptor: `hills-400.jpg 400w`. The browser combines this with how wide the image will be displayed (from `sizes`, next section) and the device pixel ratio to choose a file. `src` remains as the fallback for old browsers.

SRCSET WITH WIDTH DESCRIPTORS — HTML

```
<img src="hills-800.jpg"
     srcset="hills-400.jpg 400w,
             hills-800.jpg 800w,
             hills-1200.jpg 1200w"
     sizes="100vw"
     width="1200" height="800"
     alt="Green hills under a blue sky">
```

The same page loaded in Chromium at different viewport widths and pixel densities. The browser needs (display width × device pixel ratio) pixels and picks the smallest candidate that is at least that wide:

WHICH FILE DID CHROMIUM DOWNLOAD? index.html

```
<!-- the img above, loaded in Chromium at five viewport sizes and pixel ratios -->
```

MEASURED IN CHROMIUM

```
viewport, DPR              needs        chose
phone 360px, DPR 1         360px        hills-400.jpg
phone 360px, DPR 2         720px        hills-800.jpg
phone 412px, DPR 3         1236px       hills-1200.jpg
tablet 768px, DPR 1        768px        hills-800.jpg
laptop 1280px, DPR 1       1280px       hills-1200.jpg
```

NOTE

Browsers may choose differently when a larger file is already cached, or (in some browsers) on slow connections or with data saver on. That freedom is the point: you describe the options, the browser decides.

13.1.2 sizes attribute

The browser chooses images **before** CSS is downloaded and laid out, so it does not yet know how wide the image will be. `sizes` tells it: a comma-separated list of **media condition + slot width**, with a final default. With `sizes="100vw"` the browser assumes full width, which over-downloads for images in columns.

SIZES DESCRIBING THE LAYOUT HTML

```
<!-- Full width on phones, half width on tablets, a 400px column on laptops -->
<img src="hills-800.jpg"
     srcset="hills-400.jpg 400w, hills-800.jpg 800w, hills-1200.jpg 1200w"
     sizes="(max-width: 600px) 100vw,
            (max-width: 1000px) 50vw,
            400px"
     width="1200" height="800" alt="Green hills">
```

THE EFFECT OF SIZES index.html

```
<!-- the img with sizes above, loaded at five viewport sizes and pixel ratios -->
```

MEASURED IN CHROMIUM

```
viewport, DPR              slot       needs      chose
360px, DPR 2               360px      720px      hills-800.jpg
800px, DPR 1               400px      400px      hills-400.jpg
800px, DPR 2               400px      800px      hills-800.jpg
1280px, DPR 1              400px      400px      hills-400.jpg
1280px, DPR 2              400px      800px      hills-800.jpg
```

On a 1280px laptop the image is shown only 400px wide, so the browser downloads the 400w file instead of the 1200w file it would take with `sizes="100vw"`: 5.0 KB instead of 24.9 KB for this (very simple) test image, and typically several hundred KB for a real photograph.

13.1.3 Pixel-density descriptors (1x, 2x)

For images that are always shown at the **same CSS size** (a logo, an avatar, an icon), list files by pixel density with `x` descriptors instead. No `sizes` is needed.

DENSITY DESCRIPTORS HTML

```
<img src="avatar-80.png"
     srcset="avatar-80.png 1x, avatar-160.png 2x, avatar-240.png 3x"
     width="80" height="80" alt="Asha Rao">
```

Use	Descriptor	Needs sizes?
Image width changes with the layout (photos, banners, cards)	`w` (400w, 800w)	yes
Image always the same CSS size (logos, avatars, icons)	`x` (1x, 2x, 3x)	no
Never mix both in one srcset		

13.1.4 <picture> for art direction and modern formats

`<picture>` wraps one `<img>` and any number of `<source>` elements. The browser takes the **first source that matches** (by `media` and/or `type`) and puts its file into the img; the img provides alt, width, height and the final fallback. Two uses:

- **Art direction**: different *crops* for different screens (a wide landscape on desktop, a tight square on phones) using `media`.
- **Modern formats**: offer AVIF and WebP (much smaller) with JPEG as the fallback, using `type`.

FORMAT SWITCHING HTML

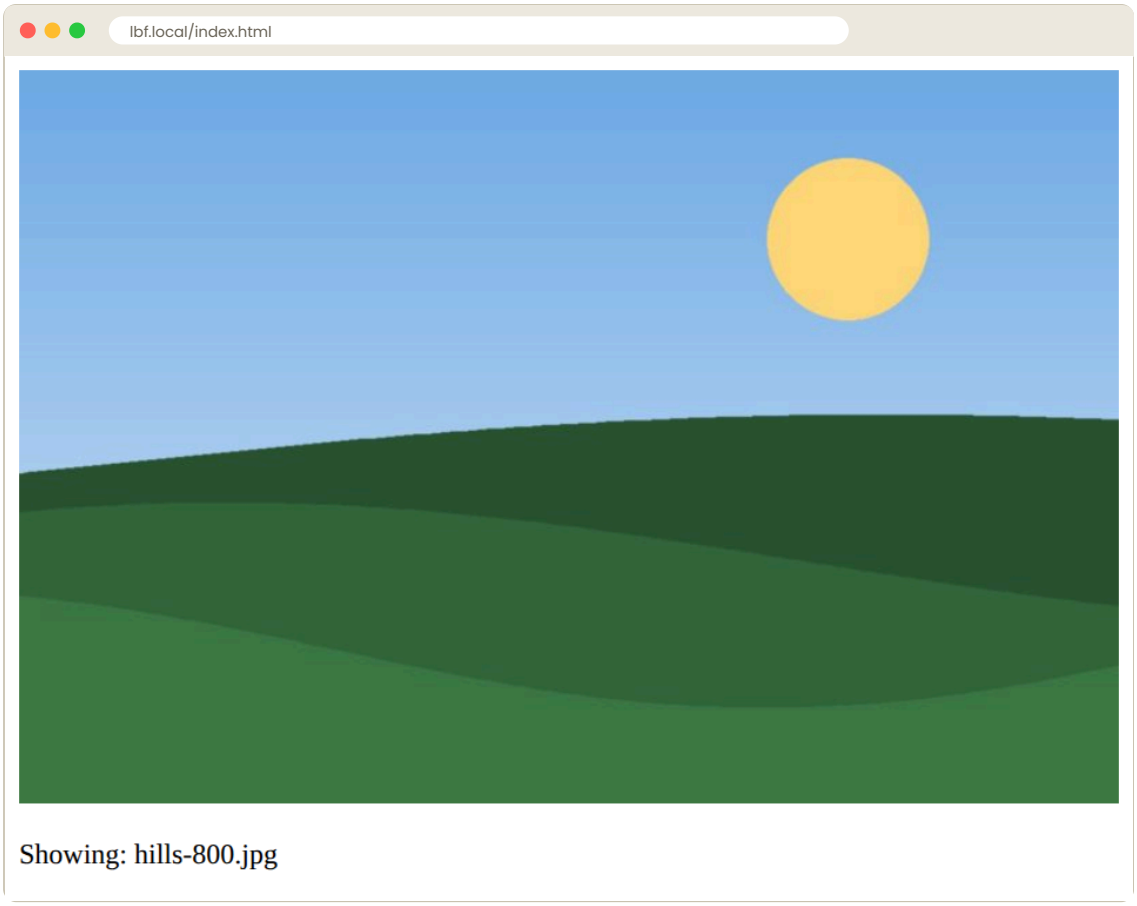
```
<picture>
  <source type="image/avif" srcset="hills-400.avif 400w, hills-800.avif 800w, hills-1200.avif 1200w"
sizes="100vw">
  <source type="image/webp" srcset="hills-400.webp 400w, hills-800.webp 800w, hills-1200.webp 1200w"
sizes="100vw">
  <img src="hills-800.jpg" srcset="hills-400.jpg 400w, hills-800.jpg 800w, hills-1200.jpg 1200w"
sizes="100vw"
       width="1200" height="800" alt="Green hills under a blue sky">
</picture>
```

File (1200px wide test image)	Size
hills-1200.jpg	24.9 KB
hills-1200.webp	5.2 KB
hills-1200.avif	5.1 KB

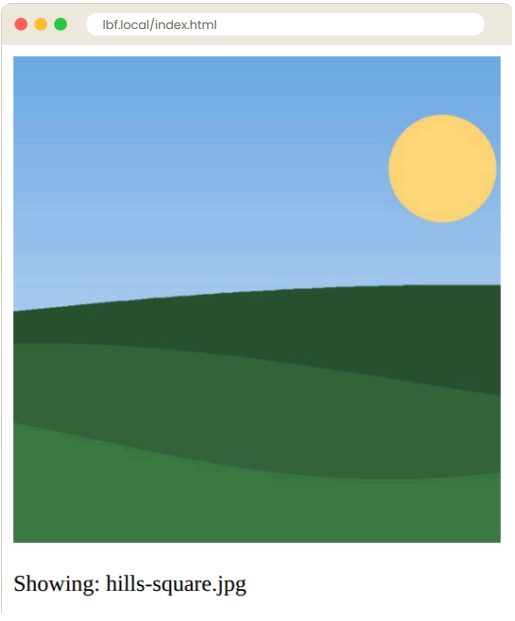
(These sizes are measured on the book's simple illustration; for real photographs WebP is typically 25-35 percent smaller than JPEG and AVIF about 50 percent smaller at similar quality.) Art direction with `media`:

ART DIRECTION: A SQUARE CROP ON NARROW SCREENS HTML

```
<picture>
  <source media="(max-width: 500px)" srcset="hills-square.jpg">
  <img src="hills-800.jpg" alt="Green hills under a blue sky" style="width:100%;height:auto">
</picture>
<p id="o"></p>
<script>addEventListener("load", () => o.textContent = "Showing: " +
document.querySelector("img").currentSrc.split("/").pop());</script>
```



AT 640PX WIDE THE LANDSCAPE VERSION IS USED



THE SAME PAGE AT 360PX WIDE: THE BROWSER SWITCHED TO THE SQUARE CROP

13.2 Loading strategies

13.2.1 loading="lazy" for images and iframes

`loading="lazy"` tells the browser to fetch an image or iframe only when it gets **close to the viewport**. Images far down a long page are never downloaded if the user does not scroll there. Do **not** lazy-load the main image visible on first load (the hero / LCP image), because lazy loading delays it.

REQUESTS MADE BY A LAZY-LOADING GALLERY index.html

```
<!-- 24 photos, each 400px tall with 300px gaps (a page about 17,000px long) -->
<img src="hills-800.jpg?photo=1" loading="lazy" width="600" height="400" alt="Photo 1">
...
```

MEASURED IN CHROMIUM

```
viewport 700px tall       images downloaded on load: 6 of 24
same page, no lazy        images downloaded on load: 24 of 24
```

NOTE

Chromium loads lazy images that are within a generous distance of the viewport (over 1000 px on fast connections), so the first few below the fold are still fetched early, and scrolling feels instant.

13.2.2 decoding="async"

After downloading, an image must be **decoded** (decompressed into pixels). `decoding="async"` allows the browser to decode it off the main thread and show the rest of the content without waiting; `sync` asks to decode it together with surrounding content; `auto` (default) lets the browser choose. Modern browsers already decode most images asynchronously, so the attribute has a small effect; it is harmless on large, non-critical images.

13.2.3 fetchpriority

Browsers assign every request a priority. `fetchpriority="high"` raises it for the most important resource, usually the **LCP hero image** (which otherwise starts at low priority until layout shows it is visible); `fetchpriority="low"` lowers it for things like carousel images that are not shown first. It works on img, link (preload) and script, and on `fetch()` calls with `priority`.

PRIORITISING THE HERO IMAGE HTML

```
<!-- The hero image: discovered early, loaded at high priority, never lazy -->
<img src="hero-1200.jpg" srcset="hero-800.jpg 800w, hero-1200.jpg 1200w" sizes="100vw"
     width="1200" height="600" alt="Students at LBF" fetchpriority="high">

<!-- Hidden carousel slides: load later -->
<img src="slide-2.jpg" alt="..." loading="lazy" fetchpriority="low">
```

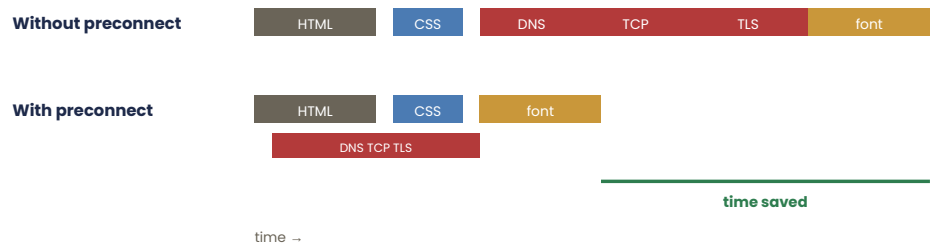
13.2.4 Resource hints: preload, prefetch, preconnect, dns-prefetch

Hint	What the browser does	Use for
`<link rel="preload">`	downloads a resource needed by **this** page early, at high priority (needs `as`)	web fonts, the hero image referenced only from CSS, critical scripts
`<link rel="prefetch">`	downloads at idle, low priority, for a **likely next** page	the next step of a checkout, the article the user will probably open
`<link rel="preconnect">`	opens the connection (DNS + TCP + TLS) to another origin early	Google Fonts, your CDN, API origin (only 2-3 important origins)
`<link rel="dns-prefetch">`	only resolves the domain name	many third-party origins; cheaper fallback for preconnect
`<link rel="modulepreload">`	preloads and parses a JavaScript module	modules needed at start

RESOURCE HINTS IN THE HEAD HTML

```html
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link rel="dns-prefetch" href="https://www.google-analytics.com">
<link rel="preload" href="/fonts/poppins-600.woff2" as="font" type="font/woff2" crossorigin>
<link rel="preload" href="/img/hero-1200.webp" as="image" fetchpriority="high">
<link rel="prefetch" href="/courses/html5/syllabus">
```

WATCH OUT
Every preload competes for bandwidth with everything else. Preloading too much makes pages **slower**, and Chrome warns in the console when a preloaded resource is not used within a few seconds.



PRECONNECT OVERLAPS THE CONNECTION SET-UP WITH OTHER WORK (SCHEMATIC)

13.2.5 Script loading with defer and async

As shown in 11.2.2, plain scripts in the head block parsing. Performance rules of thumb:

- Your own scripts: `defer` (or `type="module"`), placed in the head so they are discovered early.
- Independent third-party scripts (analytics, chat widgets): `async`, and load them as late as practical.
- Inline only tiny critical scripts; move everything else to cached files.
- Remove unused JavaScript: every kilobyte of JS costs download, parse and execution time, especially on low-end Android phones.

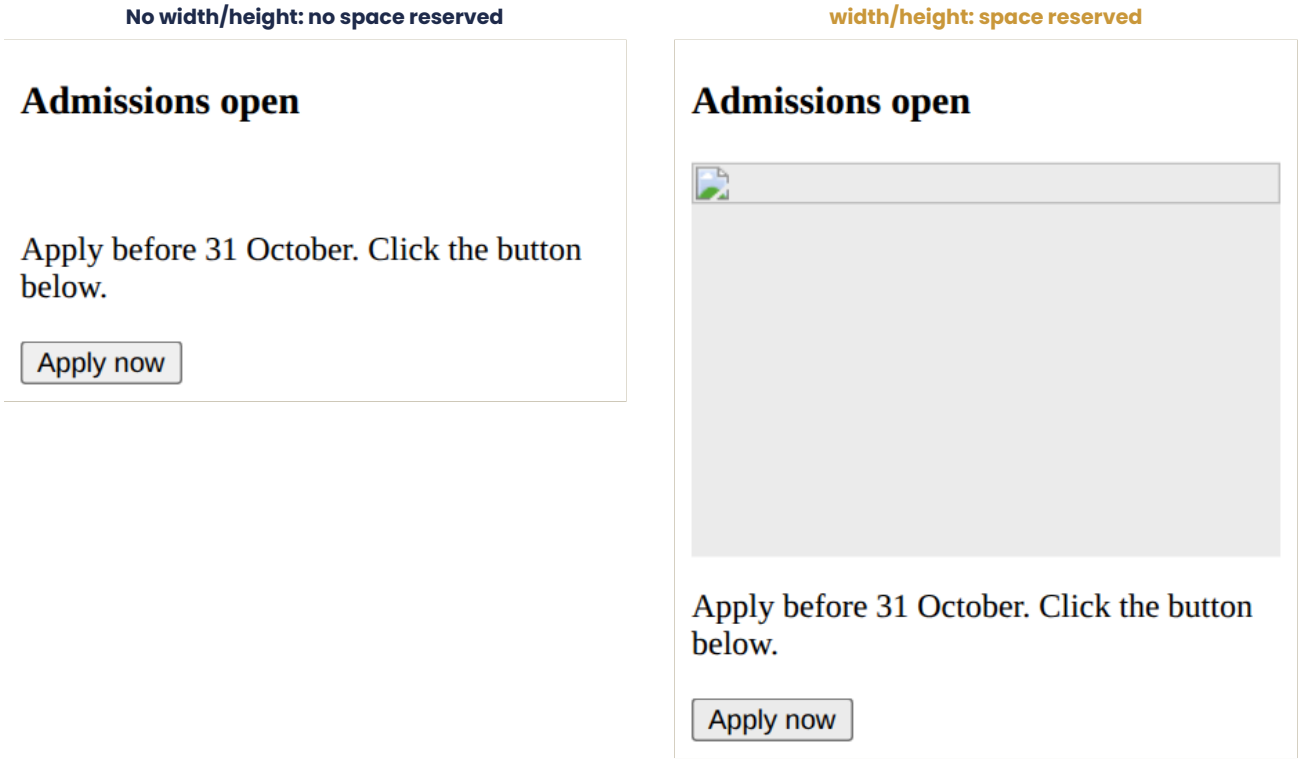
13.3 Measuring performance

13.3.1 Core Web Vitals: LCP, INP, CLS

Google's **Core Web Vitals** are three user-centred metrics that measure loading, responsiveness and visual stability. They are part of Google's page experience signals, and they are measured from real users (field data from the Chrome User Experience Report) at the 75th percentile of page loads.

Metric	Measures	Good	Poor	Common HTML fixes
LCP Largest Contentful Paint	when the largest image or text block becomes visible	≤ 2.5 s	> 4.0 s	fast server, fetchpriority="high" on the hero, no lazy hero, responsive/modern images, preload fonts
INP Interaction to Next Paint (replaced FID in March 2024)	delay from a click/tap/key to the next visual update	≤ 200 ms	> 500 ms	less JavaScript, defer scripts, break up long tasks, native elements
CLS Cumulative Layout Shift	how much visible content unexpectedly moves	≤ 0.1	> 0.25	width/height on images and iframes, reserve space for ads and embeds, font-display

Images without dimensions are the classic cause of layout shift: the text is laid out, then the image arrives and pushes it down. Setting `width` and `height` lets the browser reserve the right space (it computes the aspect ratio before the image loads):



BOTH PAGES BEFORE THE IMAGE HAS ARRIVED: ON THE LEFT THE BUTTON WILL JUMP DOWN WHEN THE IMAGE LOADS

The browser exposes these metrics to JavaScript through `PerformanceObserver`. This script ran in Chromium on a page with a large hero image and reports what was measured:

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```html
<h1>Learn Being Forward</h1>
<img src="hero.jpg" width="1200" height="800" alt="Campus" fetchpriority="high"
style="width:100%;height:auto">
```

OBSERVING LCP AND CLS main.js

```
new PerformanceObserver(list => {
  for (const e of list.getEntries())
    console.log("LCP candidate:", e.element.tagName, e.url ? e.url.split("/").pop() : "(text)",
                "at", Math.round(e.startTime), "ms");
}).observe({ type: "largest-contentful-paint", buffered: true });
let cls = 0;
new PerformanceObserver(list => list.getEntries().forEach(e => { if (!e.hadRecentInput) cls +=
e.value; }))
  .observe({ type: "layout-shift", buffered: true });
setTimeout(() => console.log("CLS so far:", cls.toFixed(3)), 600);
```

BROWSER CONSOLE

```
LCP candidate: IMG hero.jpg at 48 ms
CLS so far: 0.000
```

NOTE

Timings depend on the machine; here everything is local, so LCP is very fast. On a real site, use the field data in PageSpeed Insights or Google Search Console, and the open-source **web-vitals** JavaScript library to collect metrics from your own users.

13.3.2 Lighthouse and PageSpeed Insights

Lighthouse (Chrome DevTools > Lighthouse) runs a **lab** test: it loads the page on a simulated mid-range phone with a throttled network and reports a 0-100 Performance score built from metrics such as First Contentful Paint, LCP, Total Blocking Time (a lab stand-in for INP), CLS and Speed Index, followed by specific opportunities. **PageSpeed Insights** (pagespeed.web.dev) runs Lighthouse on Google's servers and also shows **field data** from real Chrome users when the site has enough traffic.

Lighthouse opportunity	HTML-level fix
Properly size images	srcset + sizes
Serve images in next-gen formats	picture with AVIF/WebP sources
Defer offscreen images	loading="lazy"
Largest Contentful Paint image was lazily loaded	remove loading="lazy" from the hero, add fetchpriority="high"
Eliminate render-blocking resources	defer/async scripts, media on non-critical CSS
Image elements do not have explicit width and height	add width and height
Preconnect to required origins	link rel="preconnect"
Avoid enormous network payloads	compress, remove unused JS/CSS, lazy-load embeds

TIP

Always test in an **Incognito** window (extensions slow pages down and skew results), use the **Mobile** preset, and run it a few times: lab scores vary by a few points between runs.

PRACTICE CHECKPOINT

Optimise the **photo gallery** with srcset, sizes, a WebP/AVIF picture fallback and lazy loading, and improve its Lighthouse performance score.

SAMPLE SOLUTION: AN OPTIMISED GALLERY

HTML

```
<!-- gallery.html (one figure; repeat for each photo) -->
<main>
  <h1>Mysuru in pictures</h1>

  <!-- First image is above the fold: high priority, not lazy -->
  <figure>
    <picture>
      <source type="image/avif" srcset="img/palace-400.avif 400w, img/palace-800.avif 800w,
img/palace-1200.avif 1200w"
              sizes="(max-width: 700px) 100vw, 700px">
      <source type="image/webp" srcset="img/palace-400.webp 400w, img/palace-800.webp 800w,
img/palace-1200.webp 1200w"
              sizes="(max-width: 700px) 100vw, 700px">
      <img src="img/palace-800.jpg"
           srcset="img/palace-400.jpg 400w, img/palace-800.jpg 800w, img/palace-1200.jpg 1200w"
           sizes="(max-width: 700px) 100vw, 700px"
           width="1200" height="800" alt="Mysuru Palace illuminated at night" fetchpriority="high">
    </picture>
    <figcaption>Mysuru Palace on a Sunday evening</figcaption>
  </figure>

  <!-- Every later image: lazy, async decoding -->
  <figure>
    <picture>
      <source type="image/avif" srcset="img/hills-400.avif 400w, img/hills-800.avif 800w" sizes="(max-
width: 700px) 100vw, 700px">
      <source type="image/webp" srcset="img/hills-400.webp 400w, img/hills-800.webp 800w" sizes="(max-
width: 700px) 100vw, 700px">
      <img src="img/hills-800.jpg" srcset="img/hills-400.jpg 400w, img/hills-800.jpg 800w"
           sizes="(max-width: 700px) 100vw, 700px" width="1200" height="800"
           alt="Chamundi Hills seen from the city" loading="lazy" decoding="async">
    </picture>
    <figcaption>Chamundi Hills</figcaption>
  </figure>
</main>
```

Generating the files is quick with a command-line tool. For example, with ImageMagick or Squoosh CLI, create 400, 800 and 1200-pixel versions of each photo in JPEG, WebP and AVIF; then compare the Lighthouse Performance score and the "Total byte weight" before and after. A typical photo gallery drops from several megabytes to a few hundred kilobytes on phones.

MODULE 14 · ADVANCED

Interactive elements and modern HTML

Minimum duration **2 days** | Level **Advanced**

Recent HTML features provide dialogs, popovers, accordions and components with little or no JavaScript. This module covers details and summary, exclusive accordions with the name attribute, the dialog element with show(), showModal() and close(), the ::backdrop pseudo-element, forms with method="dialog", the popover attribute with popovertarget and popovertargetaction, invoker commands (command and commandfor), the inert, contenteditable, spellcheck, translate, draggable and hidden="until-found" attributes, and templates and Web Components: template and slot, custom elements, Shadow DOM and Declarative Shadow DOM. Every widget is shown in its real state in Chromium 141 after real clicks.

IN THIS MODULE

14.1 Disclosure widgets
14.2 Dialogs
14.3 Popovers
14.4 Other modern attributes
14.5 Templates and Web Components

14.1 Disclosure widgets

14.1.1 <details> and <summary>

`<details>` creates a **disclosure widget**: content hidden until the user opens it. Its first child `<summary>` is the always-visible clickable label (with a triangle marker). The `open` attribute shows it open initially and is added and removed as the user toggles it; the `toggle` event fires each time. Everything (mouse, keyboard with Enter/Space, screen-reader announcement "expanded/collapsed", and find-in-page) works with zero JavaScript.

FAQ WITH DETAILS AND SUMMARY — HTML

```html
<details>
  <summary>What is the course duration?</summary>
  <p>The HTML5 course takes 28 days minimum: 9 beginner, 9 intermediate and 10 advanced.</p>
</details>
<details open>
  <summary>Is there a certificate?</summary>
  <p>Yes. You receive a certificate after completing the capstone project.</p>
</details>
<details>
  <summary>Can I attend online?</summary>
  <p>Yes, every batch is available in the classroom and online.</p>
</details>
```



THE FIRST ITEM WAS CLICKED OPEN; THE SECOND HAS THE OPEN ATTRIBUTE

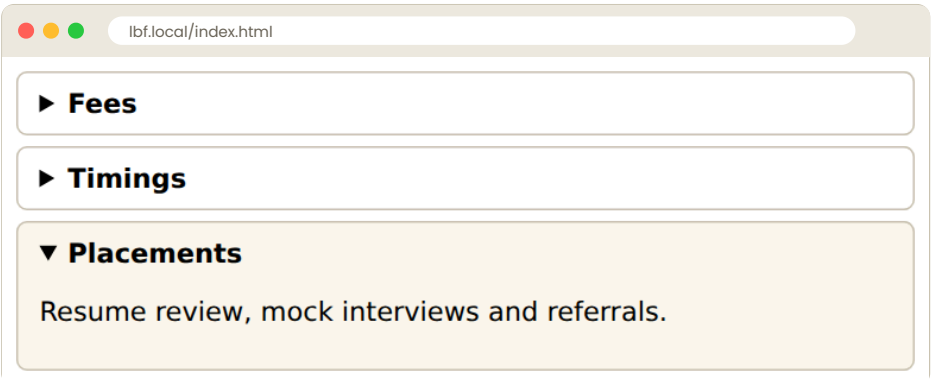
The marker can be styled or replaced with CSS (`summary::marker` or `list-style: none` plus your own icon), and JavaScript can read and set `details.open`.

14.1.2 Exclusive accordions with the name attribute

Give several `<details>` elements the **same** `name` and they form an **exclusive accordion**: opening one automatically closes the others in the group, like radio buttons. This is supported in all major browsers since 2024 and used to need JavaScript.

AN EXCLUSIVE ACCORDION HTML

```
<details name="faq" open>
  <summary>Fees</summary><p>Rs 9,999, payable in two instalments.</p>
</details>
<details name="faq">
  <summary>Timings</summary><p>Weekday batch 6-8 pm, weekend batch 10 am-1 pm.</p>
</details>
<details name="faq">
  <summary>Placements</summary><p>Resume review, mock interviews and referrals.</p>
</details>
```



AFTER CLICKING "PLACEMENTS": IT OPENED AND "FEES" CLOSED AUTOMATICALLY

14.2 Dialogs

14.2.1 <dialog> element

`<dialog>` is a native pop-up box: alerts, confirmations, login forms, image lightboxes. It is hidden until opened. A dialog can be **non-modal** (the rest of the page stays usable) or **modal** (the rest of the page becomes inert, a backdrop covers it, and focus is trapped inside). The browser handles the hard accessibility parts: the dialog role, moving focus in, Esc to close, returning focus to the trigger and the top layer (always above everything, no z-index fights).

14.2.2 show(), showModal(), close()

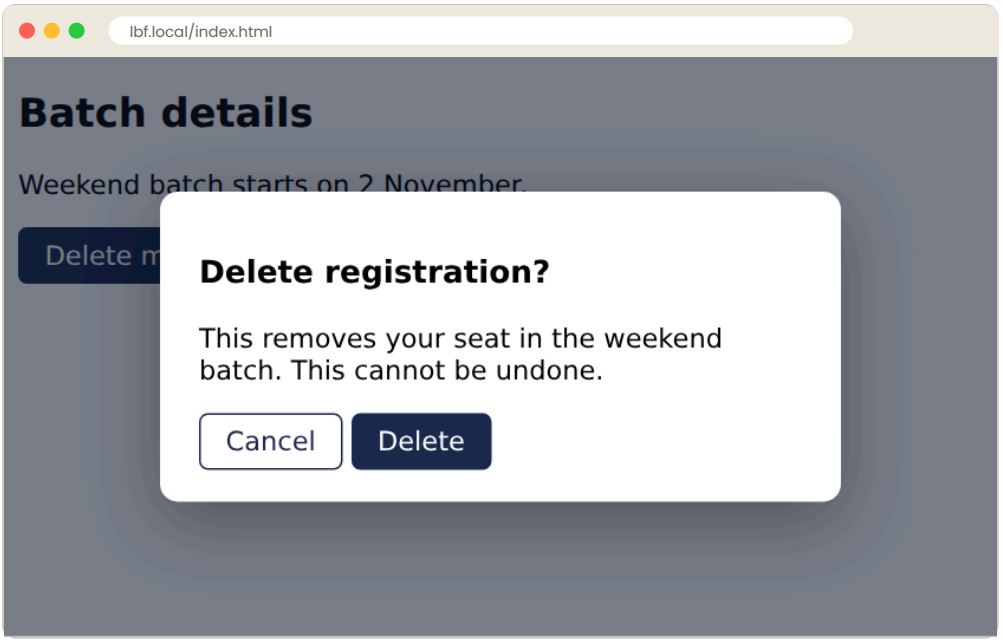
Method / attribute	Effect
`dialog.show()`	opens as **non-modal**; page remains interactive
`dialog.showModal()`	opens as **modal** in the top layer with a ::backdrop; Esc closes; outside is inert
`dialog.close(value)`	closes; optional value stored in `dialog.returnValue`
`open` attribute	reflects state; do not add it manually to open a modal (use showModal)
`close` / `cancel` events	`close` after closing; `cancel` when Esc is pressed (can be prevented)
`closedby` attribute	`any` also closes on a click outside (light dismiss), `closerequest` (Esc only), `none`; new in Chromium 134

A MODAL CONFIRMATION DIALOG HTML

```html
<h2>Batch details</h2>
<p>Weekend batch starts on 2 November.</p>
<button id="open">Delete my registration</button>

<dialog id="confirm" aria-labelledby="dt">
  <h3 id="dt">Delete registration?</h3>
  <p>This removes your seat in the weekend batch. This cannot be undone.</p>
  <button class="ghost" id="cancel" autofocus>Cancel</button>
  <button id="ok">Delete</button>
</dialog>

<script>
  const d = document.getElementById("confirm");
  document.getElementById("open").onclick = () => d.showModal();
  document.getElementById("cancel").onclick = () => d.close("cancel");
  document.getElementById("ok").onclick = () => d.close("delete");
</script>
```



AFTER CLICKING THE BUTTON: THE MODAL IS IN THE TOP LAYER, THE PAGE IS DIMMED BY ::BACKDROP AND FOCUS IS ON CANCEL

14.2.3 ::backdrop

A modal dialog (and a popover, and full-screen elements) gets a `::backdrop` pseudo-element that covers the whole viewport beneath it. Style it to dim or blur the page:

STYLING THE BACKDROP text

```
dialog::backdrop {
  background: rgb(10 20 40 / 0.55);
  backdrop-filter: blur(3px);
}
```

14.2.4 Forms with method="dialog"

A form inside a dialog with `method="dialog"` does not send anything to a server: submitting it **closes the dialog**, and the `value` of the button that was pressed becomes `dialog.returnValue`. A complete working confirm or choice dialog therefore needs only a few lines of script to open it and read the answer.

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```
<dialog id="rate">
  <form method="dialog">
    <p>How was today's class?</p>
    <button value="good">Good</button>
    <button value="ok">OK</button>
    <button value="bad">Needs work</button>
  </form>
</dialog>
```

A RATING DIALOG WITH METHOD="DIALOG" main.js

```
const d = document.getElementById("rate");
d.addEventListener("close", () => console.log("Dialog closed, returnValue =",
JSON.stringify(d.returnValue)));
d.showModal();
console.log("Dialog open?", d.open);
```

BROWSER CONSOLE

```
Dialog open? true
Dialog closed, returnValue = "good"
```

Simulated user actions, in order: clicks `button[value=good]`; waits 200 ms.

14.3 Popovers

14.3.1 The popover attribute

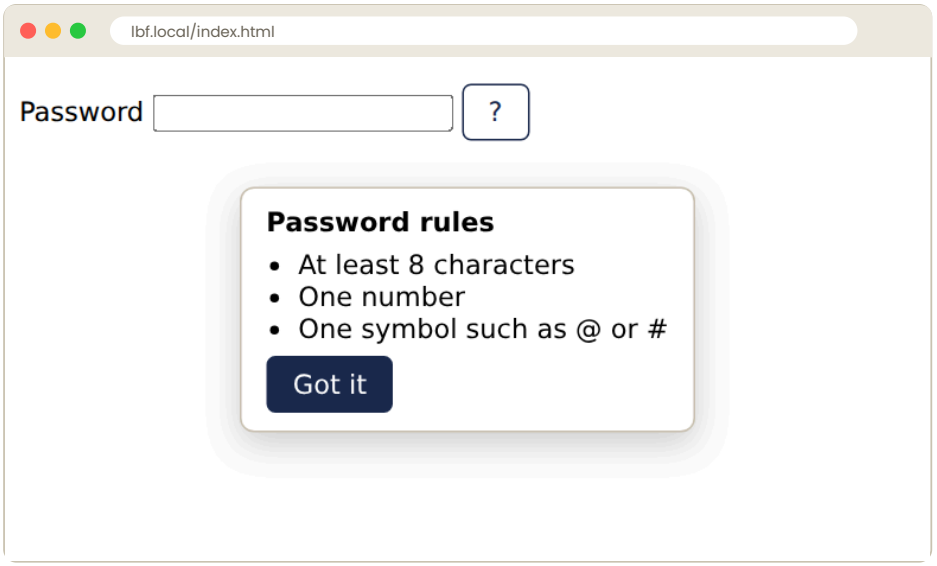
The `popover` attribute (all major browsers since 2024) turns any element into a **popover**: hidden by default, shown in the top layer above everything when triggered. The default `popover` (= `"auto"`) supports **light dismiss**: clicking outside or pressing Esc closes it, and opening one auto popover closes other unrelated ones. `popover="manual"` stays open until explicitly closed (toasts). `popover="hint"` (newer, Chromium) is for tooltips that do not close auto popovers.

14.3.2 popovertarget and popovertargetaction

A button with `popovertarget="id"` controls the popover **without any JavaScript**; `popovertargetaction` can be `toggle` (default), `show` or `hide`. The browser also adds the right accessibility relationship (aria-expanded and aria-details) to the button.

A HELP POPOVER WITHOUT JAVASCRIPT HTML

```
<p>Password <input type="password" size="18">
  <button class="ghost" popovertarget="pw-help" aria-label="Password rules">?</button></p>
<div id="pw-help" popover>
  <strong>Password rules</strong>
  <ul style="margin:6px 0;padding-left:18px">
    <li>At least 8 characters</li><li>One number</li><li>One symbol such as @ or #</li>
  </ul>
  <button popovertarget="pw-help" popovertargetaction="hide">Got it</button>
</div>
```



AFTER CLICKING "?": THE POPOVER IS SHOWN IN THE TOP LAYER (CENTRED BY DEFAULT; CSS ANCHOR POSITIONING CAN PLACE IT NEXT TO THE BUTTON)

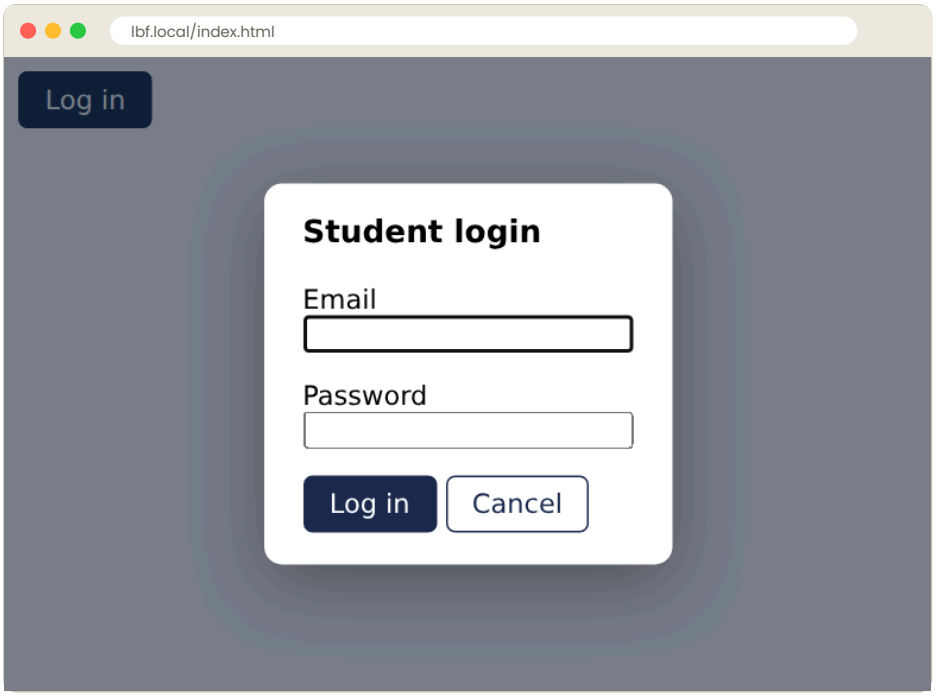
	popover	<dialog> with showModal()
Blocks the rest of the page	no	yes (modal)
Light dismiss (click outside)	yes (auto)	only with closedby="any"
Needs JavaScript to open	no (popovertarget)	not with invoker commands (14.3.3)
Use for	menus, tooltips, help, pickers, toasts	tasks requiring attention: confirmations, forms

14.3.3 Invoker commands (command and commandfor, overview)

Invoker commands generalise popovertarget: a button with `commandfor="id"` and `command="..."` performs an action on another element declaratively. Built-in commands include `show-modal`, `close` and `request-close` for dialogs, and `toggle-popover`, `show-popover`, `hide-popover` for popovers. Custom commands start with `--` and are handled with the `command` event. Shipped in Chromium 135 (2025) and Firefox 144 and Safari 26 later in 2025.

A LOGIN MODAL WITH ZERO JAVASCRIPT HTML

```html
<button commandfor="login" command="show-modal">Log in</button>
<dialog id="login" closedby="any">
  <form method="dialog">
    <h3 style="margin-top:0">Student login</h3>
    <p><label>Email<br><input type="email" autocomplete="username"></label></p>
    <p><label>Password<br><input type="password" autocomplete="current-password"></label></p>
    <button>Log in</button>
    <button type="button" class="ghost" commandfor="login" command="close">Cancel</button>
  </form>
</dialog>
```



AFTER CLICKING "LOG IN": THE DIALOG OPENED AS A MODAL THROUGH COMMAND="SHOW-MODAL"

14.4 Other modern attributes

14.4.1 inert

The boolean `inert` attribute makes an element and all its descendants **non-interactive**: they cannot be clicked, focused, selected or found by find-in-page, and they are removed from the accessibility tree. Use it on content behind a custom overlay, off-screen slides of a carousel, or a section that is temporarily unavailable. (A modal dialog applies this behaviour to the rest of the page automatically.)

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```html
<section id="step1"><input placeholder="Name"><button>Next</button></section>
<section id="step2"><input placeholder="Address"><button>Pay</button><a href="#">Terms</a></section>
```

INERT REMOVES A WHOLE REGION FROM INTERACTION main.js

```js
const tabbable = () => [...document.querySelectorAll("a, button, input")]
  .filter(el => { el.focus(); return document.activeElement === el; })
  .map(el => el.textContent || el.placeholder);
console.log("Focusable before:", tabbable().join(", "));
document.getElementById("step2").inert = true;
console.log("Focusable after step2.inert = true:", tabbable().join(", "));
```

BROWSER CONSOLE

```
Focusable before: Name, Next, Address, Pay, Terms
Focusable after step2.inert = true: Name, Next
```

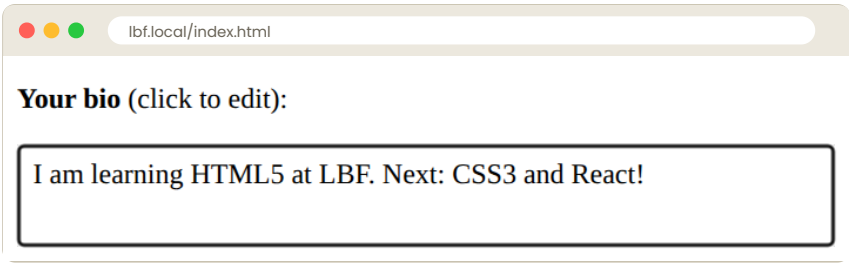
14.4.2 contenteditable

`contenteditable="true"` lets the user edit an element's content directly in the page, like a mini word processor; `"plaintext-only"` (all browsers since 2025) allows text but no formatting. It is the foundation of rich-text editors (Google Docs, Notion, CMS editors). The edited HTML is in `element.innerHTML`; always sanitise it before saving or showing it to other users (Module 16).

AN EDITABLE REGION

HTML

```
<p><b>Your bio</b> (click to edit):</p>
<div id="bio" contenteditable="plaintext-only" style="border:1px dashed #999;padding:8px;min-height:40px">I am learning HTML5 at LBF.</div>
```



AFTER CLICKING AT THE END AND TYPING: THE TEXT WAS EDITED IN PLACE

14.4.3 spellcheck, translate, draggable

Attribute	Values	Use
`spellcheck`	true / false	turn off red squiggles for code, usernames, product codes; turn on for comment boxes
`translate`	yes / no	`translate="no"` stops Google Translate and browser translation changing brand names and code: `<span translate="no">Learn Being Forward</span>`
`draggable`	true / false / auto	makes an element draggable for the Drag and Drop API (Module 15); links and images are draggable by default
`autocapitalize`	off, sentences, words, characters	mobile keyboard capitalisation
`writingsuggestions`	true / false	browser-provided writing suggestions (new)

14.4.4 hidden="until-found"

Plain `hidden` removes content completely. `hidden="until-found"` hides it visually but keeps it **searchable**: when the user's find-in-page (Ctrl+F) or a link to a text fragment matches text inside, the browser reveals it automatically and fires a `beforematch` event so the page can update its UI (for example, mark the accordion section as open). Closed `<details>` elements behave this way too.

SEARCHABLE HIDDEN CONTENT

HTML

```
<h3>Policies</h3>
<div id="refund" hidden="until-found">
  Full refund within 7 days of joining.
</div>

<script>
  // Fired just before the browser reveals the section for find-in-page or a #:~:text= link
  document.getElementById("refund").addEventListener("beforematch", e => {
    e.target.previousElementSibling.classList.add("expanded");   // update the heading's icon
  });
</script>
```

Pressing Ctrl+F and typing "refund", or opening a link such as `/faq#:~:text=refund`, reveals the section: the browser removes the `hidden` attribute, fires `beforematch` and scrolls the match into view. Search engines can also index the hidden text.

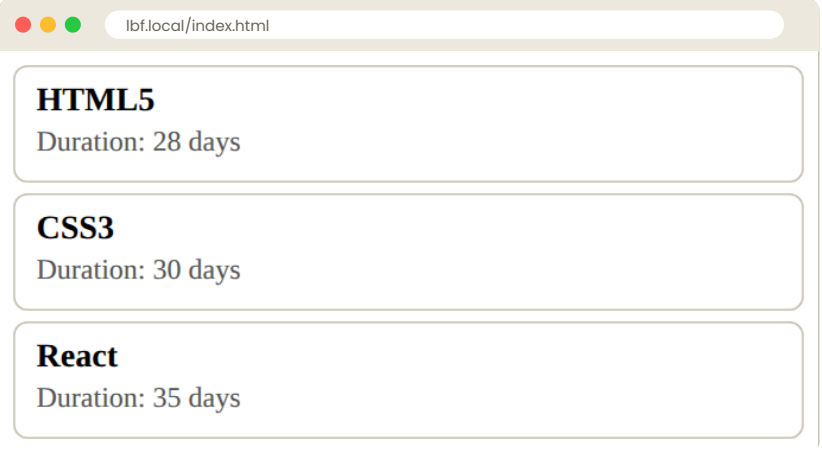
14.5 Templates and Web Components

14.5.1 <template> and <slot>

`<template>` holds HTML that is **parsed but not rendered**: its images do not load, scripts do not run and it is not part of the page. JavaScript clones its `content` to stamp out copies, for example one card per item of data. `<slot>` marks a place inside a Web Component's shadow DOM where the user's own content will appear (14.5.2).

STAMPING CARDS FROM A TEMPLATE HTML

```
<template id="course-card">
  <article style="border:1px solid #c9c2b0;border-radius:8px;padding:8px 12px;margin:6px 0">
    <h3 class="name" style="margin:0"></h3>
    <p class="meta" style="margin:4px 0;color:#555"></p>
  </article>
</template>
<div id="list"></div>
<script>
  const courses = [["HTML5", "28 days"], ["CSS3", "30 days"], ["React", "35 days"]];
  const tpl = document.getElementById("course-card");
  for (const [name, days] of courses) {
    const card = tpl.content.cloneNode(true);
    card.querySelector(".name").textContent = name;
    card.querySelector(".meta").textContent = "Duration: " + days;
    document.getElementById("list").append(card);
  }
</script>
```



THREE CARDS CREATED FROM ONE TEMPLATE

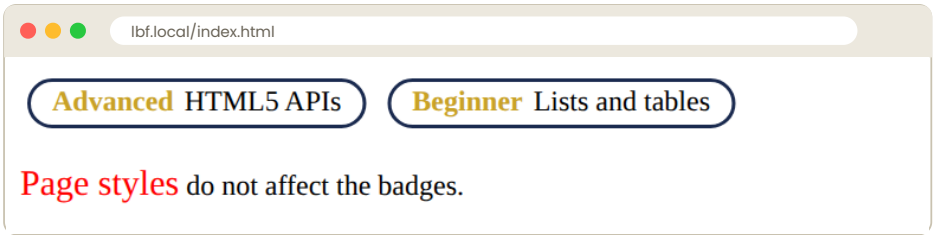
14.5.2 Custom elements and Shadow DOM (overview)

Web Components are a set of browser standards for building your own reusable HTML elements that work in any framework or none:

- **Custom elements**: define a new tag (its name must contain a hyphen, such as `<course-badge>`) with a JavaScript class extending `HTMLElement`, registered with `customElements.define()`. Lifecycle callbacks: `connectedCallback`, `disconnectedCallback`, `attributeChangedCallback`.
- **Shadow DOM**: a hidden, encapsulated DOM tree attached to an element. Its styles do not leak out and page styles do not leak in, so the component looks the same everywhere.
- **Templates and slots**: the component's markup, with slots where user content goes.

A CUSTOM ELEMENT WITH SHADOW DOM AND A SLOT HTML

```
<course-badge level="Advanced">HTML5 APIs</course-badge>
<course-badge level="Beginner">Lists and tables</course-badge>
<p><span class="title">Page styles</span> do not affect the badges.</p>
<style> .title { color: red; font-size: 20px; } </style>
<script>
  class CourseBadge extends HTMLElement {
    connectedCallback() {
      const root = this.attachShadow({ mode: "open" });
      root.innerHTML = `
        <style>
          :host { display: inline-flex; gap: 6px; border: 2px solid #1a2a4f; border-radius: 20px;
padding: 3px 12px; margin: 4px; }
          .title { color: #c9a227; font-weight: 700; }
        </style>
        <span class="title">${this.getAttribute("level")}</span>
        <slot></slot>`;
    }
  }
  customElements.define("course-badge", CourseBadge);
</script>
```



THE BADGES USE THEIR OWN .TITLE STYLE (GOLD); THE PAGE'S RED .TITLE RULE STAYS OUTSIDE THE SHADOW ROOT

14.5.3 Declarative Shadow DOM (overview)

Shadow roots normally need JavaScript (`attachShadow`). **Declarative Shadow DOM** writes one directly in HTML with `<template shadowrootmode="open">`, so server-rendered components display correctly before (or without) any JavaScript. Supported in all major browsers since 2024.

DECLARATIVE SHADOW DOM: A COMPONENT WITH NO JAVASCRIPT HTML

```
<user-card>
  <template shadowrootmode="open">
    <style>
      div { display: flex; gap: 10px; align-items: center; border: 1px solid #c9c2b0; border-radius:
10px; padding: 10px; width: 260px; }
      ::slotted(img) { border-radius: 50%; }
      b { color: #1a2a4f; }
    </style>
    <div><slot name="photo"></slot><span><b><slot name="name"></slot></b><br><slot name="role"></slot>
</span></div>
  </template>
  <img slot="photo" src="avatar.png" width="48" height="48" alt="">
  <span slot="name">Asha Rao</span>
  <span slot="role">Front-end developer</span>
</user-card>
```



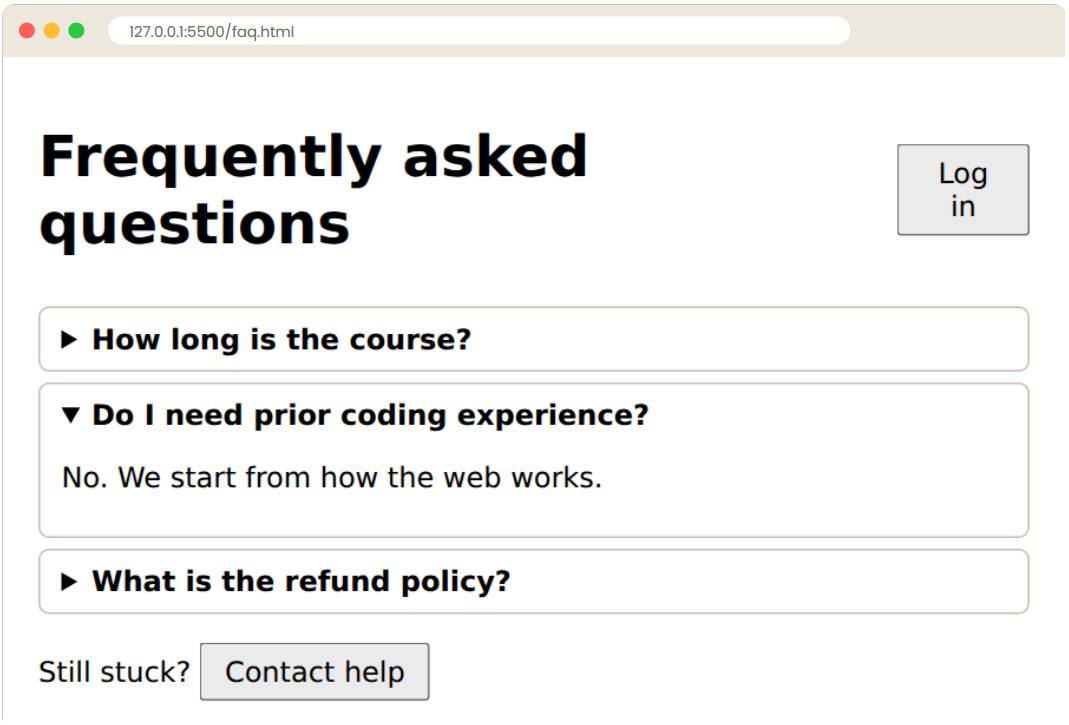
THE SHADOW ROOT AND NAMED SLOTS WERE CREATED BY THE HTML PARSER

PRACTICE CHECKPOINT

An **FAQ page** using an exclusive accordion, a **login modal** with dialog, and a **help popover**, all working without custom JavaScript where possible.

FAQ.HTML: SAMPLE SOLUTION (NO CUSTOM JAVASCRIPT AT ALL) HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>FAQ | Learn Being Forward</title>
  <style>
    body { font-family: system-ui, sans-serif; max-width: 560px; margin: 16px auto; }
    header { display: flex; justify-content: space-between; align-items: center; }
    details { border: 1px solid #c9c2b0; border-radius: 6px; padding: 8px 12px; margin: 6px 0; }
    summary { font-weight: 600; cursor: pointer; }
    dialog::backdrop { background: rgb(10 20 40 / .5); }
    button { font: inherit; padding: 5px 12px; }
  </style>
</head>
<body>
  <header>
    <h1>Frequently asked questions</h1>
    <button commandfor="login" command="show-modal">Log in</button>
  </header>
  <main>
    <details name="faq" open><summary>How long is the course?</summary><p>28 days minimum.</p>
</details>
    <details name="faq"><summary>Do I need prior coding experience?</summary><p>No. We start from how
the web works.</p></details>
    <details name="faq"><summary>What is the refund policy?</summary><p>Full refund within 7 days.</p>
</details>
    <p>Still stuck? <button popovertarget="help">Contact help</button></p>
    <div id="help" popover><p>Call +91 99729 34418 or email info@learnbeingforward.in</p></div>
  </main>
  <dialog id="login" closedby="any" aria-labelledby="lt">
    <form method="dialog">
      <h2 id="lt">Log in</h2>
      <p><label>Email <input type="email" autocomplete="username" required></label></p>
      <p><label>Password <input type="password" autocomplete="current-password" required></label></p>
      <button>Log in</button>
      <button type="button" commandfor="login" command="close">Cancel</button>
    </form>
  </dialog>
</body>
</html>
```



THE FAQ PAGE AFTER OPENING THE SECOND QUESTION (THE FIRST CLOSED AUTOMATICALLY)

MODULE 15 · ADVANCED

HTML5 APIs with JavaScript

Minimum duration **3 days** | Level **Advanced**

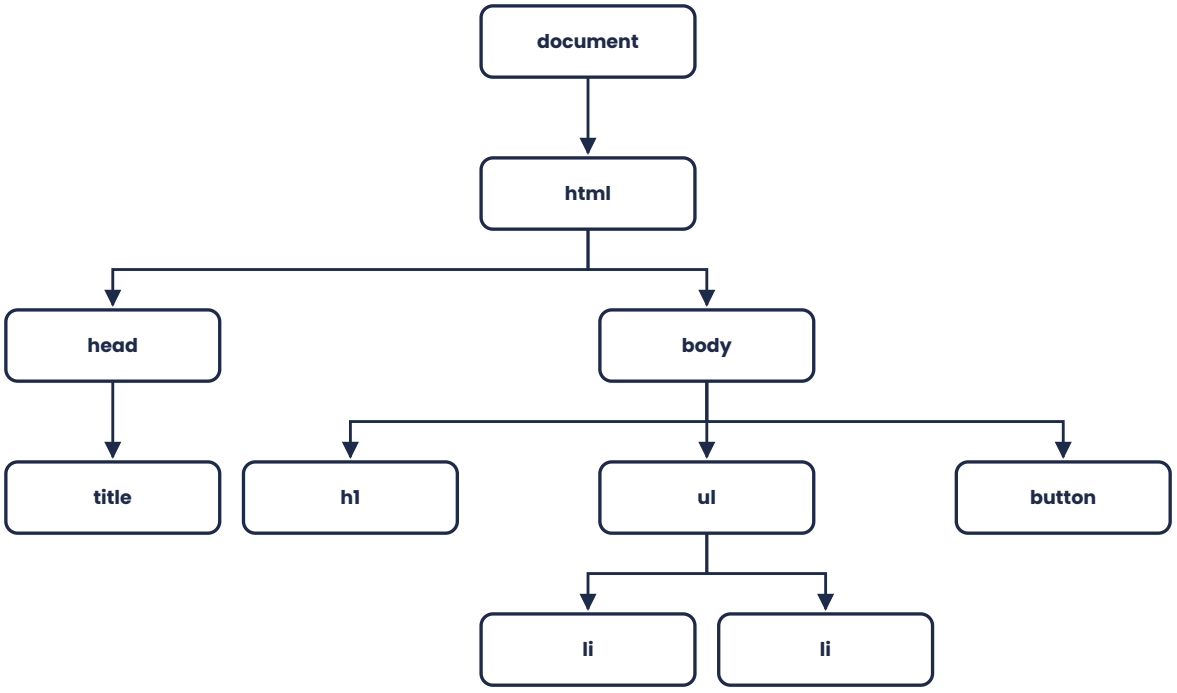
HTML5 introduced browser APIs that turn web pages into applications. Using basic JavaScript, this module covers selecting and changing DOM elements and handling events, the Canvas API (context, rectangles, lines, arcs, text, colours, gradients, images and animation with requestAnimationFrame) and when to choose canvas or SVG, Web Storage (localStorage, sessionStorage, storing objects as JSON) with an overview of IndexedDB, the Geolocation API, the Drag and Drop API, controlling audio and video from JavaScript, the Fullscreen API, the History API (pushState and popstate), and an overview of Web Workers, Notifications, Clipboard, Web Share and service workers. Every console output was produced by running the code in Chromium; drags, clicks and reloads were performed by an automated user.

IN THIS MODULE

15.1 Working with the DOM (overview)
15.2 Canvas API
15.3 Web Storage
15.4 Geolocation API
15.5 Drag and Drop API
15.6 Media APIs
15.7 History API
15.8 Other APIs (overview)

15.1 Working with the DOM (overview)

When the browser parses HTML it builds the **DOM** (Document Object Model): a tree of objects, one per element, text and attribute, that JavaScript can read and change. Every change is reflected on screen immediately. A full treatment is in the JavaScript course; this section covers what the HTML5 APIs below need.



THE DOM TREE OF A SMALL PAGE

15.1.1 Selecting and changing elements

Task	Code
Select one element (first match of a CSS selector)	`document.querySelector("#list li")`
Select all matches (a NodeList)	`document.querySelectorAll(".card")`
By id (fastest)	`document.getElementById("list")`
Change text (safe: no HTML parsed)	`el.textContent = "Hello"`
Change HTML (only with trusted content)	`el.innerHTML = "<b>Hi</b>"`
Attributes	`el.setAttribute("href", url)`, `el.getAttribute(...)`, `el.id`, `img.src`
Classes	`el.classList.add("active")`, `.remove()`, `.toggle()`, `.contains()`
Inline style	`el.style.color = "red"`
data-* attributes	`<li data-id="7">` → `el.dataset.id` is "7"
Create and insert	`const li = document.createElement("li"); list.append(li)`
Remove	`el.remove()`

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```
<h1>Our courses</h1>
<ul id="courses">
  <li data-days="28">HTML5</li>
  <li data-days="30">CSS3</li>
</ul>
```

CHANGING THE DOM main.js

```
const list = document.querySelector("#courses");
const li = document.createElement("li");
li.textContent = "React";
li.dataset.days = "35";
list.append(li);
document.querySelector("h1").textContent = "Our 3 courses";
list.firstElementChild.classList.add("popular");
for (const item of list.children) console.log(item.textContent, item.dataset.days, item.className);
```

BROWSER CONSOLE

```
HTML5 28 popular
CSS3 30
React 35
--- page shows ---
Our 3 courses
HTML5
CSS3
React
```

15.1.2 Handling events

Events are notifications that something happened: a click, a key press, a form submission, the page finishing loading. `addEventListener(type, handler)` runs a function each time; the handler receives an **event object** with details (`event.target`, `event.key`, mouse coordinates…). `event.preventDefault()` cancels the browser's default action, such as submitting a form or following a link.

Events	Fired when
click, dblclick, contextmenu	mouse button (or tap, or Enter/Space on buttons)
input, change	a form value changes (input: every keystroke; change: when committed)
submit	a form is submitted
keydown, keyup	a key is pressed / released
focus, blur, focusin, focusout	an element gains or loses focus
pointerdown, pointermove, pointerup	mouse, pen or touch
DOMContentLoaded, load	HTML parsed / everything including images loaded
scroll, resize	the page or an element scrolls / the window resizes

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```
<form>
  <input name="name" maxlength="20"> <span id="count">0/20</span>
  <button>Save</button>
</form>
```

INPUT AND SUBMIT EVENTS main.js

```
const form = document.querySelector("form");
const count = document.querySelector("#count");
form.name.addEventListener("input", e => count.textContent = e.target.value.length + "/20");
form.addEventListener("submit", e => {
  e.preventDefault();                          // stay on this page
  console.log("Submitted name:", form.name.value, "| counter shows", count.textContent);
});
```

BROWSER CONSOLE

```
Submitted name: Asha Rao | counter shows 8/20
```

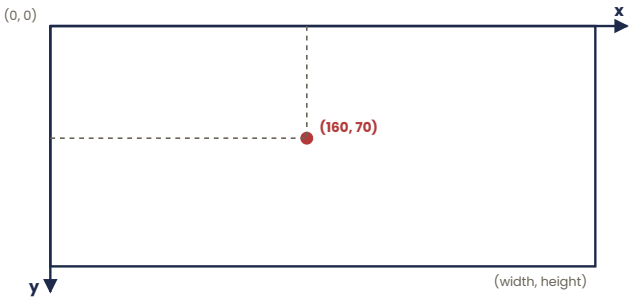
Simulated user actions, in order: types "Asha Rao" into `input[name=name]`; clicks `button`.

Event **delegation**: one listener on a parent handles events from all its children (including ones added later), using `event.target.closest(selector)`. It is used in the Kanban checkpoint.

15.2 Canvas API

15.2.1 Getting the 2D context

A `<canvas>` element is a blank bitmap. To draw, get a **rendering context** from it: `getContext("2d")` for 2D drawing (also `"webgl2"` and `"webgpu"` for 3D). Coordinates start at the **top-left** corner (0, 0); x grows right and y grows **down**. Set the size with the `width` and `height` attributes (the drawing resolution); CSS width only stretches the bitmap.



CANVAS COORDINATES: Y GROWS DOWNWARDS

15.2.2 Drawing rectangles, lines, arcs and text

Method	Draws
`fillRect(x, y, w, h)` / `strokeRect` / `clearRect`	filled / outlined rectangle; clear an area
`beginPath()`, `moveTo(x, y)`, `lineTo(x, y)`, `closePath()`	start a path, move the pen, draw a line, close the shape
`arc(x, y, r, start, end)`	circle or arc (angles in radians: a full circle is `2 * Math.PI`)
`quadraticCurveTo`, `bezierCurveTo`, `roundRect`	curves, rounded rectangles
`fill()` / `stroke()`	paint the current path
`fillText(text, x, y)` / `strokeText`	text, using `font`, `textAlign`, `textBaseline`
`fillStyle`, `strokeStyle`, `lineWidth`, `lineCap`	paint settings, apply to later drawing

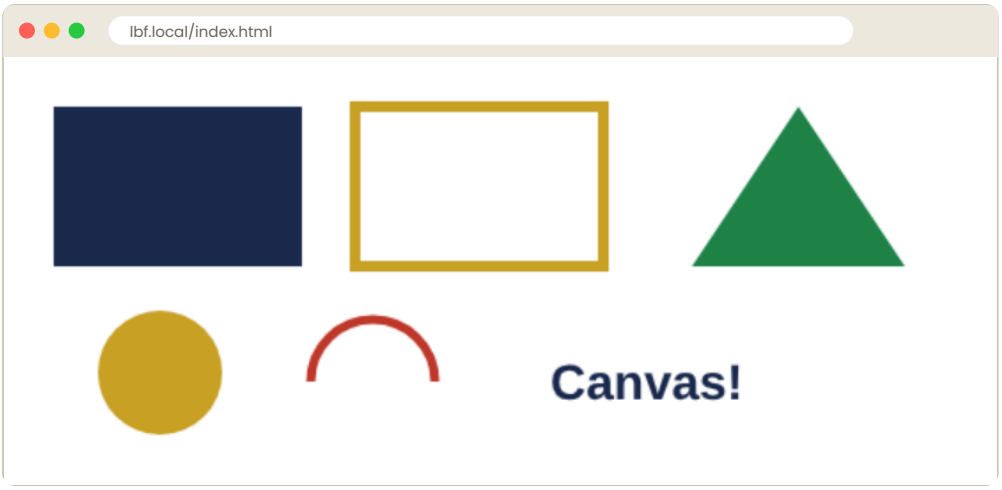
RECTANGLES, PATHS, ARCS AND TEXT HTML

```
<canvas id="c" width="520" height="220">A drawing of shapes and text.</canvas>
<script>
  const ctx = document.getElementById("c").getContext("2d");
  ctx.fillStyle = "#1a2a4f";                 // rectangle
  ctx.fillRect(20, 20, 140, 90);
  ctx.strokeStyle = "#c9a227"; ctx.lineWidth = 6;
  ctx.strokeRect(190, 20, 140, 90);

  ctx.beginPath();                           // triangle with lines
  ctx.moveTo(380, 110); ctx.lineTo(440, 20); ctx.lineTo(500, 110); ctx.closePath();
  ctx.fillStyle = "#1e8449"; ctx.fill();

  ctx.beginPath();                           // full circle
  ctx.arc(80, 170, 35, 0, 2 * Math.PI);
  ctx.fillStyle = "#c9a227"; ctx.fill();
  ctx.beginPath();                           // half circle (arc)
  ctx.arc(200, 175, 35, Math.PI, 2 * Math.PI);
  ctx.strokeStyle = "#c0392b"; ctx.lineWidth = 5; ctx.stroke();

  ctx.font = "bold 28px sans-serif";         // text
  ctx.fillStyle = "#1a2a4f";
  ctx.fillText("Canvas!", 300, 185);
</script>
```



EVERY SHAPE WAS PAINTED PIXEL BY PIXEL BY THE SCRIPT

15.2.3 Colours, gradients and images

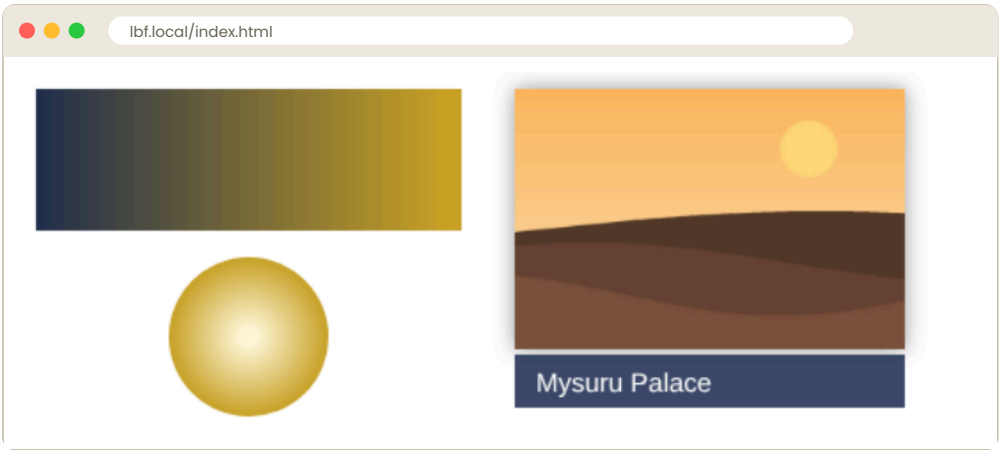
`fillStyle` and `strokeStyle` accept any CSS colour, or a **gradient** or **pattern** object. `createLinearGradient(x0, y0, x1, y1)` and `createRadialGradient(...)` return gradients to which you add colour stops. `drawImage(img, x, y, w, h)` paints an image (after it has loaded), and `globalAlpha`, `shadowBlur` and `shadowColor` add transparency and shadows.

GRADIENTS, AN IMAGE, SHADOW AND TRANSPARENCY HTML

```
<canvas id="c" width="520" height="200"></canvas>
<script>
  const ctx = document.getElementById("c").getContext("2d");
  const lin = ctx.createLinearGradient(0, 0, 240, 0);
  lin.addColorStop(0, "#1a2a4f"); lin.addColorStop(1, "#c9a227");
  ctx.fillStyle = lin; ctx.fillRect(10, 10, 240, 80);

  const rad = ctx.createRadialGradient(130, 150, 5, 130, 150, 45);
  rad.addColorStop(0, "#fff7d6"); rad.addColorStop(1, "#c9a227");
  ctx.fillStyle = rad; ctx.beginPath(); ctx.arc(130, 150, 45, 0, Math.PI * 2); ctx.fill();

  const img = new Image();
  img.onload = () => {
    ctx.shadowColor = "rgba(0,0,0,.4)"; ctx.shadowBlur = 12;
    ctx.drawImage(img, 280, 10, 220, 147);
    ctx.shadowBlur = 0; ctx.globalAlpha = 0.85;
    ctx.fillStyle = "#1a2a4f"; ctx.fillRect(280, 160, 220, 30);
    ctx.globalAlpha = 1; ctx.fillStyle = "#fff"; ctx.font = "15px sans-serif";
    ctx.fillText("Mysuru Palace", 292, 181);
  };
  img.src = "mysuru-palace.jpg";
</script>
```



HOW THE BROWSER DISPLAYS IT

15.2.4 Simple animation with requestAnimationFrame

Animation on canvas means **clear, update, redraw**, many times per second. `requestAnimationFrame(callback)` asks the browser to call the function just before the next screen refresh (usually 60 times per second, matching the display), and pauses automatically in background tabs to save battery. The callback receives a timestamp, so movement can be based on elapsed time rather than frame count.

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

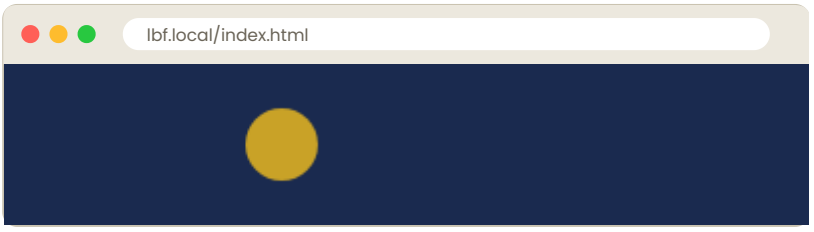
```
<canvas id="c" width="400" height="80" style="background:#1a2a4f"></canvas>
```

A BOUNCING BALL main.js

```
const canvas = document.getElementById("c"), ctx = canvas.getContext("2d");
let x = 20, speed = 120;            // pixels per second
let last = null, frames = 0;
function frame(time) {
  if (last !== null) x += speed * (time - last) / 1000;
  last = time; frames++;
  if (x > canvas.width - 20 || x < 20) speed = -speed;   // bounce
  ctx.clearRect(0, 0, canvas.width, canvas.height);       // 1. clear
  ctx.beginPath(); ctx.arc(x, 40, 18, 0, Math.PI * 2);    // 2. redraw at new position
  ctx.fillStyle = "#c9a227"; ctx.fill();
  if (frames < 60) requestAnimationFrame(frame);          // 3. ask for the next frame
  else console.log("Stopped after", frames, "frames; ball at x =", Math.round(x));
}
requestAnimationFrame(frame);
```

BROWSER CONSOLE

```
Stopped after 60 frames; ball at x = 138
```



THE CANVAS AFTER THE 60TH FRAME (SCREENSHOT)

15.2.5 Canvas vs SVG

	Canvas	SVG
Model	immediate mode: pixels; the browser forgets shapes once drawn	retained mode: each shape is a DOM element
Scaling	blurs when scaled up (redraw at higher resolution for HiDPI)	sharp at every size
Events	only on the canvas; you work out which shape was hit	on each shape (click, hover)
Styling	JavaScript only	CSS and attributes
Accessibility	opaque: needs fallback content or ARIA	text and titles are accessible
Performance	excellent for thousands of objects, pixels and fast animation	slows down with thousands of elements
Best for	games, image editing, particle effects, big data plots	icons, logos, diagrams, interactive charts with few elements

15.3 Web Storage

15.3.1 localStorage and sessionStorage

The Web Storage API stores **string key-value pairs** in the browser, per origin (protocol + domain + port). `localStorage` keeps data **until it is deleted** (across tabs, restarts and reboots); `sessionStorage` keeps it

only for the life of **one tab**. Both share the same simple, synchronous API, and each origin gets roughly 5 MB.

Method	Effect
`setItem(key, value)`	store (the value is converted to a string)
`getItem(key)`	read; returns `null` if the key does not exist
`removeItem(key)`	delete one key
`clear()`	delete everything for this origin
`key(i)`, `length`	iterate over keys
`storage` event	fires in **other** tabs of the same origin when localStorage changes

A VISIT COUNTER THAT SURVIVES RELOADS main.js

```
const visits = Number(localStorage.getItem("visits") || 0) + 1;
localStorage.setItem("visits", visits);
sessionStorage.setItem("tabOpenedAt", sessionStorage.getItem("tabOpenedAt") || new
Date().toISOString().slice(11, 19));
console.log("Visit number", visits, "| theme:", localStorage.getItem("theme"), "| age type:", typeof
localStorage.getItem("visits"));
if (visits === 1) localStorage.setItem("theme", "dark");
```

BROWSER CONSOLE

```
Visit number 1 | theme: null | age type: string
Visit number 2 | theme: dark | age type: string
Visit number 3 | theme: dark | age type: string
```

Simulated user actions, in order: reloads the page; reloads the page.

The page was loaded three times (two reloads). The count persisted, `theme` was saved on the first visit and read back on the next ones, and every value came back as a **string**.

WATCH OUT

Never store secrets (passwords, tokens, card details) in Web Storage: any script on the page, including a third-party script or an XSS attack, can read it. Storage is also per browser and per device, and users can clear it at any time.

15.3.2 Storing objects as JSON

Because storage holds strings, objects and arrays must be converted with `JSON.stringify` when saving and `JSON.parse` when loading. Storing an object directly gives the useless string "[object Object]".

SAVING AND LOADING AN ARRAY OF OBJECTS main.js

```
const cart = [{ course: "HTML5", fee: 9999 }, { course: "CSS3", fee: 8999 }];
localStorage.setItem("bad", cart);                    // wrong
localStorage.setItem("cart", JSON.stringify(cart));   // right
console.log("bad :", localStorage.getItem("bad"));
console.log("raw :", localStorage.getItem("cart"));
const loaded = JSON.parse(localStorage.getItem("cart") ?? "[]");
console.log("total fee:", loaded.reduce((sum, item) => sum + item.fee, 0));
```

BROWSER CONSOLE

```
bad : [object Object],[object Object]
raw : [{"course":"HTML5","fee":9999},{"course":"CSS3","fee":8999}]
total fee: 18998
```

15.3.3 IndexedDB (overview)

IndexedDB is a full, asynchronous, transactional database inside the browser: it stores structured data (objects, files, blobs) with indexes for fast queries, and can hold hundreds of megabytes or more. It powers offline apps such as Gmail offline and Google Docs. Its raw API is verbose, so most developers use a small wrapper library such as **idb** or **Dexie**.

	Cookies	localStorage	sessionStorage	IndexedDB
Capacity	~4 KB each	~5 MB	~5 MB	large (share of disk)
Lifetime	until expiry date	until deleted	until the tab closes	until deleted
Sent to the server	with every request	no	no	no
API	string parsing	sync, strings	sync, strings	async, objects, indexes
Use for	sessions, auth (HttpOnly)	preferences, small caches	per-tab form drafts	offline data, large datasets

INDEXEDDB: OPEN, STORE AND READ BACK main.js

```
const req = indexedDB.open("lbf", 1);
req.onupgradeneeded = () => req.result.createObjectStore("students", { keyPath: "id" });
req.onsuccess = () => {
  const db = req.result;
  const tx = db.transaction("students", "readwrite");
  tx.objectStore("students").put({ id: 1, name: "Asha", course: "HTML5" });
  tx.objectStore("students").put({ id: 2, name: "Ravi", course: "React" });
  tx.oncomplete = () => {
    db.transaction("students").objectStore("students").getAll().onsuccess = e =>
      console.log("Stored in IndexedDB:", JSON.stringify(e.target.result));
  };
};
```

BROWSER CONSOLE

```
Stored in IndexedDB: [{"id":1,"name":"Asha","course":"HTML5"},{"id":2,"name":"Ravi","course":"React"}]
```

15.4 Geolocation API

15.4.1 Getting the user's position and permissions

`navigator.geolocation.getCurrentPosition(success, error, options)` asks for the device's location (from GPS, Wi-Fi or IP). The browser **always asks the user for permission** first, and the API only works in a **secure context** (HTTPS or localhost). `watchPosition` reports updates as the user moves; `clearWatch` stops it. Ask only when the user does something that needs it (a "Find nearest centre" button), never on page load.

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```html
<button>Find my distance to campus</button>
```

FIND THE NEAREST CAMPUS main.js

```
document.querySelector("button").addEventListener("click", () => {
  navigator.geolocation.getCurrentPosition(
    pos => {
      const { latitude, longitude, accuracy } = pos.coords;
      console.log(`You are at ${latitude.toFixed(4)}, ${longitude.toFixed(4)} (within ${accuracy}
m)`);
      const mysuru = [12.2958, 76.6394];                    // LBF Mysuru campus (approx.)
      const km = distance(latitude, longitude, ...mysuru);
      console.log(`Distance to the Mysuru campus: ${km.toFixed(0)} km`);
    },
    err => console.log("Error", err.code, err.message),     // 1 = PERMISSION_DENIED, 2, 3 = TIMEOUT
    { enableHighAccuracy: false, timeout: 10000, maximumAge: 60000 });
});
function distance(lat1, lon1, lat2, lon2) {                 // haversine formula
  const r = d => d * Math.PI / 180, R = 6371;
  const a = Math.sin(r(lat2 - lat1) / 2) ** 2 + Math.cos(r(lat1)) * Math.cos(r(lat2)) *
Math.sin(r(lon2 - lon1) / 2) ** 2;
  return 2 * R * Math.asin(Math.sqrt(a));
}
```

BROWSER CONSOLE

```
You are at 12.9716, 77.5946 (within 20 m)
Distance to the Mysuru campus: 128 km
```

The test browser was given permission and a location in Bengaluru (12.9716, 77.5946). Simulated user action: clicks the button.

Error code	Name	Meaning
1	PERMISSION_DENIED	the user (or a policy) refused
2	POSITION_UNAVAILABLE	the device could not determine a position
3	TIMEOUT	no answer within `timeout` ms

15.5 Drag and Drop API

15.5.1 draggable, dragstart, dragover, drop

Native drag and drop needs an element with `draggable="true"` (images and links are draggable by default) and a drop target. The **dataTransfer** object carries data from the drag source to the target.

Event	Fires on	Typical code
`dragstart`	the dragged element	`e.dataTransfer.setData("text/plain", el.id)`
`dragenter` / `dragleave`	a target as the pointer enters/leaves	add/remove a highlight class
`dragover`	the target, continuously	`e.preventDefault()` : **required** to allow dropping
`drop`	the target, on release	`e.dataTransfer.getData("text/plain")`, move the element
`dragend`	the dragged element	clean up

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```
<style>.col{display:inline-block;vertical-align:top;width:160px;min-height:110px;border:2px dashed
#999;margin:4px;padding:6px;font:14px sans-serif}
.col h4{margin:0 0 6px}[draggable]{background:#1a2a4f;color:#fff;padding:6px;margin:4px 0;border-
radius:4px;cursor:grab}</style>
<div class="col" data-name="To do"><h4>To do</h4><div draggable="true" id="t1">Read Module 15</div>
<div draggable="true" id="t2">Build Kanban</div></div>
<div class="col" data-name="Done"><h4>Done</h4></div>
```

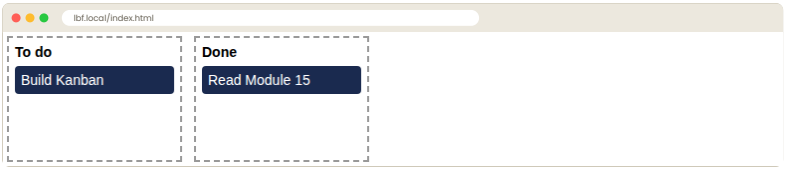
MOVING CARDS BETWEEN COLUMNS main.js

```
document.querySelectorAll("[draggable]").forEach(card =>
  card.addEventListener("dragstart", e => e.dataTransfer.setData("text/plain", card.id)));
document.querySelectorAll(".col").forEach(col => {
  col.addEventListener("dragover", e => e.preventDefault());      // allow drop
  col.addEventListener("drop", e => {
    e.preventDefault();
    const card = document.getElementById(e.dataTransfer.getData("text/plain"));
    col.append(card);
    console.log(`Moved "${card.textContent}" to ${col.dataset.name}`);
  });
});
```

BROWSER CONSOLE

```
Moved "Read Module 15" to Done
```

Simulated user actions, in order: drags `#t1` onto `.col[data-name=Done]`.



AFTER THE AUTOMATED DRAG: "READ MODULE 15" IS NOW IN THE DONE COLUMN

NOTE

The HTML Drag and Drop API does not work with touch on many mobile browsers and is not keyboard accessible by itself. Production Kanban boards add Pointer Events for touch and buttons or keyboard shortcuts ("Move to Done") for accessibility.

15.6 Media APIs

15.6.1 Controlling audio and video with JavaScript

`<audio>` and `<video>` elements are **HTMLMediaElement** objects with methods, properties and events, so you can build custom players, playlists, chapter buttons and synchronised content.

Methods and properties	Events
`play()` (returns a Promise), `pause()`, `load()`	`play`, `pause`, `ended`
`currentTime` (seconds, read/write), `duration`	`timeupdate` (about 4 times a second), `seeked`
`paused`, `ended`, `volume` (0–1), `muted`	`volumechange`
`playbackRate` (1 = normal, 1.5, 2…)	`ratechange`
`readyState`, `buffered`	`loadedmetadata`, `canplay`, `waiting`, `error`

THE PAGE THIS SCRIPT RUNS IN (HTML) — HTML

```
<video src="intro.webm" width="320" height="180"></video>
```

A SCRIPTED VIDEO PLAYER — main.js

```
const v = document.querySelector("video");
v.addEventListener("loadedmetadata", async () => {
  console.log("duration:", v.duration, "s | size:", v.videoWidth + "x" + v.videoHeight);
  v.muted = true;                       // muted playback is always allowed to start
  v.playbackRate = 2;
  await v.play();
  console.log("playing:", !v.paused, "| rate:", v.playbackRate);
  setTimeout(() => {
    v.pause();
    console.log("paused at", v.currentTime.toFixed(1), "s | ended:", v.ended);
  }, 1000);
});
v.addEventListener("ratechange", () => console.log("ratechange event: now", v.playbackRate + "x"));
```

BROWSER CONSOLE

```
duration: 8.008 s | size: 640x360
ratechange event: now 2x
playing: true | rate: 2
paused at 1.9 s | ended: false
```

15.6.2 Fullscreen API

`element.requestFullscreen()` shows any element (a video, a canvas game, an image gallery, a slide) in full screen; `document.exitFullscreen()` leaves; `document.fullscreenElement` tells what is full screen, and the `fullscreenchange` event fires on every change. It must be called in response to a user gesture (a click or key press), and the user can always leave with Esc. CSS can style the state with the `:fullscreen` pseudo-class.

THE PAGE THIS SCRIPT RUNS IN (HTML) — HTML

```
<div id="slides">Slide 1</div>
<button>Present</button>
```

ENTERING AND LEAVING FULL SCREEN main.js

```
const box = document.getElementById("slides");
document.querySelector("button").addEventListener("click", async () => {
  await box.requestFullscreen();
  console.log("in full screen:", document.fullscreenElement.id, "| matches :fullscreen?",
box.matches(":fullscreen"));
  await document.exitFullscreen();
  console.log("after exitFullscreen():", document.fullscreenElement);
});
```

BROWSER CONSOLE

```
in full screen: slides | matches :fullscreen? true
after exitFullscreen(): null
```

Simulated user actions, in order: clicks `button`; waits 500 ms.

15.7 History API

15.7.1 pushState and popstate (single-page navigation basics)

Single-page applications (React, Angular and Vue routers) change the URL **without loading a new page**, so that each view has its own address, can be bookmarked and works with the Back button. The History API makes this possible:

- `history.pushState(state, "", url)` adds a new entry to the session history and changes the address bar, **without** a request to the server.
- `history.replaceState(...)` changes the current entry instead of adding one.
- The `popstate` event fires when the user presses Back/Forward; `event.state` holds the state object stored with that entry.
- The server must return the app for every route, otherwise refreshing `/courses` gives a 404.

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```
<nav><a href="/">Home</a> <a href="/courses">Courses</a> <a href="/contact">Contact</a></nav>
<main id="view">Home page</main>
```

A TINY CLIENT-SIDE ROUTER main.js

```
const view = document.getElementById("view");
const pages = { "/": "Home page", "/courses": "All courses", "/contact": "Contact us" };
function show(path) { view.textContent = pages[path] ?? "Not found"; }
document.querySelector("nav").addEventListener("click", e => {
  if (!e.target.matches("a")) return;
  e.preventDefault();                                  // no page load
  const path = new URL(e.target.href).pathname;
  history.pushState({ path }, "", path);
  show(path);
  console.log("pushState ->", location.pathname, "| shows:", view.textContent, "| history length",
history.length);
});
addEventListener("popstate", e => {
  show(e.state?.path ?? "/");
  console.log("popstate  ->", location.pathname, "| shows:", view.textContent);
});
```

BROWSER CONSOLE

```
pushState -> /courses | shows: All courses | history length 3
pushState -> /contact | shows: Contact us | history length 4
popstate  -> /courses | shows: All courses
popstate  -> /index.html | shows: Home page
```

Simulated user actions: clicks Courses, clicks Contact, then presses Back twice.

15.8 Other APIs (overview)

15.8.1 Web Workers

JavaScript in a page runs on one **main thread** that also handles clicks and rendering; a long calculation freezes the page. A **Web Worker** runs a script on a background thread. The page and the worker cannot share variables or touch each other's DOM (workers have no DOM); they communicate by sending messages with `postMessage` and the `message` event.

COUNTING PRIMES IN A WORKER main.js

```
const worker = new Worker("primes.js");
worker.onmessage = e => console.log("Worker answered:", e.data.count, "primes below", e.data.limit);
worker.postMessage(2_000_000);
console.log("Main thread is still free: this line prints first");
```

BROWSER CONSOLE

```
Main thread is still free: this line prints first
Worker answered: 148933 primes below 2000000
```

PRIMES.JS JavaScript

```
// primes.js (runs in the worker)
onmessage = e => {
  const limit = e.data;
  const sieve = new Uint8Array(limit); let count = 0;
  for (let i = 2; i < limit; i++)
    if (!sieve[i]) { count++; for (let j = i * i; j < limit; j += i) sieve[j] = 1; }
  postMessage({ limit, count });
};
```

15.8.2 Notifications

The **Notifications API** shows system notifications outside the page. Call `Notification.requestPermission()` from a user action (it returns "granted", "denied" or "default"), then `new Notification(title, { body, icon })`. With a service worker and the Push API, notifications can arrive even when the site is closed. Ask sparingly: users who are asked on page load usually click "Block", and the site can never ask again.

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```html
<button>Remind me before class</button>
```

ASKING FOR PERMISSION, THEN NOTIFYING main.js

```
document.querySelector("button").addEventListener("click", async () => {
  const result = await Notification.requestPermission();
  console.log("permission:", result);
  if (result === "granted") {
    const n = new Notification("Class starts in 15 minutes", { body: "HTML5 APIs, Room 2 / Zoom", tag:
"class-reminder" });
    console.log("notification created:", n.title, "/", n.body);
  }
});
```

BROWSER CONSOLE

```
permission: granted
notification created: Class starts in 15 minutes / HTML5 APIs, Room 2 / Zoom
```

The test browser had notification permission granted in advance. Simulated user action: clicks the button.

15.8.3 Clipboard and Web Share

The async **Clipboard API** (`navigator.clipboard.writeText()` / `readText()`) copies and pastes text in a secure context; writing needs a user gesture, reading needs permission. The **Web Share API** (`navigator.share({ title, text, url })`) opens the device's native share sheet (WhatsApp, email, Nearby Share…). It is supported on phones and on Windows, macOS and ChromeOS browsers, but not everywhere, so always feature-detect.

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```html
<button id="copy">Copy coupon</button> <button id="share">Share</button>
```

COPY BUTTON AND SHARE WITH A FALLBACK main.js

```
document.querySelector("#copy").addEventListener("click", async () => {
  await navigator.clipboard.writeText("LBF-HTML5-EARLYBIRD");
  console.log("copied; clipboard now holds:", await navigator.clipboard.readText());
});
document.querySelector("#share").addEventListener("click", async () => {
  if (navigator.share && navigator.canShare?.({ url: location.href })) {
    await navigator.share({ title: "HTML5 course", url: location.href });
  } else {
    console.log("Web Share not available here: falling back to copying the link");
    await navigator.clipboard.writeText(location.href);
  }
});
```

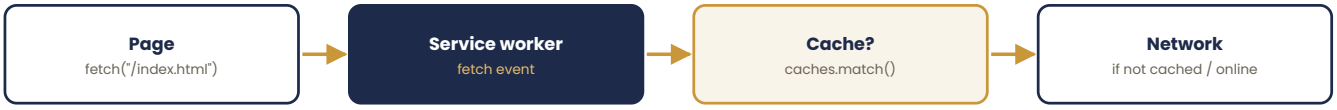
BROWSER CONSOLE

```
copied; clipboard now holds: LBF-HTML5-EARLYBIRD
Web Share not available here: falling back to copying the link
```

Simulated user actions: clicks Copy coupon, then Share (in headless Linux Chromium, where no share sheet exists).

15.8.4 Service workers and offline pages

A **service worker** is a special worker that sits **between the page and the network**: it can intercept every request (the `fetch` event) and answer from a cache, so the site loads instantly and works offline. Together with a web app manifest, it makes a **Progressive Web App (PWA)** installable. Service workers require HTTPS (or localhost) and have their own lifecycle: **register** → **install** (pre-cache files) → **activate** (clean old caches) → **fetch**.



A SERVICE WORKER INTERCEPTS REQUESTS AND CAN ANSWER FROM THE CACHE

A MINIMAL OFFLINE-CAPABLE SERVICE WORKER JavaScript

```
// main.js: register once
if ("serviceWorker" in navigator) {
  navigator.serviceWorker.register("/sw.js");
}

// sw.js: cache the app shell, then serve cache-first with a network fallback
const CACHE = "lbf-v1";
self.addEventListener("install", e => {
  e.waitUntil(caches.open(CACHE).then(c => c.addAll(["/", "/index.html", "/css/style.css",
"/offline.html"])));
});
self.addEventListener("activate", e => {
  e.waitUntil(caches.keys().then(keys => Promise.all(keys.filter(k => k !== CACHE).map(k =>
caches.delete(k)))));
});
self.addEventListener("fetch", e => {
  e.respondWith(
    caches.match(e.request).then(hit => hit || fetch(e.request).catch(() =>
caches.match("/offline.html")))
  );
});
```

TIP

Inspect service workers, caches and storage in DevTools > **Application**. Tick "Offline" in the Network panel to test offline behaviour. Libraries like Google's **Workbox** generate robust service workers for production.

PRACTICE CHECKPOINT

A **drag-and-drop Kanban board** that saves to localStorage, and a simple **drawing pad** built on canvas.

THE PAGE THIS SCRIPT RUNS IN (HTML)

HTML

```
<style>
  body{font-family:system-ui,sans-serif}
  .board{display:flex;gap:8px}
  .col{flex:1;background:#f4f1ea;border-radius:8px;padding:8px;min-height:150px}
  .col h3{margin:0 0 6px;font-size:15px}
  .card{background:#fff;border-left:4px solid #c9a227;padding:6px 8px;margin:6px 0;border-
radius:4px;cursor:grab;box-shadow:0 1px 2px #0002}
  .col.over{outline:2px dashed #1a2a4f}
</style>
<form id="add"><input name="t" placeholder="New task" required> <button>Add</button></form>
<div class="board">
  <section class="col" data-col="todo"><h3>To do</h3></section>
  <section class="col" data-col="doing"><h3>Doing</h3></section>
  <section class="col" data-col="done"><h3>Done</h3></section>
</div>
```

KANBAN.JS: SAMPLE SOLUTION

main.js

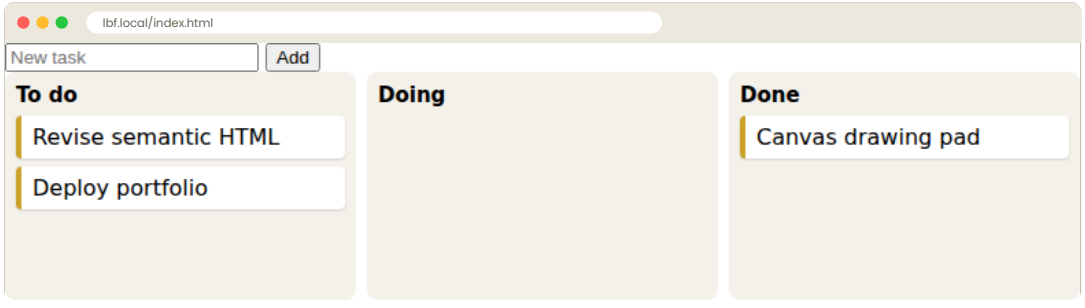
```
const KEY = "kanban";
let tasks = JSON.parse(localStorage.getItem(KEY) ?? "null") ??
  [{ id: 1, text: "Revise semantic HTML", col: "todo" }, { id: 2, text: "Canvas drawing pad", col:
"doing" }];
const save = () => localStorage.setItem(KEY, JSON.stringify(tasks));

function render() {
  document.querySelectorAll(".card").forEach(c => c.remove());
  for (const t of tasks) {
    const card = document.createElement("div");
    card.className = "card"; card.draggable = true; card.dataset.id = t.id; card.textContent = t.text;
    document.querySelector(`[data-col="${t.col}"]`).append(card);
  }
}
document.addEventListener("dragstart", e => e.dataTransfer.setData("text/plain",
e.target.dataset.id));
document.querySelectorAll(".col").forEach(col => {
  col.addEventListener("dragover", e => { e.preventDefault(); col.classList.add("over"); });
  col.addEventListener("dragleave", () => col.classList.remove("over"));
  col.addEventListener("drop", e => {
    e.preventDefault(); col.classList.remove("over");
    const task = tasks.find(t => t.id == e.dataTransfer.getData("text/plain"));
    task.col = col.dataset.col; save(); render();
  });
});
document.getElementById("add").addEventListener("submit", e => {
  e.preventDefault();
  tasks.push({ id: Date.now(), text: e.target.t.value, col: "todo" });
  e.target.reset(); save(); render();
});
render();
console.log("Loaded from storage:", tasks.map(t => t.text + " [" + t.col + "]").join(", "));
```

BROWSER CONSOLE

```
Loaded from storage: Revise semantic HTML [todo], Canvas drawing pad [doing]
Loaded from storage: Revise semantic HTML [todo], Canvas drawing pad [done], Deploy portfolio [todo]
```

Simulated user actions: adds the task "Deploy portfolio", drags "Canvas drawing pad" to Done, then reloads the page. The console line shown is printed on each load; the last one proves the board was restored from localStorage.



THE KANBAN BOARD AFTER THE RELOAD, RESTORED FROM LOCALSTORAGE

THE PAGE THIS SCRIPT RUNS IN (HTML)

HTML

```
<canvas id="pad" width="420" height="180" style="border:1px solid #999;touch-action:none"></canvas>
<p><input type="color" id="color" value="#1a2a4f"> <button id="clear">Clear</button> <button
id="save">Save PNG</button></p>
```

DRAWING-PAD.JS: SAMPLE SOLUTION

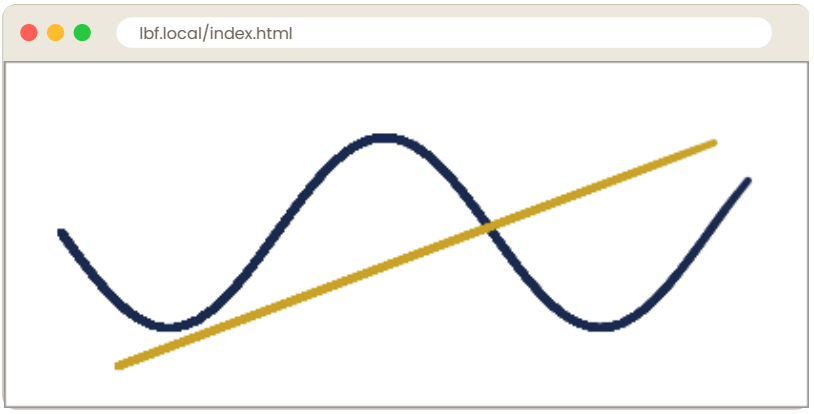
main.js

```
const canvas = document.getElementById("pad"), ctx = canvas.getContext("2d");
ctx.lineWidth = 4; ctx.lineCap = "round"; ctx.strokeStyle = "#1a2a4f";
let drawing = false;
canvas.addEventListener("pointerdown", e => { drawing = true; ctx.beginPath(); ctx.moveTo(e.offsetX,
e.offsetY); });
canvas.addEventListener("pointermove", e => { if (drawing) { ctx.lineTo(e.offsetX, e.offsetY);
ctx.stroke(); } });
addEventListener("pointerup", () => drawing = false);
document.getElementById("color").addEventListener("input", e => ctx.strokeStyle = e.target.value);
document.getElementById("clear").addEventListener("click", () => ctx.clearRect(0, 0, canvas.width,
canvas.height));
document.getElementById("save").addEventListener("click", () => {
  const url = canvas.toDataURL("image/png");            // could be used as a download link href
  console.log("Saved drawing as PNG data URL,", url.length, "characters");
});
```

BROWSER CONSOLE

```
Saved drawing as PNG data URL, 18414 characters
```

Simulated user actions: draws a navy wave and a gold line with pointer events, then clicks Save PNG.



THE DRAWING PAD AFTER THE SIMULATED STROKES

MODULE 16 · ADVANCED

Best practices, validation and tooling

Minimum duration **1 day** | Level **Advanced**

Professional HTML is valid, secure, maintainable and checked by tools. This module covers the W3C Markup Validation Service and the most common validation errors, consistent formatting with Prettier, linting with HTMLHint, naming conventions for classes and ids, separation of concerns, security in HTML (rel="noopener", iframe sandbox, escaping user content and XSS, HTTPS and mixed content), cross-browser compatibility with Can I Use and MDN, progressive enhancement and graceful degradation, and finally Git, GitHub and deploying a static site to GitHub Pages, Netlify or Vercel with an optional custom domain.

IN THIS MODULE

16.1 Validation
16.2 Code quality
16.3 Security in HTML
16.4 Cross-browser compatibility
16.5 Version control and deployment

16.1 Validation

16.1.1 W3C Markup Validation Service

Browsers are extremely forgiving: they silently repair broken HTML, but each browser may repair it differently, and screen readers and search engines see the repaired result. A **validator** checks markup against the HTML standard and lists every error with its line number. The W3C service (**validator.w3.org**, and the newer Nu checker at validator.w3.org/nu) accepts a URL, an uploaded file or pasted code; the same engine (vnu) runs locally and in VS Code extensions.

1. Open validator.w3.org/nu and choose **text input**, **file upload** or **address**.
2. Click **Check**.
3. Fix **errors** first (red), from the top: one early error, such as an unclosed tag, often causes several later ones.
4. Read **warnings** (yellow): not invalid, but usually worth fixing.
5. Repeat until the message is "Document checking completed. No errors or warnings to show."

16.1.2 Common validation errors

The page below contains many beginner mistakes. Next to it is the report format the validator produces for it.

BROKEN.HTML HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>My page</title>
</head>
<body>
  <h1>Welcome<h1>
  <img src="logo.png">
  <p>Courses: <ul><li>HTML</li></ul></p>
  <a href="contact.html"><button>Contact</button></a>
  <div id="main"></div><div id="main"></div>
  <input type="text" name="q" required="true">
  <p>Open <b><i>daily</b></i></p>
  <center>Old tag</center>
</body>
</html>
```

NU HTML CHECKER (ABRIDGED REPORT) index.html

```
validator.w3.org/nu  >  Check by file upload  >  broken.html
```

VALIDATOR REPORT

```
Warning: Consider adding a lang attribute to the html start tag to declare the language of this
document.
Error: Element head is missing a required instance of child element ... (meta charset is recommended
first).  line 3
Error: Unclosed element h1.
line 7
Error: An img element must have an alt attribute, except under certain conditions.
line 8
Error: No p element in scope but a p end tag seen.
line 9
Error: The element button must not appear as a descendant of the a element.
line 10
Error: Duplicate ID main.
line 11
Error: Bad value true for attribute required on element input.
line 12
Error: End tag b violates nesting rules.
line 13
Error: The center element is obsolete. Use CSS instead.
line 14
```

Error	Cause	Fix
Unclosed element	`<h1>Welcome<h1>`	close with `</h1>`
img missing alt	`<img src>` without alt	add meaningful alt or `alt=""`
No p element in scope	a block (ul, div, table) inside <p> closes the p automatically	end the paragraph before the list
Interactive inside interactive	button inside a, a inside a, button inside button	one element: a link styled as a button, or a button
Duplicate ID	the same id twice	ids are unique; use classes for repeats
Bad value for boolean attribute	`required="true"`, `disabled="false"`	just `required` (presence = true; to turn off, remove it)

Error	Cause	Fix
Misnested tags	`<b><i>...</b></i>`	close in reverse order
Obsolete element	center, font, marquee, frameset, big	CSS
Stray end tag / stray doctype	extra , a second doctype	remove

How did Chromium actually repair the broken list paragraph? The DOM it built shows why the validator complains: the paragraph was closed before the list, and an empty extra paragraph was created from the stray `</p>`:

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```
<div id="t"><p>Courses: <ul><li>HTML</li></ul></p></div>
```

THE DOM CHROMIUM BUILT FROM INVALID MARKUP main.js

```
console.log(document.getElementById("t").innerHTML);
```

BROWSER CONSOLE

```
<p>Courses: </p><ul><li>HTML</li></ul><p></p>
```

16.2 Code quality

16.2.1 Consistent formatting with Prettier

Prettier is an opinionated code formatter for HTML, CSS, JavaScript, JSON and Markdown. It reprints your code with consistent indentation, line breaks and quotes, so teams never argue about style and diffs show only real changes. In VS Code install the Prettier extension, set it as the default formatter and enable **Format On Save**. A `.prettierrc` file in the project shares the settings. Here Prettier 3 reformats a messy file:

MESSY.HTML HTML

```
<!doctype html><html lang=en><head><meta charset=utf-8><title>LBF</title></head>
<body><nav><ul><li><a href="/">Home</a></li><li><a href="/courses">Courses</a></li></ul></nav>
<main><h1>HTML5   course</h1><p>Learn    HTML5 in <b>28 days</b> with live classes, projects and
placement support in Mysuru.</p>
<img src=hero.jpg alt="Students" width=800 height=400></main></body></html>
```

RUNNING PRETTIER ON A MESSY FILE

terminal

```
prettier messy.html
```

TERMINAL

```
$ prettier messy.html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>LBF</title>
  </head>
  <body>
    <nav>
      <ul>
        <li><a href="/">Home</a></li>
        <li><a href="/courses">Courses</a></li>
      </ul>
    </nav>
    <main>
      <h1>HTML5 course</h1>
      <p>
        Learn HTML5 in <b>28 days</b> with live classes, projects and placement
        support in Mysuru.
      </p>
      <img src="hero.jpg" alt="Students" width="800" height="400" />
    </main>
  </body>
</html>
```

.PRETTIERRC

text

```
{
  "printWidth": 100,
  "singleQuote": false,
  "htmlWhitespaceSensitivity": "css"
}
```

16.2.2 Linting with HTMLHint (overview)

A **linter** checks code for likely mistakes and policy violations rather than formatting. **HTMLHint** (npm package `htmlhint`, with a VS Code extension) checks rules such as missing alt, duplicate ids, unpaired tags, lowercase tag names and missing doctype, configured in a `.htmlhintrc`. Run it in CI (Continuous Integration) so problems are caught before deployment.

HTMLHINT ON THE BROKEN PAGE terminal

```
npm install --save-dev htmlhint
npx htmlhint "**/*.html"
```

```
TERMINAL
   broken.html
      L7 |  <h1>Welcome<h1>
            ^ Tag must be paired, missing: [ </h1> ], start tag match failed [ <h1> ] on line 7. (tag-
pair)
      L8 |  <img src="logo.png">
            ^ An alt attribute must be present on <img> elements. (alt-require)
      L11 |  <div id="main"></div><div id="main"></div>
                                    ^ The id value [ main ] must be unique. (id-unique)

Scanned 1 files, found 3 errors in 1 files
```

.HTMLHINTRC text

```
{
  "doctype-first": true,
  "tag-pair": true,
  "id-unique": true,
  "alt-require": true,
  "attr-lowercase": true,
  "attr-value-double-quotes": true,
  "tagname-lowercase": true,
  "inline-style-disabled": true
}
```

16.2.3 Naming conventions for classes and IDs

- Use **lower-case, hyphenated** names (kebab-case): `course-card`, `main-nav`, not `CourseCard` or `course_card`.
- Name by **purpose, not appearance**: `.btn-primary`, `.alert-error` instead of `.blue-button`, `.red-text` (which become wrong when the design changes).
- Use **ids** sparingly: for fragment links, label `for`, ARIA references and unique JavaScript hooks. Style with **classes**.
- Many teams use the **BEM** convention (Block__Element--Modifier): `.card`, `.card__title`, `.card--featured` (covered in the CSS3 course).
- Optionally prefix JavaScript-only hooks: `class="js-open-menu"` or use `data-*` attributes, so restyling never breaks scripts.

BEM-STYLE CLASS NAMES HTML

```
<article class="course-card course-card--featured" data-course-id="html5">
  <h3 class="course-card__title">HTML5 Web Foundations</h3>
  <p class="course-card__meta">28 days &middot; Beginner to Advanced</p>
  <a class="btn btn--primary" href="/courses/html5">View syllabus</a>
</article>
```

16.2.4 Separation of concerns (no inline styles or scripts)

Keep **structure** (HTML), **presentation** (CSS) and **behaviour** (JavaScript) in separate files. Inline `style="..."` attributes cannot be reused, cannot use media queries and are hard to override; inline

`onclick="..."` handlers mix logic into markup and are blocked by a strict Content Security Policy. External files are also cached by the browser.

Mixed (avoid)	Separated
`<p style="color:navy;font-weight:bold" onclick="greet()">`	HTML: `<p class="welcome">`
	CSS: `.welcome { color: navy; font-weight: bold; }`
	JS: `document.querySelector(".welcome").addEventListener("click", greet)`

A CLEAN PROJECT STRUCTURE text

```
my-site/
├── index.html
├── about.html
├── contact.html
├── css/
│   └── style.css
├── js/
│   └── main.js
├── images/
│   ├── logo.svg
│   └── hero-800.webp
├── favicon.ico
└── README.md
```

16.3 Security in HTML

16.3.1 rel="noopener" for external links

A page opened with `target="_blank"` used to receive a reference to the opener page (`window.opener`) and could redirect it to a phishing copy ("reverse tabnabbing"). Since 2021 browsers treat `target="_blank"` as `noopener` by default, but adding `rel="noopener noreferrer"` remains good practice for older browsers and to state intent; `noreferrer` also hides your page's URL from the target site.

WHAT A NEW TAB CAN SEE main.js

```
const w = window.open("about:blank", "_blank");
console.log("opener visible to the new tab (default _blank):", w === null ? "no (window.open returned null with noopener)" : w.opener === window);
const w2 = window.open("about:blank", "_blank", "noopener");
console.log("with noopener, window.open returns:", w2);
```

BROWSER CONSOLE

```
opener visible to the new tab (default _blank): true
with noopener, window.open returns: null
```

Run in Chromium 141. window.open (unlike a link) still gives the new tab an opener unless "noopener" is passed.

SAFE EXTERNAL LINK HTML

```
<a href="https://github.com/asharao" target="_blank" rel="noopener noreferrer">GitHub profile (opens in a new tab)</a>
```

16.3.2 iframe sandbox

As shown in 10.3.2, `sandbox` on an iframe disables scripts, forms, pop-ups, top-level navigation and same-origin access, and each token re-enables one capability. Always sandbox third-party or user-generated content you frame, granting only what it needs; combine with `allow` (permissions) and `referrerpolicy`. Conversely, protect **your** pages from being framed by others (clickjacking) with the HTTP header `Content-Security-Policy: frame-ancestors 'self'`.

16.3.3 Escaping user content and XSS awareness

Cross-site scripting (XSS) happens when text supplied by a user (a comment, a name, a search term) is inserted into a page as HTML, so that a script inside it runs in other users' browsers and can steal sessions or act as them. The rule: **treat user input as text, never as HTML**. In JavaScript use `textContent` (not `innerHTML`); on the server, escape `< > & " '` when building HTML (template engines do this automatically); if rich HTML must be allowed, sanitise it with a library such as DOMPurify.

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```html
<div id="unsafe"></div>
<div id="safe"></div>
```

INNERHTML VS TEXTCONTENT WITH A MALICIOUS COMMENT main.js

```js
const comment = '<img src=x onerror="console.log(\'XSS! stolen cookie: \' + document.cookie)">';
document.cookie = "session=abc123";
document.getElementById("unsafe").innerHTML = comment;     // parsed as HTML: the onerror script runs
document.getElementById("safe").textContent = comment;     // shown as plain text: harmless
```

BROWSER CONSOLE

```
XSS! stolen cookie: session=abc123
--- page shows ---
<img src=x onerror="console.log('XSS! stolen cookie: ' + document.cookie)">
```

The console shows the attacker's code running from the `innerHTML` version, while the `textContent` version simply displays the characters. A **Content Security Policy** header (or `<meta http-equiv="Content-Security-Policy">`) that forbids inline scripts adds a second layer of defence.

A CONTENT SECURITY POLICY HTML

```html
<meta http-equiv="Content-Security-Policy"
      content="default-src 'self'; img-src 'self' data:; script-src 'self'; style-src 'self'
https://fonts.googleapis.com">
```

16.3.4 HTTPS and mixed content

HTTPS encrypts traffic between browser and server, proves the server's identity and prevents tampering (such as networks injecting ads). It is required for geolocation, service workers, clipboard, camera and many other APIs, is a ranking signal, and browsers label HTTP pages "Not secure". Hosts such as GitHub Pages, Netlify and Vercel provide free certificates (Let's Encrypt) automatically.

Mixed content is an HTTPS page loading resources over plain HTTP. Browsers **block** active mixed content (scripts, stylesheets, iframes) and automatically upgrade or block passive content (images, audio, video). Fix it by using `https://` (or relative) URLs for every resource.

MIXED CONTENT IN THE CONSOLE

index.html

```
<!-- page served from https://asharao.dev -->
<script src="http://cdn.example.com/lib.js"></script>
<img src="http://images.example.com/photo.jpg" alt="">
```

BROWSER CONSOLE (CHROME)

```
✖ Mixed Content: The page at 'https://asharao.dev/' was loaded over HTTPS, but requested an insecure
  script 'http://cdn.example.com/lib.js'. This request has been blocked; the content must be served
over HTTPS.
⚠ Mixed Content: The page at 'https://asharao.dev/' was loaded over HTTPS, but requested an insecure
  element 'http://images.example.com/photo.jpg'. This request was automatically upgraded to HTTPS.
```

16.4 Cross-browser compatibility

16.4.1 Checking support on Can I Use and MDN

Before relying on a newer feature, check where it works. **caniuse.com** shows a support table per browser version with global usage percentages (search "dialog", "popover", "loading lazy"). **MDN Web Docs** (developer.mozilla.org) has a "Browser compatibility" table at the bottom of every element and API page, and labels features with **Baseline**: "Widely available" (supported in all major browsers for at least 30 months) or "Newly available".

Feature (this book)	Status in 2026	Fallback if unsupported
<dialog>, details name, inert, loading=lazy	Baseline widely available	not needed
popover, <search>, :user-invalid, Declarative Shadow DOM	Baseline (2024)	content still shows / plain form
Invoker commands (command, commandfor)	Baseline newly available (late 2025)	small script calling showModal()
closedby on dialog, popover="hint"	Chromium only at the time of writing	Esc still closes; ignore
Web Share	mobile + some desktop	copy link button

Feature detection in JavaScript lets a page use a feature only where it exists:

FEATURE DETECTION main.js

```
const supports = {
  dialog: typeof HTMLDialogElement === "function",
  popover: HTMLElement.prototype.hasOwnProperty("popover"),
  invokerCommands: "command" in HTMLButtonElement.prototype,
  share: "share" in navigator,
  viewTransitions: "startViewTransition" in document,
  userInvalid: CSS.supports("selector(:user-invalid)"),
};
for (const [feature, ok] of Object.entries(supports)) console.log(feature.padEnd(16), ok);
```

BROWSER CONSOLE

```
dialog           true
popover          true
invokerCommands  true
share            false
viewTransitions  true
userInvalid      true
```

16.4.2 Progressive enhancement and graceful degradation

Progressive enhancement starts from a baseline that works everywhere (semantic HTML: links navigate, forms submit to the server) and adds CSS and JavaScript as **layers** on top for browsers that support them. **Graceful degradation** starts from the full modern experience and makes sure it fails acceptably in older browsers. Progressive enhancement is the more robust approach, because JavaScript can fail for many reasons: a slow network, an error in a third-party script, a corporate proxy, an old phone.



PROGRESSIVE ENHANCEMENT BUILDS UPWARDS FROM HTML THAT WORKS ON ITS OWN

AN ENHANCED SEARCH FORM HTML

```
<!-- Works with no JavaScript: a normal GET form and real links -->
<form action="/search" class="js-live-search">
  <label for="q">Search courses</label>
  <input id="q" name="q" type="search">
  <button>Search</button>
</form>
<script type="module">
  // Enhancement: if JS runs, show results as you type instead of reloading
  const form = document.querySelector(".js-live-search");
  form?.addEventListener("input", async () => { /* fetch and show results */ });
</script>
```

16.5 Version control and deployment

16.5.1 Git and GitHub basics

Git records snapshots (**commits**) of a project so you can see what changed, undo mistakes, work on **branches** and collaborate. **GitHub** hosts Git repositories online, shows your work to employers, and can

publish websites. The daily cycle is **edit** → **add** → **commit** → **push**. These commands really ran in a fresh folder:

INDEX.HTML HTML

```
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8"><title>Asha Rao</title></head>
<body><h1>Asha Rao</h1></body></html>
```

CSS/STYLE.CSS text

```
body { font-family: system-ui; }
```

A FIRST REPOSITORY terminal

```
git init -b main
git config user.name "Asha Rao" && git config user.email "asha@example.com"
git status --short
git add index.html css/style.css
git commit -m "Add home page and styles"
echo '<p>New section</p>' >> index.html
git diff --stat
git commit -am "Add new section to home page"
git log --oneline
```

TERMINAL

```
$ git init -b main
Initialized empty Git repository in .git/
$ git config user.name "Asha Rao" && git config user.email "asha@example.com"
$ git status --short
?? css/
?? index.html
$ git add index.html css/style.css
$ git commit -m "Add home page and styles"
[main (root-commit) c9f12d7] Add home page and styles
 2 files changed, 4 insertions(+)
 create mode 100644 css/style.css
 create mode 100644 index.html
$ echo '<p>New section</p>' >> index.html
$ git diff --stat
 index.html | 1 +
 1 file changed, 1 insertion(+)
$ git commit -am "Add new section to home page"
[main c650dbc] Add new section to home page
 1 file changed, 1 insertion(+)
$ git log --oneline
c650dbc Add new section to home page
c9f12d7 Add home page and styles
```

Command	Does
`git init`	start a repository in the current folder
`git status`	show changed, staged and untracked files
`git add <files>` / `git add .`	stage changes for the next commit
`git commit -m "message"`	save a snapshot with a message
`git log --oneline`	history

Command	Does
`git remote add origin <url>`	connect to a GitHub repository
`git push -u origin main`	upload commits
`git pull`	download and merge others' commits
`git switch -c feature`	create and switch to a branch

PUBLISHING TO GITHUB

terminal

```
git remote add origin https://github.com/asharao/portfolio.git
git push -u origin main
```

TERMINAL

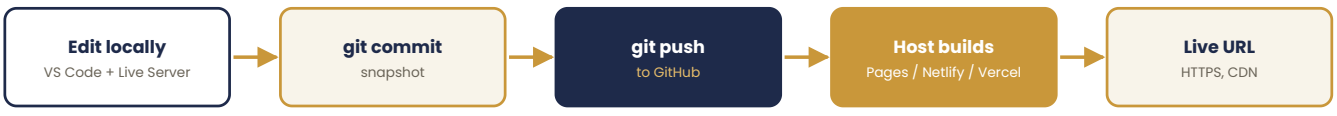
```
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Writing objects: 100% (7/7), 712 bytes | 712.00 KiB/s, done.
To https://github.com/asharao/portfolio.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

TIP

Add a `README.md` describing the project, and a `.gitignore` listing files that should not be committed (`node_modules/` , `.DS_Store` , `.env`). Write commit messages in the imperative: "Add contact form", "Fix broken image path".

16.5.2 Hosting static sites on GitHub Pages, Netlify or Vercel

Host	How to deploy	URL you get	Notes
GitHub Pages	repository Settings > Pages > Deploy from branch: main, folder / (root)	username.github.io/repo	free; perfect for portfolios; index.html must be at the root of the chosen folder
Netlify	drag the folder onto app.netlify.com/drop, or connect the GitHub repo	name.netlify.app	automatic deploys on every push, deploy previews, forms, redirects
Vercel	import the GitHub repository at vercel.com/new	name.vercel.app	automatic deploys, preview URLs per branch; popular for React/Next.js
Cloudflare Pages	connect the repo in the dashboard	name.pages.dev	fast global CDN



CONTINUOUS DEPLOYMENT: EVERY PUSH UPDATES THE LIVE SITE

WATCH OUT

URLs on a live server are **case-sensitive** (Linux): `Images/Logo.PNG` works on Windows but breaks online if the file is `images/logo.png`. Use lower-case file names everywhere. Also check every page link after deploying to a subfolder such as `/portfolio/`: links starting with `/` point to the domain root.

16.5.3 Custom domains (overview)

A custom domain (`asharao.dev`, `asharao.in`) looks professional. Buy it from a registrar (for example GoDaddy, Namecheap, Hostinger, Cloudflare), add it in the host's settings, then create the **DNS records** the host asks for: an **A** or **ALIAS** record for the root domain pointing to the host's IP addresses, and a **CNAME** for `www` pointing to your default host name. DNS changes can take from minutes to a few hours. The host then issues the HTTPS certificate automatically; turn on "Enforce HTTPS".

Type	Name	Value (example for GitHub Pages)
A	@	185.199.108.153 (and .109, .110, .111)
CNAME	www	asharao.github.io
(file)	CNAME in the repository	asharao.dev

PRACTICE CHECKPOINT

Validate all earlier pages with zero errors, put them in a **GitHub repository**, and **deploy** them to a live URL.

1. Run every page through validator.w3.org/nu (or the VS Code validator extension) and fix all errors and warnings.
2. Format with Prettier and fix any HTMLHint warnings.
3. Check accessibility with Lighthouse (score 100 is achievable for these pages) and keyboard-only navigation.
4. Create a repository named `html5-projects` on GitHub; `git init`, add, commit and push.
5. Enable GitHub Pages (or import into Netlify/Vercel) and open the live URL on your phone.
6. Test every link and image on the live site, then add the live URL and repository link to your résumé and LinkedIn.

INDEX.HTML HTML

```
<!doctype html><html lang="en"><head><meta charset="utf-8"><title>Home</title></head><body><h1>Asha Rao</h1></body></html>
```

ABOUT.HTML HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>About</title>
  </head>
  <body>
    <h1>About</h1>
  </body>
</html>
```

FINAL PRE-DEPLOY CHECK (FORMATTING AND FIRST COMMIT)

terminal

```
prettier --check "*.html"
prettier --write "*.html" > /dev/null && prettier --check "*.html"
git init -q -b main && git add . && git -c user.name="Asha" -c user.email="a@x.in" commit -qm
"Validated HTML5 projects"
git log --stat --oneline
```

TERMINAL

```
$ prettier --check "*.html"
Checking formatting...
[warn] about.html
[warn] index.html
[warn] Code style issues found in 2 files. Run Prettier with --write to fix.
$ prettier --write "*.html" > /dev/null && prettier --check "*.html"
Checking formatting...
All matched files use Prettier code style!
$ git init -q -b main && git add . && git -c user.name="Asha" -c user.email="a@x.in" commit -qm
"Validated HTML5 projects"
$ git log --stat --oneline
e08a2b6 Validated HTML5 projects
 about.html | 10 ++++++++++
 index.html | 10 ++++++++++
 2 files changed, 20 insertions(+)
```

MODULE 17 · ADVANCED

Projects and interview preparation

Minimum duration **3 days** | Level **Advanced**

The final module brings everything together in portfolio projects and prepares you for interviews. It specifies five capstone projects (a personal portfolio, a restaurant or college website, an online résumé with structured data, a fully validated survey or quiz, and a product landing page), with requirements, file structure, key code and a screenshot of a reference build for each. It then gives a core-topics checklist, answers to the questions interviewers ask most, a method for explaining your projects, and the next steps: CSS3, JavaScript and front-end frameworks.

IN THIS MODULE

17.1 Capstone projects
17.2 Interview preparation
17.3 Next steps

17.1 Capstone projects

Each project below lists the **requirements** a reviewer will check. Build them without CSS frameworks; a little CSS for readability is fine, but the marks are for the HTML. Every project must pass the W3C validator with zero errors, score 100 for Accessibility in Lighthouse, work with the keyboard alone, and be deployed to a live URL (Module 16).

Project	Main skills shown	Days
1. Personal portfolio (multi-page)	semantics, navigation, accessibility, metadata, deployment	1
2. Restaurant or college website	tables, booking form with validation, images, maps	0.5
3. Online résumé	structured data (JSON-LD Person), print-friendly semantics	0.5
4. Survey or quiz form	every input type, validation, fieldset/legend, output	0.5
5. Product landing page	media, responsive images, SEO and social metadata, performance	0.5

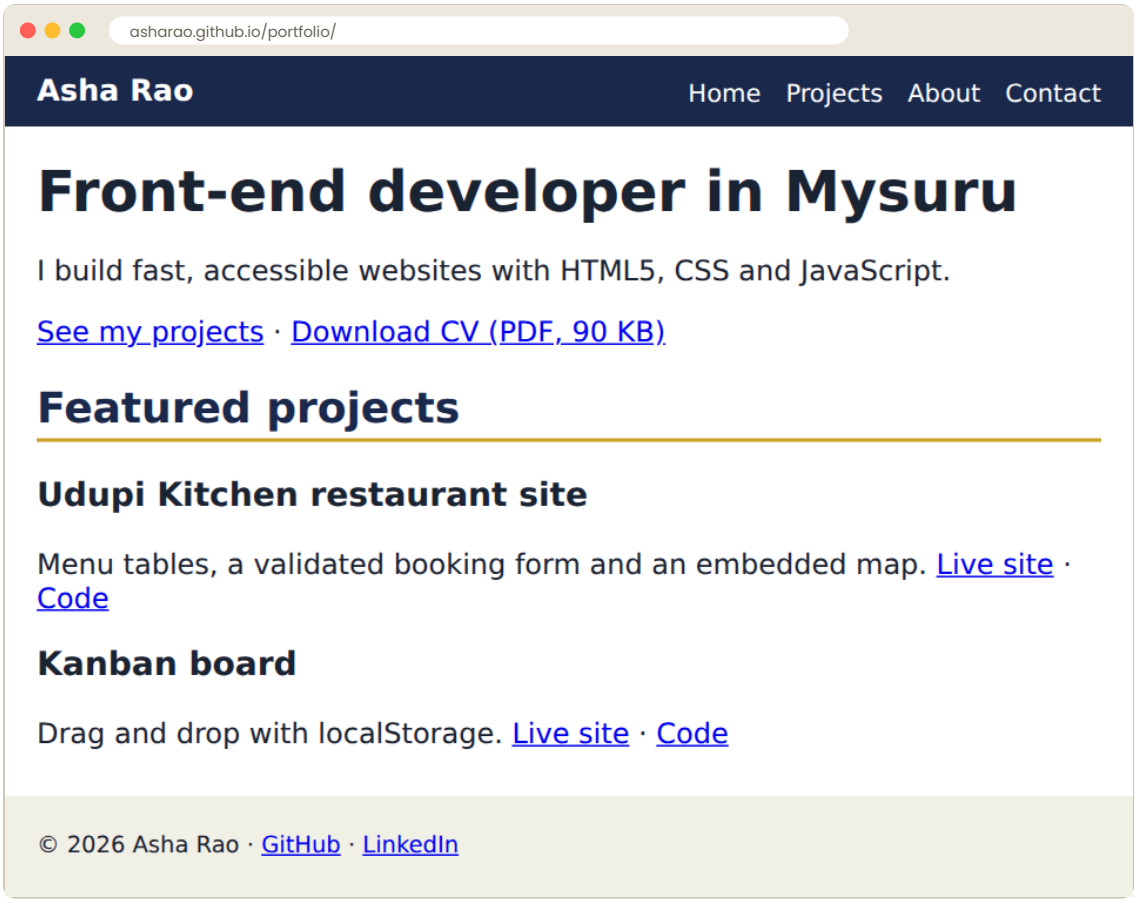
17.1.1 Personal portfolio website (multi-page, semantic, accessible)

- Pages: `index.html` (intro and featured projects), `projects.html`, `about.html`, `contact.html`.
- Shared header with logo link and `<nav aria-label="Main">`; current page marked with `aria-current="page"`; skip link.
- Each project as an `<article>` with h3, screenshot (alt text), short description, tech list and links to live demo and code.
- About page with a photo (`<figure>`), skills as a list, education as a timeline (`<ol>` with `<time>`).
- Contact page with an accessible form (name, email, subject select, message, consent checkbox) posting to a form service such as Formspree or Netlify Forms.

- Complete head on every page: unique title and description, canonical, Open Graph, favicon; JSON-LD Person on About.

INDEX.HTML (BODY) HTML

```
<a class="skip" href="#main">Skip to main content</a>
<header>
  <a href="index.html" style="margin:0;font-weight:700;font-size:17px">Asha Rao</a>
  <nav aria-label="Main"><a href="index.html" aria-current="page">Home</a><a
href="projects.html">Projects</a><a href="about.html">About</a><a href="contact.html">Contact</a>
</nav>
</header>
<main id="main">
  <section aria-labelledby="intro">
    <h1 id="intro">Front-end developer in Mysuru</h1>
    <p>I build fast, accessible websites with HTML5, CSS and JavaScript.</p>
    <p><a href="projects.html">See my projects</a> &middot; <a href="assets/asha-rao-cv.pdf">Download
CV (PDF, 90 KB)</a></p>
  </section>
  <section aria-labelledby="featured">
    <h2 id="featured">Featured projects</h2>
    <article>
      <h3>Udupi Kitchen restaurant site</h3>
      <p>Menu tables, a validated booking form and an embedded map. <a
href="https://asharao.github.io/udupi-kitchen/">Live site</a> &middot; <a
href="https://github.com/asharao/udupi-kitchen">Code</a></p>
    </article>
    <article>
      <h3>Kanban board</h3>
      <p>Drag and drop with localStorage. <a href="#">Live site</a> &middot; <a href="#">Code</a></p>
    </article>
  </section>
</main>
<footer><p>&copy; 2026 Asha Rao &middot; <a href="https://github.com/asharao">GitHub</a> &middot; <a
href="https://www.linkedin.com/in/asharao">LinkedIn</a></p></footer>
```



PORTFOLIO HOME PAGE (LIGHT CSS FOR READABILITY)

17.1.2 Restaurant or college website with menu tables and a booking form

- Home page with a hero image (responsive `srcset`, `fetchpriority="high"`), opening hours in a `<dl>` or table, address in `<address>` and an embedded map (lazy iframe with title).
- Menu (or course list) as tables with `<caption>`, `<thead>`, row headers with `scope`, and prices; or a definition list per category.
- Booking (or admission enquiry) form: name, phone with pattern, email, date with min = today, time with min/max opening hours, guests (number 1-12), seating radio group in a fieldset, special requests textarea, consent checkbox.

MENU.HTML AND BOOKING FORM (EXCERPT) HTML

```
<main>
  <h1>Udupi Kitchen</h1>
  <table>
    <caption style="font-weight:700;text-align:left;padding:4px 0">Breakfast menu (7 am to 11 am)
</caption>
    <thead><tr><th scope="col">Dish</th><th scope="col">Description</th><th scope="col">Price
(&#8377;)</th></tr></thead>
    <tbody>
      <tr><th scope="row" style="background:none;color:inherit;text-align:left">Masala dosa</th>
<td>Crisp dosa, potato palya, chutney</td><td>90</td></tr>
      <tr><th scope="row" style="background:none;color:inherit;text-align:left">Idli vada</th><td>2
idlis, 1 vada, sambar</td><td>70</td></tr>
      <tr><th scope="row" style="background:none;color:inherit;text-align:left">Filter coffee</th>
<td>Served in a davara</td><td>30</td></tr>
    </tbody>
  </table>
  <h2>Book a table</h2>
  <form action="https://formspree.io/f/your-id" method="post">
    <p><label>Name <input name="name" autocomplete="name" required></label>
       <label>Phone <input name="phone" type="tel" pattern="[6-9][0-9]{9}" required></label></p>
    <p><label>Date <input type="date" name="date" min="2026-10-08" required></label>
       <label>Time <input type="time" name="time" min="07:00" max="22:00" step="900" required></label>
       <label>Guests <input type="number" name="guests" min="1" max="12" value="2"></label></p>
    <fieldset><legend>Seating</legend>
      <label><input type="radio" name="seat" value="indoor" checked> Indoor</label>
      <label><input type="radio" name="seat" value="garden"> Garden</label></fieldset>
    <p><button>Request booking</button></p>
  </form>
</main>
```



MENU TABLE AND BOOKING FORM

17.1.3 Online resume page with structured data

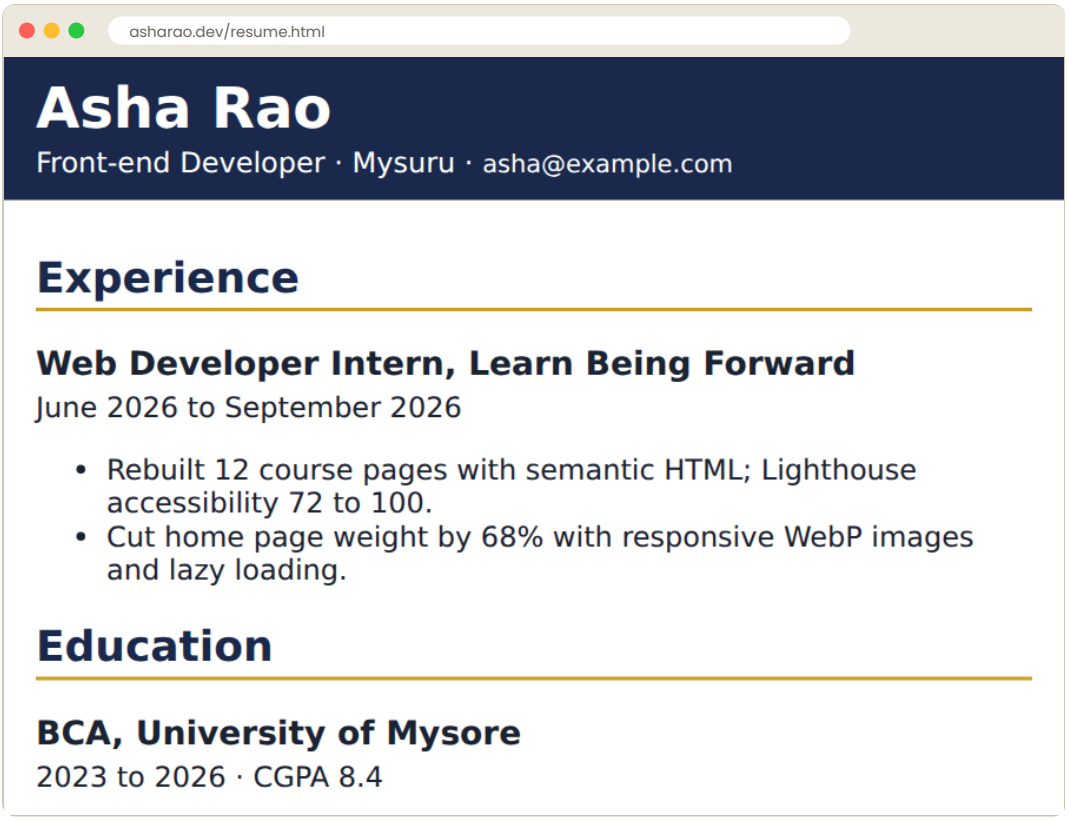
- One page: header with name, title and contact links (`mailto:`, `tel:`), sections for Summary, Skills, Experience, Projects, Education, Certifications.
- Each job/education entry as an `<article>` with h3, organisation, dates with `<time datetime>` and a list of achievements.
- JSON-LD `Person` with name, jobTitle, address, email, url, sameAs (GitHub, LinkedIn), alumniOf, knowsAbout.
- Prints cleanly to PDF (the CSS course adds `@media print`; in HTML keep it to a single column and real text, not images).

RESUME.HTML: STRUCTURED DATA HTML

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Person",
  "name": "Asha Rao",
  "jobTitle": "Front-end Developer",
  "email": "mailto:asha@example.com",
  "url": "https://asharao.dev",
  "address": { "@type": "PostalAddress", "addressLocality": "Mysuru", "addressRegion": "Karnataka",
"addressCountry": "IN" },
  "alumniOf": { "@type": "CollegeOrUniversity", "name": "University of Mysore" },
  "knowsAbout": ["HTML5", "CSS3", "JavaScript", "Accessibility", "SEO"],
  "sameAs": ["https://github.com/asharao", "https://www.linkedin.com/in/asharao"]
}
</script>
```

RESUME.HTML (EXCERPT) HTML

```
<header style="display:block"><h1 style="margin:0">Asha Rao</h1><p style="margin:2px 0">Front-end
Developer &middot; Mysuru &middot;
  <a href="mailto:asha@example.com" style="margin:0">asha@example.com</a></p></header>
<main>
  <section aria-labelledby="exp"><h2 id="exp">Experience</h2>
    <article><h3 style="margin:4px 0">Web Developer Intern, Learn Being Forward</h3>
      <p style="margin:2px 0"><time datetime="2026-06">June 2026</time> to <time datetime="2026-
09">September 2026</time></p>
      <ul><li>Rebuilt 12 course pages with semantic HTML; Lighthouse accessibility 72 to 100.</li>
          <li>Cut home page weight by 68% with responsive WebP images and lazy loading.</li></ul>
</article>
  </section>
  <section aria-labelledby="edu"><h2 id="edu">Education</h2>
    <article><h3 style="margin:4px 0">BCA, University of Mysore</h3><p style="margin:2px 0"><time
datetime="2023">2023</time> to <time datetime="2026">2026</time> &middot; CGPA 8.4</p></article>
  </section>
</main>
```



THE RÉSUMÉ PAGE

17.1.4 Survey or quiz form with full validation

- At least eight question types: text, email, number with range, date, radio group, checkbox group (with a "choose at least one" rule), select, range with live `<output>`, textarea with maxlength and a character counter.
- Each question group in a fieldset with legend; required markers explained once; hints linked with aria-describedby.
- Styling with `:user-invalid`; errors in text; a custom rule with `setCustomValidity` (for example, at least one checkbox).
- For a quiz: on submit, prevent the default, score the answers in JavaScript and show the result in an `<output>` or a `<dialog>`.

THE PAGE THIS SCRIPT RUNS IN (HTML) HTML

```
<form>
  <fieldset><legend>1. Which element marks the main navigation?</legend>
    <label><input type="radio" name="q1" value="a" required> &lt;menu&gt;</label>
    <label><input type="radio" name="q1" value="b"> &lt;nav&gt;</label>
    <label><input type="radio" name="q1" value="c"> &lt;navigation&gt;</label></fieldset>
  <fieldset><legend>2. Which attribute makes an image load only near the viewport?</legend>
    <label><input type="radio" name="q2" value="a" required> defer</label>
    <label><input type="radio" name="q2" value="b"> async</label>
    <label><input type="radio" name="q2" value="c"> loading="lazy"</label></fieldset>
  <button>Submit</button>
  <output name="result"></output>
</form>
```

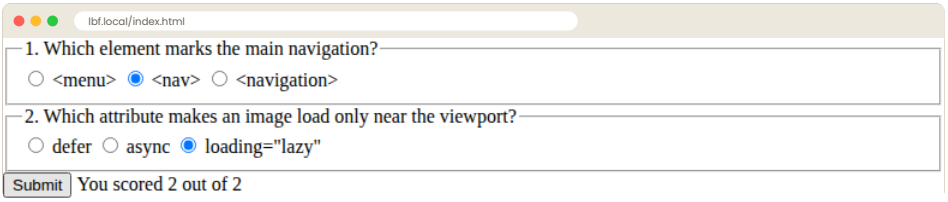
QUIZ.JS: SCORING WITH BUILT-IN VALIDATION main.js

```
const form = document.querySelector("form");
const answers = { q1: "b", q2: "c" };
form.addEventListener("submit", e => {
  e.preventDefault();
  if (!form.reportValidity()) return;
  const score = Object.entries(answers).filter(([q, a]) => form[q].value === a).length;
  form.result.value = `You scored ${score} out of ${Object.keys(answers).length}`;
  console.log(form.result.value);
});
const reported = new Set();                       // each radio in a group fires "invalid": log once
per group
form.addEventListener("invalid", e => {
  if (!reported.has(e.target.name)) console.log("invalid:", e.target.name, "-",
e.target.validationMessage);
  reported.add(e.target.name);
}, true);
```

BROWSER CONSOLE

```
invalid: q1 - Please select one of these options.
invalid: q2 - Please select one of these options.
You scored 1 out of 2
You scored 2 out of 2
```

Simulated user actions: submits with nothing chosen, answers q1 correctly and q2 wrongly and submits, then corrects q2 and submits again.



THE QUIZ AFTER THE FINAL SUBMISSION

17.1.5 Product landing page with media and SEO metadata

- Sections: hero (h1, value proposition, primary call to action), features (list or article cards with inline SVG icons), demo video with captions, testimonials (`<blockquote>` + `<cite>`), pricing (table or cards), FAQ (exclusive accordion), final call to action, footer.
- Hero image with `<picture>` (AVIF/WebP/JPEG), width/height, fetchpriority="high"; every other image lazy.
- Full head: title under 60 characters, meta description, canonical, Open Graph and X card with a 1200×630 image, theme-color, JSON-LD Product with offers (and FAQPage for the FAQ).
- Lighthouse targets: Performance 90+, Accessibility 100, Best Practices 100, SEO 100.

PRODUCT STRUCTURED DATA HTML

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Product",
  "name": "LBF StudyPad",
  "description": "A distraction-free note-taking tablet for students.",
  "image": "https://studypad.example/og/studypad.jpg",
  "brand": { "@type": "Brand", "name": "LBF" },
  "offers": { "@type": "Offer", "price": "14999", "priceCurrency": "INR", "availability":
"https://schema.org/InStock" }
}
</script>
```

17.2 Interview preparation

17.2.1 Core topics checklist

Topic	Be able to explain	Module
Doctype	why `<!DOCTYPE html>` is needed (standards mode vs quirks mode)	2
Document structure	html, head, body; what belongs in head; charset first	2, 11
Semantic elements	header, nav, main, article, section, aside, footer, search; landmarks	9
Block vs inline	display behaviour, nesting rules, examples of each	2
Forms and validation	GET vs POST, name/value, input types, constraint attributes, :user-invalid, server-side validation	7, 8
Meta tags	viewport, description, robots, Open Graph, canonical	11
Accessibility	alt text, labels, heading order, keyboard, ARIA rules, WCAG AA	12
defer vs async	parsing, order of execution, modules	11, 13
Images and performance	srcset/sizes, picture, lazy loading, Core Web Vitals	13
HTML5 APIs	storage, canvas, geolocation, drag and drop, history, workers	15
Security	XSS and escaping, noopener, sandbox, HTTPS	16

17.2.2 Common questions

Q. What is the difference between `<div>` and `<span>`?
Both are generic containers with no meaning. `div` is block-level (starts on a new line, takes the full width) and groups larger sections; `span` is inline (flows within text) and wraps a phrase. Use them only when no semantic element fits.

Q. When do you use `<article>` vs `<section>`?
`article` is self-contained and makes sense on its own (blog post, product card, comment). `section` is a thematic part of a larger whole, normally with a heading (chapters, "Features", "Pricing"). If neither applies and it is just for styling, use a div.

Q. What is the difference between `alt` and `title`?
`alt` is the text alternative that replaces an image for screen readers, when the image fails to load, and for search engines; it is required on every img. `title` is advisory extra information shown as a tooltip on mouse hover; it is not reliably read by assistive technology and invisible on touch screens, so never put essential information only in title.

Q. localStorage vs sessionStorage vs cookies?
localStorage: about 5 MB per origin, persists until deleted, never sent to the server. sessionStorage: same API but cleared when the tab closes, and separate per tab. Cookies: about 4 KB, sent with every HTTP request to their domain, can have an expiry, and can be HttpOnly (unreadable by JavaScript) and Secure, which makes them the right choice for session identifiers.

Q. What does the doctype do?
It tells the browser to render in standards mode. Without it browsers use quirks mode, which emulates old, inconsistent behaviours (for example a different box model) and breaks modern layouts.

Q. defer vs async?
Both download without blocking parsing. async runs as soon as it arrives, in no particular order, possibly before parsing finishes; defer runs after the document is parsed, before DOMContentLoaded, in document order. Use defer for your app scripts and async for independent scripts such as analytics. Module scripts are deferred by default.

Q. What are semantic elements and why use them?
Elements that describe their meaning (nav, main, article, button, time). They give accessibility (landmarks, roles), better SEO, built-in keyboard behaviour and more readable code, without extra attributes.

Q. How do you make a website accessible?
Semantic HTML first; text alternatives; labelled forms with clear errors; logical heading order and landmarks; full keyboard support with visible focus; sufficient contrast; captions; ARIA only where native HTML cannot express something; test with keyboard, a screen reader and Lighthouse/axe against WCAG 2.2 AA.

Q. What is the difference between GET and POST?
GET puts the data in the URL query string: visible, bookmarkable, cached, size-limited; for reading and searching. POST sends data in the request body; for creating or changing data, logins and file uploads. Neither is encryption; HTTPS is.

Q. What is the purpose of the viewport meta tag?
It tells mobile browsers to set the layout width to the device width and the initial zoom to 1, instead of rendering a 980px desktop page and shrinking it. Without it, responsive CSS does not work on phones.

Q. What are data-* attributes?
Custom attributes for storing extra information on elements (`data-course-id="7"`) that is not visible and has no meaning to the browser. JavaScript reads them through `element.dataset.courseId`; CSS can select them with `[data-state="open"]`.

Q. What is the difference between `<b>` and `<strong>` (and i vs em)?
`strong` marks strong importance and `em` stress emphasis that changes meaning; `b` and `i` draw attention or mark an alternate voice (keywords, technical terms, foreign words) without extra importance. All may look the same; choose by meaning.

Q. How do srcset and sizes work?
srcset lists image files with their widths (400w, 800w) or densities (2x); sizes tells the browser how wide

the image will be displayed at each breakpoint. The browser multiplies the slot width by the device pixel ratio and picks the smallest sufficient file.

Q. What is the difference between `<section>` and `<div>` for accessibility?
A div exposes nothing. A section with an accessible name (aria-labelledby on its heading) becomes a region landmark that screen-reader users can jump to.

Q. What does `<meta charset="UTF-8">` do and where should it go?
It declares the character encoding so every language and symbol displays correctly. It must be within the first 1024 bytes, so it is the first element in head.

Q. What is the difference between an id and a class?
An id must be unique in the page and is used for fragment links, label for, ARIA references and unique JavaScript hooks; a class can be shared by many elements and is the normal hook for CSS. Ids also have much higher CSS specificity.

Q. What are void (self-closing) elements?
Elements that cannot have content and have no end tag: br, hr, img, input, meta, link, source, track, wbr, area, base, col, embed. In HTML the trailing slash (`<br />`) is allowed but ignored.

Q. How does the browser render a page?
Download HTML, parse it into the DOM; download and parse CSS into the CSSOM; combine them into the render tree; layout (compute positions and sizes); paint pixels; composite layers. Render-blocking CSS and parser-blocking scripts delay this, which is why scripts use defer.

Q. What is the difference between `<canvas>` and SVG?
Canvas is a scriptable bitmap: fast for many objects and pixel work but not accessible or scalable by itself. SVG is vector markup in the DOM: sharp at any size, styleable with CSS, each shape can have events and accessible text; slower with thousands of elements.

Q. What is the difference between `<dialog>`.show() and showModal()?
show() opens a non-modal dialog and leaves the page interactive. showModal() opens it in the top layer with a backdrop, makes the rest of the page inert, traps focus and enables Esc to close.

17.2.3 Explaining your projects

Interviewers almost always ask "Tell me about a project you built". Prepare a two-minute answer for each project using **STAR**: Situation, Task, Action, Result. Mention specific HTML decisions and measurable results.

Step	Example (restaurant website)
Situation	A local restaurant had only a Facebook page; customers phoned to ask the menu and book tables.
Task	Build a fast, accessible website with the menu and an online booking form.
Action	Semantic multi-page site; menu as captioned tables with row headers; booking form with date/time limits, phone pattern and server-side checks through Formspree; responsive WebP images with lazy loading; Open Graph tags so WhatsApp shares show a preview; deployed on GitHub Pages with a custom domain.
Result	Lighthouse 98 / 100 / 100 / 100; zero validator errors; 40 bookings in the first month; I learned how much HTML alone does for accessibility.

- Keep the live link and repository ready on your phone and laptop.
- Be ready to open DevTools and explain your markup, the accessibility tree and a Lighthouse report.

- Know one thing you would improve next and why: it shows you can evaluate your own work.

17.3 Next steps

17.3.1 CSS3 for styling and layout

HTML gives structure and meaning; **CSS** gives design: colours, typography, the box model, Flexbox and Grid layouts, responsive design with media queries, transitions and animations. The LBF CSS3 course picks up exactly where this book ends, styling the portfolio you have built.

17.3.2 JavaScript for interactivity

Module 15 used JavaScript at a basic level. The **JavaScript** course covers the language properly: variables, functions, objects, arrays, the DOM in depth, events, asynchronous code with promises and async/await, fetch and APIs, modules and tooling.

17.3.3 Front-end frameworks

Large applications use a **framework** or library that builds the DOM from data and components: **React** (most widely used, components written in JSX), **Angular** (a complete TypeScript framework used by many enterprises) and **Vue** (approachable, with HTML-based templates). All of them still output HTML, so everything in this book (semantics, forms, accessibility, metadata) applies directly. Utility-first CSS such as **Tailwind CSS** is a popular partner for all three.



THE FRONT-END LEARNING PATH

PRACTICE CHECKPOINT

Build and deploy a **fully semantic, accessible and validated personal portfolio** site with metadata, ready to be styled in the CSS course.

Final checklist before you call it done:

Area	Check
Validity	W3C validator: no errors or warnings on every page
Semantics	one h1 per page, no skipped levels; header, nav, main, footer; articles for projects
Accessibility	Lighthouse 100; keyboard-only run through every page; skip link; alt text reviewed by reading it aloud
Forms	labels, autocomplete tokens, validation, helpful errors; working submission
Metadata	unique title and description; canonical; Open Graph tested in a sharing debugger; JSON-LD Person
Performance	responsive images, lazy loading below the fold, width/height everywhere, defer on scripts
Security	HTTPS, rel="noopener noreferrer" on external links, no inline scripts
Deployment	live URL works on a phone; all links and images load; repository with README

Congratulations on completing HTML5 Web Foundations. You can now write HTML that is valid, semantic, accessible, fast and search-friendly, the foundation on which every CSS, JavaScript and framework skill is built.